



SaltoGestão

Código Fonte Completo

Sistema ERP para Gestão Empresarial

FastAPI + Next.js + Flet Desktop + PostgreSQL/SQLite

Corumbá Sistemas

Janeiro 2026

Desenvolvido por: Corumbá Sistemas
88 arquivos de código fonte

Índice de Arquivos

#	Arquivo	Seção
1	main.py	1. Ponto de Entrada e Configuracao
2	dashboard.py	1. Ponto de Entrada e Configuracao
3	clientes.py	1. Ponto de Entrada e Configuracao
4	produtos.py	1. Ponto de Entrada e Configuracao
5	licenca.py	1. Ponto de Entrada e Configuracao
6	auth.py	1. Ponto de Entrada e Configuracao
7	web/app.py	1. Ponto de Entrada e Configuracao
8	web/index.html	1. Ponto de Entrada e Configuracao
9	docker-compose.yml	1. Ponto de Entrada e Configuracao
10	backendPy/database.py	2. Backend - Nucleo (database, models, security)
11	backendPy/models.py	2. Backend - Nucleo (database, models, security)
12	backendPy/models_core.py	2. Backend - Nucleo (database, models, security)
13	backendPy/models_historico.py	2. Backend - Nucleo (database, models, security)
14	backendPy/schemas.py	2. Backend - Nucleo (database, models, security)
15	backendPy/security.py	2. Backend - Nucleo (database, models, security)
16	backendPy/historico.py	2. Backend - Nucleo (database, models, security)
17	backendPy/licenca.py	2. Backend - Nucleo (database, models, security)
18	backendPy/main.py	3. Backend - App Principal
19	backendPy/dev.py	3. Backend - App Principal
20	backendPy/alembic/env.py	3. Backend - App Principal
21	backendPy/routers/_init_.py	4. Backend - Roters (API Endpoints)
22	backendPy/routers/login_operacional.py	4. Backend - Roters (API Endpoints)
23	backendPy/routers/auth.py	4. Backend - Roters (API Endpoints)
24	backendPy/routers/admin_auth.py	4. Backend - Roters (API Endpoints)
25	backendPy/routers/admin.py	4. Backend - Roters (API Endpoints)
26	backendPy/routers/produtos.py	4. Backend - Roters (API Endpoints)
27	backendPy/routers/clientes.py	4. Backend - Roters (API Endpoints)
28	backendPy/routers/vendas.py	4. Backend - Roters (API Endpoints)
29	backendPy/routers/caixa.py	4. Backend - Roters (API Endpoints)
30	backendPy/routers/usuarios.py	4. Backend - Roters (API Endpoints)
31	backendPy/routers/fornecedores.py	4. Backend - Roters (API Endpoints)
32	backendPy/routers/dashboard.py	4. Backend - Roters (API Endpoints)
33	backendPy/routers/painel.py	4. Backend - Roters (API Endpoints)
34	backendPy/routers/historico.py	4. Backend - Roters (API Endpoints)
35	backendPy/routers/router_admin_clientes.py	4. Backend - Roters (API Endpoints)
36	backendPy/seed_admin.py	5. Backend - Seeds (Dados de Teste)
37	backendPy/seed_clientes.py	5. Backend - Seeds (Dados de Teste)
38	backendPy/seed_inicial.py	5. Backend - Seeds (Dados de Teste)
39	backendPy/seed_produtos.py	5. Backend - Seeds (Dados de Teste)
40	backendPy/seed_test.py	5. Backend - Seeds (Dados de Teste)
41	backendPy/seed_test_local.py	5. Backend - Seeds (Dados de Teste)
42	backendPy/templates/painel.html	6. Backend - Templates e Static
43	backendPy/static/desktop/main.py	6. Backend - Templates e Static
44	backendPy/requirements.txt	6. Backend - Templates e Static
45	frontend-web/app/layout.tsx	7. Frontend Web - Next.js (Tiposcript/React)
46	frontend-web/app/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
47	frontend-web/app/login/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
48	frontend-web/app/globals.css	7. Frontend Web - Next.js (Tiposcript/React)
49	frontend-web/app/dashboard/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
50	frontend-web/app/dashboard/mesapronta/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)

51	frontend-web/app/dashboard/salto-gestao/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
52	frontend-web/app/dashboard/salto-gestao/metricas/route.ts	7. Frontend Web - Next.js (Tiposcript/React)
53	frontend-web/app/admin/login/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
54	frontend-web/app/admin/dashboard/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
55	frontend-web/app/admin/clientes/criar/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
56	frontend-web/app/admin/clientes/listar/page.tsx	7. Frontend Web - Next.js (Tiposcript/React)
57	frontend-web/app/api/login/route.ts	8. Frontend Web - API Routes (Proxy)
58	frontend-web/app/api/admin/auth/login/route.ts	8. Frontend Web - API Routes (Proxy)
59	frontend-web/app/api/admin/clientes/route.ts	8. Frontend Web - API Routes (Proxy)
60	frontend-web/app/api/admin/clientes/criar/route.ts	8. Frontend Web - API Routes (Proxy)
61	frontend-web/app/api/admin/dashboard/route.ts	8. Frontend Web - API Routes (Proxy)
62	frontend-web/app/api/admin/produtos/route.ts	8. Frontend Web - API Routes (Proxy)
63	frontend-web/app/api/cliente/funcionarios/route.ts	8. Frontend Web - API Routes (Proxy)
64	frontend-web/app/api/cliente/funcionarios/cadastrar/route.ts	8. Frontend Web - API Routes (Proxy)
65	frontend-web/app/api/cliente/funcionarios/[id]/route.ts	8. Frontend Web - API Routes (Proxy)
66	frontend-web/app/api/cliente/licenca/route.ts	8. Frontend Web - API Routes (Proxy)
67	frontend-web/app/api/painel/produtos/route.ts	8. Frontend Web - API Routes (Proxy)
68	frontend-web/app/api/proxy/mesapronta/[...slug]/route.ts	8. Frontend Web - API Routes (Proxy)
69	frontend-web/app/api/seed/route.ts	8. Frontend Web - API Routes (Proxy)
70	frontend-web/app/api/usuarios/route.ts	8. Frontend Web - API Routes (Proxy)
71	frontend-web/next.config.ts	9. Frontend Web - Configuracao
72	frontend-web/package.json	9. Frontend Web - Configuracao
73	frontend-web/tsconfig.json	9. Frontend Web - Configuracao
74	frontend-web/AGENTS.md	9. Frontend Web - Configuracao
75	frontend-web/next-env.d.ts	9. Frontend Web - Configuracao
76	desktop/main.py	10. Desktop - Flet (Python)
77	desktop/main_flet.py	10. Desktop - Flet (Python)
78	desktop/main_v2.py	10. Desktop - Flet (Python)
79	desktop/design_system.py	10. Desktop - Flet (Python)
80	desktop/testar_100_clientes.py	10. Desktop - Flet (Python)
81	desktop/utils/_init_.py	10. Desktop - Flet (Python)
82	desktop/utils/api_client.py	10. Desktop - Flet (Python)
83	desktop/utils/config.py	10. Desktop - Flet (Python)
84	desktop/README.md	10. Desktop - Flet (Python)
85	desktop/requirements.txt	10. Desktop - Flet (Python)
86	testar_100_clientes_api.py	11. Testes e Documentacao
87	documentos/README.md	11. Testes e Documentacao
88	sites/README.md	11. Testes e Documentacao

1. Ponto de Entrada e Configuracao

main.py

main.py • 1 linhas

[ARQUIVO NAO ENCONTRADO: main.py]

dashboard.py

dashboard.py • 113 linhas

```
"""
routers/dashboard.py - Metricas e informacoes consolidadas
"""
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from datetime import datetime, timedelta

from database import get_db_salto, get_db_core
from models import Venda, Produto, Cliente, EstoqueMovimentacao
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/dashboard", tags=["Dashboard"])

@router.get("/")
def get_dashboard(
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    hoje = datetime.utcnow().date()
    ontem = hoje - timedelta(days=1)
    inicio_mes = hoje.replace(day=1)

    def _date_str(d):
        return d.isoformat()

    # ---- Vendas hoje ----
    vendas_hoje_q = db.query(func.sum(Venda.total), func.count(Venda.id), func.avg(Venda.total)).filter(
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA",
        func.date(Venda.data_venda) == hoje
    ).first()
    vendas_hoje = float(vendas_hoje_q[0] or 0)
    qtd_vendas_hoje = int(vendas_hoje_q[1] or 0)
    ticket_medio_hoje = float(vendas_hoje_q[2] or 0)

    # ---- Vendas ontem ----
    vendas_ontem_q = db.query(func.sum(Venda.total)).filter(
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA",
        func.date(Venda.data_venda) == ontem
    ).first()
    vendas_ontem = float(vendas_ontem_q[0] or 0)

    # ---- Vendas mes ----
    vendas_mes_q = db.query(func.sum(Venda.total)).filter(
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA",
        func.date(Venda.data_venda) >= inicio_mes
    ).first()
    vendas_mes = float(vendas_mes_q[0] or 0)

    # ---- Estoque baixo ----
    produtos_estoque_baixo = db.query(func.count(Produto.id)).filter(
        Produto.empresa_documento == empresa_documento,
        Produto.ativo == True,
        Produto.estoque_atual <= Produto.estoque_minimo
    ).scalar() or 0

    # ---- Clientes ativos ----
    clientes_ativos = db.query(func.count(Cliente.id)).filter(
        Cliente.empresa_documento == empresa_documento,
        Cliente.ativo == True
    ).scalar() or 0

    # ---- Vendedores do dia (who made sales today) ----
    vendedores_dia = db.query(
        Venda.vendedor_nome,
        func.count(Venda.id).label("qtd_vendas"),
        func.sum(Venda.total).label("total_vendido")
    ).filter(
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA",
        func.date(Venda.data_venda) == hoje
    ).group_by(Venda.vendedor_nome).order_by(func.sum(Venda.total).desc()).all()
```

```
# ---- Ultimas vendas ----
ultimas = db.query(Venda).filter(
    Venda.empresa_documento == empresa_documento,
    Venda.status == "CONCLUIDA"
).order_by(Venda.data_venda.desc()).limit(10).all()

return {
    "vendas_hoje": vendas_hoje,
    "vendas_ontem": vendas_ontem,
    "vendas_mes": vendas_mes,
    "quantidade_vendas_hoje": qtd_vendas_hoje,
    "ticket_medio_hoje": round(ticket_medio_hoje, 2),
    "produtos_estoque_baixo": produtos_estoque_baixo,
    "clientes_ativos": clientes_ativos,
    "vendedores_do_dia": [
        {
            "nome": v[0] or "Sem vendedor",
            "qtd_vendas": int(v[1]),
            "total_vendido": float(v[2])
        }
        for v in vendedores_dia
    ],
    "ultimas_vendas": [
        {
            "id": v.id,
            "total": float(v.total),
            "vendedor": v.vendedor_nome,
            "cliente_id": v.cliente_id,
            "data": str(v.data_venda)
        }
        for v in ultimas
    ]
}
```

clientes.py

clientes.py • 206 linhas

```
"""
routers/clientes.py - CRUD de clientes + historico de compras + fidelidade
Multi-tenant: todas as operacoes filtram por empresa_documento.
"""

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from typing import Optional
from datetime import datetime, timedelta

from database import get_db_salto, get_db_core
from models import Cliente, Venda, VendaItem
from schemas import ClienteCreate, ClienteUpdate, ClienteResponse, ClienteHistoricoResponse
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/clientes", tags=["Clientes"])

@router.post("/", response_model=ClienteResponse)
def criar_cliente(
    cliente: ClienteCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)):
    obj = Cliente(empresa_documento=empresa_documento, **cliente.model_dump())
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj

@router.get("/", response_model=list[ClienteResponse])
def listar_clientes(
    empresa_documento: str = Depends(get_empresa_from_user),
    busca: Optional[str] = None,
    ativo: Optional[bool] = True,
    ordem: str = "nome", # nome, pontos, recente
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)):
    query = db.query(Cliente).filter(Cliente.empresa_documento == empresa_documento)
    if ativo is not None:
        query = query.filter(Cliente.ativo == ativo)
    if busca:
        query = query.filter(
            Cliente.nome.ilike(f"%{busca}%") |
            Cliente.telefone.ilike(f"%{busca}%") |
            Cliente.cpf_cnpj.ilike(f"%{busca}%") |
            Cliente.email.ilike(f"%{busca}%")
        )
    if ordem == "pontos":
        query = query.order_by(Cliente.pontos_fidelidade.desc())
    elif ordem == "recente":
        query = query.order_by(Cliente.id.desc())
    else:
```

```

        query = query.order_by(Cliente.nome)
        return query.all()

@router.get("/{cliente_id}", response_model=ClienteResponse)
def get_cliente(
    cliente_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    return obj

@router.put("/{cliente_id}", response_model=ClienteResponse)
def atualizar_cliente(
    cliente_id: int,
    dados: ClienteUpdate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    for key, value in dados.model_dump(exclude_unset=True).items():
        setattr(obj, key, value)
    db.commit()
    db.refresh(obj)
    return obj

@router.delete("/{cliente_id}")
def desativar_cliente(
    cliente_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    obj.ativo = False
    db.commit()
    return {"status": "Cliente desativado"}

# =====
# HISTORICO DE COMPRAS + FIDELIDADE
# =====

@router.get("/{cliente_id}/historico")
def historico_compras(
    cliente_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    limite: int = 30,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Retorna historico de compras do cliente com detalhes."""
    cliente = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")

    vendas = db.query(Venda).filter(
        Venda.cliente_id == cliente_id,
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA"
    ).order_by(Venda.data_venda.desc()).limit(limite).all()

    total_gasto = db.query(func.sum(Venda.total)).filter(
        Venda.cliente_id == cliente_id,
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA"
    ).scalar() or 0

    return {
        "cliente": cliente.nome,
        "total_gasto": float(total_gasto),
        "pontos_fidelidade": cliente.pontos_fidelidade,
    }

```

```

        "compras": [
            {
                "venda_id": v.id,
                "data": str(v.data_venda),
                "total": float(v.total),
                "forma_pagamento": v.forma_pagamento,
                "itens": [
                    {
                        "produto": item.produto_nome,
                        "quantidade": float(item.quantidade),
                        "preco": float(item.preco_unitario),
                        "subtotal": float(item.subtotal)
                    }
                    for item in v.itens
                ]
            }
            for v in vendas
        ]
    }

@router.post("/{cliente_id}/pontos")
def adicionar_pontos(
    cliente_id: int,
    pontos: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Adiciona pontos de fidelidade ao cliente."""
    cliente = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    cliente.pontos_fidelidade += pontos
    db.commit()
    return {"pontos_totais": cliente.pontos_fidelidade}

@router.get("/fidelidade/ranking")
def ranking_fidelidade(
    empresa_documento: str = Depends(get_empresa_from_user),
    limite: int = 20,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Top clientes por pontuacao de fidelidade."""
    return db.query(Cliente).filter(
        Cliente.empresa_documento == empresa_documento,
        Cliente.ativo == True,
        Cliente.pontos_fidelidade > 0
    ).order_by(Cliente.pontos_fidelidade.desc()).limit(limite).all()

```

produtos.py

produtos.py • 198 linhas

```

"""
routers/produtos.py - CRUD de produtos + gestao de estoque
Multi-tenant: todas as operacoes filtram por empresa_documento (via JWT)
"""

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from typing import Optional

from database import get_db_salto, get_db_core
from models import Produto, EstoqueMovimentacao
from schemas import ProdutoCreate, ProdutoUpdate, ProdutoResponse, EstoqueMovimentacaoCreate
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/produtos", tags=["Produtos"])

def _get_empresa(current_user):
    return current_user.empresa_documento

@router.post("/", response_model=ProdutoResponse)
def criar_produto(
    prod: ProdutoCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = Produto(empresa_documento=empresa_documento, **prod.model_dump())
    db.add(produto)
    db.commit()
    db.refresh(produto)

    # Registrar movimentacao de estoque inicial se > 0
    if produto.estoque_atual > 0:

```



```

        mov = EstoqueMovimentacao(
            empresa_documento=empresa_documento,
            produto_id=produto.id,
            tipo="ENTRADA",
            quantidade=produto.estoque_atual,
            motivo="Cadastro inicial",
            usuario=current_user.nome
        )
        db.add(mov)
        db.commit()
    return produto

@router.get("/", response_model=list[ProdutoResponse])
def listar_produtos(
    empresa_documento: str = Depends(get_empresa_from_user),
    busca: Optional[str] = None,
    ativo: Optional[bool] = True,
    estoque_baixo: Optional[bool] = None,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    query = db.query(Produto).filter(Produto.empresa_documento == empresa_documento)
    if ativo is not None:
        query = query.filter(Produto.ativo == ativo)
    if busca:
        query = query.filter(
            Produto.nome.ilike(f"%{busca}%") |
            Produto.sku.ilike(f"%{busca}%") |
            Produto.codigo_barras.ilike(f"%{busca}%")
        )
    if estoque_baixo:
        query = query.filter(Produto.estoque_atual <= Produto.estoque_minimo)
    return query.order_by(Produto.nome).all()

@router.get("/{produto_id}", response_model=ProdutoResponse)
def get_produto(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")
    return produto

@router.put("/{produto_id}", response_model=ProdutoResponse)
def atualizar_produto(
    produto_id: int,
    dados: ProdutoUpdate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")

    update_data = dados.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(produto, key, value)
    db.commit()
    db.refresh(produto)
    return produto

@router.delete("/{produto_id}")
def desativar_produto(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")
    produto.ativo = False
    db.commit()
    return {"status": "Produto desativado"}

# =====
# MOVIMENTACAO DE ESTOQUE

```

```
# =====
@router.post("/movimentacao")
def movimentar_estoque(
    mov: EstoqueMovimentacaoCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == mov.produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")

    if mov.tipo == "SAIDA" and produto.estoque_atual < mov.quantidade:
        raise HTTPException(status_code=400, detail="Estoque insuficiente.")

    # Atualizar estoque
    if mov.tipo == "ENTRADA":
        produto.estoque_atual += mov.quantidade
    elif mov.tipo == "SAIDA":
        produto.estoque_atual -= mov.quantidade
    elif mov.tipo == "AJUSTE":
        produto.estoque_atual = mov.quantidade # Ajuste define o valor absoluto

    # Registrar movimentacao
    reg = EstoqueMovimentacao(
        empresa_documento=empresa_documento,
        produto_id=mov.produto_id,
        tipo=mov.tipo,
        quantidade=mov.quantidade,
        motivo=mov.motivo,
        usuario=current_user.nome
    )
    db.add(reg)
    db.commit()
    return {"status": "Movimentacao registrada", "estoque_atual": float(produto.estoque_atual)}

@router.get("/{produto_id}/movimentacoes")
def historico_movimentacoes(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    return db.query(EstoqueMovimentacao).filter(
        EstoqueMovimentacao.produto_id == produto_id,
        EstoqueMovimentacao.empresa_documento == empresa_documento
    ).order_by(EstoqueMovimentacao.data_movimentacao.desc()).limit(50).all()

@router.get("/estoque/alertas")
def alertas_estoque(
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Retorna produtos com estoque abaixo do minimo."""
    return db.query(Produto).filter(
        Produto.empresa_documento == empresa_documento,
        Produto.ativo == True,
        Produto.estoque_atual <= Produto.estoque_minimo
    ).all()
```

licenca.py

licenca.py • 56 linhas

```
"""
licenca.py - Gerenciamento de licencas do Corumbá Sistemas

Fluxo de licenciamento:
1. Admin recebe chave de licenca (ex: CORUMBA-A1B2-C3D4-E5F6)
2. Na primeira execucao, sistema pede a chave
3. Chave validada -> licenca ATIVA -> acesso total sem senha
4. Em cada login, verifica se licenca ainda esta ativa
5. Se EXPIRADA/SUSPENSA -> pede nova chave para reativar
6. Isolamento: cada CNPJ tem sua propria licenca e seus proprios dados
"""
import hashlib
import secrets
import string
from datetime import datetime, timedelta
from typing import Optional

def gerar_chave_licenca() -> str:
    """Gera uma chave de licenca unica no formato CORUMBA-XXXX-XXXX-XXXX."""
    partes = []
    for _ in range(3):
        parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
        partes.append(parte)
```

```

return f"CORUMBA-{'-'.join(partes)}"

def gerar_hash_chave(chave: str) -> str:
    """Gera hash da chave para armazenamento seguro (nao guarda a chave pura)."""
    return hashlib.sha256(chave.encode()).hexdigest()

def validar_formato_chave(chave: str) -> bool:
    """Valida se a chave esta no formato correto: CORUMBA-XXXX-XXXX-XXXX."""
    if not chave or not chave.startswith("CORUMBA-"):
        return False
    partes = chave.split("-")
    if len(partes) != 4:
        return False
    for parte in partes[1:]:
        if len(parte) != 4 or not all(c in string.ascii_uppercase + string.digits for c in parte):
            return False
    return True

def verificar_licenca_ativa(status: str) -> bool:
    """Verifica se a licenca permite acesso."""
    return status == "ATIVA"

def licenca_expirou(data_expiracao: Optional[datetime]) -> bool:
    """Verifica se a licenca expirou."""
    if data_expiracao is None:
        return False # Sem expiracao = valida para sempre
    return datetime.utcnow() > data_expiracao

```

auth.py

auth.py • 307 linhas

```

"""
routers/auth.py - Autenticação por licença + login de colaboradores

FLUXO ADMIN (primeiro acesso / reativacao):
1. Admin insere a CHAVE DE LICENCA (ex: CORUMBA-A1B2-C3D4-E5F6)
2. Sistema valida a chave, cria/ativa a licenca
3. Licenca ATIVA = acesso total, sistema abre direto sem pedir senha
4. Licenca EXPIRADA/SUSPENSA = pede nova chave para reativar

FLUXO COLABORADOR (dia a dia):
1. Colaborador insere CODIGO + SENHA
2. Sistema verifica se a licenca da empresa dele esta ATIVA
3. Se ATIVA: login normal, acesso aos dados da empresa dele
4. Se INATIVA: acesso bloqueado, pede admin reativar com nova chave

ISOLAMENTO MULTI-TENANT:
- Cada CNPJ tem sua propria licenca
- Usuarios de empresas diferentes podem ter mesmos codigos (1001, 1002, etc.)
- Dados de um CNPJ NUNCA se misturam com outro
"""
from datetime import datetime
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel

from database import get_db_core
from models_core import Usuario, Empresa, Licenca
from schemas import LoginResponse
from security import hash_senha, verificar_senha, criar_token
from licenca import validar_formato_chave, verificar_licenca_ativa, licenca_expirou

router = APIRouter(prefix="/auth", tags=["Autenticacao"])

# ===== SCHEMAS =====

class LoginAdminRequest(BaseModel):
    """Login do admin com chave de licenca (sem senha)."""
    chave_licenca: str # Ex: CORUMBA-A1B2-C3D4-E5F6

class LoginColaboradorRequest(BaseModel):
    """Login do colaborador com codigo + senha."""
    codigo: str #Codigo do colaborador (ex: 1001, V01)
    senha: str

class AtivarLicencaRequest(BaseModel):
    """Ativa uma licenca existente com a chave correta."""
    chave_licenca: str

class CriarLicencaRequest(BaseModel):
    """Cria uma nova licenca (uso interno/admin master)."""
    empresa_documento: str
    chave_licenca: str = None # Se None, gera automaticamente
    plano: str = "BASICO"
    dias_validade: int = None # None = sem expiracao

```

```
# ===== ENDPOINTS =====

@router.post("/admin/login")
def login_admin(req: LoginAdminRequest, db: Session = Depends(get_db_core)):
    """
    Login do administrador via CHAVE DE LICENCA.
    Se a licenca esta ATIVA, abre direto sem pedir senha.
    Se EXPIRADA/SUSPENSA, retorna erro com mensagem para reativar.
    """
    # Validar formato da chave
    if not validar_formato_chave(req.chave_licenca):
        raise HTTPException(status_code=400, detail="Formato de chave invalido. Use: CORUMBA-XXXX-XXXX-XXXX")

    # Buscar licenca pela chave
    licenca = db.query(Licenca).filter(Licenca.chave == req.chave_licenca).first()
    if not licenca:
        raise HTTPException(status_code=404, detail="Chave de licenca nao encontrada.")

    # Verificar se expirou
    if licenca_expirou(licenca.data_expiracao):
        licenca.status = "EXPIRADA"
        db.commit()
        raise HTTPException(status_code=403, detail="Licenca expirada. Insira uma nova chave para reativar.")

    # Verificar status
    if not verificar_licenca_ativa(licenca.status):
        raise HTTPException(status_code=403, detail=f"Licenca {licenca.status}. Insira uma nova chave para reativar.")

    # Buscar o admin da empresa
    admin = db.query(Usuario).filter(
        Usuario.empresa_documento == licenca.empresa_documento,
        Usuario.papel == "ADMIN",
        Usuario.ativo == True
    ).first()

    if not admin:
        raise HTTPException(status_code=404, detail="Administrador nao encontrado para esta licenca.")

    # Atualizar verificacao
    licenca.data_ultima_verificacao = datetime.utcnow()
    admin.ultimo_acesso = datetime.utcnow()
    db.commit()

    # Gerar token JWT
    token = criar_token({
        "sub": admin.username,
        "empresa_documento": admin.empresa_documento,
        "papel": admin.papel,
        "usuario_id": admin.id,
        "nome": admin.nome,
        "licenca_ativa": True
    })

    return {
        "access_token": token,
        "token_type": "bearer",
        "nome": admin.nome,
        "username": admin.username,
        "papel": admin.papel,
        "empresa_documento": admin.empresa_documento,
        "licenca_status": licenca.status,
        "plano": licenca.plano,
        "message": "Licenca validada com sucesso. Acesso liberado."
    }

@router.post("/colaborador/login")
def login_colaborador(req: LoginColaboradorRequest, db: Session = Depends(get_db_core)):
    """
    Login do colaborador com CODIGO + SENHA.
    Verifica se a licenca da empresa dele esta ATIVA antes de permitir acesso.
    """
    # Buscar colaborador por codigo (em qualquer empresa)
    user = db.query(Usuario).filter(
        Usuario.codigo == req.codigo,
        Usuario.ativo == True
    ).first()

    if not user:
        raise HTTPException(status_code=401, detail="Codigo de colaborador nao encontrado.")

    # Verificar senha
    if not verificar_senha(req.senha, user.senha_hash):
        raise HTTPException(status_code=401, detail="Senha incorreta.")

    # Verificar se a licenca da empresa esta ativa
    licenca = db.query(Licenca).filter(
        Licenca.empresa_documento == user.empresa_documento
    ).first()

    if not licenca:
        raise HTTPException(status_code=403, detail="Nenhuma licenca encontrada para esta empresa. Contate o administrador.")

    if licenca_expirou(licenca.data_expiracao):
```

```

        licenca.status = "EXPIRADA"
        db.commit()
        raise HTTPException(status_code=403, detail="Licenca da empresa expirada. Contate o administrador para reativar.")

    if not verificar_licenca_ativa(licenca.status):
        raise HTTPException(status_code=403, detail=f"Licenca da empresa {licenca.status}. Contate o administrador.")

    # Atualizar acessos
    licenca.data_ultima_verificacao = datetime.utcnow()
    user.ultimo_acesso = datetime.utcnow()
    db.commit()

    # Gerar token JWT
    token = criar_token({
        "sub": user.username,
        "empresa_documento": user.empresa_documento,
        "papel": user.papel,
        "usuario_id": user.id,
        "nome": user.nome,
        "licenca_ativa": True
    })

    return {
        "access_token": token,
        "token_type": "bearer",
        "nome": user.nome,
        "username": user.username,
        "papel": user.papel,
        "empresa_documento": user.empresa_documento,
        "licenca_status": licenca.status,
        "plano": licenca.plano
    }

@router.post("/licenca/ativar")
def ativar_licenca(req: AtivarLicencaRequest, db: Session = Depends(get_db_core)):
    """
    Ativa ou reativa uma licenca com a chave.
    Usado quando a licenca expirou ou foi suspensa.
    """
    if not validar_formato_chave(req.chave_licenca):
        raise HTTPException(status_code=400, detail="Formato de chave invalido.")

    licenca = db.query(Licenca).filter(Licenca.chave == req.chave_licenca).first()
    if not licenca:
        raise HTTPException(status_code=404, detail="Chave de licenca nao encontrada.")

    # Reativar
    licenca.status = "ATIVA"
    licenca.data_ativacao = datetime.utcnow()
    licenca.data_ultima_verificacao = datetime.utcnow()
    db.commit()

    return {
        "status": "Licenca ativada com sucesso",
        "empresa_documento": licenca.empresa_documento,
        "plano": licenca.plano,
        "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else "Sem expiracao"
    }

@router.post("/licenca/criar")
def criar_licenca(req: CriarLicencaRequest, db: Session = Depends(get_db_core)):
    """
    Cria uma nova licenca para uma empresa.
    Se nao informar chave, gera automaticamente.
    """
    # Verificar se empresa existe
    empresa = db.query(Empresa).filter(Empresa.documento == req.empresa_documento).first()
    if not empresa:
        raise HTTPException(status_code=404, detail="Empresa nao encontrada.")

    # Gerar chave se nao informada
    chave = req.chave_licenca
    if not chave:
        from licenca import gerar_chave_licenca
        chave = gerar_chave_licenca()

    # Verificar se chave ja existe
    existing = db.query(Licenca).filter(Licenca.chave == chave).first()
    if existing:
        raise HTTPException(status_code=400, detail="Chave de licenca ja existe.")

    # Calcular expiracao
    data_expiracao = None
    if req.dias_validade:
        data_expiracao = datetime.utcnow() + timedelta(days=req.dias_validade)

    licenca = Licenca(
        empresa_documento=req.empresa_documento,
        chave=chave,
        status="ATIVA",
        plano=req.plano,
        data_expiracao=data_expiracao,
        ativo=True
    )

```

```

    )
    db.add(licenca)
    db.commit()
    db.refresh(licenca)

    return {
        "status": "Licenca criada",
        "chave": licenca.chave,
        "empresa": req.empresa_documento,
        "plano": licenca.plano,
        "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else "Sem expiracao"
    }

@router.get("/licenca/verificar/{empresa_documento}")
def verificar_licenca(empresa_documento: str, db: Session = Depends(get_db_core)):
    """Verifica o status da licenca de uma empresa."""
    licenca = db.query(Licenca).filter(
        Licenca.empresa_documento == empresa_documento
    ).first()

    if not licenca:
        return {"status": "SEM_LICENCA", "ativa": False}

    # Auto-expirar se passou da data
    if licenca.expirou(licenca.data_expiracao) and licenca.status == "ATIVA":
        licenca.status = "EXPIRADA"
        db.commit()

    return {
        "status": licenca.status,
        "ativa": verificar_licenca_ativa(licenca.status),
        "plano": licenca.plano,
        "data_ativacao": str(licenca.data_ativacao),
        "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else None,
        "max_usuarios": licenca.max_usuarios,
        "max_produtos": licenca.max_produtos
    }

@router.get("/licenca/status")
def status_licenca(chave: str, db: Session = Depends(get_db_core)):
    """Consulta status de uma licenca pela chave."""
    licenca = db.query(Licenca).filter(Licenca.chave == chave).first()
    if not licenca:
        raise HTTPException(status_code=404, detail="Licenca nao encontrada.")

    return {
        "chave": licenca.chave,
        "empresa_documento": licenca.empresa_documento,
        "status": licenca.status,
        "ativa": verificar_licenca_ativa(licenca.status),
        "plano": licenca.plano,
        "data_ativacao": str(licenca.data_ativacao),
        "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else None,
        "ultima_verificacao": str(licenca.data_ultima_verificacao) if licenca.data_ultima_verificacao else None
    }

```

app.py

web/app.py • 721 linhas

```

"""
web/app.py - Painele Web SaltoGestão (alternativa ao desktop tkinter)
Serve HTML/CSS/JS que conecta à API FastAPI via fetch()
"""
from fastapi import FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.middleware.cors import CORSMiddleware
import os

app = FastAPI(title="SaltoGestão Web", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Caminho para pasta static
STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "static")
os.makedirs(STATIC_DIR, exist_ok=True)
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

@app.get("/", response_class=HTMLResponse)
def painel(request: Request):
    """Painele web principal - SaltoGestão"""
    return HTMLResponse(content=PAGE_HTML)

```

```

# =====
# PÁGINA HTML COMPLETA (single-page app)
# =====
PAGE_HTML = """
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>🏢 SaltoGestão - Painel Administrativo</title>
<style>
:root {
  --bg: #0b1a30; --bg2: #0f2044; --bg3: #142a55;
  --card: #112240; --border: #1e3a5f;
  --primary: #3b82f6; --primary-hover: #2563eb;
  --success: #10b981; --warning: #f59e0b; --danger: #ef4444;
  --text: #e2e8f0; --text2: #94a3b8; --text3: #64748b;
}
* { margin:0; padding:0; box-sizing:border-box; }
body { font-family: 'Segoe UI', system-ui, sans-serif; background: var(--bg); color: var(--text); min-height:100vh; }

/* LOGIN */
.login-bg { display:flex; align-items:center; justify-content:center; min-height:100vh; background: linear-gradient(135deg, #0b1a30 0%, #1a1a4e 100%); }
.login-box { background: var(--card); border:1px solid var(--border); border-radius:16px; padding:40px; width:420px; box-shadow: 0 20px 60px rgba(0,0,0,0.3); }
.login-box h1 { font-size:28px; color:var(--primary); text-align:center; margin-bottom:4px; }
.login-box .sub { color:var(--text3); text-align:center; margin-bottom:30px; font-size:14px; }
.form-group { margin-bottom:16px; }
.form-group label { display:block; color:var(--text2); font-size:13px; margin-bottom:6px; }
.form-group input, .form-group select {
  width:100%; padding:12px 16px; border-radius:8px; border:1px solid var(--border);
  background:var(--bg); color:var(--text); font-size:14px; outline:none; transition:.2s;
}
.form-group input:focus, .form-group select:focus { border-color:var(--primary); }
.btn { width:100%; padding:14px; border:none; border-radius:10px; font-size:16px; font-weight:600; cursor:pointer; transition:.2s; }
.btn-primary { background:var(--primary); color:#fff; }
.btn-primary:hover { background:var(--primary-hover); }
.btn-success { background:var(--success); color:#fff; }
.btn-danger { background:var(--danger); color:#fff; }
.btn-sm { padding:8px 14px; font-size:13px; width:auto; }
.error-msg { color:var(--danger); font-size:13px; margin-top:12px; text-align:center; }
.success-msg { color:var(--success); font-size:13px; margin-top:12px; text-align:center; }

/* LAYOUT PRINCIPAL */
.app { display:none; }
.sidebar { position:fixed; left:0; top:0; width:240px; height:100vh; background:var(--card); border-right:1px solid var(--border); padding:24px 0; z-index:100; }
.sidebar h2 { color:var(--primary); font-size:20px; padding:0 24px; margin-bottom:8px; }
.sidebar .user-info { color:var(--text3); font-size:12px; padding:0 24px; margin-bottom:30px; }
.sidebar nav a { display:flex; align-items:center; gap:10px; padding:12px 24px; color:var(--text2); text-decoration:none; font-size:14px; transition:.2s; border-left:3px solid transparent; }
.sidebar nav a:hover, .sidebar nav a.active { background:var(--bg2); color:var(--text); border-left-color:var(--primary); }
.main { margin-left:240px; padding:24px; min-height:100vh; }

/* HEADER */
.header { display:flex; justify-content:space-between; align-items:center; margin-bottom:24px; }
.header h1 { font-size:24px; }
.header .stats { display:flex; gap:16px; }
.stat-card { background:var(--card); border:1px solid var(--border); border-radius:12px; padding:16px 24px; text-align:center; }
.stat-card .num { font-size:28px; font-weight:700; color:var(--primary); }
.stat-card .label { font-size:12px; color:var(--text3); margin-top:4px; }

/* TABLE */
.table-wrap { background:var(--card); border:1px solid var(--border); border-radius:12px; overflow:hidden; }
.table-wrap .table-header { display:flex; justify-content:space-between; align-items:center; padding:16px 20px; border-bottom:1px solid var(--border); }
.table-wrap .table-header h3 { font-size:16px; }
.search-box { display:flex; gap:8px; }
.search-box input { padding:8px 14px; border-radius:8px; border:1px solid var(--border); background:var(--bg); color:var(--text); font-size:13px; width:240px; outline:none; }
table { width:100%; border-collapse:collapse; }
th { text-align:left; padding:12px 16px; font-size:12px; color:var(--text3); text-transform:uppercase; border-bottom:1px solid var(--border); background:var(--bg2); }
td { padding:12px 16px; font-size:13px; border-bottom:1px solid rgba(30,58,95,0.5); }
tr:hover { background:var(--bg2); }
.badge { display:inline-block; padding:3px 10px; border-radius:20px; font-size:11px; font-weight:600; }
.badge-success { background:rgba(16,185,129,0.15); color:var(--success); }
.badge-danger { background:rgba(239,68,68,0.15); color:var(--danger); }
.badge-info { background:rgba(59,130,246,0.15); color:var(--primary); }
.actions { display:flex; gap:6px; }

/* MODAL */
.modal-overlay { display:none; position:fixed; inset:0; background:rgba(0,0,0,0.6); z-index:200; align-items:center; justify-content:center; }
.modal-overlay.open { display:flex; }
.modal { background:var(--card); border:1px solid var(--border); border-radius:16px; padding:32px; width:520px; max-height:80vh; overflow-y:auto; }
.modal h3 { margin-bottom:20px; font-size:18px; }
.modal .form-group { margin-bottom:14px; }
.modal .actions { justify-content:flex-end; margin-top:20px; gap:8px; }

/* PAGINATION */
.pagination { display:flex; justify-content:center; gap:6px; margin-top:16px; }
.pagination button { padding:8px 14px; border:1px solid var(--border); background:var(--bg); color:var(--text); border-radius:8px; cursor:pointer; font-size:13px; }

```

```

.pagination button.active { background:var(--primary); border-color:var(--primary); }
.pagination button:hover:not(.active) { background:var(--bg2); }

/* NOTIFICATION */
.toast { position:fixed; top:20px; right:20px; padding:14px 24px; border-radius:10px; font-size:14px; z-index:300; animation: slideIn .3s ease; }
.toast-success { background:var(--success); color:#fff; }
.toast-error { background:var(--danger); color:#fff; }
@keyframes slideIn { from { transform:translateX(100%); opacity:0; } to { transform:translateX(0); opacity:1; } }
</style>
</head>
<body>

<!-- LOGIN -->
<div id="loginPage" class="login-bg">
  <div class="login-box">
    <h1><img alt="Logo" data-bbox="140 212 155 222"/> SaltoGestão</h1>
    <div class="sub">Sistema de Gestão Comercial</div>

    <div class="form-group">
      <label>Servidor da API:</label>
      <input type="text" id="apiUrl" value="http://191.252.210.235:8080" placeholder="http://localhost:8080">
    </div>

    <div class="form-group">
      <label>Usuário (username):</label>
      <input type="text" id="username" placeholder="admin_empresa" value="admin_empresa">
    </div>

    <div class="form-group">
      <label>Senha:</label>
      <input type="password" id="senha" placeholder="*****" value="empresa123">
    </div>

    <button class="btn btn-primary" onclick="fazerLogin()"><img alt="Login icon" data-bbox="430 375 445 385"/> Entrar</button>

    <div id="loginError" class="error-msg"></div>
  </div>
</div>

<!-- APP -->
<div id="app" class="app">
  <!-- SIDEBAR -->
  <div class="sidebar">
    <h2><img alt="Logo" data-bbox="140 465 155 475"/> SaltoGestão</h2>
    <div class="user-info" id="userInfo">Carregando...</div>
    <nav>
      <a href="#" class="active" onclick="showPage('dashboard', event)"><img alt="Dashboard icon" data-bbox="505 492 520 502"/> Dashboard</a>
      <a href="#" onclick="showPage('clientes', event)"><img alt="Clientes icon" data-bbox="505 502 520 512"/> Clientes</a>
      <a href="#" onclick="showPage('produtos', event)"><img alt="Produtos icon" data-bbox="505 512 520 522"/> Produtos</a>
      <a href="#" onclick="showPage('vendas', event)"><img alt="Vendas icon" data-bbox="505 522 520 532"/> Vendas</a>
      <a href="#" onclick="showPage('fornecedores', event)"><img alt="Fornecedores icon" data-bbox="505 532 520 542"/> Fornecedores</a>
      <a href="#" onclick="showPage('relatorios', event)"><img alt="Relatórios icon" data-bbox="505 542 520 552"/> Relatórios</a>
      <a href="#" onclick="sair()" style="margin-top:30px; color:var(--danger);"><img alt="Sair icon" data-bbox="505 552 520 562"/> Sair</a>
    </nav>
  </div>

  <!-- MAIN CONTENT -->
  <div class="main">

    <!-- DASHBOARD -->
    <div id="page-dashboard" class="page">
      <div class="header">
        <h1><img alt="Logo" data-bbox="165 638 180 648"/> Dashboard</h1>
      </div>
      <div class="stats">
        <div class="stat-card"><div class="num" id="stClientes">0</div><div class="label">Clientes</div></div>
        <div class="stat-card"><div class="num" id="stProdutos">0</div><div class="label">Produtos</div></div>
        <div class="stat-card"><div class="num" id="stVendas">0</div><div class="label">Vendas</div></div>
        <div class="stat-card"><div class="num" id="stFornecedores">0</div><div class="label">Fornecedores</div></div>
      </div>
      <div class="table-wrap" style="margin-top:24px;">
        <div class="table-header"><h3>Últimos clientes cadastrados</h3></div>
        <table>
          <thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>CPF/CNPJ</th><th>Status</th></tr></thead>
          <tbody id="dashboardClientes"></tbody>
        </table>
      </div>
    </div>

    <!-- CLIENTES -->
    <div id="page-clientes" class="page" style="display:none;">
      <div class="header">
        <h1><img alt="Logo" data-bbox="165 818 180 828"/> Clientes</h1>
      </div>
      <div class="table-wrap">
        <div class="table-header">
          <div class="search-box">
            <input type="text" id="searchClientes" placeholder="🔍 Buscar cliente..." oninput="buscarClientes()" />
            <select id="filterClientes" onchange="buscarClientes()">
              <option value="true">Ativos</option>
              <option value="false">Inativos</option>
              <option value="">Todos</option>
            </select>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```



```

        <button class="btn btn-success btn-sm" onclick="abrirModalCliente()">+ Novo Cliente</button>
    </div>
    <table>
        <thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>E-mail</th><th>CPF/CNPJ</th><th>Cidade</th><th>Pontos</th><th>Status</th><th>Ações</th></tr></thead>
        <tbody id="tbodyClientes"></tbody>
    </table>
    <div class="pagination" id="pagClientes"></div>
</div>
</div>

<!-- PRODUTOS -->
<div id="page-produtos" class="page" style="display:none;">
    <div class="header"><h1>🛒 Produtos</h1></div>
    <div class="table-wrap">
        <div class="table-header">
            <div class="search-box">
                <input type="text" id="searchProdutos" placeholder="🔍 Buscar produto..." oninput="buscarProdutos()" />
            </div>
            <button class="btn btn-success btn-sm" onclick="abrirModalProduto()">+ Novo Produto</button>
        </div>
        <table>
            <thead><tr><th>ID</th><th>Nome</th><th>SKU</th><th>Preço Venda</th><th>Estoque</th><th>Unidade</th><th>Status</th><th>Ações</th></tr></thead>
            <tbody id="tbodyProdutos"></tbody>
        </table>
    </div>
</div>

<!-- VENDAS -->
<div id="page-vendas" class="page" style="display:none;">
    <div class="header"><h1>💰 Vendas</h1></div>
    <div class="table-wrap">
        <div class="table-header">
            <div class="search-box">
                <input type="text" placeholder="🔍 Buscar venda..." />
            </div>
            <button class="btn btn-success btn-sm" onclick="abrirModalVenda()">+ Nova Venda</button>
        </div>
        <table>
            <thead><tr><th>ID</th><th>Data</th><th>Cliente</th><th>Total</th><th>Pagamento</th><th>Status</th><th>Ações</th></tr></thead>
            <tbody id="tbodyVendas"></tbody>
        </table>
    </div>
</div>

<!-- FORNECEDORES -->
<div id="page-fornecedores" class="page" style="display:none;">
    <div class="header"><h1>🏢 Fornecedores</h1></div>
    <div class="table-wrap">
        <div class="table-header">
            <div class="search-box">
                <input type="text" placeholder="🔍 Buscar fornecedor..." />
            </div>
            <button class="btn btn-success btn-sm" onclick="abrirModalFornecedor()">+ Novo Fornecedor</button>
        </div>
        <table>
            <thead><tr><th>ID</th><th>Nome Fantasia</th><th>CNPJ</th><th>Telefone</th><th>E-mail</th><th>Status</th><th>Ações</th></tr></thead>
            <tbody id="tbodyFornecedores"></tbody>
        </table>
    </div>
</div>

<!-- RELATÓRIOS -->
<div id="page-relatorios" class="page" style="display:none;">
    <div class="header"><h1>📊 Relatórios</h1></div>
    <div class="stats">
        <div class="stat-card"><div class="num" id="rClientes">0</div><div class="label">Total Clientes</div></div>
        <div class="stat-card"><div class="num" id="rProdutos">0</div><div class="label">Total Produtos</div></div>
        <div class="stat-card"><div class="num" id="rVendas">0</div><div class="label">Total Vendas</div></div>
        <div class="stat-card"><div class="num" id="rFornecedores">0</div><div class="label">Total Fornecedores</div></div>
    </div>
    <div class="table-wrap" style="margin-top:24px;">
        <div class="table-header"><h3>Top 10 Clientes por Fidelidade</h3></div>
        <table>
            <thead><tr><th>#</th><th>Cliente</th><th>Pontos</th><th>Total Gasto</th></tr></thead>
            <tbody id="tbodyRanking"></tbody>
        </table>
    </div>
</div>
</div>
</div>

<!-- MODAL CLIENTE -->
<div id="modalCliente" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalCliente')">
    <div class="modal">
        <h3 id="modalClienteTitulo">Novo Cliente</h3>
        <input type="hidden" id="cId" />
        <div class="form-group"><label>Nome:</label><input type="text" id="cNome" placeholder="Nome completo"></div>
        <div class="form-group"><label>Telefone:</label><input type="text" id="cTelefone" placeholder="(62) 99999-0000"></div>
        <div class="form-group"><label>E-mail:</label><input type="text" id="cEmail" placeholder="email@exemplo.com"></div>
        <div class="form-group"><label>CPF/CNPJ:</label><input type="text" id="cCpfCnpj" placeholder="000.000.000-00"></div>
        <div class="form-group"><label>Endereço:</label><input type="text" id="cEndereco" placeholder="Rua, número"></div>
        <div class="form-group"><label>Cidade:</label><input type="text" id="cCidade" placeholder="Cidade"></div>
        <div class="form-group"><label>Estado:</label><input type="text" id="cEstado" placeholder="GO" maxlength="2"></div>
    </div>
</div>

```

```

<div class="form-group"><label>CEP:</label><input type="text" id="cCep" placeholder="00000-000"></div>
<div class="actions">
  <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalCliente')">Cancelar</button>
  <button class="btn btn-success btn-sm" onclick="salvarCliente()"><img alt="salvar icon" data-bbox="495 95 505 105"/> Salvar</button>
</div>
</div>
</div>

<!-- MODAL PRODUTO -->
<div id="modalProduto" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalProduto')">
  <div class="modal">
    <h3 id="modalProdutoTitulo">Novo Produto</h3>
    <input type="hidden" id="pId" />
    <div class="form-group"><label>Nome:</label><input type="text" id="pNome" placeholder="Nome do produto"></div>
    <div class="form-group"><label>Descrição:</label><input type="text" id="pDescricao" placeholder="Descrição"></div>
    <div class="form-group"><label>SKU:</label><input type="text" id="pSku" placeholder="SKU-001"></div>
    <div class="form-group"><label>Preço Venda:</label><input type="number" id="pPrecoVenda" step="0.01" placeholder="0.00"></div>
    <div class="form-group"><label>Preço Custo:</label><input type="number" id="pPrecoCusto" step="0.01" placeholder="0.00"></div>
    <div class="form-group"><label>Estoque Atual:</label><input type="number" id="pEstoque" step="1" placeholder="0"></div>
    <div class="form-group"><label>Estoque Mínimo:</label><input type="number" id="pEstoqueMin" step="1" placeholder="0"></div>
    <div class="form-group"><label>Unidade:</label>
      <select id="pUnidade">
        <option value="UN">UN</option><option value="KG">KG</option>
        <option value="LT">LT</option><option value="CX">CX</option>
      </select>
    </div>
    <div class="actions">
      <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalProduto')">Cancelar</button>
      <button class="btn btn-success btn-sm" onclick="salvarProduto()"><img alt="salvar icon" data-bbox="495 315 505 325"/> Salvar</button>
    </div>
  </div>
</div>

<!-- MODAL FORNECEDOR -->
<div id="modalFornecedor" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalFornecedor')">
  <div class="modal">
    <h3>Novo Fornecedor</h3>
    <input type="hidden" id="fId" />
    <div class="form-group"><label>Nome Fantasia:</label><input type="text" id="fNome" placeholder="Nome do fornecedor"></div>
    <div class="form-group"><label>Razão Social:</label><input type="text" id="fRazao" placeholder="Razão social"></div>
    <div class="form-group"><label>CNPJ/CPF:</label><input type="text" id="fCnpj" placeholder="00.000.000/0000-00"></div>
    <div class="form-group"><label>Telefone:</label><input type="text" id="fTelefone" placeholder="(62) 99999-0000"></div>
    <div class="form-group"><label>E-mail:</label><input type="email" id="fEmail" placeholder="email@exemplo.com"></div>
    <div class="form-group"><label>Endereço:</label><input type="text" id="fEndereco" placeholder="Rua, número"></div>
    <div class="actions">
      <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalFornecedor')">Cancelar</button>
      <button class="btn btn-success btn-sm" onclick="salvarFornecedor()"><img alt="salvar icon" data-bbox="495 480 505 490"/> Salvar</button>
    </div>
  </div>
</div>

<script>
// =====
// ESTADO GLOBAL
// =====
const S = {
  token: localStorage.getItem('token') || null,
  usuario: JSON.parse(localStorage.getItem('usuario') || 'null'),
  apiUrl: localStorage.getItem('apiUrl') || 'http://191.252.210.235:8080',
  clientes: [],
  produtos: [],
  vendas: [],
  fornecedores: [],
  page: 1,
  perPage: 20,
  totalClientes: 0
};

// =====
// API HELPER
// =====
async function api(method, path, body) {
  const opts = {
    method,
    headers: { 'Content-Type': 'application/json' }
  };
  if (S.token) opts.headers['Authorization'] = 'Bearer ' + S.token;
  if (body) opts.body = JSON.stringify(body);
  const r = await fetch(S.apiUrl + path, opts);
  if (r.status === 401) { sair(); return null; }
  const t = await r.text();
  return t ? JSON.parse(t) : null;
}

function toast(msg, type) {
  const d = document.createElement('div');
  d.className = 'toast toast-' + (type || 'success');
  d.textContent = msg;
  document.body.appendChild(d);
  setTimeout(() => d.remove(), 3000);
}

// =====
// LOGIN
// =====

```

```

function fazerLogin() {
  const apiUrl = document.getElementById('apiUrl').value.trim();
  const username = document.getElementById('username').value.trim();
  const senha = document.getElementById('senha').value.trim();
  const err = document.getElementById('loginError');
  err.textContent = '';

  S.apiUrl = apiUrl;
  localStorage.setItem('apiUrl', apiUrl);

  const body = username.includes('@') ? { email: username, senha } : { username, senha };

  fetch(apiUrl + '/api/login', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify(body)
  })
  .then(r => r.json())
  .then(d => {
    if (d.access_token) {
      S.token = d.access_token;
      S.usuario = d.usuario || d.admin;
      localStorage.setItem('token', S.token);
      localStorage.setItem('usuario', JSON.stringify(S.usuario));
      entrar();
    } else {
      err.textContent = '✖' + (d.detail || 'Falha no login');
    }
  })
  .catch(e => { err.textContent = '✖ Erro de conexão: ' + e.message; });
}

function entrar() {
  document.getElementById('loginPage').style.display = 'none';
  document.getElementById('app').style.display = 'block';
  document.getElementById('userInfo').textContent = S.usuario.nome + ' (' + S.usuario.papel + ')';
  carregarDashboard();
}

function sair() {
  S.token = null; S.usuario = null;
  localStorage.removeItem('token'); localStorage.removeItem('usuario');
  document.getElementById('app').style.display = 'none';
  document.getElementById('loginPage').style.display = 'flex';
}

// =====
// NAVEGAÇÃO
// =====
function showPage(page, e) {
  if (e) e.preventDefault();
  document.querySelectorAll('.page').forEach(p => p.style.display = 'none');
  document.querySelectorAll('.sidebar nav a').forEach(a => a.classList.remove('active'));
  document.getElementById('page-' + page).style.display = 'block';
  if (e) e.target.classList.add('active');

  if (page === 'dashboard') carregarDashboard();
  if (page === 'clientes') carregarClientes();
  if (page === 'produtos') carregarProdutos();
  if (page === 'vendas') carregarVendas();
  if (page === 'fornecedores') carregarFornecedores();
  if (page === 'relatorios') carregarRelatorios();
}

// =====
// DASHBOARD
// =====
async function carregarDashboard() {
  const [c, p, v, f] = await Promise.all([
    api('GET', '/api/clientes/?ativo=true&ordem=recente&limite=5'),
    api('GET', '/api/produtos/'),
    api('GET', '/api/vendas/'),
    api('GET', '/api/fornecedores/')
  ]);

  if (c) {
    document.getElementById('stClientes').textContent = c.length;
    const tbody = document.getElementById('dashboardClientes');
    tbody.innerHTML = c.slice(0, 5).map(x => `<tr>
      <td>${x.id}</td><td>${x.nome || '-'}</td><td>${x.telefone || '-'}</td>
      <td>${x.cpf_cnpj || '-'}</td>
      <td><span class="badge ${x.ativo ? 'badge-success' : 'badge-danger'}">${x.ativo ? 'Ativo' : 'Inativo'}</span></td>
    `).join('');
  }
  if (p) document.getElementById('stProdutos').textContent = p.length;
  if (v) document.getElementById('stVendas').textContent = v.length;
  if (f) document.getElementById('stFornecedores').textContent = f.length;
}

// =====
// CLIENTES
// =====
async function carregarClientes() {
  const busca = document.getElementById('searchClientes').value;
  const ativo = document.getElementById('filterClientes').value;

```

```

let url = '/api/clientes/?';
if (ativo) url += 'ativo=' + ativo + '&';
if (busca) url += 'busca=' + encodeURIComponent(busca) + '&';
url += 'ordem=nome&limite=100';

S.clientes = await api('GET', url) || [];
renderClientes(S.clientes);
}

function renderClientes(lista) {
  const tbody = document.getElementById('tbodyClientes');
  if (!lista.length) { tbody.innerHTML = '<tr><td colspan="9" style="text-align:center;color:var(--text3);padding:40px;">Nenhum cliente encontrado</td></tr>'; return; }
  tbody.innerHTML = lista.map(c => `<tr>
    <td>${c.id}</td>
    <td>${c.nome || '-'}</td>
    <td>${c.telefone || '-'}</td>
    <td>${c.email || '-'}</td>
    <td>${c.cpf_cnpj || '-'}</td>
    <td>${c.cidade || '-'}</td>
    <td><span class="badge badge-info">${c.pontos_fidelidade || 0}</span></td>
    <td><span class="badge ${c.ativo ? 'badge-success' : 'badge-danger'}">${c.ativo ? 'Ativo' : 'Inativo'}</span></td>
    <td>
      <button class="btn btn-sm btn-primary" onclick="editarCliente(${c.id})"><img alt="pencil icon" /></button>
      <button class="btn btn-sm btn-danger" onclick="excluirCliente(${c.id})"><img alt="trash icon" /></button>
    </td>
  `).join('');
}

function buscarClientes() { carregarClientes(); }

function abrirModalCliente() {
  document.getElementById('modalClienteTitulo').textContent = 'Novo Cliente';
  ['cId', 'cNome', 'cTelefone', 'cEmail', 'cCpfCnpj', 'cEndereco', 'cCidade', 'cEstado', 'cCep'].forEach(id => document.getElementById(id).value = '');
  document.getElementById('modalCliente').classList.add('open');
}

async function editarCliente(id) {
  const c = S.clientes.find(x => x.id === id);
  if (!c) return;
  document.getElementById('modalClienteTitulo').textContent = 'Editar Cliente #' + id;
  for (const k of ['Id', 'Nome', 'Telefone', 'Email', 'CpfCnpj', 'Endereco', 'Cidade', 'Estado', 'Cep']) {
    const el = document.getElementById('c' + k);
    if (el) el.value = c[k.toLowerCase()] || '';
  }
  document.getElementById('modalCliente').classList.add('open');
}

async function salvarCliente() {
  const btn = document.querySelector('#modalCliente .btn-success');
  btn.disabled = true; btn.textContent = '<img alt="loading spinner" /> Salvando...';

  const dados = {
    nome: document.getElementById('cNome').value.trim(),
    telefone: document.getElementById('cTelefone').value.trim() || null,
    email: document.getElementById('cEmail').value.trim() || null,
    cpf_cnpj: document.getElementById('cCpfCnpj').value.trim() || null,
    endereco: document.getElementById('cEndereco').value.trim() || null,
    cidade: document.getElementById('cCidade').value.trim() || null,
    estado: document.getElementById('cEstado').value.trim() || null,
    cep: document.getElementById('cCep').value.trim() || null
  };
  if (!dados.nome) { toast('Nome é obrigatório!', 'error'); btn.disabled = false; btn.textContent = '<img alt="save icon" /> Salvar'; return; }

  const id = document.getElementById('cId').value;
  if (id) {
    await api('PUT', '/api/clientes/' + id, dados);
    toast('Cliente atualizado! <img alt="checkmark icon" />');
  } else {
    await api('POST', '/api/clientes/', dados);
    toast('Cliente cadastrado! <img alt="checkmark icon" />');
  }
  fecharModal('modalCliente');
  carregarClientes();
  btn.disabled = false; btn.textContent = '<img alt="save icon" /> Salvar';
}

async function excluirCliente(id) {
  if (!confirm('Deseja realmente excluir o cliente #' + id + '?')) return;
  await api('DELETE', '/api/clientes/' + id);
  toast('Cliente desativado! <img alt="checkmark icon" />');
  carregarClientes();
}

// =====
// PRODUTOS
// =====
async function carregarProdutos() {
  S.produtos = await api('GET', '/api/produtos/') || [];
  renderProdutos(S.produtos);
}

function renderProdutos(lista) {
  const tbody = document.getElementById('tbodyProdutos');

```

```

if (!lista.length) { tbody.innerHTML = '<tr><td colspan="8" style="text-align:center;color:var(--text3);padding:40px;">Nenhum produto</td></tr>'; return; }
tbody.innerHTML = lista.map(p => `<tr>
<td>${p.id}</td><td>${p.nome || '-'}</td><td>${p.sku || '-'}</td>
<td>R$ ${parseFloat(p.preco_venda).toFixed(2)}</td>
<td>${p.estoque_atual || 0}</td><td>${p.unidade || 'UN'}</td>
<td><span class="badge ${p.ativo ? 'badge-success' : 'badge-danger'}">${p.ativo ? 'Ativo' : 'Inativo'}</span></td>
<td>
<button class="btn btn-sm btn-primary">✎</button>
<button class="btn btn-sm btn-danger">🗑️</button>
</td>
`
).join('');
}

function buscarProdutos() { carregarProdutos(); }

function abrirModalProduto() {
document.getElementById('modalProdutoTitulo').textContent = 'Novo Produto';
['pId', 'pNome', 'pDescricao', 'pSku', 'pPrecoVenda', 'pPrecoCusto', 'pEstoque', 'pEstoqueMin'].forEach(id => document.getElementById(id).value = '');
document.getElementById('pUnidade').value = 'UN';
document.getElementById('modalProduto').classList.add('open');
}

async function salvarProduto() {
const dados = {
nome: document.getElementById('pNome').value.trim(),
descricao: document.getElementById('pDescricao').value.trim() || null,
sku: document.getElementById('pSku').value.trim() || null,
preco_venda: parseFloat(document.getElementById('pPrecoVenda').value) || 0,
preco_custo: parseFloat(document.getElementById('pPrecoCusto').value) || 0,
estoque_atual: parseFloat(document.getElementById('pEstoque').value) || 0,
estoque_minimo: parseFloat(document.getElementById('pEstoqueMin').value) || 0,
unidade: document.getElementById('pUnidade').value
};
if (!dados.nome) { toast('Nome é obrigatório!', 'error'); return; }
await api('POST', '/api/produtos/', dados);
toast('Produto cadastrado! ✅');
fecharModal('modalProduto');
carregarProdutos();
}

// =====
// VENDAS
// =====
async function carregarVendas() {
S.vendas = await api('GET', '/api/vendas/') || [];
const tbody = document.getElementById('tbodyVendas');
if (!S.vendas.length) { tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;color:var(--text3);padding:40px;">Nenhuma venda</td></tr>'; return; }
tbody.innerHTML = S.vendas.map(v => `<tr>
<td>${v.id}</td><td>${v.data_venda ? v.data_venda.substring(0,10) : '-'}</td>
<td>${v.cliente_nome || '-'}</td><td>R$ ${parseFloat(v.total || 0).toFixed(2)}</td>
<td>${v.forma_pagamento || '-'}</td>
<td><span class="badge ${v.status === 'CONCLUIDA' ? 'badge-success' : 'badge-danger'}">${v.status}</span></td>
<td><button class="btn btn-sm btn-primary">📄</button></td>
`
).join('');
}

// =====
// FORNECEDORES
// =====
async function carregarFornecedores() {
S.fornecedores = await api('GET', '/api/fornecedores/') || [];
const tbody = document.getElementById('tbodyFornecedores');
if (!S.fornecedores.length) { tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;color:var(--text3);padding:40px;">Nenhum fornecedor</td></tr>'; return; }
tbody.innerHTML = S.fornecedores.map(f => `<tr>
<td>${f.id}</td><td>${f.nome_fantasia || '-'}</td><td>${f.cnpj_cpf || '-'}</td>
<td>${f.telefone || '-'}</td><td>${f.email || '-'}</td>
<td><span class="badge ${f.ativo ? 'badge-success' : 'badge-danger'}">${f.ativo ? 'Ativo' : 'Inativo'}</span></td>
<td><button class="btn btn-sm btn-primary">✎</button><button class="btn btn-sm btn-danger">🗑️</button></td>
`
).join('');
}

function abrirModalFornecedor() {
['fId', 'fNome', 'fRazao', 'fCnpj', 'fTelefone', 'fEmail', 'fEndereco'].forEach(id => document.getElementById(id).value = '');
document.getElementById('modalFornecedor').classList.add('open');
}

async function salvarFornecedor() {
const dados = {
nome_fantasia: document.getElementById('fNome').value.trim(),
razao_social: document.getElementById('fRazao').value.trim() || null,
cnpj_cpf: document.getElementById('fCnpj').value.trim() || null,
telefone: document.getElementById('fTelefone').value.trim() || null,
email: document.getElementById('fEmail').value.trim() || null,
endereco: document.getElementById('fEndereco').value.trim() || null
};
if (!dados.nome_fantasia) { toast('Nome é obrigatório!', 'error'); return; }
await api('POST', '/api/fornecedores/', dados);
toast('Fornecedor cadastrado! ✅');
fecharModal('modalFornecedor');
carregarFornecedores();
}

```

```
// =====
// RELATÓRIOS
// =====
async function carregarRelatorios() {
  const [c, p, v, f, ranking] = await Promise.all([
    api('GET', '/api/clientes/?limite=10000'),
    api('GET', '/api/produtos/'),
    api('GET', '/api/vendas/'),
    api('GET', '/api/fornecedores/'),
    api('GET', '/api/clientes/fidelidade/ranking?limite=10')
  ]);
  document.getElementById('rClientes').textContent = c ? c.length : 0;
  document.getElementById('rProdutos').textContent = p ? p.length : 0;
  document.getElementById('rVendas').textContent = v ? v.length : 0;
  document.getElementById('rFornecedores').textContent = f ? f.length : 0;

  const tbody = document.getElementById('tbodyRanking');
  if (ranking && ranking.length) {
    tbody.innerHTML = ranking.map((r, i) => `<tr>
      <td>${i + 1}</td><td>${r.nome || '-'}</td>
      <td><span class="badge badge-info">${r.pontos_fidelidade || 0}</span></td>
      <td></td>
    `).join('');
  } else {
    tbody.innerHTML = `<tr><td colspan="4" style="text-align:center;color:var(--text3);padding:20px;">Nenhum dado de fidelidade</td></tr>`;
  }
}

// =====
// HELPERS
// =====
function fecharModal(id) { document.getElementById(id).classList.remove('open'); }

// Auto-login se tem token
if (S.token && S.usuario) entrar();
</script>
</body>
</html>
"""
```

index.html

web/index.html • 771 linhas

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>🏢 SaltoGestão - Painel Administrativo</title>
<style>
:root {
  --bg: #0b1a30; --bg2: #0f2044; --bg3: #142a55;
  --card: #112240; --border: #1e3a5f;
  --primary: #3b82f6; --primary-hover: #2563eb;
  --success: #10b981; --warning: #f59e0b; --danger: #ef4444;
  --text: #e2e8f0; --text2: #94a3b8; --text3: #64748b;
}
* { margin:0; padding:0; box-sizing:border-box; }
body { font-family: 'Segoe UI', system-ui, sans-serif; background: var(--bg); color: var(--text); min-height:100vh; }

/* LOGIN */
.login-bg { display:flex; align-items:center; justify-content:center; min-height:100vh; background: linear-gradient(135deg, #0b1a30 0%, #1a1a4e 100%); }
.login-box { background: var(--card); border:1px solid var(--border); border-radius:16px; padding:40px; width:420px; box-shadow: 0 20px 60px rgba(0,0,0,0.3); }
.login-box h1 { font-size:28px; color:var(--primary); text-align:center; margin-bottom:4px; }
.login-box .sub { color:var(--text3); text-align:center; margin-bottom:30px; font-size:14px; }
.form-group { margin-bottom:16px; }
.form-group label { display:block; color:var(--text2); font-size:13px; margin-bottom:6px; }
.form-group input, .form-group select {
  width:100%; padding:12px 16px; border-radius:8px; border:1px solid var(--border);
  background:var(--bg); color:var(--text); font-size:14px; outline:none; transition:.2s;
}
.form-group input:focus, .form-group select:focus { border-color:var(--primary); }
.btn { width:100%; padding:14px; border:none; border-radius:10px; font-size:16px; font-weight:600; cursor:pointer; transition:.2s; }
.btn-primary { background:var(--primary); color:#fff; }
.btn-primary:hover { background:var(--primary-hover); }
.btn-success { background:var(--success); color:#fff; }
.btn-danger { background:var(--danger); color:#fff; }
.btn-sm { padding:8px 14px; font-size:13px; width:auto; }
.error-msg { color:var(--danger); font-size:13px; margin-top:12px; text-align:center; }

/* LAYOUT PRINCIPAL */
.app { display:none; }
.sidebar { position:fixed; left:0; top:0; width:240px; height:100vh; background:var(--card); border-right:1px solid var(--border); padding:
24px 0; z-index:100; }
.sidebar h2 { color:var(--primary); font-size:20px; padding:0 24px; margin-bottom:8px; }
.sidebar .user-info { color:var(--text3); font-size:12px; padding:0 24px; margin-bottom:30px; }
.sidebar nav a { display:flex; align-items:center; gap:10px; padding:12px 24px; color:var(--text2); text-decoration:none; font-size:14px;
transition:.2s; border-left:3px solid transparent; }
.sidebar nav a:hover, .sidebar nav a.active { background:var(--bg2); color:var(--text); border-left-color:var(--primary); }
.main { margin-left:240px; padding:24px; min-height:100vh; }
```

```

/* HEADER */
.header { display:flex; justify-content:space-between; align-items:center; margin-bottom:24px; }
.header h1 { font-size:24px; }
.stats { display:flex; gap:16px; flex-wrap:wrap; }
.stat-card { background:var(--card); border:1px solid var(--border); border-radius:12px; padding:16px 24px; text-align:center; }
.stat-card .num { font-size:28px; font-weight:700; color:var(--primary); }
.stat-card .label { font-size:12px; color:var(--text3); margin-top:4px; }

/* TABLE */
.table-wrap { background:var(--card); border:1px solid var(--border); border-radius:12px; overflow:hidden; }
.table-wrap .table-header { display:flex; justify-content:space-between; align-items:center; padding:16px 20px; border-bottom:1px solid var(--border); flex-wrap:wrap; gap:10px; }
.table-wrap .table-header h3 { font-size:16px; }
.search-box { display:flex; gap:8px; flex-wrap:wrap; }
.search-box input { padding:8px 14px; border-radius:8px; border:1px solid var(--border); background:var(--bg); color:var(--text); font-size:13px; width:220px; outline:none; }
table { width:100%; border-collapse:collapse; }
th { text-align:left; padding:12px 16px; font-size:12px; color:var(--text3); text-transform:uppercase; border-bottom:1px solid var(--border); background:var(--bg2); }
td { padding:12px 16px; font-size:13px; border-bottom:1px solid rgba(30,58,95,0.5); }
tr:hover { background:var(--bg2); }
.badge { display:inline-block; padding:3px 10px; border-radius:20px; font-size:11px; font-weight:600; }
.badge-success { background:rgba(16,185,129,0.15); color:var(--success); }
.badge-danger { background:rgba(239,68,68,0.15); color:var(--danger); }
.badge-info { background:rgba(59,130,246,0.15); color:var(--primary); }
.actions { display:flex; gap:6px; }

/* MODAL */
.modal-overlay { display:none; position:fixed; inset:0; background:rgba(0,0,0,0.6); z-index:200; align-items:center; justify-content:center; }
.modal-overlay.open { display:flex; }
.modal { background:var(--card); border:1px solid var(--border); border-radius:16px; padding:32px; width:520px; max-height:80vh; overflow-y:auto; }
.modal h3 { margin-bottom:20px; font-size:18px; }
.modal .actions { display:flex; justify-content:flex-end; gap:8px; margin-top:20px; }

/* PAGINATION */
.pagination { display:flex; justify-content:center; gap:6px; margin-top:16px; flex-wrap:wrap; }
.pagination button { padding:8px 14px; border:1px solid var(--border); background:var(--bg); color:var(--text); border-radius:8px; cursor:pointer; font-size:13px; }
.pagination button.active { background:var(--primary); border-color:var(--primary); }
.pagination button:hover:not(.active) { background:var(--bg2); }

/* TOAST */
.toast { position:fixed; top:20px; right:20px; padding:14px 24px; border-radius:10px; font-size:14px; z-index:300; animation: slideIn .3s ease; }
.toast-success { background:var(--success); color:#fff; }
.toast-error { background:var(--danger); color:#fff; }
@keyframes slideIn { from { transform:translateX(100%); opacity:0; } to { transform:translateX(0); opacity:1; } }
</style>
</head>
<body>

<!-- LOGIN -->
<div id="loginPage" class="login-bg">
  <div class="login-box">
    <h1><img alt="SaltoGestão logo" data-bbox="140 565 155 575"/> SaltoGestão</h1>
    <div class="sub">Sistema de Gestão Comercial - Painel Web</div>

    <div class="form-group">
      <label>Servidor da API:</label>
      <input type="text" id="apiUrl" value="http://191.252.210.235:8080" placeholder="http://localhost:8080">
    </div>

    <div class="form-group">
      <label>Usuário (username ou email):</label>
      <input type="text" id="username" placeholder="admin_empresa" value="admin_empresa">
    </div>

    <div class="form-group">
      <label>Senha:</label>
      <input type="password" id="senha" placeholder="*****" value="empresa123">
    </div>

    <button class="btn btn-primary" onclick="fazerLogin()"><img alt="Entrar button icon" data-bbox="435 725 450 735"/> Entrar</button>

    <div id="loginError" class="error-msg"></div>
  </div>
</div>

<!-- APP -->
<div id="app" class="app">
  <div class="sidebar">
    <h2><img alt="SaltoGestão logo" data-bbox="140 805 155 815"/> SaltoGestão</h2>
    <div class="user-info" id="userInfo">Carregando...</div>
    <nav>
      <a href="#" class="active" onclick="showPage('dashboard', event)"><img alt="Dashboard icon" data-bbox="195 835 210 845"/> Dashboard</a>
      <a href="#" onclick="showPage('clientes', event)"><img alt="Clientes icon" data-bbox="215 845 230 855"/> Clientes</a>
      <a href="#" onclick="showPage('produtos', event)"><img alt="Produtos icon" data-bbox="235 855 250 865"/> Produtos</a>
      <a href="#" onclick="showPage('vendas', event)"><img alt="Vendas icon" data-bbox="255 865 270 875"/> Vendas</a>
      <a href="#" onclick="showPage('fornecedores', event)"><img alt="Fornecedores icon" data-bbox="275 875 290 885"/> Fornecedores</a>
      <a href="#" onclick="showPage('relatorios', event)"><img alt="Relatórios icon" data-bbox="295 885 310 895"/> Relatórios</a>
      <a href="#" onclick="showPage('testar100', event)"><img alt="Teste 100 icon" data-bbox="315 895 330 905"/> Teste 100 Clientes</a>
      <hr style="border-color:var(--border);margin:10px 24px;">
      <a href="#" onclick="showPage('admin-clientes', event)"><img alt="Admin Clientes icon" data-bbox="445 905 460 915"/> Admin Clientes</a>
      <a href="#" onclick="showPage('cliente-portal', event)"><img alt="Portal Cliente icon" data-bbox="465 915 480 925"/> Portal Cliente</a>
    </nav>
  </div>
  <div class="main">
    <div class="header">
      <div class="stats">
        <div class="stat-card">
          <div class="num">10</div>
          <div class="label">Clientes</div>
        </div>
        <div class="stat-card">
          <div class="num">20</div>
          <div class="label">Produtos</div>
        </div>
        <div class="stat-card">
          <div class="num">30</div>
          <div class="label">Vendas</div>
        </div>
        <div class="stat-card">
          <div class="num">40</div>
          <div class="label">Fornecedores</div>
        </div>
        <div class="stat-card">
          <div class="num">50</div>
          <div class="label">Relatórios</div>
        </div>
        <div class="stat-card">
          <div class="num">60</div>
          <div class="label">Teste 100</div>
        </div>
      </div>
      <div class="table">
        <table>
          <thead>
            <tr>
              <th>ID</th>
              <th>Nome</th>
              <th>Email</th>
              <th>Telefone</th>
              <th>Status</th>
              <th>Ações</th>
            </tr>
          </thead>
          <tbody>
            <tr>
              <td>1</td>
              <td>João Silva</td>
              <td>josilva@empresa.com</td>
              <td>(11) 1234-5678</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>2</td>
              <td>Maria Santos</td>
              <td>maria@empresa.com</td>
              <td>(11) 9876-5432</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>3</td>
              <td>Carlos Pereira</td>
              <td>carlos@empresa.com</td>
              <td>(11) 5555-1111</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>4</td>
              <td>Ana Costa</td>
              <td>ana@empresa.com</td>
              <td>(11) 2222-3333</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>5</td>
              <td>Pedro Almeida</td>
              <td>pedro@empresa.com</td>
              <td>(11) 4444-6666</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>6</td>
              <td>Lucas Oliveira</td>
              <td>lucas@empresa.com</td>
              <td>(11) 7777-8888</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>7</td>
              <td>Gabriela Lima</td>
              <td>gabriela@empresa.com</td>
              <td>(11) 9999-0000</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>8</td>
              <td>Rafael Costa</td>
              <td>rafael@empresa.com</td>
              <td>(11) 1111-2222</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>9</td>
              <td>Beatriz Silva</td>
              <td>beatriz@empresa.com</td>
              <td>(11) 3333-4444</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
            <tr>
              <td>10</td>
              <td>Vinicius Pereira</td>
              <td>vinicius@empresa.com</td>
              <td>(11) 6666-7777</td>
              <td>Ativo</td>
              <div>
                <button class="btn btn-primary">Editar</button>
                <button class="btn btn-danger">Excluir</button>
              </div>
            </tr>
          </tbody>
        </table>
      </div>
    </div>
  </div>
</div>

```

```

    <a href="#" onclick="sair()" style="margin-top:30px; color:var(--danger);"> Sair</a>
  </nav>
</div>

<div class="main">

  <!-- DASHBOARD -->
  <div id="page-dashboard" class="page">
    <div class="header"><h1> Dashboard</h1></div>
    <div class="stats">
      <div class="stat-card"><div class="num" id="stClientes">0</div><div class="label">Clientes</div></div>
      <div class="stat-card"><div class="num" id="stProdutos">0</div><div class="label">Produtos</div></div>
      <div class="stat-card"><div class="num" id="stVendas">0</div><div class="label">Vendas</div></div>
      <div class="stat-card"><div class="num" id="stFornecedores">0</div><div class="label">Fornecedores</div></div>
    </div>
    <div class="table-wrap" style="margin-top:24px;">
      <div class="table-header"><h3>Últimos clientes cadastrados</h3></div>
      <table><thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>CPF/CNPJ</th><th>Status</th></tr></thead>
      <tbody id="dashboardClientes"></tbody></table>
    </div>
  </div>

  <!-- CLIENTES -->
  <div id="page-clientes" class="page" style="display:none;">
    <div class="header"><h1> Clientes</h1></div>
    <div class="table-wrap">
      <div class="table-header">
        <div class="search-box">
          <input type="text" id="searchClientes" placeholder="🔍 Buscar..." oninput="buscarClientes()">
          <select id="filterClientes" onchange="buscarClientes()">
            <option value="true">Ativos</option>
            <option value="false">Inativos</option>
            <option value="">Todos</option>
          </select>
        </div>
        <button class="btn btn-success btn-sm" onclick="abrirModalCliente()">+ Novo Cliente</button>
      </div>
      <table><thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>E-mail</th><th>CPF/CNPJ</th><th>Cidade</th><th>Pontos</th><th>Status</th><th>Ações</th></tr></thead>
      <tbody id="tbodyClientes"></tbody></table>
    </div>
  </div>

  <!-- PRODUTOS -->
  <div id="page-produtos" class="page" style="display:none;">
    <div class="header"><h1> Produtos</h1></div>
    <div class="table-wrap">
      <div class="table-header">
        <div class="search-box"><input type="text" placeholder="🔍 Buscar..."></div>
        <button class="btn btn-success btn-sm">+ Novo Produto</button>
      </div>
      <table><thead><tr><th>ID</th><th>Nome</th><th>SKU</th><th>Preço</th><th>Estoque</th><th>Status</th></tr></thead>
      <tbody id="tbodyProdutos"></tbody></table>
    </div>
  </div>

  <!-- VENDAS -->
  <div id="page-vendas" class="page" style="display:none;">
    <div class="header"><h1> Vendas</h1></div>
    <div class="table-wrap">
      <div class="table-header"><div class="search-box"><input type="text" placeholder="🔍 Buscar..."></div><button class="btn btn-success btn-sm">+ Nova Venda</button></div>
      <table><thead><tr><th>ID</th><th>Data</th><th>Cliente</th><th>Total</th><th>Status</th></tr></thead>
      <tbody id="tbodyVendas"></tbody></table>
    </div>
  </div>

  <!-- FORNECEDORES -->
  <div id="page-fornecedores" class="page" style="display:none;">
    <div class="header"><h1> Fornecedores</h1></div>
    <div class="table-wrap">
      <div class="table-header"><div class="search-box"><input type="text" placeholder="🔍 Buscar..."></div><button class="btn btn-success btn-sm">+ Novo Fornecedor</button></div>
      <table><thead><tr><th>ID</th><th>Nome</th><th>CNPJ</th><th>Telefone</th><th>Status</th></tr></thead>
      <tbody id="tbodyFornecedores"></tbody></table>
    </div>
  </div>

  <!-- RELATÓRIOS -->
  <div id="page-relatorios" class="page" style="display:none;">
    <div class="header"><h1> Relatórios</h1></div>
    <div class="stats">
      <div class="stat-card"><div class="num" id="rClientes">0</div><div class="label">Total Clientes</div></div>
      <div class="stat-card"><div class="num" id="rProdutos">0</div><div class="label">Total Produtos</div></div>
      <div class="stat-card"><div class="num" id="rVendas">0</div><div class="label">Total Vendas</div></div>
      <div class="stat-card"><div class="num" id="rFornecedores">0</div><div class="label">Total Fornecedores</div></div>
    </div>
  </div>

  <!-- TESTE 100 CLIENTES -->
  <div id="page-testar100" class="page" style="display:none;">
    <div class="header"><h1> Teste de 100 Clientes</h1></div>
    <div class="table-wrap">
      <div class="table-header">
        <div>

```



```

        <p style="color:var(--text3);font-size:13px;">Insere 100 clientes de teste via API para validar o sistema.</p>
    </div>
    <button class="btn btn-success btn-sm" onclick="executarTeste100()">▶ Executar Teste</button>
  </div>
  <div style="padding:24px;">
    <div id="testeResultado" style="font-family:monospace;font-size:13px;color:var(--text2);">Clique em "Executar Teste" para
    iniciar...</div>
  </div>
</div>
</div>

<!-- ===== -->
<!-- PORTAL ADMIN - CADASTRO DE CLIENTES (Corumba Sistemas) -->
<!-- ===== -->
<div id="page-admin-clientes" class="page" style="display:none;">
  <div class="header"><h1>🏠 Portal Admin - Cadastro de Clientes</h1></div>
  <div class="stats">
    <div class="stat-card"><div class="num" id="saTotal">0</div><div class="label">Clientes</div></div>
    <div class="stat-card"><div class="num" id="saAtivos">0</div><div class="label">Ativos</div></div>
    <div class="stat-card"><div class="num" id="saExpirados">0</div><div class="label">Expirados</div></div>
  </div>
  <div class="table-wrap" style="margin-top:24px;">
    <div class="table-header">
      <div class="search-box"><input type="text" id="searchAdminClientes" placeholder="🔍 Buscar..." oninput="buscarAdminClientes()"></div>
      <button class="btn btn-success btn-sm" onclick="abrirModalAdminCliente()">+ Novo Cliente</button>
    </div>
    <table>
      <thead><tr><th>ID</th><th>Empresa</th><th>CNPJ</th><th>Plano</th><th>Código Ativação</th><th>Licença</th><th>Status</th><th>Ações</th></tr></thead>
      <tbody id="tbodyAdminClientes"></tbody>
    </table>
  </div>
</div>

<!-- PORTAL DO CLIENTE - ATIVAÇÃO + EQUIPE -->
<div id="page-cliente-portal" class="page" style="display:none;">
  <div class="header"><h1>👤 Portal do Cliente</h1></div>

  <!-- Tela de Ativação (aparece se não ativou) -->
  <div id="cliente-ativacao" style="display:none;">
    <div class="login-box" style="margin:0 auto;max-width:500px;">
      <h1>Ativação de Licença</h1>
      <div class="sub">Digite o código de ativação recebido</div>
      <div class="form-group"><label>CNPJ da Empresa</label><input type="text" id="cliCnpj" placeholder="00.000.000/0000-00"></div>
      <div class="form-group"><label>Código de Ativação</label><input type="text" id="cliCodigo" placeholder="XXXX-XXXX-XXXX" style="text-align:center;font-family:monospace;font-size:18px;"></div>
      <div class="form-group"><label>Nome do Administrador</label><input type="text" id="cliAdminNome" placeholder="Seu nome"></div>
      <div class="form-group"><label>Senha do Administrador</label><input type="password" id="cliAdminSenha" placeholder="Senha para login"></div>
    </div>
    <div id="cliAtivacaoMsg" class="error-msg"></div>
    <button class="btn btn-success" onclick="clienteAtivar()">🔒 Ativar Licença</button>
  </div>

  <!-- Tela de Gestão (aparece após ativação) -->
  <div id="cliente-gestao" style="display:none;">
    <div class="stats">
      <div class="stat-card"><div class="num" id="cliDias">-</div><div class="label">Dias Restantes</div></div>
      <div class="stat-card"><div class="num" id="cliFuncionarios">0</div><div class="label">Funcionários</div></div>
      <div class="stat-card"><div class="num" id="cliPlano">-</div><div class="label">Plano</div></div>
    </div>

    <!-- Gestão de Equipe -->
    <div class="table-wrap" style="margin-top:24px;">
      <div class="table-header"><h3>👥 Gestão de Equipe</h3><button class="btn btn-success btn-sm" onclick="abrirModalFuncionario()">+ Novo Funcionário</button></div>
      <table>
        <thead><tr><th>Nome</th><th>Usuário</th><th>Papel</th><th>Status</th><th>Ações</th></tr></thead>
        <tbody id="tbodyFuncionarios"></tbody>
      </table>
    </div>

    <!-- Módulos Contratados -->
    <div class="table-wrap" style="margin-top:24px;">
      <div class="table-header"><h3>📦 Módulos do Sistema</h3></div>
      <div style="padding:20px;display:flex;gap:16px;flex-wrap:wrap;">
        <div class="stat-card"><div class="num" style="font-size:18px;">📊 SaltoGestão</div><div class="label">Gestão de Clientes</div></div>
        <div class="stat-card"><div class="num" style="font-size:18px;">🍽️ MesaPronta</div><div class="label">PDV Restaurante</div></div>
        <div class="stat-card"><div class="num" style="font-size:18px;">🍷 ConnectFood</div><div class="label">Integração iFood/99Food</div></div>
      </div>
    </div>
  </div>

  <!-- MODAL NOVO CLIENTE (Admin) -->
  <div id="modalAdminCliente" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalAdminCliente')">
    <div class="modal" style="max-width:500px;">
      <h3 id="modalAdminClienteTitulo">Novo Cliente</h3>
      <input type="hidden" id="acId">
      <div class="form-group"><label>Nome da Empresa:</label><input type="text" id="acNome"></div>
      <div class="form-group"><label>CNPJ:</label><input type="text" id="acDocumento" placeholder="00.000.000/0000-00"></div>
    </div>
  </div>

```

```

<div class="form-group"><label>E-mail:</label><input type="email" id="acEmail"></div>
<div class="form-group"><label>Telefone:</label><input type="text" id="acTelefone"></div>
<div class="form-group"><label>Cidade:</label><input type="text" id="acCidade"></div>
<div class="form-group"><label>Estado:</label><input type="text" id="acEstado" maxlength="2"></div>
<div class="form-group"><label>Plano:</label>
  <select id="acPlano">
    <option value="mensal">Mensal (30 dias)</option>
    <option value="trimestral">Trimestral (90 dias)</option>
    <option value="anual">Anual (365 dias)</option>
  </select>
</div>
<div class="form-group"><label>Máx. Usuários:</label><input type="number" id="acMaxUsuarios" value="5" min="1" max="50"></div>
<div id="acMsg" class="error-msg"></div>
<div class="actions">
  <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalAdminCliente')">Cancelar</button>
  <button class="btn btn-success btn-sm" onclick="salvarAdminCliente()">📌 Salvar & Gerar Código</button>
</div>
</div>
</div>
</div>

<!-- MODAL FUNCIONÁRIO (Cliente) -->
<div id="modalFuncionario" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalFuncionario')">
  <div class="modal" style="max-width:450px;">
    <h3>Novo Funcionário</h3>
    <input type="hidden" id="fcId">
    <div class="form-group"><label>Nome:</label><input type="text" id="fcNome"></div>
    <div class="form-group"><label>Usuário:</label><input type="text" id="fcUsername" placeholder="login do funcionário"></div>
    <div class="form-group"><label>Senha:</label><input type="password" id="fcSenha" placeholder="senha padrão"></div>
    <div class="form-group"><label>Papel:</label>
      <select id="fcPapel">
        <option value="ADMIN">Administrador</option>
        <option value="GARCOM">Garçom</option>
        <option value="COZINHA">Cozinha</option>
        <option value="CAIXA">Caixa</option>
      </select>
    </div>
    <div id="fcMsg" class="error-msg"></div>
    <div class="actions">
      <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalFuncionario')">Cancelar</button>
      <button class="btn btn-success btn-sm" onclick="salvarFuncionario()">📌 Salvar</button>
    </div>
  </div>
</div>

<div id="modalCliente" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalCliente')">
  <div class="modal">
    <h3 id="modalClienteTitulo">Novo Cliente</h3>
    <input type="hidden" id="cId">
    <div class="form-group"><label>Nome:</label><input type="text" id="cNome"></div>
    <div class="form-group"><label>Telefone:</label><input type="text" id="cTelefone"></div>
    <div class="form-group"><label>E-mail:</label><input type="email" id="cEmail"></div>
    <div class="form-group"><label>CPF/CNPJ:</label><input type="text" id="cCpfCnpj"></div>
    <div class="form-group"><label>Endereço:</label><input type="text" id="cEndereco"></div>
    <div class="form-group"><label>Cidade:</label><input type="text" id="cCidade"></div>
    <div class="form-group"><label>Estado:</label><input type="text" id="cEstado" maxlength="2"></div>
    <div class="form-group"><label>CEP:</label><input type="text" id="cCep"></div>
    <div class="actions">
      <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalCliente')">Cancelar</button>
      <button class="btn btn-success btn-sm" onclick="salvarCliente()">📌 Salvar</button>
    </div>
  </div>
</div>
</div>

<script>
const S = {
  token: localStorage.getItem('token') || null,
  usuario: JSON.parse(localStorage.getItem('usuario') || 'null'),
  apiUrl: localStorage.getItem('apiUrl') || 'http://191.252.210.235:8080',
  clientes: [],
  produtos: [],
  vendas: [],
  fornecedores: []
};

async function api(method, path, body) {
  const opts = { method, headers: {'Content-Type':'application/json'} };
  if (S.token) opts.headers['Authorization'] = 'Bearer ' + S.token;
  if (body) opts.body = JSON.stringify(body);
  const r = await fetch(S.apiUrl + path, opts);
  if (r.status === 401) { sair(); return null; }
  const t = await r.text();
  return t ? JSON.parse(t) : null;
}

function toast(msg, type) {
  const d = document.createElement('div');
  d.className = 'toast toast-' + (type || 'success');
  d.textContent = msg;
  document.body.appendChild(d);
  setTimeout(() => d.remove(), 3000);
}

function fazerLogin() {
  S.apiUrl = document.getElementById('apiUrl').value.trim();
  localStorage.setItem('apiUrl', S.apiUrl);
  const username = document.getElementById('username').value.trim();

```

```

const senha = document.getElementById('senha').value.trim();
const body = username.includes('@') ? {email:username, senha} : {username, senha};
fetch(S.apiUrl + '/api/login', {method:'POST', headers:{'Content-Type':'application/json'}, body:JSON.stringify(body)})
.then(r=>r.json()).then(d=>{
  if (d.access_token) { S.token=d.access_token; S.usuario=d.usuario||d.admin; localStorage.setItem('token',S.token);
  localStorage.setItem('usuario',JSON.stringify(S.usuario)); entrar(); }
  else document.getElementById('loginError').textContent = '❌ ' + (d.detail||'Falha no login');
}).catch(e=>{ document.getElementById('loginError').textContent = '❌ Erro: ' + e.message; });
}

function entrar() {
  document.getElementById('loginPage').style.display='none';
  document.getElementById('app').style.display='block';
  document.getElementById('userInfo').textContent = S.usuario.nome + ' (' + S.usuario.papel + ')';
  carregarDashboard();
}

function sair() {
  S.token=S.usuario=null; localStorage.removeItem('token'); localStorage.removeItem('usuario');
  document.getElementById('app').style.display='none';
  document.getElementById('loginPage').style.display='flex';
}

function showPage(page, e) {
  if (e) e.preventDefault();
  document.querySelectorAll('.page').forEach(p=>p.style.display='none');
  document.querySelectorAll('.sidebar nav a').forEach(a=>a.classList.remove('active'));
  document.getElementById('page-'+page).style.display='block';
  if (e && e.target) e.target.classList.add('active');
  if (page==='dashboard') carregarDashboard();
  if (page==='clientes') carregarClientes();
  if (page==='produtos') carregarProdutos();
  if (page==='vendas') carregarVendas();
  if (page==='fornecedores') carregarFornecedores();
  if (page==='relatorios') carregarRelatorios();
  if (page==='admin-clientes') carregarAdminClientes();
  if (page==='cliente-portal') carregarPortalCliente();
}

async function carregarDashboard() {
  const [c,p,v,f] = await Promise.all([
    api('GET','/api/clientes/?ativo=true&ordem=recente&limite=5'),
    api('GET','/api/produtos/'),
    api('GET','/api/vendas/'),
    api('GET','/api/fornecedores/')
  ]);
  if(c){ document.getElementById('stClientes').textContent=c.length;
  document.getElementById('dashboardClientes').innerHTML = c.slice(0,5).map(x=><tr><td>${x.id}</td><td>${x.nome||'-'}</td><td>${x.telefone||'-'}</td><td>${x.cpf_cnpj||'-'}</td><td><span class="badge ${x.ativo?'badge-success':'badge-danger'}">${x.ativo?'Ativo':'Inativo'}</span></td></tr>).join('');
  }
  if(p) document.getElementById('stProdutos').textContent=p.length;
  if(v) document.getElementById('stVendas').textContent=v.length;
  if(f) document.getElementById('stFornecedores').textContent=f.length;
}

async function carregarClientes() {
  const busca = document.getElementById('searchClientes').value;
  const ativo = document.getElementById('filterClientes').value;
  let url = '/api/clientes/?';
  if(ativo) url+=`ativo=${ativo}&`;
  if(busca) url+=`busca=${encodeURIComponent(busca)}&`;
  url+=`ordem=nome&limite=100`;
  S.clientes = await api('GET',url) || [];
  renderClientes(S.clientes);
}

function renderClientes(list) {
  const t = document.getElementById('tbodyClientes');
  if(!list.length){ t.innerHTML=<tr><td colspan="9" style="text-align:center;color:var(--text3);padding:40px;">Nenhum cliente encontrado</td></tr>; return; }
  t.innerHTML = list.map(c=><tr><td>${c.id}</td><td>${c.nome||'-'}</td><td>${c.telefone||'-'}</td><td>${c.email||'-'}</td><td>${c.cpf_cnpj||'-'}</td><td>${c.cidade||'-'}</td><td><span class="badge badge-info">${c.pontos_fidelidade||0}</span></td><td><span class="badge ${c.ativo?'badge-success':'badge-danger'}">${c.ativo?'Ativo':'Inativo'}</span></td><td><div class="actions"><button class="btn btn-sm btn-primary" onclick="editarCliente(${c.id})">✎</button><button class="btn btn-sm btn-danger" onclick="excluirCliente(${c.id})">🗑️</button></div></td></tr>).join('');
}

function buscarClientes(){ carregarClientes(); }

function abrirModalCliente() {
  document.getElementById('modalClienteTitulo').textContent='Novo Cliente';
  ['cId','cNome','cTelefone','cEmail','cCpfCnpj','cEndereco','cCidade','cEstado','cCep'].forEach(id=>document.getElementById(id).value='');
  document.getElementById('modalCliente').classList.add('open');
}

async function editarCliente(id) {
  const c=S.clientes.find(x=>x.id===id); if(!c) return;
  document.getElementById('modalClienteTitulo').textContent='Editar #'+id;
  for(const k of ['Id','Nome','Telefone','Email','CpfCnpj','Endereco','Cidade','Estado','Cep']){ const el=document.getElementById('c'+k);
  if(el) el.value=c[k.toLowerCase()]||''; }
  document.getElementById('modalCliente').classList.add('open');
}

async function salvarCliente() {

```

```

const dados = {
  nome:document.getElementById('cNome').value.trim(),
  telefone:document.getElementById('cTelefone').value.trim()||null,
  email:document.getElementById('cEmail').value.trim()||null,
  cpf_cnpj:document.getElementById('cCpfCnpj').value.trim()||null,
  endereco:document.getElementById('cEndereco').value.trim()||null,
  cidade:document.getElementById('cCidade').value.trim()||null,
  estado:document.getElementById('cEstado').value.trim()||null,
  cep:document.getElementById('cCep').value.trim()||null
};
if(!dados.nome){toast('Nome obrigatório!', 'error');return;}
const id=document.getElementById('cId').value;
if(id){ await api('PUT', '/api/clientes/'+id,dados); toast('Atualizado! ✅'); }
else{ await api('POST', '/api/clientes/', dados); toast('Cadastrado! ✅'); }
fecharModal('modalCliente');
carregarClientes();
}

async function excluirCliente(id){ if(!confirm('Excluir cliente #' +id+'?')) return; await api('DELETE', '/api/clientes/'+id);
toast('Desativado! ✅'); carregarClientes(); }

async function carregarProdutos(){ S.produtos=await api('GET', '/api/produtos/')||[]; const t=document.getElementById('tbodyProdutos'); if(!
S.produtos.length){t.innerHTML=<tr><td colspan="7" style="text-align:center;color:var(--text3);padding:40px;">Nenhum produto</td></
tr>;return;} t.innerHTML=S.produtos.map(p=><tr><td>${p.id}</td><td>${p.nome||'-'}</td><td>${p.sku||'-'}</td><td>R$${
parseFloat(p.preco_venda).toFixed(2)}</td><td>${p.estoque_atual||0}</td><td><span class="badge ${p.ativo?'badge-success':'badge-danger'}">${
p.ativo?'Ativo':'Inativo'}</span></td></tr>`).join(''); }

async function carregarVendas(){ S.vendas=await api('GET', '/api/vendas/')||[]; const t=document.getElementById('tbodyVendas'); if(!
S.vendas.length){t.innerHTML=<tr><td colspan="6" style="text-align:center;color:var(--text3);padding:40px;">Nenhuma venda</td></
tr>;return;} t.innerHTML=S.vendas.map(v=><tr><td>${v.id}</td><td>${v.data_venda?v.data_venda.substring(0,10):-'}</td><td>${
v.cliente_nome||'-'}</td><td>R$${parseFloat(v.total||0).toFixed(2)}</td><td><span class="badge ${v.status==='CONCLUIDA'? 'badge-
success':'badge-danger'}">${v.status}</span></td></tr>`).join(''); }

async function carregarFornecedores(){ S.fornecedores=await api('GET', '/api/fornecedores/')||[]; const
t=document.getElementById('tbodyFornecedores'); if(!S.fornecedores.length){t.innerHTML=<tr><td colspan="6" style="text-
align:center;color:var(--text3);padding:40px;">Nenhum fornecedor</td></tr>;return;} t.innerHTML=S.fornecedores.map(f=><tr><td>${f.id}</
td><td>${f.nome_fantasia||'-'}</td><td>${f.cnpj_cpf||'-'}</td><td>${f.telefone||'-'}</td><td><span class="badge ${f.ativo?'badge-
success':'badge-danger'}">${f.ativo?'Ativo':'Inativo'}</span></td></tr>`).join(''); }

async function carregarRelatorios(){
  const [c,p,v,f]=await Promise.all([api('GET', '/api/clientes/?limite=10000'),api('GET', '/api/produtos/'),api('GET', '/api/
vendas/'),api('GET', '/api/fornecedores/')]);
  if(c)document.getElementById('rClientes').textContent=c.length;
  if(p)document.getElementById('rProdutos').textContent=p.length;
  if(v)document.getElementById('rVendas').textContent=v.length;
  if(f)document.getElementById('rFornecedores').textContent=f.length;
}

// =====
// TESTE 100 CLIENTES
// =====
async function executarTeste100() {
  const out = document.getElementById('testeResultado');
  out.innerHTML = '🕒 Executando teste...<br>';

  const nomes =
  ['João','Maria','José','Ana','Pedro','Carla','Lucas','Fernanda','Marcos','Juliana','Rafael','Patricia','Bruno','Camila','Diego','Amanda','Gustavo
  H.','Vitória','Bruno H.','Luna','Cauã','Marina','Kaïque','Valentim','Yuri','Antonella'];
  const sobren =
  ['Silva','Santos','Oliveira','Souza','Rodrigues','Ferreira','Alves','Pereira','Lima','Gomes','Costa','Ribeiro','Martins','Carvalho','Araújo','Mel
  const bairros = ['Centro','Jardim América','Setor Oeste','Setor Bueno','Jardim Goiás','Campinas','Vila Aurora','Jardim Europa'];
  const cidades = ['Aparecida de Goiânia','Goiânia','Anápolis'];
  const usados = new Set();

  let ok=0, fail=0;
  for(let i=0;i<100;i++){
    const nome = nomes[i%nomes.length]+' '+sobren[i%sobren.length]+' '+sobren[(i*3+7)%sobren.length];
    let cpf; do{ cpf = String(Math.floor(1e10+Math.random()*9e10)); }while(usados.has(cpf.substring(0,9)));
    usados.add(cpf.substring(0,9));
    const dados = {
      nome: nome.trim(),
      telefone: '629'+String(Math.floor(1e7+Math.random()*9e7)),
      email: nome.toLowerCase().replace(/ /g, '')+(i+1)+'@email.com',
      cpf_cnpj: cpf,
      endereco: 'Rua '+ (i+1) +', '+(i*10),
      cidade: cidades[i%cidades.length],
      estado: 'GO',
      cep: String(Math.floor(1e8+Math.random()*9e8))
    };
    try{
      const r = await api('POST', '/api/clientes/', dados);
      if(r && r.id){ ok++; }else{ fail++; }
    } catch(e){ fail++; }
    if((i+1)%20===0) out.innerHTML += `📊 ${i+1}/100... (✅${ok} ❌${fail})<br>`;
  }
  out.innerHTML += `<br>📊 <b>Resultado: ${ok}/100 OK, ${fail} falhas</b>`;
  if(ok===100){ out.innerHTML += `<br>🎉 <b style="color:#10b981">PASSOU! Sistema operacional!</b>`; }
  toast(ok+' /100 clientes inseridos!', ok===100?'success':'error');
  // Recarregar dashboard
  carregarDashboard();
}

// =====
// PORTAL ADMIN - CADASTRO DE CLIENTES
// =====

```

```

async function carregarAdminClientes() {
  const data = await api('GET', '/api/admin/clientes');
  if (!data) return;
  const lista = data.clientes || [];
  document.getElementById('saTotal').textContent = lista.length;
  document.getElementById('saAtivos').textContent = lista.filter(c => c.licenca_status === 'ativa').length;
  document.getElementById('saExpirados').textContent = lista.filter(c => c.licenca_status === 'expirada').length;
  renderAdminClientes(lista);
}

function renderAdminClientes(lista) {
  const t = document.getElementById('tbodyAdminClientes');
  if (!lista.length) { t.innerHTML = '<tr><td colspan="8" style="text-align:center;color:var(--text3);padding:40px;">Nenhum cliente cadastrado</td></tr>'; return; }
  t.innerHTML = lista.map(c => {
    const licBadge = c.licenca_status === 'ativa' ? 'badge-success' : (c.licenca_status === 'expirada' ? 'badge-danger' : 'badge-warning');
    const statusBadge = c.ativo ? 'badge-success' : 'badge-danger';
    return `<tr>
      <td>${c.id}</td><td>${c.nome||'-'}</td><td>${c.documento||'-'}</td><td>${c.plano||'-'}</td>
      <td>${c.codigo_ativacao||'-'}<span style="color:var(--text3)">Pendente</span>'</td>
      <td><span class="badge ${licBadge}">${c.licenca_status||'N/A'}</span></td>
      <td><span class="badge ${statusBadge}">${c.ativo?'Ativo':'Inativo'}</span></td>
      <td class="actions">
        <button class="btn btn-sm btn-primary" onclick="copiarCodigo('${c.codigo_ativacao||'-'}')">📋</button>
        <button class="btn btn-sm btn-danger" onclick="excluirAdminCliente(${c.id}, '${c.nome}')">🗑️</button>
      </td>
    </tr>`;
  }).join('');
}

function buscarAdminClientes() {
  const q = document.getElementById('searchAdminClientes').value.toLowerCase();
  // Re-carregar e filtrar
  carregarAdminClientes().then(() => {
    const rows = document.querySelectorAll('#tbodyAdminClientes tr');
    rows.forEach(r => { r.style.display = r.textContent.toLowerCase().includes(q) ? '' : 'none'; });
  });
}

function abrirModalAdminCliente() {
  document.getElementById('modalAdminClienteTitulo').textContent = 'Novo Cliente';
  ['acId', 'acNome', 'acDocumento', 'acEmail', 'acTelefone', 'acCidade', 'acEstado'].forEach(id => document.getElementById(id).value = '');
  document.getElementById('acPlano').value = 'mensal';
  document.getElementById('acMaxUsuarios').value = '5';
  document.getElementById('acMsg').textContent = '';
  document.getElementById('modalAdminCliente').classList.add('open');
}

async function salvarAdminCliente() {
  const dados = {
    nome: document.getElementById('acNome').value.trim(),
    documento: document.getElementById('acDocumento').value.trim(),
    email: document.getElementById('acEmail').value.trim(),
    telefone: document.getElementById('acTelefone').value.trim(),
    cidade: document.getElementById('acCidade').value.trim(),
    estado: document.getElementById('acEstado').value.trim(),
    plano: document.getElementById('acPlano').value,
    max_usuarios: parseInt(document.getElementById('acMaxUsuarios').value) || 5
  };
  if (!dados.nome || !dados.documento) {
    document.getElementById('acMsg').textContent = 'Nome e CNPJ obrigatórios!';
    return;
  }
  const r = await api('POST', '/api/admin/clientes', dados);
  if (r && r.sucesso) {
    toast(`Cliente cadastrado! Código: ${r.codigo_ativacao}`, 'success');
    fecharModal('modalAdminCliente');
    carregarAdminClientes();
  } else {
    document.getElementById('acMsg').textContent = r?.erro || 'Erro ao cadastrar';
  }
}

function copiarCodigo(codigo) {
  if (!codigo) return;
  navigator.clipboard.writeText(codigo);
  toast(`Código ${codigo} copiado!`, 'success');
}

async function excluirAdminCliente(id, nome) {
  if (!confirm(`Excluir cliente "${nome}" e todos os dados?`)) return;
  await api('DELETE', '/api/admin/clientes/${id}');
  toast('Excluído!', 'success');
  carregarAdminClientes();
}

// =====
// PORTAL DO CLIENTE - ATIVAÇÃO + EQUIPE
// =====
let clienteCnpj = localStorage.getItem('clienteCnpj') || '';

async function carregarPortalCliente() {
  if (!clienteCnpj) {
    // Mostrar tela de ativação
    document.getElementById('cliente-ativacao').style.display = 'block';
  }
}

```

```

    document.getElementById('cliente-gestao').style.display = 'none';
    return;
}

// Verificar se já está ativo
const r = await api('POST', '/api/licenca/validar', { cnpj: clienteCnpj });
if (r && r.valida) {
    document.getElementById('cliente-ativacao').style.display = 'none';
    document.getElementById('cliente-gestao').style.display = 'block';
    document.getElementById('cliDias').textContent = r.dias_restantes || 0;
    document.getElementById('cliPlano').textContent = r.plano || '-';
    carregarFuncionarios();
} else {
    document.getElementById('cliente-ativacao').style.display = 'block';
    document.getElementById('cliente-gestao').style.display = 'none';
}
}

async function clienteAtivar() {
    const cnpj = document.getElementById('cliCnpj').value.trim();
    const codigo = document.getElementById('cliCodigo').value.trim();
    const adminNome = document.getElementById('cliAdminNome').value.trim();
    const adminSenha = document.getElementById('cliAdminSenha').value;

    if (!cnpj || !codigo) {
        document.getElementById('cliAtivacaoMsg').textContent = 'CNPJ e código obrigatórios!';
        return;
    }

    const r = await api('POST', '/api/cliente/ativar', { cnpj, codigo, admin_nome: adminNome, admin_senha: adminSenha });
    if (r && r.sucesso) {
        clienteCnpj = cnpj;
        localStorage.setItem('clienteCnpj', cnpj);
        toast('Ativado! ${r.dias_ativacao} dias de licença.', 'success');
        carregarPortalCliente();
    } else {
        document.getElementById('cliAtivacaoMsg').textContent = r?.erro || 'Erro na ativação';
    }
}

async function carregarFuncionarios() {
    if (!clienteCnpj) return;
    const data = await api('GET', '/api/cliente/${clienteCnpj}/funcionarios');
    if (!data) return;
    const lista = data.funcionarios || [];
    document.getElementById('cliFuncionarios').textContent = lista.length;

    const t = document.getElementById('tbodyFuncionarios');
    if (!lista.length) { t.innerHTML = '<tr><td colspan="5" style="text-align:center;color:var(--text3);padding:30px;>Nenhum funcionário cadastrado</td></tr>'; return; }
    t.innerHTML = lista.map(f => `<tr>
        <td>${f.nome}</td><td>${f.username}</td>
        <td><span class="badge">${f.papel}</span></td>
        <td><span class="badge ${f.ativo?'badge-success':'badge-danger'}">${f.ativo?'Ativo':'Inativo'}</span></td>
        <td class="actions">
            <button class="btn btn-sm btn-danger" onclick="excluirFuncionario(${f.id})"><img alt="trash icon" /></button>
        </td>
    </tr>`).join('');
}

function abrirModalFuncionario() {
    ['fcId', 'fcNome', 'fcUsername', 'fcSenha'].forEach(id => document.getElementById(id).value = '');
    document.getElementById('fcPapel').value = 'GARCOM';
    document.getElementById('fcMsg').textContent = '';
    document.getElementById('modalFuncionario').classList.add('open');
}

async function salvarFuncionario() {
    if (!clienteCnpj) return;
    const dados = {
        nome: document.getElementById('fcNome').value.trim(),
        username: document.getElementById('fcUsername').value.trim(),
        senha: document.getElementById('fcSenha').value || '123456',
        papel: document.getElementById('fcPapel').value
    };
    if (!dados.nome || !dados.username) {
        document.getElementById('fcMsg').textContent = 'Nome e usuário obrigatórios!';
        return;
    }
    const r = await api('POST', '/api/cliente/${clienteCnpj}/funcionarios', dados);
    if (r && r.sucesso) {
        toast('Funcionário cadastrado!', 'success');
        fecharModal('modalFuncionario');
        carregarFuncionarios();
    } else {
        document.getElementById('fcMsg').textContent = r?.erro || 'Erro ao cadastrar';
    }
}

async function excluirFuncionario(id) {
    if (!confirm('Excluir funcionário?')) return;
    await api('DELETE', '/api/cliente/${clienteCnpj}/funcionarios/${id}');
    toast('Excluído!', 'success');
    carregarFuncionarios();
}

```

```
// FIM PORTAL CLIENTE

function fecharModal(id){ document.getElementById(id).classList.remove('open'); }
if(S.token && S.usuario) entrar();
</script>
</body>
</html>
```



docker-compose.yml

docker-compose.yml • 37 linhas

```
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    container_name: corumba-postgres
    restart: always
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: corumba2025
      POSTGRES_DB: corumba
    volumes:
      - postgres_data:/var/lib/postgresql/data
    # Permitir conexões de qualquer máquina da rede local
    command: >
      postgres
      -c listen_addresses='*'
      -c max_connections=100

  # Opcional: pgAdmin para visualizar o banco no browser
  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: corumba-pgadmin
    restart: always
    ports:
      - "5050:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@corumba.com
      PGADMIN_DEFAULT_PASSWORD: admin123
    depends_on:
      - postgres

volumes:
  postgres_data:
```

2. Backend - Nucleo (database, models, security)

 database.py

backendPy/database.py • 104 linhas

```
"""
database.py - Configuração do banco de dados e sessões SQLAlchemy

Suporta 3 ambientes automaticamente:
- development (sua máquina) → SQLite local
- production (VPS dedicada) → PostgreSQL centralizado
- testing (CI/automação) → SQLite em memória
"""
import os
import sys
from sqlalchemy import create_engine, Column, Integer, String, event
from sqlalchemy.orm import sessionmaker, declarative_base

def _build_engine(url: str):
    """Cria engine compatível com SQLite e PostgreSQL."""
    if url.startswith("sqlite"):
        eng = create_engine(url, connect_args={"check_same_thread": False})
        # Otimizações SQLite para PDV
        @event.listens_for(eng, "connect")
        def _sqlite_pragma(dbapi_conn, connection_record):
            dbapi_conn.execute("PRAGMA journal_mode=WAL")
            dbapi_conn.execute("PRAGMA foreign_keys=ON")
            dbapi_conn.execute("PRAGMA synchronous=NORMAL")
            dbapi_conn.execute("PRAGMA cache_size=10000")
        return eng
    else:
        return create_engine(
            url,
            pool_pre_ping=True,
            pool_size=20,
            max_overflow=30,
            pool_timeout=30,
            pool_recycle=3600,
            echo=False,
        )

# =====
# AMBIENTE E URL DO BANCO
# =====
APP_ENV = os.getenv("APP_ENV", "development")

if "DATABASE_URL" in os.environ:
    DB_URL = os.environ["DATABASE_URL"]
elif APP_ENV == "production":
    DB_URL = os.getenv("DATABASE_URL", "postgres://psycopg2://corumba:corumba2025@localhost:5432/corumba")
elif APP_ENV == "testing":
    DB_URL = "sqlite:///file:memory:?cache=shared&mode=memory"
else: # development
    DB_URL = "sqlite:///./salto_principal.db"

print(f"[database] Ambiente: {APP_ENV} | Banco: {DB_URL.split('/:')[0]}")

engine = _build_engine(DB_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# =====
# BASES DECLARATIVAS (metadados separados)
# =====
# BaseCore: models do sistema Corumbá (Cliente, Usuario, AdminCorumba, Produto catálogo, etc.)
# BaseSalto: models operacionais do SaltoGestão (clientes, produtos, vendas, etc.)
# Em dev SQLite ambas usam o mesmo engine/arquivo; em produção PostgreSQL usariam bancos separados
BaseCore = declarative_base()
BaseSalto = declarative_base()

# =====
# MIXIN MULTI-TENANT (para models operacionais)
# =====
class TenantMixin:
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)

# =====
# DEPENDENCIES PARA FASTAPI
# =====
def get_db():
    """Sessão do banco (core)."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```



```
def get_db_salto():
    """Sessão do banco operacional SaltoGestão."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def get_db_core():
    """Sessão do banco core (Corumbá)."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

models.py

backendPy/models.py • 216 linhas

```
"""
models.py - Modelos SQLAlchemy do banco operacional (BaseSalto)
Todas as tabelas herdam de TenantMixin para isolamento multi-tenant por CNPJ.

Modelos do banco core (Usuario, Empresa) estao em models_core.py
"""
import datetime
from sqlalchemy import (
    Column, Integer, String, Numeric, Boolean, ForeignKey,
    DateTime, Text, Enum as SAEnum
)
from sqlalchemy.orm import relationship
from database import BaseSalto, TenantMixin

# =====
# PRODUTOS
# =====
class Produto(BaseSalto, TenantMixin):
    __tablename__ = "op_produtos"

    nome = Column(String(255), nullable=False)
    descricao = Column(Text, nullable=True)
    sku = Column(String(50), index=True)
    codigo_barras = Column(String(50), index=True)
    preco_venda = Column(Numeric(10, 2), nullable=False, default=0)
    preco_custo = Column(Numeric(10, 2), default=0)
    estoque_atual = Column(Numeric(10, 3), default=0)
    estoque_minimo = Column(Numeric(10, 3), default=0)
    unidade = Column(String(10), default="UN") # UN, KG, LT, CX, etc.
    ncm = Column(String(8))
    cfop = Column(String(4))
    cest = Column(String(7))
    ativo = Column(Boolean, default=True)

    # Relacoes
    movimentacoes = relationship("EstoqueMovimentacao", back_populates="produto", cascade="all, delete-orphan")
    venda_itens = relationship("VendaItem", back_populates="produto")

# =====
# CLIENTES
# =====
class Cliente(BaseSalto, TenantMixin):
    __tablename__ = "op_clientes"

    nome = Column(String(255), nullable=False)
    telefone = Column(String(20), index=True)
    email = Column(String(255), index=True)
    cpf_cnpj = Column(String(20), index=True)
    endereco = Column(String(500))
    cidade = Column(String(100))
    estado = Column(String(2))
    cep = Column(String(10))
    data_nascimento = Column(String(10))
    pontos_fidelidade = Column(Integer, default=0)
    ativo = Column(Boolean, default=True)

    # Relacoes
    vendas = relationship("Venda", back_populates="cliente")

# =====
# FORNECEDORES
# =====
class Fornecedor(BaseSalto, TenantMixin):
    __tablename__ = "op_fornecedores"

    nome_fantasia = Column(String(255), nullable=False)
    razao_social = Column(String(255))
    cnpj_cpf = Column(String(20), index=True)
    telefone = Column(String(20))
```

```

email = Column(String(255))
endereco = Column(String(500))
ativo = Column(Boolean, default=True)

# =====
# VENDAS E ITENS
# =====
class Venda(BaseSalto, TenantMixin):
    __tablename__ = "op_vendas"

    cliente_id = Column(Integer, ForeignKey("op_clientes.id"), nullable=True)
    vendedor_id = Column(Integer, nullable=True)
    vendedor_nome = Column(String(255), nullable=True)
    operador = Column(String(255), nullable=True)
    total = Column(Numeric(10, 2), nullable=False, default=0)
    desconto = Column(Numeric(10, 2), default=0)
    valor_recebido = Column(Numeric(10, 2), default=0)
    troco = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(String(50))
    status = Column(String(20), default="CONCLUIDA") # CONCLUIDA, CANCELADA
    observacao = Column(Text, nullable=True)
    data_venda = Column(DateTime, default=datetime.datetime.utcnow)

    # Relacoes
    itens = relationship("VendaItem", back_populates="venda", cascade="all, delete-orphan", lazy="joined")
    cliente = relationship("Cliente", back_populates="vendas", lazy="joined")

class VendaItem(BaseSalto, TenantMixin):
    __tablename__ = "op_venda_itens"

    venda_id = Column(Integer, ForeignKey("op_vendas.id"), nullable=False)
    produto_id = Column(Integer, ForeignKey("op_produtos.id"), nullable=False)
    produto_nome = Column(String(255)) # _snapshot do nome no momento da venda
    quantidade = Column(Numeric(10, 3), nullable=False)
    preco_unitario = Column(Numeric(10, 2), nullable=False)
    subtotal = Column(Numeric(10, 2), nullable=False)

    venda = relationship("Venda", back_populates="itens")
    produto = relationship("Produto", back_populates="venda_itens", lazy="joined")

# =====
# ESTOQUE - MOVIMENTACOES
# =====
class EstoqueMovimentacao(BaseSalto, TenantMixin):
    __tablename__ = "op_estoque_movimentacoes"

    produto_id = Column(Integer, ForeignKey("op_produtos.id"), nullable=False)
    tipo = Column(String(20), nullable=False) # ENTRADA, SAIDA, AJUSTE, INVENTARIO
    quantidade = Column(Numeric(10, 3), nullable=False)
    motivo = Column(String(255), nullable=True)
    usuario = Column(String(255), nullable=True)
    data_movimentacao = Column(DateTime, default=datetime.datetime.utcnow)

    produto = relationship("Produto", back_populates="movimentacoes")

# =====
# CAIXA - SESSOES E TRANSACOES
# =====
class CaixaSessao(BaseSalto, TenantMixin):
    __tablename__ = "op_caixa_sessoes"

    operador = Column(String(255), nullable=False)
    usuario_id = Column(Integer, nullable=True)
    status = Column(String(20), default="ABERTO") # ABERTO, FECHADO
    saldo_inicial = Column(Numeric(10, 2), default=0)
    saldo_final = Column(Numeric(10, 2), nullable=True)
    total_vendas = Column(Numeric(10, 2), default=0)
    total_entradas = Column(Numeric(10, 2), default=0)
    total_saidas = Column(Numeric(10, 2), default=0)
    data_abertura = Column(DateTime, default=datetime.datetime.utcnow)
    data_fechamento = Column(DateTime, nullable=True)

    transacoes = relationship("CaixaTransacao", back_populates="sessao", cascade="all, delete-orphan")

class CaixaTransacao(BaseSalto, TenantMixin):
    __tablename__ = "op_caixa_transacoes"

    sessao_id = Column(Integer, ForeignKey("op_caixa_sessoes.id"), nullable=False)
    tipo = Column(String(20), nullable=False) # VENDA, ENTRADA, SAIDA, SANGRIA, DESCONTO, CANCELAMENTO
    valor = Column(Numeric(10, 2), nullable=False)
    descricao = Column(String(255), nullable=True)
    venda_id = Column(Integer, nullable=True)
    operador = Column(String(255), nullable=True)
    autorizado_por = Column(String(255), nullable=True) # Para descontos/sangrias
    data_transacao = Column(DateTime, default=datetime.datetime.utcnow)

    sessao = relationship("CaixaSessao", back_populates="transacoes")

# =====

```

```
# FLUXO DE CAIXA (resumo diario consolidado)
# =====
class FluxoCaixa(BaseSalto, TenantMixin):
    __tablename__ = "op_fluxo_caixa"

    data_referencia = Column(String(10), nullable=False) # YYYY-MM-DD
    total_vendas = Column(Numeric(10, 2), default=0)
    total_entradas = Column(Numeric(10, 2), default=0)
    total_saidas = Column(Numeric(10, 2), default=0)
    saldo_dia = Column(Numeric(10, 2), default=0)
    quantidade_vendas = Column(Integer, default=0)
    ticket_medio = Column(Numeric(10, 2), default=0)
    atualizado_em = Column(DateTime, default=datetime.datetime.utcnow)

# =====
# HISTÓRICO DE VENDAS - Auditoria completa
# =====
class HistoricoVenda(BaseSalto, TenantMixin):
    __tablename__ = "op_historico_vendas"

    # id e empresa_documento vem de TenantMixin (PK + isolamento multi-tenant)
    venda_id = Column(Integer, nullable=True)
    numero_pedido = Column(String(50), index=True)
    usuario_id = Column(Integer, nullable=True)
    usuario_nome = Column(String(255), nullable=True)
    usuario_papel = Column(String(30), nullable=True)
    cliente_id = Column(Integer, ForeignKey("op_clientes.id"), nullable=True)
    cliente_nome = Column(String(255), nullable=True)
    valor_total = Column(Numeric(10, 2), default=0)
    valor_desconto = Column(Numeric(10, 2), default=0)
    valor_recebido = Column(Numeric(10, 2), default=0)
    troco = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(String(50), nullable=True)
    itens_json = Column(Text, nullable=True)
    status = Column(String(20), default="CONCLUIDA")
    tipo_operacao = Column(String(30), default="VENDA")
    origem = Column(String(50), default="PDV")
    observacao = Column(Text, nullable=True)
    data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
    data_alteracao = Column(DateTime, nullable=True)
    data_cancelamento = Column(DateTime, nullable=True)
    motivo_cancelamento = Column(Text, nullable=True)
    erro_detalhes = Column(Text, nullable=True)
    ip_origem = Column(String(50), nullable=True)
    sessao_id = Column(String(100), nullable=True)
    versao = Column(Integer, default=1)
    metadados_json = Column(Text, nullable=True)
```

models_core.py

backendPy/models_core.py • 146 linhas

```
"""
models_core.py - Modelos SQLAlchemy do banco CORE

Arquitetura:
- Cliente: pessoa/empresa que compra serviços da Corumbá Sistemas
- Assinatura: vincula um cliente aos produtos que ele adquiriu
- Produto: catálogo de serviços/produtos oferecidos pela Corumbá (Salto PDV, MesaPronta, etc.)
- ChaveAtivacao: chave única usada no download para validar assinatura
"""
import datetime
from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, Numeric, ForeignKey, Table
from sqlalchemy.orm import relationship
from database import BaseCore

# =====
# CLIENTE (quem compra da Corumbá Sistemas)
# =====
class Cliente(BaseCore):
    __tablename__ = "clientes"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    nome = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, index=True, nullable=False)
    telefone = Column(String(20))
    cpf_cnpj = Column(String(20), unique=True, index=True, nullable=False)
    senha_hash = Column(String(255), nullable=False)
    ativo = Column(Boolean, default=True)
    data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
    ultimo_acesso = Column(DateTime, nullable=True)

    # Relacionamentos
    assinaturas = relationship("Assinatura", back_populates="cliente", cascade="all, delete-orphan")
    chaves = relationship("ChaveAtivacao", back_populates="cliente", cascade="all, delete-orphan")

# =====
# PRODUTO/SERVIÇO (catálogo da Corumbá Sistemas)
# =====
class Produto(BaseCore):
    __tablename__ = "produtos_catalogo"
```

```

id = Column(Integer, primary_key=True, index=True, autoincrement=True)
nome = Column(String(255), nullable=False) # Ex: "Salto PDV", "MesaPronta"
descricao = Column(Text, nullable=True)
preco_mensal = Column(Numeric(10, 2), nullable=False, default=0)
preco_anual = Column(Numeric(10, 2), nullable=True)
ativo = Column(Boolean, default=True)
icone = Column(String(50), nullable=True) # Emoji ou classe CSS
ordem = Column(Integer, default=0) # Ordem de exibição no painel
data_criacao = Column(DateTime, default=datetime.datetime.utcnow)

# =====
# ASSINATURA (liga cliente aos produtos adquiridos)
# =====
class Assinatura(BaseCore):
    __tablename__ = "assinaturas"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
    produto_id = Column(Integer, ForeignKey("produtos_catalogo.id"), nullable=False)
    status = Column(String(20), default="ATIVA") # ATIVA, INADIMIENTE, CANCELADA, EXPIRADA
    data_aquisicao = Column(DateTime, default=datetime.datetime.utcnow)
    data_expiracao = Column(DateTime, nullable=True) # NULL = sem expiracao
    valor_mensal = Column(Numeric(10, 2), nullable=False, default=0)
    dia_vencimento = Column(Integer, nullable=True) # Dia do mês de vencimento (1-31)

    # Relacionamentos
    cliente = relationship("Cliente", back_populates="assinaturas")
    produto = relationship("Produto")

# =====
# CHAVE DE ATIVAÇÃO (usada no download do desktop)
# =====
class ChaveAtivacao(BaseCore):
    __tablename__ = "chaves_ativacao"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
    chave = Column(String(50), unique=True, index=True, nullable=False)
    produto_id = Column(Integer, ForeignKey("produtos_catalogo.id"), nullable=True) # NULL = vale para todos
    usada = Column(Boolean, default=False)
    data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
    data_uso = Column(DateTime, nullable=True)

    # Relacionamentos
    cliente = relationship("Cliente", back_populates="chaves")
    produto = relationship("Produto")

# =====
# EMPRESA (dados da empresa do cliente, preenchidos após ativação)
# =====
class Empresa(BaseCore):
    __tablename__ = "empresas"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=True)
    razao_social = Column(String(255), nullable=False)
    nome_fantasia = Column(String(255))
    documento = Column(String(20), unique=True, index=True, nullable=False) # CNPJ
    email = Column(String(255), index=True)
    telefone = Column(String(20))
    endereco = Column(String(500))
    ativo = Column(Boolean, default=True)
    data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)

# =====
# USUÁRIO (colaboradores da empresa do cliente)
# =====
class Usuario(BaseCore):
    __tablename__ = "usuarios"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    empresa_id = Column(Integer, ForeignKey("empresas.id"), nullable=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=True)
    empresa_documento = Column(String(20), index=True, nullable=False, default="00.000.000/0001-00")
    nome = Column(String(255), nullable=False)
    username = Column(String(100), index=True, nullable=False)
    email = Column(String(255), index=True, nullable=True)
    senha_hash = Column(String(255), nullable=False)
    papel = Column(String(30), default="OPERADOR")
    ativo = Column(Boolean, default=True)
    codigo = Column(String(20), nullable=True)
    data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
    ultimo_acesso = Column(DateTime, nullable=True)

# =====
# ADMINISTRADOR CORUMBÁ (vendedor/funcionário da Corumbá Sistemas)
# =====
class AdminCorumba(BaseCore):
    __tablename__ = "admins_corumba"

```

```

id = Column(Integer, primary_key=True, index=True, autoincrement=True)
nome = Column(String(255), nullable=False)
email = Column(String(255), unique=True, index=True, nullable=False)
senha_hash = Column(String(255), nullable=False)
papel = Column(String(30), default="VENDEDOR") # ADMIN, VENDEDOR, SUPORTE
ativo = Column(Boolean, default=True)
data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
ultimo_acesso = Column(DateTime, nullable=True)

```

models_historico.py

backendPy/models_historico.py • 73 linhas

```

"""
models.py - Adicionar modelo de HistoricoVendas

Registra TODA operação de venda para auditoria e rastreabilidade.
Cada venda gera múltiplos registros de histórico:
- Criação da venda
- Cada item adicionado
- Alterações de status
- Cancelamentos
- Edições
- Logs de erro
"""
# Adicionar ao final de models.py

import datetime
from sqlalchemy import Column, Integer, String, Numeric, Boolean, ForeignKey, DateTime, Text, JSON
from sqlalchemy.orm import relationship
from database import BaseSalto, TenantMixin

class HistoricoVenda(BaseSalto, TenantMixin):
    __tablename__ = "historico_vendas"

    # id e empresa_documento vem de TenantMixin
    numero_pedido = Column(String(50), index=True) # Número legível (ex: PED-20260601-0001)

    # Dados da empresa e usuário
    # empresa_documento vem de TenantMixin
    usuario_id = Column(Integer, nullable=True)
    usuario_nome = Column(String(255), nullable=True)
    usuario_papel = Column(String(30), nullable=True)

    # Dados do cliente
    cliente_id = Column(Integer, nullable=True)
    cliente_nome = Column(String(255), nullable=True)

    # Dados financeiros
    valor_total = Column(Numeric(10, 2), default=0)
    valor_desconto = Column(Numeric(10, 2), default=0)
    valor_recebido = Column(Numeric(10, 2), default=0)
    troco = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(String(50), nullable=True)

    # Itens da venda (JSON para flexibilidade)
    itens_json = Column(Text, nullable=True) # JSON com produtos, quantidades, preços

    # Status e tipo de operação
    status = Column(String(20), default="CONCLUIDA") # CONCLUIDA, CANCELADA, PENDENTE, ERRO
    tipo_operacao = Column(String(30), default="VENDA") # VENDA, CANCELAMENTO, EDICAO, TROCA, DEVOLUCAO

    # Rastreamento
    origem = Column(String(50), default="PDV") # PDV, WEB, API, MOBILE
    observacao = Column(Text, nullable=True)

    # Dados de controle
    data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
    data_alteracao = Column(DateTime, nullable=True)
    data_cancelamento = Column(DateTime, nullable=True)
    motivo_cancelamento = Column(Text, nullable=True)

    # Logs de erro (se houve falha)
    erro_detalhes = Column(Text, nullable=True)

    # IP e sessão (para auditoria)
    ip_origem = Column(String(50), nullable=True)
    sessao_id = Column(String(100), nullable=True)

    # Versão dos dados (para controle de concorrência)
    versao = Column(Integer, default=1)

    # Dados extras flexíveis
    metadados_json = Column(Text, nullable=True) # JSON com dados adicionais

```

schemas.py

backendPy/schemas.py • 344 linhas

```

"""
schemas.py - Schemas Pydantic para validacao de entrada/saida da API
"""
from pydantic import BaseModel, Field

```

```

from typing import Optional, List
from datetime import datetime

# =====
# AUTENTICACAO
# =====
class LoginRequest(BaseModel):
    username: str # Pode ser username ou codigo do colaborador
    senha: str

class LoginResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    nome: str
    username: str
    papel: str
    empresa_documento: str

class TokenData(BaseModel):
    username: str
    empresa_documento: str
    papel: str
    usuario_id: int

# =====
# USUARIOS (Colaboradores)
# =====
class UsuarioCreate(BaseModel):
    nome: str
    username: str
    senha: str
    papel: str = "OPERADOR"
    codigo: Optional[str] = None

class UsuarioUpdate(BaseModel):
    nome: Optional[str] = None
    senha: Optional[str] = None
    papel: Optional[str] = None
    codigo: Optional[str] = None
    ativo: Optional[bool] = None

class UsuarioResponse(BaseModel):
    id: int
    nome: str
    username: str
    papel: str
    codigo: Optional[str]
    ativo: bool
    empresa_documento: str

    class Config:
        from_attributes = True

# =====
# PRODUTOS
# =====
class ProdutoCreate(BaseModel):
    nome: str
    descricao: Optional[str] = None
    sku: Optional[str] = None
    codigo_barras: Optional[str] = None
    preco_venda: float = Field(ge=0)
    preco_custo: float = Field(ge=0, default=0)
    estoque_atual: float = Field(ge=0, default=0)
    estoque_minimo: float = Field(ge=0, default=0)
    unidade: str = "UN"
    ncm: Optional[str] = None
    cfop: Optional[str] = None
    cest: Optional[str] = None

class ProdutoUpdate(BaseModel):
    nome: Optional[str] = None
    descricao: Optional[str] = None
    sku: Optional[str] = None
    codigo_barras: Optional[str] = None
    preco_venda: Optional[float] = None
    preco_custo: Optional[float] = None
    estoque_minimo: Optional[float] = None
    unidade: Optional[str] = None
    ativo: Optional[bool] = None
    ncm: Optional[str] = None
    cfop: Optional[str] = None
    cest: Optional[str] = None

class ProdutoResponse(BaseModel):
    id: int

```

```

nome: str
descricao: Optional[str]
sku: Optional[str]
codigo_barras: Optional[str]
preco_venda: float
preco_custo: float
estoque_atual: float
estoque_minimo: float
unidade: str
ativo: bool
ncm: Optional[str]
cfop: Optional[str]
cest: Optional[str]

class Config:
    from_attributes = True

# =====
# ESTOQUE MOVIMENTACAO
# =====
class EstoqueMovimentacaoCreate(BaseModel):
    produto_id: int
    tipo: str # ENTRADA, SAIDA, AJUSTE
    quantidade: float
    motivo: Optional[str] = None

class EstoqueMovimentacaoResponse(BaseModel):
    id: int
    produto_id: int
    tipo: str
    quantidade: float
    motivo: Optional[str]
    usuario: Optional[str]
    data_transacao: Optional[datetime]

class Config:
    from_attributes = True

# =====
# CLIENTES
# =====
class ClienteCreate(BaseModel):
    nome: str
    telefone: Optional[str] = None
    email: Optional[str] = None
    cpf_cnpj: Optional[str] = None
    endereco: Optional[str] = None
    cidade: Optional[str] = None
    estado: Optional[str] = None
    cep: Optional[str] = None
    data_nascimento: Optional[str] = None

class ClienteUpdate(BaseModel):
    nome: Optional[str] = None
    telefone: Optional[str] = None
    email: Optional[str] = None
    cpf_cnpj: Optional[str] = None
    endereco: Optional[str] = None
    cidade: Optional[str] = None
    estado: Optional[str] = None
    cep: Optional[str] = None
    data_nascimento: Optional[str] = None
    ativo: Optional[bool] = None

class ClienteResponse(BaseModel):
    id: int
    nome: str
    telefone: Optional[str]
    email: Optional[str]
    cpf_cnpj: Optional[str]
    endereco: Optional[str]
    cidade: Optional[str]
    estado: Optional[str]
    cep: Optional[str]
    data_nascimento: Optional[str]
    pontos_fidelidade: int
    ativo: bool

class Config:
    from_attributes = True

class ClienteHistoricoResponse(BaseModel):
    compras: List[dict]
    total_gasto: float
    pontos: int

# =====
# FORNECEDORES

```

```

# =====
class FornecedorCreate(BaseModel):
    nome_fantasia: str
    razao_social: Optional[str] = None
    cnpj_cpf: Optional[str] = None
    telefone: Optional[str] = None
    email: Optional[str] = None
    endereco: Optional[str] = None

class FornecedorResponse(BaseModel):
    id: int
    nome_fantasia: str
    razao_social: Optional[str]
    cnpj_cpf: Optional[str]
    telefone: Optional[str]
    email: Optional[str]
    endereco: Optional[str]
    ativo: bool

    class Config:
        from_attributes = True

# =====
# VENDAS
# =====
class ItemVendaCreate(BaseModel):
    produto_id: int
    quantidade: float

class PagamentoVenda(BaseModel):
    forma: str
    valor: float

class DescontoVendaRequest(BaseModel):
    valor_desconto: float = Field(ge=0)
    motivo: Optional[str] = None
    senha_admin: str # Exige senha admin para desconto

class VendaCreate(BaseModel):
    cliente_id: Optional[int] = None
    vendedor_id: Optional[int] = None
    vendedor_nome: Optional[str] = None
    itens: List[ItemVendaCreate]
    pagamentos: List[PagamentoVenda]
    observacao: Optional[str] = None

class VendaCancelRequest(BaseModel):
    senha_admin: str
    motivo: Optional[str] = None

class VendaItemResponse(BaseModel):
    id: int
    produto_id: int
    produto_nome: Optional[str]
    quantidade: float
    preco_unitario: float
    subtotal: float

    class Config:
        from_attributes = True

class VendaResponse(BaseModel):
    id: int
    cliente_id: Optional[int]
    vendedor_id: Optional[int]
    vendedor_nome: Optional[str]
    operador: Optional[str]
    total: float
    desconto: float
    valor_recebido: float
    troco: float
    forma_pagamento: Optional[str]
    status: str
    observacao: Optional[str]
    data_venda: Optional[datetime]
    itens: List[VendaItemResponse]

    class Config:
        from_attributes = True

# =====
# CAIXA
# =====
class CaixaAbrirRequest(BaseModel):
    operador: str
    saldo_inicial: float = 0

```



```

class CaixaFecharRequest(BaseModel):
    operador: str
    saldo_final: float

class CaixaTransacaoRequest(BaseModel):
    tipo: str # ENTRADA, SAIDA, SANGRIA
    valor: float = Field(gt=0)
    descricao: Optional[str] = None
    senha_admin: Optional[str] = None # Exigido para SAIDA e SANGRIA

class CaixaStatusResponse(BaseModel):
    status: str
    sessao_id: Optional[int]
    operador: Optional[str]
    saldo_inicial: Optional[float]
    total_vendas: Optional[float]
    total_entradas: Optional[float]
    total_saidas: Optional[float]
    saldo_esperado: Optional[float]
    data_abertura: Optional[datetime]
    transacoes: List[dict] = []

# =====
# DASHBOARD
# =====
class DashboardResponse(BaseModel):
    vendas_hoje: float
    vendas_ontem: float
    vendas_mes: float
    quantidade_vendas_hoje: int
    ticket_medio_hoje: float
    produtos_estoque_baixo: int
    clientes_ativos: int
    vendedores_do_dia: List[dict]
    ultimas_vendas: List[dict]

# =====
# EMPRESA (validacao de CNPJ)
# =====
class EmpresaValidaRequest(BaseModel):
    documento: str # CNPJ

class EmpresaValidaResponse(BaseModel):
    valido: bool
    nome_fantasia: Optional[str] = None
    plano: Optional[str] = None
    ativo: bool

```

security.py

backendPy/security.py • 144 linhas

```

"""
security.py - Utilitários de segurança: hash de senha (PBKDF2-SHA256), JWT, dependências FastAPI
Usa hash nativo do Python (hashlib) – compatível com Python 3.13 sem dependências externas
"""
import os
import hashlib
import hmac
import secrets
from datetime import datetime, timedelta
from typing import Optional
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from sqlalchemy.orm import Session

from database import get_db, SessionLocal, APP_ENV
from models_core import Cliente, Usuario

# =====
# CONFIGURAÇÃO
# =====
SECRET_KEY = os.getenv("SECRET_KEY")
if not SECRET_KEY:
    if APP_ENV == "production":
        raise RuntimeError(
            "SECRET_KEY must be set in production! "
            "Generate one with: python -c \"import secrets; print(secrets.token_hex(32))\""
        )
    SECRET_KEY = "dev-local-key-change-in-production"
import warnings
warnings.warn("Using default SECRET_KEY – set SECRET_KEY env var in production!", stacklevel=2)
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_DAYS = 30
PBKDF2_ITERATIONS = 600_000 # OWASP 2024 recommendation

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/cliente/login")

```

```

# =====
# HASH DE SENHA (PBKDF2-SHA256 + salt)
# =====
def hash_senha(senha: str) -> str:
    """Gera hash PBKDF2-SHA256 com salt aleatório. Formato: salt_hex$hash_hex"""
    salt = secrets.token_bytes(32)
    dk = hashlib.pbkdf2_hmac("sha256", senha.encode("utf-8"), salt, PBKDF2_ITERATIONS)
    return salt.hex() + "$" + dk.hex()

def verificar_senha(senha_plana: str, senha_hash: str) -> bool:
    """Verifica senha contra hash PBKDF2. Formato esperado: salt_hex$hash_hex"""
    try:
        salt_hex, hash_hex = senha_hash.split("$", 1)
        salt = bytes.fromhex(salt_hex)
        expected = bytes.fromhex(hash_hex)
        dk = hashlib.pbkdf2_hmac("sha256", senha_plana.encode("utf-8"), salt, PBKDF2_ITERATIONS)
        return hmac.compare_digest(dk, expected)
    except (ValueError, AttributeError):
        return False

# =====
# JWT
# =====
def criar_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(days=ACCESS_TOKEN_EXPIRE_DAYS))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decodificar_token(token: str) -> dict:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token inválido ou expirado",
            headers={"WWW-Authenticate": "Bearer"},
        )

# =====
# DEPENDENCY: obter cliente atual
# =====
async def get_current_cliente(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> Cliente:
    payload = decodificar_token(token)
    email: str = payload.get("sub")
    if email is None:
        raise HTTPException(status_code=401, detail="Token inválido")

    cliente = db.query(Cliente).filter(Cliente.email == email).first()
    if cliente is None or not cliente.ativo:
        raise HTTPException(status_code=401, detail="Cliente inativo ou não encontrado")
    return cliente

# =====
# DEPENDENCY: obter usuário colaborador atual (SaltoGestão desktop)
# =====
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> Usuario:
    """Obtém o usuário colaborador (vendedor/operador) a partir do token JWT."""
    core_db = SessionLocal()
    try:
        payload = decodificar_token(token)
        username: str = payload.get("username") or payload.get("sub")
        empresa_documento: str = payload.get("empresa_documento")
        if username is None:
            raise HTTPException(status_code=401, detail="Token inválido")
        # Busca no banco core via models_core
        user = core_db.query(Usuario).filter(Usuario.username == username).first()
        if user is None or not user.ativo:
            raise HTTPException(status_code=401, detail="Usuário inativo ou não encontrado")
        return user
    finally:
        core_db.close()

async def get_empresa_from_user(
    current_user: Usuario = Depends(get_current_user),
) -> str:
    """Extrai o CNPJ da empresa do usuário autenticado."""
    return current_user.empresa_documento

```

```
def require_role(*roles):
    """Dependência que exige que o usuário tenha um dos papéis listados."""
    async def _check(
        current_user: Usuario = Depends(get_current_user),
    ) -> Usuario:
        if current_user.papel not in roles:
            raise HTTPException(
                status_code=403,
                detail=f"Papel '{current_user.papel}' não autorizado. Necessário: {'', '.join(roles)}",
            )
        return current_user
    return _check
```

historico.py

backendPy/historico.py • 184 linhas

```
"""
historico.py - Módulo de Histórico de Vendas

Registra TODA operação de venda para auditoria e rastreabilidade.
Chamado automaticamente pelo router de vendas em cada operação.
"""
import json
import datetime
from typing import Optional, List, Dict, Any
from sqlalchemy.orm import Session

from database import get_db_salto
from models import HistoricoVenda

def gerar_numero_pedido(empresa_documento: str) -> str:
    """Gera número de pedido único: PED-YYYYMMDD-XXXXX."""
    agora = datetime.datetime.utcnow()
    prefixo = f"PED-{agora.strftime('%Y%m%d')}"
    # Os últimos 5 chars do CNPJ + timestamp para unicidade
    sufixo = f"{empresa_documento[-5:]}{agora.strftime('%H%M%S%f')[:8]}"
    return f"{prefixo}-{sufixo}"

def registrar_venda(
    db: Session,
    empresa_documento: str,
    usuario_id: Optional[int],
    usuario_nome: Optional[str],
    usuario_papel: Optional[str],
    venda_id: Optional[int],
    itens: List[Dict[str, Any]],
    pagamentos: List[Dict[str, Any]],
    cliente_id: Optional[int] = None,
    cliente_nome: Optional[str] = None,
    valor_total: float = 0,
    valor_desconto: float = 0,
    valor_recebido: float = 0,
    troco: float = 0,
    forma_pagamento: str = "",
    numero_pedido: Optional[str] = None,
    origem: str = "PDV",
    observacao: Optional[str] = None,
    ip_origem: Optional[str] = None,
) -> HistoricoVenda:
    """Registra uma venda no histórico."""
    if not numero_pedido:
        numero_pedido = gerar_numero_pedido(empresa_documento)

    # Calcular totais se não informados
    if valor_total == 0 and itens:
        valor_total = sum(
            float(item.get('preco_unitario', 0)) * float(item.get('quantidade', 1))
            for item in itens
        )

    historico = HistoricoVenda(
        empresa_documento=empresa_documento,
        venda_id=venda_id,
        numero_pedido=numero_pedido,
        usuario_id=usuario_id,
        usuario_nome=usuario_nome,
        usuario_papel=usuario_papel,
        cliente_id=cliente_id,
        cliente_nome=cliente_nome,
        valor_total=valor_total,
        valor_desconto=valor_desconto,
        valor_recebido=valor_recebido,
        troco=troco,
        forma_pagamento=forma_pagamento,
        itens_json=json.dumps(itens, default=str),
        status="CONCLUIDA",
        tipo_operacao="VENDA",
        origem=origem,
        observacao=observacao,
        ip_origem=ip_origem,
        data_criacao=datetime.datetime.utcnow(),
    )
```

```

db.add(historico)
db.flush()
return historico

def registrar_cancelamento(
    db: Session,
    empresa_documento: str,
    venda_id: int,
    usuario_id: Optional[int],
    usuario_nome: Optional[str],
    motivo: Optional[str] = None,
) -> Optional[HistoricoVenda]:
    """Registra o cancelamento de uma venda no histórico."""
    # Buscar o histórico original
    original = db.query(HistoricoVenda).filter(
        HistoricoVenda.empresa_documento == empresa_documento,
        HistoricoVenda.venda_id == venda_id,
        HistoricoVenda.status == "CONCLUIDA"
    ).order_by(HistoricoVenda.data_criacao.desc()).first()

    if original:
        # Atualizar status do original
        original.status = "CANCELADA"
        original.data_cancelamento = datetime.datetime.utcnow()
        original.motivo_cancelamento = motivo

        # Registrar log de cancelamento
        cancelamento = HistoricoVenda(
            empresa_documento=empresa_documento,
            venda_id=venda_id,
            numero_pedido=original.numero_pedido,
            usuario_id=usuario_id,
            usuario_nome=usuario_nome,
            usuario_papel=original.usuario_papel,
            cliente_id=original.cliente_id,
            cliente_nome=original.cliente_nome,
            valor_total=original.valor_total,
            valor_desconto=original.valor_desconto,
            forma_pagamento=original.forma_pagamento,
            itens_json=original.itens_json,
            status="CANCELADA",
            tipo_operacao="CANCELAMENTO",
            origem=original.origem,
            observacao=f"Cancelamento: {motivo or 'Sem motivo informado'}",
            data_cancelamento=datetime.datetime.utcnow(),
            motivo_cancelamento=motivo,
        )
        db.add(cancelamento)
        db.flush()
        return cancelamento
    return None

def registrar_erro_venda(
    db: Session,
    empresa_documento: str,
    usuario_id: Optional[int],
    usuario_nome: Optional[str],
    erro: str,
    dados_tentativa: Optional[Dict] = None,
) -> HistoricoVenda:
    """Registra uma tentativa de venda que falhou."""
    historico = HistoricoVenda(
        empresa_documento=empresa_documento,
        numero_pedido=gerar_numero_pedido(empresa_documento),
        usuario_id=usuario_id,
        usuario_nome=usuario_nome,
        erro_detalhes=erro,
        status="ERRO",
        tipo_operacao="ERRO",
        metadados_json=json.dumps(dados_tentativa, default=str) if dados_tentativa else None,
        data_criacao=datetime.datetime.utcnow(),
    )
    db.add(historico)
    db.flush()
    return historico

def registrar_edicao_venda(
    db: Session,
    empresa_documento: str,
    venda_id: int,
    usuario_id: int,
    usuario_nome: str,
    campo_alterado: str,
    valor_anterior: str,
    valor_novo: str,
) -> HistoricoVenda:
    """Registra uma edição em uma venda existente."""
    historico = HistoricoVenda(
        empresa_documento=empresa_documento,
        venda_id=venda_id,
        numero_pedido=f"EDIT-{gerar_numero_pedido(empresa_documento)}",
        usuario_id=usuario_id,

```

```

        usuario_nome=usuario_nome,
        status="CONCLUIDA",
        tipo_operacao="EDICAO",
        observacao=f"Campo '{campo_alterado}' alterado de '{valor_anterior}' para '{valor_novo}'",
        data_alteracao=datetime.datetime.utcnow(),
        versao=2,
    )
    db.add(historico)
    db.flush()
    return historico

```

licenca.py

backendPy/licenca.py • 56 linhas

```

"""
licenca.py - Gerenciamento de licencas do Corumbá Sistemas

Fluxo de licenciamento:
1. Admin recebe chave de licenca (ex: CORUMBA-A1B2-C3D4-E5F6)
2. Na primeira execucao, sistema pede a chave
3. Chave validada -> licenca ATIVA -> acesso total sem senha
4. Em cada login, verifica se licenca ainda esta ativa
5. Se EXPIRADA/SUSPENSA -> pede nova chave para reativar
6. Isolamento: cada CNPJ tem sua propria licenca e seus proprios dados
"""
import hashlib
import secrets
import string
from datetime import datetime, timedelta
from typing import Optional

def gerar_chave_licenca() -> str:
    """Gera uma chave de licenca unica no formato CORUMBA-XXXX-XXXX-XXXX."""
    partes = []
    for _ in range(3):
        parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
        partes.append(parte)
    return f"CORUMBA-{'-'.join(partes)}"

def gerar_hash_chave(chave: str) -> str:
    """Gera hash da chave para armazenamento seguro (nao guarda a chave pura)."""
    return hashlib.sha256(chave.encode()).hexdigest()

def validar_formato_chave(chave: str) -> bool:
    """Valida se a chave esta no formato correto: CORUMBA-XXXX-XXXX-XXXX."""
    if not chave or not chave.startswith("CORUMBA-"):
        return False
    partes = chave.split("-")
    if len(partes) != 4:
        return False
    for parte in partes[1:]:
        if len(parte) != 4 or not all(c in string.ascii_uppercase + string.digits for c in parte):
            return False
    return True

def verificar_licenca_ativa(status: str) -> bool:
    """Verifica se a licenca permite acesso."""
    return status == "ATIVA"

def licenca_expirou(data_expiracao: Optional[datetime]) -> bool:
    """Verifica se a licenca expirou."""
    if data_expiracao is None:
        return False # Sem expiracao = valida para sempre
    return datetime.utcnow() > data_expiracao

```

3. Backend - App Principal

 main.py

backendPy/main.py • 317 linhas

```

"""
main.py - Ponto de entrada da API FastAPI
"""
# DOTENV PRIMEIRO - antes de qualquer import que use DATABASE_URL
from pathlib import Path
from dotenv import load_dotenv
env_path = Path(__file__).parent / ".env"
load_dotenv(dotenv_path=env_path)

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import HTMLResponse
import os

# Importar modelos
import models_core

# Importar routers (depois do load_dotenv!)
from routers import auth, painel, admin, admin_auth, login_operacional
from routers import clientes, produtos, vendas, caixa, dashboard, fornecedores, historico, usuarios
from routers.router_admin_clientes import router as admin_clientes_router

app = FastAPI(
    title="Corumbá Sistemas API",
    description="API REST para gestão de clientes e assinaturas",
    version="3.0.0"
)

# Arquivos estáticos (pasta prints para testes de conexão)
_PRINTS_DIR = os.path.join(os.path.dirname(__file__), "..", "prints")
os.makedirs(_PRINTS_DIR, exist_ok=True)
app.mount("/prints", StaticFiles(directory=_PRINTS_DIR, name="prints"))

# CORS
CORS_ORIGINS = os.getenv("CORS_ORIGINS", "http://localhost:3000,http://localhost:5173,http://191.252.210.235:3000,http://192.168.100.68:3000,*")
app.add_middleware(
    CORSMiddleware,
    allow_origins=CORS_ORIGINS.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Registrar routers
# Login operacional (colaboradores SaltoGestão) - registrado antes do auth de cliente
app.include_router(login_operacional.router, prefix="/api")
app.include_router(auth.router, prefix="/api")
app.include_router(painel.router, prefix="/api")
app.include_router(admin.router, prefix="/api")
app.include_router(admin_auth.router, prefix="/api")
# Routers operacionais (SaltoGestão desktop)
app.include_router(clientes.router, prefix="/api")
app.include_router(produtos.router, prefix="/api")
app.include_router(vendas.router, prefix="/api")
app.include_router(caixa.router, prefix="/api")
app.include_router(dashboard.router, prefix="/api")
app.include_router(fornecedores.router, prefix="/api")
app.include_router(historico.router, prefix="/api")
app.include_router(usuarios.router, prefix="/api")
# Admin clientes (cadastro de clientes Corumba Sistemas)
app.include_router(admin_clientes_router)

# =====
# LICENÇAS E SYNC DO MESAPRONTA (Coruba Food)
# =====
from fastapi import APIRouter as _LicRouter
from pydantic import BaseModel as _LicBase
from datetime import datetime as _LicDt, timedelta as _LicTd
import json as _LicJson

_lic_router = _LicRouter()

class _AtivacaoReq(_LicBase):
    codigo: str
    cnpj: str

class _ValidarReq(_LicBase):
    cnpj: str

# Usar engine SEPARADA para licencas (sempre PostgreSQL na VPS)
import os as _lic_os
_lic_db_url = _lic_os.getenv("DATABASE_URL", "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba")
from sqlalchemy import create_engine as _lic_create_engine, text as _txt

```

```

_lic_db_url = "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba"
print("DB URL LICENCA:", _lic_db_url)
_lic_db_engine = _lic_create_engine(_lic_db_url, pool_pre_ping=True, pool_size=5, max_overflow=10)

@_lic_router.get("/api/health")
def _lic_health():
    return {"status": "ok", "timestamp": _LicDt.now().isoformat()}

@_lic_router.post("/api/licenca/ativar")
def _lic_ativar(req: _AtivacaoReq):
    try:
        with _lic_db_engine.connect() as conn:
            r = conn.execute(_txt("SELECT id FROM licencas WHERE codigo_ativacao=:cod AND status IN ('disponivel','ativa')"), {"cod":
req.codigo}).fetchone()
            if r:
                conn.execute(_txt("UPDATE licencas SET status='ativa', empresa_documento=:cnppj, data_ativacao=:now, data_expiracao=:exp,
updated_at=:now WHERE id=:id"),
                    {"cnppj": req.cnppj, "now": _LicDt.now(), "exp": _LicDt.now() + _LicDt(days=30), "id": r[0]})
                conn.commit()
                return {"status": "ativa", "mensagem": "Licenca ativada!"}
            return {"status": "erro", "mensagem": "Codigo invalido"}
    except Exception as e:
        return {"status": "erro", "mensagem": str(e)}

@_lic_router.post("/api/licenca/validar")
def _lic_validar(req: _ValidarReq):
    try:
        with _lic_db_engine.connect() as conn:
            r = conn.execute(_txt("SELECT id, status, data_expiracao, max_usuarios FROM licencas WHERE empresa_documento=:cnppj"), {"cnppj":
req.cnppj}).fetchone()
            if not r:
                return {"valida": False, "motivo": "Licenca nao encontrada"}
            status, exp, max_u = r[1], r[2], r[3]
            if status != "ativa":
                return {"valida": False, "motivo": "Status: " + str(status)}
            if exp and exp < _LicDt.now():
                conn.execute(_txt("UPDATE licencas SET status='expirada' WHERE id=:id"), {"id": r[0]})
                conn.commit()
                return {"valida": False, "motivo": "Expirada"}
            dias = (exp - _LicDt.now()).days if exp else 0
            return {"valida": True, "dias_restantes": dias, "max_usuarios": max_u}
    except Exception as e:
        return {"valida": False, "motivo": str(e)}

@_lic_router.post("/api/sync")
def _lic_sync(dados: dict):
    try:
        cnppj = dados.get("cnppj")
        tipo = dados.get("tipo")
        if not cnppj or not tipo:
            return {"status": "erro", "mensagem": "cnppj e tipo obrigatorios"}
        with _lic_db_engine.connect() as conn:
            conn.execute(_txt("INSERT INTO backup_clientes (empresa_documento, tipo_registro, dados_json, created_at) VALUES
(:c, :t, :d, :now)"),
                {"c": cnppj, "t": tipo, "d": _LicJson.dumps(dados, default=str), "now": _LicDt.now()})
            conn.commit()
            return {"status": "ok"}
    except Exception as e:
        return {"status": "erro", "mensagem": str(e)}

@_lic_router.post("/api/licenca/gerar-codigo")
def _lic_gerar(req: dict):
    import random, string
    codigo = "-".join(["".join(random.choices(string.ascii_uppercase + string.digits, k=4)) for _ in range(3)])
    try:
        with _lic_db_engine.connect() as conn:
            plano = req.get("tipo_plano", "mensal")
            dias = 365 if plano == "anual" else 90 if plano == "trimestral" else 30
            conn.execute(_txt("INSERT INTO licencas (empresa_nome, empresa_documento, codigo_ativacao, status, data_expiracao, max_usuarios,
tipo_plano) VALUES (:n, :d, :c, 'disponivel', :exp, :m, :p)"),
                {"n": req.get("empresa_nome", ""), "d": req.get("empresa_documento", ""), "c": codigo, "exp": _LicDt.now() +
_LicDt(days=dias), "m": req.get("max_usuarios", 5), "p": plano})
            conn.commit()
            return {"codigo_ativacao": codigo, "plano": plano, "expira_em_dias": dias}
    except Exception as e:
        return {"status": "erro", "mensagem": str(e)}

# =====
# MESA PRONTA - Auth de funcionários + Sync
# =====
from fastapi import APIRouter as _MpRouter
from pydantic import BaseModel as _MpBase

_mp_router = _MpRouter()

class _MpAuthReq(_MpBase):
    username: str
    senha: str
    cnppj: str

class _MpSyncReq(_MpBase):
    cnppj: str

@_mp_router.post("/api/mesapronta/auth")
def _mp_auth(req: _MpAuthReq):

```

```

"""Autentica funcionário do MesaPronta usando credenciais do Portal do Cliente"""
try:
    with _lic_db_engine.connect() as conn:
        # Buscar empresa pelo CNPJ
        emp = conn.execute(_txt(
            "SELECT id FROM empresas_cliente WHERE documento=:d"
        ), {"d": req.cnpj}).fetchone()
        if not emp:
            return {"sucesso": False, "erro": "Empresa não encontrada"}
        empresa_id = emp[0]

        # Verificar licença ativa
        lic = conn.execute(_txt(
            "SELECT status, data_expiracao FROM licencas WHERE empresa_id=:eid"
        ), {"eid": empresa_id}).fetchone()
        if not lic or lic[0] != "ativa":
            return {"sucesso": False, "erro": "Licença inválida ou expirada"}
        if lic[1] and lic[1] < _LicDt.now():
            return {"sucesso": False, "erro": "Licença expirada"}

        # Buscar funcionário
        func = conn.execute(_txt(
            "SELECT id, nome, username, senha, papel, ativo "
            "FROM funcionarios_cliente WHERE empresa_id=:eid AND username=:u"
        ), {"eid": empresa_id, "u": req.username}).fetchone()

        if not func:
            return {"sucesso": False, "erro": "Usuário não encontrado"}
        if not func[5]: # ativo
            return {"sucesso": False, "erro": "Usuário desativado"}

        # Verificar senha (comparação direta por enquanto - em produção usar hash)
        import hashlib
        senha_hash = hashlib.sha256(req.senha.encode()).hexdigest()
        # Aceitar senha pura ou hash
        senha_ok = (req.senha == func[3]) or (senha_hash == func[3])

        if not senha_ok:
            return {"sucesso": False, "erro": "Senha incorreta"}

        return {
            "sucesso": True,
            "funcionario": {
                "id": func[0],
                "nome": func[1],
                "username": func[2],
                "papel": func[4],
                "empresa_id": empresa_id,
                "empresa_nome": req.cnpj
            }
        }
except Exception as e:
    return {"sucesso": False, "erro": str(e)}

@_mp_router.get("/api/mesapronta/funcionarios/{cnpj}")
def _mp_listar_funcionarios(cnpj: str):
    """Lista todos os funcionários de um cliente (para sync local)"""
    try:
        with _lic_db_engine.connect() as conn:
            emp = conn.execute(_txt(
                "SELECT id FROM empresas_cliente WHERE documento=:d"
            ), {"d": cnpj}).fetchone()
            if not emp:
                return {"erro": "Empresa não encontrada"}

            rows = conn.execute(_txt(
                "SELECT id, nome, username, senha, papel, ativo, created_at "
                "FROM funcionarios_cliente WHERE empresa_id=:eid ORDER BY created_at"
            ), {"eid": emp[0]}).fetchall()

            return {
                "funcionarios": [
                    {
                        "id": r[0], "nome": r[1], "username": r[2],
                        "senha": r[3], "papel": r[4], "ativo": bool(r[5]),
                        "created_at": r[6].isoformat() if r[6] else None
                    }
                    for r in rows
                ]
            }
    except Exception as e:
        return {"erro": str(e)}

app.include_router(_mp_router)
app.include_router(_lic_router)

# FIM ROUTER LICENCAS E MESAPRONTA

@app.get("/")
def root():
    return {"message": "Corumbá Sistemas API v3.0", "status": "operante"}

```



```

@app.get("/health")
def health():
    return {"status": "ok"}

# =====
# PAINEL WEB (alternativa ao desktop tkinter)
# =====
import os as _os
_WEB_DIR = _os.path.join(_os.path.dirname(_os.path.abspath(__file__)), "..", "web")
_web_index = None

def _load_web_index():
    global _web_index
    web_html_path = _os.path.join(_WEB_DIR, "index.html")
    if _os.path.exists(web_html_path):
        with open(web_html_path) as f:
            _web_index = f.read()
    return _web_index

@app.get("/painel", response_class=HTMLResponse)
def painel_web():
    """Painel web - SaltoGestão (funciona no navegador, sem tkinter)."""
    html = _load_web_index()
    if html:
        return HTMLResponse(content=html)
    return HTMLResponse(content=_PAINEL_HTML, status_code=200)

# HTML inline como fallback
from fastapi.responses import HTMLResponse as _HTMLResp
_PAINEL_HTML = """<!DOCTYPE html>
<html><head><meta charset="UTF-8"><title>SaltoGestão Web</title></head>
<body style="font-family:monospace; max-width:800px; margin:40px auto; padding:20px;">
<h1>📄 SaltoGestão - Painel Web</h1>
<p>✅ O painel web está funcionando! A API está operacional.</p>
<h3>Endpoints disponíveis:</h3>
<ul>
<li><a href="/docs">/docs</a> - Documentação Swagger</li>
<li><a href="/api/clientes">/api/clientes</a> - Clientes</li>
<li><a href="/api/produtos">/api/produtos</a> - Produtos</li>
<li><a href="/api/vendas">/api/vendas</a> - Vendas</li>
<li><a href="/api/fornecedores">/api/fornecedores</a> - Fornecedores</li>
<li><a href="/api/dashboard">/api/dashboard</a> - Dashboard</li>
</ul>
<p style="color:#666;">Para usar o painel completo, acesse via arquivo index.html na pasta web</p>
</body></html>"""

```

dev.py

backendPy/dev.py • 114 linhas

```

#!/usr/bin/env python3
"""
dev.py - Script para iniciar o SaltoGestão
Uso:
  Desenvolvimento: python dev.py
  Produção:       APP_ENV=production python dev.py
"""
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

# Detectar ambiente
APP_ENV = os.getenv("APP_ENV", "development")

# Carregar .env apropriado
env_dir = os.path.dirname(os.path.abspath(__file__))
env_files = [
    os.path.join(env_dir, f".env.{APP_ENV}"), # .env.development ou .env.production
    os.path.join(env_dir, ".env"),           # .env.local (override)
]
for env_path in env_files:
    if os.path.exists(env_path):
        print(f"📄 Carregando: {env_path}")
        with open(env_path) as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith("#") and "=" in line:
                    key, _, value = line.partition("=")
                    key = key.strip()
                    value = value.strip().strip("'").strip('"')
                    if key not in os.environ:
                        os.environ[key] = value

# Defaults para desenvolvimento
os.environ.setdefault("DATABASE_URL", "sqlite:///./salto_principal.db")
os.environ.setdefault("SECRET_KEY", "dev-local-key-change-in-production")
os.environ.setdefault("CORS_ORIGINS", "http://localhost:3000,http://localhost:5173")

# --- Instalar driver PostgreSQL automaticamente se necessário ---
_db_url = os.environ["DATABASE_URL"]
if _db_url.startswith("postgresql"):

```

```

try:
    import psycopg2 # noqa: F401
except ImportError:
    print("🔧 Instalando psycopg2-binary (driver PostgreSQL)...")
    import subprocess
    subprocess.check_call([sys.executable, "-m", "pip", "install", "psycopg2-binary", "-q"])
    print("✅ psycopg2-binary instalado!")

print("=" * 50)
print(f"🚀 Corumbá Sistemas API v3.0 - {APP_ENV.upper()}")
print("=" * 50)
print(f"DB: {os.environ['DATABASE_URL'].split(':')[0]}")
print(f"CORS: {os.environ['CORS_ORIGINS']}")
print("=" * 50)

# =====
# SEED AUTOMÁTICO (na primeira execução)
# =====
from database import SessionLocal, engine, BaseCore, BaseSalto
import models_core # Registra models do banco core (Cliente, Usuario, AdminCorumba, etc.)
import models # Registra models operacionais (Cliente, Produto, Venda, etc.)

BaseCore.metadata.create_all(bind=engine)
BaseSalto.metadata.create_all(bind=engine)
print("✅ Tabelas criadas/atualizadas (core + salto)")

_db = SessionLocal()

# 1) Seed Admin (se não existir)
from models_core import AdminCorumba
from security import hash_senha
if not _db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumbasistemas.com.br").first():
    _db.add(AdminCorumba(nome="Admin Corumbá", email="admin@corumbasistemas.com.br",
        senha_hash=hash_senha("corumba2026"), papel="admin", ativo=True))
    print("✅ Admin criado: admin@corumbasistemas.com.br / corumba2026")
if not _db.query(AdminCorumba).filter(AdminCorumba.email == "vendedor@corumbasistemas.com.br").first():
    _db.add(AdminCorumba(nome="Vendedor Corumbá", email="vendedor@corumbasistemas.com.br",
        senha_hash=hash_senha("vendedor123"), papel="vendedor", ativo=True))
    print("✅ Vendedor criado: vendedor@corumbasistemas.com.br / vendedor123")

# 2) Seed Produtos (se não existir)
from models_core import Produto
if _db.query(Produto).count() == 0:
    _db.add_all([
        Produto(nome="SaltoGestão", descricao="Sistema de gestão empresarial completo",
            preco_mensal=149.90, preco_anual=1499.90, ativo=True, ordem=1, icone="📁"),
        Produto(nome="MesaPronta", descricao="Sistema de gestão para restaurantes e lanchonetes",
            preco_mensal=249.90, preco_anual=2499.90, ativo=True, ordem=2, icone="🍔"),
        Produto(nome="ConnectFood", descricao="Integração com iFood e 99Food",
            preco_mensal=49.90, preco_anual=499.90, ativo=True, ordem=3, icone="👤"),
    ])
    print("✅ 3 produtos criados (SaltoGestão, MesaPronta, ConnectFood)")

_db.commit()
_db.close()
print("✅ Seed automático concluído")

from main import app
import uvicorn

if __name__ == "__main__":
    print(f"\n🚀 Iniciando FastAPI em http://localhost:8080")
    print(f"📖 Documentação: http://localhost:8080/docs")
    print(f"🛑 Para parar: CTRL+C\n")
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8080,
        reload=True,
        reload_dirs=[os.path.dirname(os.path.abspath(__file__))],
    )

```

env.py

backendPy/alembic/env.py • 62 linhas

```

from logging.config import fileConfig

from sqlalchemy import engine_from_config
from sqlalchemy import pool

from alembic import context

# this is the Alembic Config object, which provides
# access to the values within the .ini file in use.
config = context.config

# Interpret the config file for Python logging.
# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support

```

```

from database import BaseSalto
import models
# 🐞 Corrigido: adicionado o "t" inicial
target_metadata = BaseSalto.metadata

# other values from the config, defined by the needs of env.py,
# can be acquired:
# my_important_option = config.get_main_option("my_important_option")
# ... etc.

def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode."""
    # 🐞 Corrigido: Ele apenas procura a chave, não recebe a URL direta aqui
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()

def run_migrations_online() -> None:
    """Run migrations in 'online' mode."""
    connectable = engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:
        context.configure(
            connection=connection, target_metadata=target_metadata
        )

        with context.begin_transaction():
            context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()

```

4. Backend - Roters (API Endpoints)

 `__init__.py`

backendPy/routers/__init__.py • 2 linhas

```
# routers package
```

 `login_operacional.py`

backendPy/routers/login_operacional.py • 117 linhas

```
"""
routers/login_operacional.py - Login de colaborador (vendedor/operador) do SaltoGestão
Endpoint: POST /api/auth/login
"""
from datetime import datetime
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel

from database import get_db, SessionLocal
from models_core import Usuario, Empresa, AdminCorumba
from security import hash_senha, verificar_senha, criar_token, get_current_user

router = APIRouter(tags=["Autenticação Operacional"])

class LoginOperacionalRequest(BaseModel):
    """Login com username + senha OU email + senha."""
    username: str | None = None
    email: str | None = None
    senha: str

class LoginResponse(BaseModel):
    access_token: str = ""
    token_type: str = "bearer"
    usuario: dict

@router.post("/login")
def login_operacional(req: LoginOperacionalRequest, db: Session = Depends(get_db)):
    """
    Login de colaborador do SaltoGestão.
    Aceita username ou email.
    Também aceita login de admin Corumbá como fallback.
    """
    user = None
    user_type = None

    # 1) Tenta buscar por username no banco de usuários (COM filtro multi-tenant)
    if req.username:
        user = db.query(Usuario).filter(
            Usuario.username == req.username,
            Usuario.ativo == True
        ).first()
        if user:
            user_type = "colaborador"

    # 2) Tenta buscar por email no banco de usuários
    if not user and req.email:
        user = db.query(Usuario).filter(Usuario.email == req.email).first()
        if user:
            user_type = "colaborador"

    # 3) Fallback: tenta login de admin Corumbá
    if not user and req.email:
        user = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
        if user:
            user_type = "admin"

    if not user:
        raise HTTPException(status_code=401, detail="Usuário não encontrado.")

    # Verifica senha
    senha_ok = verificar_senha(req.senha, user.senha_hash)
    if not senha_ok:
        raise HTTPException(status_code=401, detail="Senha incorreta.")

    if not user.ativo:
        raise HTTPException(status_code=403, detail="Conta desativada.")

    # Gera token
    if user_type == "colaborador":
        token = criar_token({
            "sub": user.username,
            "username": user.username,
            "nome": user.nome,
            "papel": user.papel,
```

```

        "empresa_documento": user.empresa_documento,
        "tipo": "colaborador",
        "usuario_id": user.id,
    })
    return {
        "access_token": token,
        "token_type": "bearer",
        "usuario": {
            "id": user.id,
            "nome": user.nome,
            "username": user.username,
            "email": getattr(user, 'email', ''),
            "papel": user.papel,
            "empresa_documento": user.empresa_documento,
            "tipo": "colaborador",
        }
    }
}
else: # admin
    token = criar_token({
        "sub": user.email,
        "username": user.email,
        "nome": user.nome,
        "papel": "ADMIN",
        "empresa_documento": "00.000.000/0001-00",
        "tipo": "admin",
        "admin_id": user.id,
    })
    return {
        "access_token": token,
        "token_type": "bearer",
        "usuario": {
            "id": user.id,
            "nome": user.nome,
            "email": user.email,
            "papel": "ADMIN",
            "tipo": "admin",
        }
    }
}

```

auth.py

backendPy/routers/auth.py • 259 linhas

```

"""
routers/auth.py - Autenticação do cliente Corumbá Sistemas

FLUXO:
1. Vendedor da Corumbá cadastra o cliente (nome, email, telefone, CPF/CNPJ, senha)
2. Cliente faz login com email + senha
3. No painel, cliente vê produtos (adquiridos = ativos, não adquiridos = bloqueados)
4. Ao clicar em produto não adquirido, abre informações + botão de compra
5. Se inadimplente, produtos ficam bloqueados até inserir nova chave de ativação
6. Chave de ativação é usada UMA VEZ no download do desktop
"""
import secrets
import string
from datetime import datetime
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel

from database import get_db
from models_core import Cliente, Produto, Assinatura, ChaveAtivacao
from security import hash_senha, verificar_senha, criar_token, get_current_cliente

router = APIRouter(prefix="/auth", tags=["Autenticação"])

# ===== SCHEMAS =====

class CadastroClienteRequest(BaseModel):
    """Dados para o vendedor da Corumbá cadastrar um novo cliente."""
    nome: str
    email: str
    telefone: str
    cpf_cnpj: str
    senha: str

class LoginClienteRequest(BaseModel):
    """Login do cliente com email + senha."""
    email: str
    senha: str

class AtivarChaveRequest(BaseModel):
    """Cliente insere chave de ativação (uma única vez, no download)."""
    chave: str

class CriarChaveRequest(BaseModel):
    """Vendedor/administrador gera uma chave de ativação para o cliente."""
    cliente_id: int
    produto_id: int = None # None = vale para todos os produtos

```

```
# ===== ENDPOINTS =====

@router.post("/cliente/cadastro")
def cadastro_cliente(req: CadastroClienteRequest, db: Session = Depends(get_db)):
    """
    Cadastra um novo cliente (feito pelo vendedor da Corumbá Sistemas).
    """
    # Verificar se email já existe
    existing = db.query(Cliente).filter(Cliente.email == req.email).first()
    if existing:
        raise HTTPException(status_code=400, detail="E-mail já cadastrado.")

    # Verificar se CPF/CNPJ já existe
    existing_doc = db.query(Cliente).filter(Cliente.cpf_cnpj == req.cpf_cnpj).first()
    if existing_doc:
        raise HTTPException(status_code=400, detail="CPF/CNPJ já cadastrado.")

    cliente = Cliente(
        nome=req.nome,
        email=req.email,
        telefone=req.telefone,
        cpf_cnpj=req.cpf_cnpj,
        senha_hash=hash_senha(req.senha),
        ativo=True,
    )
    db.add(cliente)
    db.commit()
    db.refresh(cliente)

    return {
        "status": "Cliente cadastrado com sucesso",
        "cliente_id": cliente.id,
        "nome": cliente.nome,
        "email": cliente.email,
    }

@router.post("/cliente/login")
def login_cliente(req: LoginClienteRequest, db: Session = Depends(get_db)):
    """
    Login do cliente com email + senha.
    Retorna JWT token para acesso ao painel.
    """
    cliente = db.query(Cliente).filter(Cliente.email == req.email).first()
    if not cliente:
        raise HTTPException(status_code=401, detail="E-mail não encontrado.")

    if not cliente.ativo:
        raise HTTPException(status_code=403, detail="Conta desativada. Contate a Corumbá Sistemas.")

    if not verificar_senha(req.senha, cliente.senha_hash):
        raise HTTPException(status_code=401, detail="Senha incorreta.")

    # Atualizar último acesso
    cliente.ultimo_acesso = datetime.utcnow()
    db.commit()

    # Verificar status das assinaturas
    assinaturas = db.query(Assinatura).filter(
        Assinatura.cliente_id == cliente.id
    ).all()

    produtos_adquiridos = []
    tem_inadimplente = False
    for a in assinaturas:
        if a.status == "ATIVA":
            produtos_adquiridos.append(a.produto_id)
        elif a.status == "INADIMIENTE":
            tem_inadimplente = True

    token = criar_token({
        "sub": cliente.email,
        "cliente_id": cliente.id,
        "nome": cliente.nome,
    })

    return {
        "access_token": token,
        "token_type": "bearer",
        "cliente": {
            "id": cliente.id,
            "nome": cliente.nome,
            "email": cliente.email,
            "telefone": cliente.telefone,
            "cpf_cnpj": cliente.cpf_cnpj,
        },
        "produtos_adquiridos": produtos_adquiridos,
        "tem_inadimplencia": tem_inadimplente,
    }

@router.get("/cliente/me")
def get_me(cliente: Cliente = Depends(get_current_cliente)):
    """Retorna dados do cliente logado."""
```

```

return {
    "id": cliente.id,
    "nome": cliente.nome,
    "email": cliente.email,
    "telefone": cliente.telefone,
    "cpf_cnpj": cliente.cpf_cnpj,
}

@router.post("/chave/ativar")
def ativar_chave(
    req: AtivarChaveRequest,
    cliente: Cliente = Depends(get_current_cliente),
    db: Session = Depends(get_db)
):
    """
    Cliente insere chave de ativação (uma única vez, no download do desktop).
    Se a assinatura estiver inadimplente, a chave reativa o acesso.
    """
    chave = db.query(ChaveAtivacao).filter(
        ChaveAtivacao.chave == req.chave,
        ChaveAtivacao.cliente_id == cliente.id,
    ).first()

    if not chave:
        raise HTTPException(status_code=404, detail="Chave de ativação não encontrada.")

    if chave.usada:
        raise HTTPException(status_code=400, detail="Esta chave já foi utilizada.")

    # Marcar chave como usada
    chave.usada = True
    chave.data_uso = datetime.utcnow()

    # Ativar/reativar assinatura do produto
    if chave.produto_id:
        assinatura = db.query(Assinatura).filter(
            Assinatura.cliente_id == cliente.id,
            Assinatura.produto_id == chave.produto_id,
        ).first()
        if assinatura:
            assinatura.status = "ATIVA"
    else:
        # Chave genérica – ativar todas as assinaturas inadimplentes
        assinaturas = db.query(Assinatura).filter(
            Assinatura.cliente_id == cliente.id,
            Assinatura.status == "INADIMLENTE",
        ).all()
        for a in assinaturas:
            a.status = "ATIVA"

    db.commit()

    return {
        "status": "Chave ativada com sucesso!",
        "message": "Seus produtos foram liberados. Você já pode fazer o download.",
    }

@router.post("/chave/gerar")
def gerar_chave(
    req: CriarChaveRequest,
    db: Session = Depends(get_db)
):
    """
    Administrador/vendedor gera uma chave de ativação para o cliente.
    """
    cliente = db.query(Cliente).filter(Cliente.id == req.cliente_id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado.")

    # Gerar chave aleatória
    partes = []
    for _ in range(3):
        parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
        partes.append(parte)
    chave_str = f"CORUMBA-{'-'.join(partes)}"

    chave = ChaveAtivacao(
        cliente_id=req.cliente_id,
        chave=chave_str,
        produto_id=req.produto_id,
        usada=False,
    )
    db.add(chave)

    # Criar assinatura se não existir
    if req.produto_id:
        existing = db.query(Assinatura).filter(
            Assinatura.cliente_id == req.cliente_id,
            Assinatura.produto_id == req.produto_id,
        ).first()
        if not existing:
            produto = db.query(Produto).filter(Produto.id == req.produto_id).first()
            assinatura = Assinatura(

```

```

        cliente_id=req.cliente_id,
        produto_id=req.produto_id,
        status="ATIVA",
        valor_mensal=produto.preco_mensal if produto else 0,
    )
    db.add(assinatura)

db.commit()
db.refresh(chave)

return {
    "status": "Chave gerada com sucesso",
    "chave": chave_str,
    "cliente": cliente.nome,
    "produto_id": req.produto_id,
}

```



admin_auth.py

backendPy/routers/admin_auth.py • 124 linhas

```

"""
routers/admin_auth.py - Autenticação do vendedor/administrador Corumbá Sistemas
"""
import secrets
import string
from datetime import datetime
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel

from database import get_db
from models_core import AdminCorumba, Cliente, Produto, Assinatura
from security import hash_senha, verificar_senha, criar_token

router = APIRouter(prefix="/admin/auth", tags=["Autenticação Admin"])

# ===== SCHEMAS =====

class AdminLoginRequest(BaseModel):
    email: str
    senha: str

class AdminCadastroRequest(BaseModel):
    """Cadastro de novo vendedor/administrador (apenas ADMIN pode criar)."""
    nome: str
    email: str
    senha: str
    papel: str = "VENDEDOR"

# ===== ENDPOINTS =====

@router.post("/login")
def admin_login(req: AdminLoginRequest, db: Session = Depends(get_db)):
    """Login do vendedor/administrador Corumbá."""
    admin = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()

    if not admin:
        raise HTTPException(status_code=401, detail="E-mail não encontrado.")

    if not admin.ativo:
        raise HTTPException(status_code=403, detail="Conta desativada.")

    if not verificar_senha(req.senha, admin.senha_hash):
        raise HTTPException(status_code=401, detail="Senha incorreta.")

    admin.ultimo_acesso = datetime.utcnow()
    db.commit()

    token = criar_token({
        "sub": admin.email,
        "admin_id": admin.id,
        "nome": admin.nome,
        "papel": admin.papel,
        "tipo": "admin_corumba",
    })

    return {
        "access_token": token,
        "token_type": "bearer",
        "admin": {
            "id": admin.id,
            "nome": admin.nome,
            "email": admin.email,
            "papel": admin.papel,
        },
    }

@router.post("/cadastro")
def admin_cadastro(req: AdminCadastroRequest, db: Session = Depends(get_db)):
    """

```



```

Cadastro de novo administrador/vendedor.
Em produção, isso deveria ser restrito a admins já logados.
"""
existing = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
if existing:
    raise HTTPException(status_code=400, detail="E-mail já cadastrado.")

if req.papel not in ("ADMIN", "VENDEDOR", "SUPORTE"):
    raise HTTPException(status_code=400, detail="Papel inválido. Use: ADMIN, VENDEDOR, SUPORTE.")

admin = AdminCorumba(
    nome=req.nome,
    email=req.email,
    senha_hash=hash_senha(req.senha),
    papel=req.papel,
    ativo=True,
)
db.add(admin)
db.commit()
db.refresh(admin)

return {
    "status": "Administrador cadastrado com sucesso",
    "admin_id": admin.id,
    "nome": admin.nome,
    "email": admin.email,
    "papel": admin.papel,
}

@router.get("/me")
def admin_me(
    email: str = None,
    db: Session = Depends(get_db)
):
    """Retorna dados do admin logado (para verificação de token)."""
    if not email:
        raise HTTPException(status_code=401, detail="Não autenticado.")

    admin = db.query(AdminCorumba).filter(AdminCorumba.email == email).first()
    if not admin:
        raise HTTPException(status_code=404, detail="Admin não encontrado.")

    return {
        "id": admin.id,
        "nome": admin.nome,
        "email": admin.email,
        "papel": admin.papel,
    }

```

admin.py

backendPy/routers/admin.py • 343 linhas

```

"""
routers/admin.py - Área administrativa do vendedor Corumbá Sistemas
- Cadastrar clientes
- Gerar chaves de ativação
- Gerenciar assinaturas
- Ver status de inadimplência
"""
import secrets
import string
from datetime import datetime
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel

from database import get_db
from models_core import Cliente, Produto, Assinatura, ChaveAtivacao
from security import hash_senha

router = APIRouter(prefix="/admin", tags=["Administração Corumbá"])

# ===== PRODUTOS DO CATÁLOGO =====

@router.get("/produtos")
def listar_produtos_catalogo(db: Session = Depends(get_db)):
    """Lista todos os produtos do catálogo ativos."""
    produtos = db.query(Produto).filter(Produto.ativo == True).order_by(Produto.ordem).all()
    return [
        {
            "id": p.id,
            "nome": p.nome,
            "descricao": p.descricao,
            "preco_mensal": float(p.preco_mensal) if p.preco_mensal else 0,
            "preco_anual": float(p.preco_anual) if p.preco_anual else None,
            "icone": p.icone,
            "ordem": p.ordem,
        }
        for p in produtos
    ]

```

```

# ===== CLIENTES =====

class CadastrarClienteRequest(BaseModel):
    """Vendedor da Corumbá cadastra um novo cliente."""
    nome: str
    email: str
    telefone: str
    cpf_cnpj: str
    senha: str
    produto_ids: list[int] = [] # IDs dos produtos que o cliente está adquirindo agora

class CriarChaveRequest(BaseModel):
    """Gera uma ou mais chaves de ativação para um cliente."""
    cliente_id: int
    produto_ids: list[int] = [] # Lista de produtos. Vazio = chave genérica para todos
    quantidade_chaves: int = 1

class AlterarStatusAssinaturaRequest(BaseModel):
    """Altera o status de uma assinatura (ATIVA, INADIMENTE, CANCELADA)."""
    novo_status: str

# ===== ENDPOINTS =====

@router.post("/clientes/criar")
def criar_cliente(req: CadastrarClienteRequest, db: Session = Depends(get_db)):
    """
    Vendedor da Corumbá cadastra um novo cliente no sistema.
    """
    # Validações
    existing = db.query(Cliente).filter(Cliente.email == req.email).first()
    if existing:
        raise HTTPException(status_code=400, detail="E-mail já cadastrado.")

    existing_doc = db.query(Cliente).filter(Cliente.cpf_cnpj == req.cpf_cnpj).first()
    if existing_doc:
        raise HTTPException(status_code=400, detail="CPF/CNPJ já cadastrado.")

    cliente = Cliente(
        nome=req.nome,
        email=req.email,
        telefone=req.telefone,
        cpf_cnpj=req.cpf_cnpj,
        senha_hash=hash_senha(req.senha),
        ativo=True,
    )
    db.add(cliente)
    db.flush()

    # Criar assinaturas iniciais
    assinaturas_criadas = []
    for pid in req.produto_ids:
        produto = db.query(Produto).filter(Produto.id == pid, Produto.ativo == True).first()
        if produto:
            assinatura = Assinatura(
                cliente_id=cliente.id,
                produto_id=produto.id,
                status="ATIVA",
                valor_mensal=produto.preco_mensal,
            )
            db.add(assinatura)
            assinaturas_criadas.append(produto.nome)

    db.commit()
    db.refresh(cliente)

    return {
        "status": "Cliente cadastrado com sucesso",
        "cliente_id": cliente.id,
        "nome": cliente.nome,
        "email": cliente.email,
        "cpf_cnpj": cliente.cpf_cnpj,
        "produtos_ativos": assinaturas_criadas,
        "proximo_passo": "Gere chaves de ativação para o cliente fazer download do desktop",
    }

@router.get("/clientes")
def listar_clientes(
    busca: str = None,
    status: str = None, # ATIVO, INADIMENTE
    db: Session = Depends(get_db)
):
    """Lista todos os clientes cadastrados."""
    query = db.query(Cliente)

    if busca:
        query = query.filter(
            Cliente.nome.ilike(f"%{busca}%")
            | Cliente.email.ilike(f"%{busca}%")
            | Cliente.cpf_cnpj.ilike(f"%{busca}%")
        )

```

```

clientes = query.order_by(Cliente.nome).all()

resultado = []
for c in clientes:
    assinaturas = db.query(Assinatura).filter(Assinatura.cliente_id == c.id).all()
    produtos_info = []
    tem_inadimplencia = False
    for a in assinaturas:
        produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
        produtos_info.append({
            "nome": produto.nome if produto else "Desconhecido",
            "status": a.status,
            "valor": float(a.valor_mensal),
        })
        if a.status == "INADIMIENTE":
            tem_inadimplencia = True

    resultado.append({
        "id": c.id,
        "nome": c.nome,
        "email": c.email,
        "telefone": c.telefone,
        "cpf_cnpj": c.cpf_cnpj,
        "ativo": c.ativo,
        "tem_inadimplencia": tem_inadimplencia,
        "produtos": produtos_info,
        "total_produtos": len(produtos_info),
    })

# Filtrar por status se solicitado
if status == "INADIMIENTE":
    resultado = [r for r in resultado if r["tem_inadimplencia"]]
elif status == "ATIVO":
    resultado = [r for r in resultado if not r["tem_inadimplencia"]]

return resultado

@router.get("/clientes/{cliente_id}")
def detalhe_cliente(cliente_id: int, db: Session = Depends(get_db)):
    """Detalhes de um cliente específico."""
    cliente = db.query(Cliente).filter(Cliente.id == cliente_id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado.")

    assinaturas = db.query(Assinatura).filter(Assinatura.cliente_id == cliente_id).all()
    produtos_info = []
    for a in assinaturas:
        produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
        produtos_info.append({
            "produto_id": a.produto_id,
            "nome": produto.nome if produto else "Desconhecido",
            "status": a.status,
            "valor": float(a.valor_mensal),
            "data_aquisicao": str(a.data_aquisicao),
            "data_expiracao": str(a.data_expiracao) if a.data_expiracao else None,
        })

    chaves = db.query(ChaveAtivacao).filter(ChaveAtivacao.cliente_id == cliente_id).all()
    chaves_info = []
    for k in chaves:
        produto = db.query(Produto).filter(Produto.id == k.produto_id).first() if k.produto_id else None
        chaves_info.append({
            "chave": k.chave,
            "produto": produto.nome if produto else "Genérica",
            "usada": k.usada,
            "data_criacao": str(k.data_criacao),
            "data_uso": str(k.data_uso) if k.data_uso else None,
        })

    return {
        "id": cliente.id,
        "nome": cliente.nome,
        "email": cliente.email,
        "telefone": cliente.telefone,
        "cpf_cnpj": cliente.cpf_cnpj,
        "ativo": cliente.ativo,
        "data_cadastro": str(cliente.data_cadastro),
        "ultimo_acesso": str(cliente.ultimo_acesso) if cliente.ultimo_acesso else None,
        "produtos": produtos_info,
        "chaves": chaves_info,
    }

@router.post("/chaves/gerar")
def gerar_chaves(req: CriarChaveRequest, db: Session = Depends(get_db)):
    """
    Gera chaves de ativação para um cliente.
    O cliente usa a chave UMA VEZ no download do desktop.
    """
    cliente = db.query(Cliente).filter(Cliente.id == req.cliente_id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado.")

    chaves_criadas = []

```

```

for _ in range(req.quantidade_chaves):
    for pid in (req.produto_ids or [None]):
        # Gerar chave única
        partes = []
        for _ in range(3):
            parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
            partes.append(parte)
        chave_str = f"CORUMBA-{'-'.join(partes)}"

        chave = ChaveAtivacao(
            cliente_id=req.cliente_id,
            chave=chave_str,
            produto_id=pid,
            usada=False,
        )
        db.add(chave)
        chaves_criadas.append(chave_str)

    # Se tem produto específico, garantir que existe assinatura
    if pid:
        existing = db.query(Assinatura).filter(
            Assinatura.cliente_id == req.cliente_id,
            Assinatura.produto_id == pid,
        ).first()
        if not existing:
            produto = db.query(Produto).filter(Produto.id == pid).first()
            if produto:
                assinatura = Assinatura(
                    cliente_id=req.cliente_id,
                    produto_id=pid,
                    status="ATIVA",
                    valor_mensal=produto.preco_mensal,
                )
                db.add(assinatura)

db.commit()

return {
    "status": f"{len(chaves_criadas)} chave(s) gerada(s)",
    "chaves": chaves_criadas,
    "cliente": cliente.nome,
}

@router.put("/assinaturas/{assinatura_id}/status")
def alterar_statusassinatura(
    assinatura_id: int,
    req: AlterarStatusAssinaturaRequest,
    db: Session = Depends(get_db)
):
    """Altera o status de uma assinatura."""
    assinatura = db.query(Assinatura).filter(Assinatura.id == assinatura_id).first()
    if not assinatura:
        raise HTTPException(status_code=404, detail="Assinatura não encontrada.")

    if req.novo_status not in ("ATIVA", "INADIMIENTE", "CANCELADA", "EXPIRADA"):
        raise HTTPException(status_code=400, detail="Status inválido. Use: ATIVA, INADIMIENTE, CANCELADA, EXPIRADA.")

    assinatura.status = req.novo_status
    db.commit()

    produto = db.query(Produto).filter(Produto.id == assinatura.produto_id).first()
    cliente = db.query(Cliente).filter(Cliente.id == assinatura.cliente_id).first()

    return {
        "status": "Alterado com sucesso",
        "cliente": cliente.nome if cliente else "?",
        "produto": produto.nome if produto else "?",
        "novo_status": req.novo_status,
    }

@router.get("/dashboard")
def dashboard_admin(db: Session = Depends(get_db)):
    """Dashboard administrativo com métricas gerais."""
    total_clientes = db.query(Cliente).count()
    clientes_ativos = db.query(Cliente).filter(Cliente.ativo == True).count()

    total_assinaturas = db.query(Assinatura).count()
    assinaturas_ativas = db.query(Assinatura).filter(Assinatura.status == "ATIVA").count()
    assinaturas_inadimplentes = db.query(Assinatura).filter(Assinatura.status == "INADIMIENTE").count()
    assinaturas_canceladas = db.query(Assinatura).filter(Assinatura.status == "CANCELADA").count()

    total_chaves = db.query(ChaveAtivacao).count()
    chaves_usadas = db.query(ChaveAtivacao).filter(ChaveAtivacao.usada == True).count()
    chaves_pendentes = db.query(ChaveAtivacao).filter(ChaveAtivacao.usada == False).count()

    # Receita mensal estimada
    from sqlalchemy import func
    receita_mensal = db.query(func.sum(Assinatura.valor_mensal)).filter(
        Assinatura.status == "ATIVA"
    ).scalar()

    return {
        "clientes": {

```

```

        "total": total_clientes,
        "ativos": clientes_ativos,
    },
    "assinaturas": {
        "total": total_assinaturas,
        "ativos": assinaturas_ativas,
        "inadimplentes": assinaturas_inadimplentes,
        "canceladas": assinaturas_canceladas,
    },
    "chaves_ativacao": {
        "total": total_chaves,
        "usadas": chaves_usadas,
        "pendentes": chaves_pendentes,
    },
    "receita_mensal_estimada": float(receita_mensal) if receita_mensal else 0,
}

```

produtos.py

backendPy/routers/produtos.py • 198 linhas

```

"""
routers/produtos.py - CRUD de produtos + gestao de estoque
Multi-tenant: todas as operacoes filtram por empresa_documento (via JWT)
"""

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from typing import Optional

from database import get_db_salto, get_db_core
from models import Produto, EstoqueMovimentacao
from schemas import ProdutoCreate, ProdutoUpdate, ProdutoResponse, EstoqueMovimentacaoCreate
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/produtos", tags=["Produtos"])

def _get_empresa(current_user):
    return current_user.empresa_documento

@router.post("/", response_model=ProdutoResponse)
def criar_produto(
    prod: ProdutoCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = Produto(empresa_documento=empresa_documento, **prod.model_dump())
    db.add(produto)
    db.commit()
    db.refresh(produto)

    # Registrar movimentacao de estoque inicial se > 0
    if produto.estoque_atual > 0:
        mov = EstoqueMovimentacao(
            empresa_documento=empresa_documento,
            produto_id=produto.id,
            tipo="ENTRADA",
            quantidade=produto.estoque_atual,
            motivo="Cadastro inicial",
            usuario=current_user.nome
        )
        db.add(mov)
        db.commit()
    return produto

@router.get("/", response_model=list[ProdutoResponse])
def listar_produtos(
    empresa_documento: str = Depends(get_empresa_from_user),
    busca: Optional[str] = None,
    ativo: Optional[bool] = True,
    estoque_baixo: Optional[bool] = None,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    query = db.query(Produto).filter(Produto.empresa_documento == empresa_documento)
    if ativo is not None:
        query = query.filter(Produto.ativo == ativo)
    if busca:
        query = query.filter(
            Produto.nome.ilike(f"%{busca}%") |
            Produto.sku.ilike(f"%{busca}%") |
            Produto.codigo_barras.ilike(f"%{busca}%")
        )
    if estoque_baixo:
        query = query.filter(Produto.estoque_atual <= Produto.estoque_minimo)
    return query.order_by(Produto.nome).all()

@router.get("/{produto_id}", response_model=ProdutoResponse)
def get_produto(

```

```

    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")
    return produto

@router.put("/{produto_id}", response_model=ProdutoResponse)
def atualizar_produto(
    produto_id: int,
    dados: ProdutoUpdate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")

    update_data = dados.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(produto, key, value)
    db.commit()
    db.refresh(produto)
    return produto

@router.delete("/{produto_id}")
def desativar_produto(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")
    produto.ativo = False
    db.commit()
    return {"status": "Produto desativado"}

# =====
# MOVIMENTACAO DE ESTOQUE
# =====

@router.post("/movimentacao")
def movimentar_estoque(
    mov: EstoqueMovimentacaoCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    produto = db.query(Produto).filter(
        Produto.id == mov.produto_id,
        Produto.empresa_documento == empresa_documento
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto nao encontrado.")

    if mov.tipo == "SAIDA" and produto.estoque_atual < mov.quantidade:
        raise HTTPException(status_code=400, detail="Estoque insuficiente.")

    # Atualizar estoque
    if mov.tipo == "ENTRADA":
        produto.estoque_atual += mov.quantidade
    elif mov.tipo == "SAIDA":
        produto.estoque_atual -= mov.quantidade
    elif mov.tipo == "AJUSTE":
        produto.estoque_atual = mov.quantidade # Ajuste define o valor absoluto

    # Registrar movimentacao
    reg = EstoqueMovimentacao(
        empresa_documento=empresa_documento,
        produto_id=mov.produto_id,
        tipo=mov.tipo,
        quantidade=mov.quantidade,
        motivo=mov.motivo,
        usuario=current_user.nome
    )
    db.add(reg)
    db.commit()

```

```

        return {"status": "Movimentacao registrada", "estoque_atual": float(produto.estoque_atual)}

@router.get("/{produto_id}/movimentacoes")
def historico_movimentacoes(
    produto_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    return db.query(EstoqueMovimentacao).filter(
        EstoqueMovimentacao.produto_id == produto_id,
        EstoqueMovimentacao.empresa_documento == empresa_documento
    ).order_by(EstoqueMovimentacao.data_movimentacao.desc()).limit(50).all()

@router.get("/estoque/alertas")
def alertas_estoque(
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Retorna produtos com estoque abaixo do minimo."""
    return db.query(Produto).filter(
        Produto.empresa_documento == empresa_documento,
        Produto.ativo == True,
        Produto.estoque_atual <= Produto.estoque_minimo
    ).all()

```

clientes.py

backendPy/routers/clientes.py • 206 linhas

```

"""
routers/clientes.py - CRUD de clientes + historico de compras + fidelidade
Multi-tenant: todas as operacoes filtram por empresa_documento.
"""

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from typing import Optional
from datetime import datetime, timedelta

from database import get_db_salto
from models import Cliente, Venda, VendaItem
from schemas import ClienteCreate, ClienteUpdate, ClienteResponse, ClienteHistoricoResponse
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/clientes", tags=["Clientes"])

@router.post("/", response_model=ClienteResponse)
def criar_cliente(
    cliente: ClienteCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)):
    obj = Cliente(empresa_documento=empresa_documento, **cliente.model_dump())
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj

@router.get("/", response_model=list[ClienteResponse])
def listar_clientes(
    empresa_documento: str = Depends(get_empresa_from_user),
    busca: Optional[str] = None,
    ativo: Optional[bool] = True,
    ordem: str = "nome", # nome, pontos, recente
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    query = db.query(Cliente).filter(Cliente.empresa_documento == empresa_documento)
    if ativo is not None:
        query = query.filter(Cliente.ativo == ativo)
    if busca:
        query = query.filter(
            Cliente.nome.ilike(f"%{busca}%") |
            Cliente.telefone.ilike(f"%{busca}%") |
            Cliente.cpf_cnpj.ilike(f"%{busca}%") |
            Cliente.email.ilike(f"%{busca}%")
        )
    if ordem == "pontos":
        query = query.order_by(Cliente.pontos_fidelidade.desc())
    elif ordem == "recente":
        query = query.order_by(Cliente.id.desc())
    else:
        query = query.order_by(Cliente.nome)
    return query.all()

@router.get("/{cliente_id}", response_model=ClienteResponse)
def get_cliente(

```

```

    cliente_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    return obj

@router.put("/{cliente_id}", response_model=ClienteResponse)
def atualizar_cliente(
    cliente_id: int,
    dados: ClienteUpdate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    for key, value in dados.model_dump(exclude_unset=True).items():
        setattr(obj, key, value)
    db.commit()
    db.refresh(obj)
    return obj

@router.delete("/{cliente_id}")
def desativar_cliente(
    cliente_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    obj.ativo = False
    db.commit()
    return {"status": "Cliente desativado"}

# =====
# HISTORICO DE COMPRAS + FIDELIDADE
# =====

@router.get("/{cliente_id}/historico")
def historico_compras(
    cliente_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    limite: int = 30,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Retorna historico de compras do cliente com detalhes."""
    cliente = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")

    vendas = db.query(Venda).filter(
        Venda.cliente_id == cliente_id,
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA"
    ).order_by(Venda.data_venda.desc()).limit(limite).all()

    total_gasto = db.query(func.sum(Venda.total)).filter(
        Venda.cliente_id == cliente_id,
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA"
    ).scalar() or 0

    return {
        "cliente": cliente.nome,
        "total_gasto": float(total_gasto),
        "pontos_fidelidade": cliente.pontos_fidelidade,
        "compras": [
            {
                "venda_id": v.id,
                "data": str(v.data_venda),
                "total": float(v.total),
                "forma_pagamento": v.forma_pagamento,
            }
        ]
    }

```



```

        "itens": [
            {
                "produto": item.produto_nome,
                "quantidade": float(item.quantidade),
                "preco": float(item.preco_unitario),
                "subtotal": float(item.subtotal)
            }
            for item in v.itens
        ]
        for v in vendas
    ]
}

@router.post("/{cliente_id}/pontos")
def adicionar_pontos(
    cliente_id: int,
    pontos: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Adiciona pontos de fidelidade ao cliente."""
    cliente = db.query(Cliente).filter(
        Cliente.id == cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
    cliente.pontos_fidelidade += pontos
    db.commit()
    return {"pontos_totais": cliente.pontos_fidelidade}

@router.get("/fidelidade/ranking")
def ranking_fidelidade(
    empresa_documento: str = Depends(get_empresa_from_user),
    limite: int = 20,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """Top clientes por pontuacao de fidelidade."""
    return db.query(Cliente).filter(
        Cliente.empresa_documento == empresa_documento,
        Cliente.ativo == True,
        Cliente.pontos_fidelidade > 0
    ).order_by(Cliente.pontos_fidelidade.desc()).limit(limite).all()

```

vendas.py

backendPy/routers/vendas.py • 449 linhas

```

"""
routers/vendas.py - Registro de vendas (PDV) + cancelamento + descontos
Multi-tenant com filtros em todas as queries.
Fluxo PDV:
1. Cliente busca produtos por nome/sku
2. Adiciona ao carrinho
3. Escolhe vendedor (por codigo)
4. Aplica desconto (se houver, exige senha admin)
5. Registra pagamento
6. Finaliza -> gera Venda + VendaItem + atualiza estoque + pontos cliente + transacao caixa
"""

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from datetime import datetime, timedelta
from typing import Optional

from database import get_db_salto, get_db_core
from models import Venda, VendaItem, Produto, Cliente, CaixaSessao, CaixaTransacao
from schemas import VendaCreate, VendaResponse, DescontoVendaRequest, VendaCancelRequest
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
from historico import registrar_venda, registrar_cancelamento, registrar_erro_venda

router = APIRouter(prefix="/vendas", tags=["Vendas"])

@router.post("/", response_model=dict)
def registrar_venda(
    venda: VendaCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    """
    Registra uma venda completa com itens, atualiza estoque e registra no caixa.
    """
    # 1. Verificar se caixa esta aberto
    caixa_aberto = db.query(CaixaSessao).filter(
        CaixaSessao.empresa_documento == empresa_documento,
        CaixaSessao.status == "ABERTO"
    ).first()

```

```

if not caixa_aberto:
    raise HTTPException(status_code=400, detail="Caixa fechado. Abra o caixa antes de vender.")

# 2. Calcular total e validar itens
total = 0
itens_venda = []
for item in venda.itens:
    produto = db.query(Produto).filter(
        Produto.id == item.produto_id,
        Produto.empresa_documento == empresa_documento,
        Produto.ativo == True
    ).first()
    if not produto:
        raise HTTPException(status_code=404, detail=f"Produto ID {item.produto_id} nao encontrado ou inativo.")
    if produto.estoque_atual < item.quantidade:
        raise HTTPException(status_code=400, detail=f"Estoque insuficiente para {produto.nome}. Disponivel: {float(produto.estoque_atual)}")

    subtotal = float(produto.preco_venda) * float(item.quantidade)
    itens_venda.append({
        "produto": produto,
        "quantidade": item.quantidade,
        "preco_unitario": float(produto.preco_venda),
        "subtotal": subtotal
    })
    total += subtotal

# 3. Determinar forma de pagamento principal (primeira forma listada)
forma_pagamento = venda.pagamentos[0].forma if venda.pagamentos else "DESCONHECIDO"
valor_recebido = sum(p.valor for p in venda.pagamentos)
troco = max(0, valor_recebido - total)

# 4. Buscar dados do vendedor
vendedor_nome = None
if venda.vendedor_id:
    # Buscar nome do vendedor pelo id (tabela usuarios no core)
    from database import get_db_core
    # Nota: usamos o id como referencia
    vendedor_nome = venda.vendedor_nome # 0 frontend ja envia o nome

# 5. Criar a venda
nova_venda = Venda(
    empresa_documento=empresa_documento,
    cliente_id=venda.cliente_id,
    vendedor_id=venda.vendedor_id,
    vendedor_nome=vendedor_nome,
    operador=current_user.nome,
    total=total,
    desconto=0,
    valor_recebido=valor_recebido,
    troco=troco,
    forma_pagamento=forma_pagamento,
    status="CONCLUIDA",
    observacao=venda.observacao,
    data_venda=datetime.utcnow()
)
db.add(nova_venda)
db.flush() # Para obter o ID

# 6. Criar itens da venda e atualizar estoque
for iv in itens_venda:
    item_db = VendaItem(
        empresa_documento=empresa_documento,
        venda_id=nova_venda.id,
        produto_id=iv["produto"].id,
        produto_nome=iv["produto"].nome,
        quantidade=iv["quantidade"],
        preco_unitario=iv["preco_unitario"],
        subtotal=iv["subtotal"]
    )
    db.add(item_db)
    # Baixar estoque
    iv["produto"].estoque_atual -= iv["quantidade"]

# 7. Registrar transacao no caixa
transacao = CaixaTransacao(
    empresa_documento=empresa_documento,
    sessao_id=caixa_aberto.id,
    tipo="VENDA",
    valor=total,
    descricao=f"Venda #{nova_venda.id}",
    venda_id=nova_venda.id,
    operador=current_user.nome
)
db.add(transacao)

# 8. Atualizar totais da sessao do caixa
caixa_aberto.total_vendas += total

# 9. Atualizar pontos do cliente (fidelidade: 1 ponto por R$ 1 gasto)
if venda.cliente_id:
    cliente = db.query(Cliente).filter(
        Cliente.id == venda.cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()

```

```

        if cliente:
            pontos_ganhos = int(total) # 1 ponto por real
            cliente.pontos_fidelidade += pontos_ganhos

    db.commit()

# 10. Registrar no historico de vendas
try:
    itens_para_historico = [
        {
            "produto_id": iv["produto"].id,
            "produto_nome": iv["produto"].nome,
            "quantidade": float(iv["quantidade"]),
            "preco_unitario": float(iv["preco_unitario"]),
            "subtotal": float(iv["subtotal"])
        }
        for iv in itens_venda
    ]

    registrar_venda(
        db=db,
        empresa_documento=empresa_documento,
        usuario_id=current_user.id,
        usuario_nome=current_user.nome,
        usuario_papel=current_user.papel,
        venda_id=nova_venda.id,
        itens=itens_para_historico,
        pagamentos=[{"forma": p.forma, "valor": float(p.valor)} for p in venda.pagamentos],
        cliente_id=venda.cliente_id,
        cliente_nome=cliente.nome if venda.cliente_id and cliente else None,
        valor_total=total,
        valor_desconto=0,
        valor_recebido=valor_recebido,
        troco=troco,
        forma_pagamento=forma_pagamento,
        origem="PDV",
        observacao=venda.observacao,
    )
    db.commit()
except Exception as e:
    # Se falhar o historico, nao bloqueia a venda
    import logging
    logging.error(f"Erro ao registrar historico de venda: {e}")

return {
    "status": "Venda registrada com sucesso",
    "venda_id": nova_venda.id,
    "total": float(total),
    "troco": float(troco),
    "forma_pagamento": forma_pagamento
}

@router.get("/", response_model=list[dict])
def listar_vendas(
    empresa_documento: str = Depends(get_empresa_from_user),
    data_inicio: Optional[str] = None,
    data_fim: Optional[str] = None,
    vendedor_id: Optional[int] = None,
    cliente_id: Optional[int] = None,
    status: Optional[str] = None,
    limite: int = Query(50, ge=1, le=200),
    offset: int = Query(0, ge=0),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    query = db.query(Venda).filter(Venda.empresa_documento == empresa_documento)

    if data_inicio:
        try:
            di = datetime.strptime(data_inicio, "%Y-%m-%d")
            query = query.filter(Venda.data_venda >= di)
        except ValueError:
            pass

    if data_fim:
        try:
            df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
            query = query.filter(Venda.data_venda < df)
        except ValueError:
            pass

    if vendedor_id:
        query = query.filter(Venda.vendedor_id == vendedor_id)
    if cliente_id:
        query = query.filter(Venda.cliente_id == cliente_id)
    if status:
        query = query.filter(Venda.status == status)

    vendas = query.order_by(Venda.data_venda.desc()).offset(offset).limit(limite).all()
    return [
        {
            "id": v.id,
            "cliente_id": v.cliente_id,
            "vendedor_nome": v.vendedor_nome,

```

```

        "operador": v.operador,
        "total": float(v.total),
        "desconto": float(v.desconto),
        "forma_pagamento": v.forma_pagamento,
        "status": v.status,
        "data_venda": str(v.data_venda),
        "quantidade_itens": len(v.itens)
    }
    for v in vendas
]

@router.get("/{venda_id}")
def get_venda(
    venda_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    venda = db.query(Venda).filter(
        Venda.id == venda_id,
        Venda.empresa_documento == empresa_documento
    ).first()
    if not venda:
        raise HTTPException(status_code=404, detail="Venda nao encontrada.")

    return {
        "id": venda.id,
        "cliente_id": venda.cliente_id,
        "vendedor_id": venda.vendedor_id,
        "vendedor_nome": venda.vendedor_nome,
        "operador": venda.operador,
        "total": float(venda.total),
        "desconto": float(venda.desconto),
        "valor_recebido": float(venda.valor_recebido),
        "troco": float(venda.troco),
        "forma_pagamento": venda.forma_pagamento,
        "status": venda.status,
        "observacao": venda.observacao,
        "data_venda": str(venda.data_venda),
        "itens": [
            {
                "produto_id": item.produto_id,
                "produto_nome": item.produto_nome,
                "quantidade": float(item.quantidade),
                "preco_unitario": float(item.preco_unitario),
                "subtotal": float(item.subtotal)
            }
            for item in venda.itens
        ]
    ]

@router.post("/{venda_id}/cancelar")
def cancelar_venda(
    venda_id: int,
    req: VendaCancelRequest,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db_core: Session = Depends(get_db_core),
    db: Session = Depends(get_db_salto)
):
    """
    Cancela uma venda. Exige senha de admin para autorizar.
    Devolve estoque, remove pontos do cliente e registra transacao de cancelamento no caixa.
    """
    # 1. Verificar senha admin
    admin_user = db_core.query(Usuario).filter(
        Usuario.empresa_documento == empresa_documento,
        Usuario.papel.in_([ "ADMIN", "GERENTE" ]),
        Usuario.ativo == True
    ).all()

    senha_valida = False
    for u in admin_user:
        if verificar_senha(req.senha_admin, u.senha_hash):
            senha_valida = True
            break

    if not senha_valida:
        raise HTTPException(status_code=403, detail="Senha de administrador invalida.")

    # 2. Buscar venda
    venda = db.query(Venda).filter(
        Venda.id == venda_id,
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA"
    ).first()
    if not venda:
        raise HTTPException(status_code=404, detail="Venda nao encontrada ou ja cancelada.")

    # 3. Devolver estoque
    for item in venda.itens:
        produto = db.query(Produto).filter(
            Produto.id == item.produto_id,

```

```

        Produto.empresa_documento == empresa_documento
    ).first()
    if produto:
        produto.estoque_antigo = produto.estoque_atual
        produto.estoque_atual += item.quantidade

# 4. Remover pontos do cliente
if venda.cliente_id:
    cliente = db.query(Cliente).filter(
        Cliente.id == venda.cliente_id,
        Cliente.empresa_documento == empresa_documento
    ).first()
    if cliente:
        pontos_perdidos = int(venda.total)
        cliente.pontos_fidelidade = max(0, cliente.pontos_fidelidade - pontos_perdidos)

# 5. Registrar transacao de cancelamento no caixa
caixa_aberto = db.query(CaixaSessao).filter(
    CaixaSessao.empresa_documento == empresa_documento,
    CaixaSessao.status == "ABERTO"
).first()
if caixa_aberto:
    transacao = CaixaTransacao(
        empresa_documento=empresa_documento,
        sessao_id=caixa_aberto.id,
        tipo="CANCELAMENTO",
        valor=venda.total,
        descricao=f"Cancelamento Venda #{venda_id}",
        venda_id=venda_id,
        operador=current_user.nome,
        autorizado_por="VERIFICADO_SENHA_ADMIN"
    )
    db.add(transacao)
    caixa_aberto.total_vendas -= venda.total

# 6. Atualizar status
venda.status = "CANCELADA"
venda.observacao = f"[venda.observacao or ''] [CANCELADA: {req.motivo or 'Sem motivo informado'}]"

# 7. Registrar cancelamento no historico
try:
    registrar_cancelamento(
        db=db,
        empresa_documento=empresa_documento,
        venda_id=venda_id,
        usuario_id=current_user.id,
        usuario_nome=current_user.nome,
        motivo=req.motivo,
    )
except Exception as e:
    import logging
    logging.error(f"Erro ao registrar cancelamento no historico: {e}")

db.commit()

return {"status": "Venda cancelada", "venda_id": venda_id, "valor_devolvido": float(venda.total)}

@router.post("/{venda_id}/desconto")
def aplicar_desconto(
    venda_id: int,
    req: DescontoVendaRequest,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db_core: Session = Depends(get_db_core),
    db: Session = Depends(get_db_salto)
):
    """
    Aplica desconto em uma venda ja concluida. Exige senha admin.
    Registra transacao de desconto no caixa.
    """
    # 1. Verificar senha admin
    admin_user = db_core.query(Usuario).filter(
        Usuario.empresa_documento == empresa_documento,
        Usuario.papel.in_([ "ADMIN", "GERENTE" ]),
        Usuario.ativo == True
    ).all()

    senha_valida = False
    for u in admin_user:
        if verificar_senha(req.senha_admin, u.senha_hash):
            senha_valida = True
            break

    if not senha_valida:
        raise HTTPException(status_code=403, detail="Senha de administrador invalida.")

    # 2. Buscar venda
    venda = db.query(Venda).filter(
        Venda.id == venda_id,
        Venda.empresa_documento == empresa_documento,
        Venda.status == "CONCLUIDA"
    ).first()
    if not venda:
        raise HTTPException(status_code=404, detail="Venda nao encontrada.")

```

```

if req.valor_desconto >= float(venda.total):
    raise HTTPException(status_code=400, detail="Desconto nao pode ser maior ou igual ao total da venda.")

venda.desconto = req.valor_desconto
venda.total = float(venda.total) - req.valor_desconto

# Registrar no caixa
caixa_aberto = db.query(CaixaSessao).filter(
    CaixaSessao.empresa_documento == empresa_documento,
    CaixaSessao.status == "ABERTO"
).first()
if caixa_aberto:
    transacao = CaixaTransacao(
        empresa_documento=empresa_documento,
        sessao_id=caixa_aberto.id,
        tipo="DESCONTO",
        valor=req.valor_desconto,
        descricao=f"Desconto Venda #{venda_id}: {req.motivo or 'Sem motivo'}",
        venda_id=venda_id,
        operador=current_user.nome,
        autorizado_por="VERIFICADO_SENHA_ADMIN"
    )
    db.add(transacao)

db.commit()
return {"status": "Desconto aplicado", "novo_total": float(venda.total), "desconto": float(venda.desconto)}

```

 **caixa.py**

backendPy/routers/caixa.py • 264 linhas

```

"""
routers/caixa.py - Gestao de caixa: abertura, fechamento, transacoes, sangria
Regras de seguranca:
- Sangria/Saida exige senha admin
- Venda automaticamente registra transacao no caixa aberto
"""
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from datetime import datetime
from typing import Optional

from database import get_db_salto, get_db_core
from models import CaixaSessao, CaixaTransacao
from schemas import CaixaAbrirRequest, CaixaFecharRequest, CaixaTransacaoRequest
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/caixa", tags=["Caixa"])

@router.post("/abrir")
def abrir_caixa(
    dados: CaixaAbrirRequest,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    caixa_aberto = db.query(CaixaSessao).filter(
        CaixaSessao.empresa_documento == empresa_documento,
        CaixaSessao.status == "ABERTO"
    ).first()
    if caixa_aberto:
        raise HTTPException(status_code=400, detail=f"Ja existe um caixa aberto por {caixa_aberto.operador}.")

    nova_sessao = CaixaSessao(
        empresa_documento=empresa_documento,
        operador=dados.operador,
        usuario_id=current_user.id,
        saldo_inicial=dados.saldo_inicial,
        status="ABERTO",
        data_abertura=datetime.utcnow()
    )
    db.add(nova_sessao)
    db.commit()
    db.refresh(nova_sessao)

    if dados.saldo_inicial > 0:
        transacao = CaixaTransacao(
            empresa_documento=empresa_documento,
            sessao_id=nova_sessao.id,
            tipo="ENTRADA",
            valor=dados.saldo_inicial,
            descricao="Troco inicial (fundo de caixa)",
            operador=dados.operador
        )
        db.add(transacao)
        db.commit()

    return {"status": "Caixa aberto", "sessao_id": nova_sessao.id, "operador": dados.operador}

@router.post("/fechar")
def fechar_caixa(

```

```

dados: CaixaFecharRequest,
empresa_documento: str = Depends(get_empresa_from_user),
current_user: Usuario = Depends(get_current_user),
db: Session = Depends(get_db_salto)
):
    caixa_aberto = db.query(CaixaSessao).filter(
        CaixaSessao.empresa_documento == empresa_documento,
        CaixaSessao.status == "ABERTO"
    ).first()
    if not caixa_aberto:
        raise HTTPException(status_code=400, detail="Nenhum caixa aberto encontrado.")

    transacoes = db.query(CaixaTransacao).filter(
        CaixaTransacao.sessao_id == caixa_aberto.id,
        CaixaTransacao.empresa_documento == empresa_documento
    ).all()

    total_vendas = sum(float(t.valor) for t in transacoes if t.tipo == "VENDA")
    total_entradas = sum(float(t.valor) for t in transacoes if t.tipo == "ENTRADA")
    total_saidas = sum(float(t.valor) for t in transacoes if t.tipo in ("SAIDA", "SANGRIA"))
    saldo_esperado = float(caixa_aberto.saldo_inicial) + total_entradas + total_vendas - total_saidas
    diferenca = float(dados.saldo_final) - saldo_esperado

    caixa_aberto.status = "FECHADO"
    caixa_aberto.saldo_final = dados.saldo_final
    caixa_aberto.total_vendas = total_vendas
    caixa_aberto.total_entradas = total_entradas
    caixa_aberto.total_saidas = total_saidas
    caixa_aberto.data_fechamento = datetime.utcnow()
    db.commit()

    resultado = {
        "status": "Caixa fechado",
        "sessao_id": caixa_aberto.id,
        "operador": caixa_aberto.operador,
        "saldo_inicial": float(caixa_aberto.saldo_inicial),
        "total_vendas": total_vendas,
        "total_entradas": total_entradas,
        "total_saidas": total_saidas,
        "saldo_esperado": saldo_esperado,
        "saldo_informado": float(dados.saldo_final),
        "diferenca": diferenca
    }
    if abs(diferenca) > 0.01:
        resultado["alerta"] = f"DIFERENCA DE R$ {diferenca:.2f} {'SOBRA' if diferenca > 0 else 'FALTA'}"
    return resultado

@router.get("/status")
def status_caixa(
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    caixa_aberto = db.query(CaixaSessao).filter(
        CaixaSessao.empresa_documento == empresa_documento,
        CaixaSessao.status == "ABERTO"
    ).first()

    if not caixa_aberto:
        return {"status": "FECHADO"}

    transacoes = db.query(CaixaTransacao).filter(
        CaixaTransacao.sessao_id == caixa_aberto.id,
        CaixaTransacao.empresa_documento == empresa_documento
    ).order_by(CaixaTransacao.data_transacao.desc()).all()

    total_vendas = sum(float(t.valor) for t in transacoes if t.tipo == "VENDA")
    total_entradas = sum(float(t.valor) for t in transacoes if t.tipo == "ENTRADA")
    total_saidas = sum(float(t.valor) for t in transacoes if t.tipo in ("SAIDA", "SANGRIA"))
    saldo_esperado = float(caixa_aberto.saldo_inicial) + total_entradas + total_vendas - total_saidas

    return {
        "status": "ABERTO",
        "sessao_id": caixa_aberto.id,
        "operador": caixa_aberto.operador,
        "saldo_inicial": float(caixa_aberto.saldo_inicial),
        "total_vendas": total_vendas,
        "total_entradas": total_entradas,
        "total_saidas": total_saidas,
        "saldo_esperado": saldo_esperado,
        "data_abertura": str(caixa_aberto.data_abertura),
        "transacoes": [
            {
                "id": t.id,
                "tipo": t.tipo,
                "valor": float(t.valor),
                "descricao": t.descricao,
                "operador": t.operador,
                "data_transacao": str(t.data_transacao)
            }
            for t in transacoes
        ]
    }

```

```

@router.post("/transacao")
def registrar_transacao(
    dados: CaixaTransacaoRequest,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db_core: Session = Depends(get_db_core),
    db: Session = Depends(get_db_salto)
):
    """
    Registra entrada, saída ou sangria no caixa.
    SAIDA e SANGRIA exigem senha de admin.
    """
    caixa_aberto = db.query(CaixaSessao).filter(
        CaixaSessao.empresa_documento == empresa_documento,
        CaixaSessao.status == "ABERTO"
    ).first()
    if not caixa_aberto:
        raise HTTPException(status_code=400, detail="Caixa fechado.")

    # Verificar senha admin para saída/sangria
    if dados.tipo in ("SAIDA", "SANGRIA"):
        if not dados.senha_admin:
            raise HTTPException(status_code=403, detail=f"Senha de administrador obrigatoria para {dados.tipo}.")

        admin_user = db_core.query(Usuario).filter(
            Usuario.empresa_documento == empresa_documento,
            Usuario.papel.in_([ "ADMIN", "GERENTE" ]),
            Usuario.ativo == True
        ).all()

        senha_valida = False
        for u in admin_user:
            if verificar_senha(dados.senha_admin, u.senha_hash):
                senha_valida = True
                break

        if not senha_valida:
            raise HTTPException(status_code=403, detail="Senha de administrador invalida.")

    transacao = CaixaTransacao(
        empresa_documento=empresa_documento,
        sessao_id=caixa_aberto.id,
        tipo=dados.tipo,
        valor=dados.valor,
        descricao=dados.descricao or f"{dados.tipo} - {current_user.nome}",
        operador=current_user.nome,
        autorizado_por="SENHA_ADMIN" if dados.tipo in ("SAIDA", "SANGRIA") else None,
        data_transacao=datetime.utcnow()
    )
    db.add(transacao)

    if dados.tipo == "ENTRADA":
        caixa_aberto.total_entradas += dados.valor
    elif dados.tipo in ("SAIDA", "SANGRIA"):
        caixa_aberto.total_saídas += dados.valor

    db.commit()
    return {"status": f"{dados.tipo} registrada", "valor": dados.valor}

@router.get("/historico")
def historico_caixa(
    empresa_documento: str = Depends(get_empresa_from_user),
    data_inicio: Optional[str] = None,
    data_fim: Optional[str] = None,
    limite: int = 20,
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_salto)
):
    """Historico de sessoes de caixa fechadas."""
    query = db.query(CaixaSessao).filter(
        CaixaSessao.empresa_documento == empresa_documento,
        CaixaSessao.status == "FECHADO"
    )

    if data_inicio:
        try:
            di = datetime.strptime(data_inicio, "%Y-%m-%d")
            query = query.filter(CaixaSessao.data_abertura >= di)
        except ValueError:
            pass

    if data_fim:
        try:
            df = datetime.strptime(data_fim, "%Y-%m-%d")
            query = query.filter(CaixaSessao.data_abertura <= df)
        except ValueError:
            pass

    sessoes = query.order_by(CaixaSessao.data_abertura.desc()).limit(limite).all()
    return [
        {
            "sessao_id": s.id,
            "operador": s.operador,

```



```

        "saldo_inicial": float(s.saldo_inicial),
        "saldo_final": float(s.saldo_final) if s.saldo_final else None,
        "total_vendas": float(s.total_vendas),
        "total_entradas": float(s.total_entradas),
        "total_saidas": float(s.total_saidas),
        "data_abertura": str(s.data_abertura),
        "data_fechamento": str(s.data_fechamento) if s.data_fechamento else None
    }
    for s in sessoes
]

```

usuarios.py

backendPy/routers/usuarios.py • 133 linhas

```

"""
routers/usuarios.py - CRUD de colaboradores/equipe
Permissões: ADMIN pode criar/editar/remover qualquer usuario.
            GERENTE pode criar apenas VENDEDOR e CAIXA.
"""
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from typing import Optional

from database import get_db_core
from models_core import Usuario
from schemas import UsuarioCreate, UsuarioUpdate, UsuarioResponse
from security import hash_senha, require_role

router = APIRouter(prefix="/usuarios", tags=["Colaboradores"])

@router.post("/", response_model=UsuarioResponse)
def criar_usuario(
    usuario: UsuarioCreate,
    empresa_documento: str,
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_core)
):
    if current_user.empresa_documento != empresa_documento:
        raise HTTPException(status_code=403, detail="So pode criar usuarios da sua empresa.")

    # Verificar se username ja existe na empresa
    existing = db.query(Usuario).filter(
        Usuario.username == usuario.username,
        Usuario.empresa_documento == empresa_documento
    ).first()
    if existing:
        raise HTTPException(status_code=400, detail="Username ja existe nesta empresa.")

    # GERENTE so pode criar VENDEDOR e CAIXA
    if current_user.papel == "GERENTE" and usuario.papel in ("ADMIN", "GERENTE"):
        raise HTTPException(status_code=403, detail="Gerente so pode criar vendedores e operadores de caixa.")

    user = Usuario(
        empresa_documento=empresa_documento,
        nome=usuario.nome,
        username=usuario.username,
        senha_hash=hash_senha(usuario.senha),
        papel=usuario.papel,
        codigo=usuario.codigo,
        ativo=True
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

@router.get("/", response_model=list[UsuarioResponse])
def listar_usuarios(
    empresa_documento: str,
    ativo: Optional[bool] = None,
    papel: Optional[str] = None,
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_core)
):
    query = db.query(Usuario).filter(Usuario.empresa_documento == empresa_documento)
    if ativo is not None:
        query = query.filter(Usuario.ativo == ativo)
    if papel:
        query = query.filter(Usuario.papel == papel)
    return query.order_by(Usuario.nome).all()

@router.get("/{usuario_id}", response_model=UsuarioResponse)
def get_usuario(
    usuario_id: int,
    empresa_documento: str,
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_core)
):
    user = db.query(Usuario).filter(
        Usuario.id == usuario_id,
        Usuario.empresa_documento == empresa_documento
    ).first()
    if not user:
        raise HTTPException(status_code=404, detail="Usuario não encontrado")
    return user

```

```

).first()
if not user:
    raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
return user

@router.put("/{usuario_id}", response_model=UsuarioResponse)
def atualizar_usuario(
    usuario_id: int,
    dados: UsuarioUpdate,
    empresa_documento: str,
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
    db: Session = Depends(get_db_core)
):
    user = db.query(Usuario).filter(
        Usuario.id == usuario_id,
        Usuario.empresa_documento == empresa_documento
    ).first()
    if not user:
        raise HTTPException(status_code=404, detail="Usuario nao encontrado.")

    update_data = dados.model_dump(exclude_unset=True)
    if "senha" in update_data:
        update_data["senha_hash"] = hash_senha(update_data.pop("senha"))

    for key, value in update_data.items():
        setattr(user, key, value)

    db.commit()
    db.refresh(user)
    return user

@router.delete("/{usuario_id}")
def desativar_usuario(
    usuario_id: int,
    empresa_documento: str,
    current_user: Usuario = Depends(require_role("ADMIN")),
    db: Session = Depends(get_db_core)
):
    user = db.query(Usuario).filter(
        Usuario.id == usuario_id,
        Usuario.empresa_documento == empresa_documento
    ).first()
    if not user:
        raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
    if user.id == current_user.id:
        raise HTTPException(status_code=400, detail="Voce nao pode desativar a si mesmo.")

    user.ativo = False
    db.commit()
    return {"status": "Usuario desativado"}

```

fornecedores.py

backendPy/routers/fornecedores.py • 104 linhas

```

"""
routers/fornecedores.py - CRUD de fornecedores
"""
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional

from database import get_db_salto, get_db_core
from models import Fornecedor
from schemas import FornecedorCreate, FornecedorResponse
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/fornecedores", tags=["Fornecedores"])

@router.post("/", response_model=FornecedorResponse)
def criar_fornecedor(
    fornecedor: FornecedorCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = Fornecedor(empresa_documento=empresa_documento, **fornecedor.model_dump())
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj

@router.get("/", response_model=list[FornecedorResponse])
def listar_fornecedores(
    empresa_documento: str = Depends(get_empresa_from_user),
    busca: Optional[str] = None,
    ativo: Optional[bool] = True,
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):

```

```

query = db.query(Fornecedor).filter(Fornecedor.empresa_documento == empresa_documento)
if ativo is not None:
    query = query.filter(Fornecedor.ativo == ativo)
if busca:
    query = query.filter(
        Fornecedor.nome_fantasia.ilike(f"%{busca}%") |
        Fornecedor.razao_social.ilike(f"%{busca}%") |
        Fornecedor.cnpj_cpf.ilike(f"%{busca}%")
    )
return query.order_by(Fornecedor.nome_fantasia).all()

@router.get("/{fornecedor_id}", response_model=FornecedorResponse)
def get_fornecedor(
    fornecedor_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Fornecedor).filter(
        Fornecedor.id == fornecedor_id,
        Fornecedor.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
    return obj

@router.put("/{fornecedor_id}", response_model=FornecedorResponse)
def atualizar_fornecedor(
    fornecedor_id: int,
    dados: FornecedorCreate,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Fornecedor).filter(
        Fornecedor.id == fornecedor_id,
        Fornecedor.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
    for key, value in dados.model_dump(exclude_unset=True).items():
        setattr(obj, key, value)
    db.commit()
    db.refresh(obj)
    return obj

@router.delete("/{fornecedor_id}")
def desativar_fornecedor(
    fornecedor_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    obj = db.query(Fornecedor).filter(
        Fornecedor.id == fornecedor_id,
        Fornecedor.empresa_documento == empresa_documento
    ).first()
    if not obj:
        raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
    obj.ativo = False
    db.commit()
    return {"status": "Fornecedor desativado"}

```

dashboard.py

backendPy/routers/dashboard.py • 113 linhas

```

"""
routers/dashboard.py - Metricas e informacoes consolidadas
"""
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func
from datetime import datetime, timedelta

from database import get_db_salto, get_db_core
from models import Venda, Produto, Cliente, EstoqueMovimentacao
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha

router = APIRouter(prefix="/dashboard", tags=["Dashboard"])

@router.get("/")
def get_dashboard(
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto)
):
    hoje = datetime.utcnow().date()
    ontem = hoje - timedelta(days=1)
    inicio_mes = hoje.replace(day=1)

```

```

def _date_str(d):
    return d.isoformat()

# ---- Vendas hoje ----
vendas_hoje_q = db.query(func.sum(Venda.total), func.count(Venda.id), func.avg(Venda.total)).filter(
    Venda.empresa_documento == empresa_documento,
    Venda.status == "CONCLUIDA",
    func.date(Venda.data_venda) == hoje
).first()
vendas_hoje = float(vendas_hoje_q[0] or 0)
qtd_vendas_hoje = int(vendas_hoje_q[1] or 0)
ticket_medio_hoje = float(vendas_hoje_q[2] or 0)

# ---- Vendas ontem ----
vendas_ontem_q = db.query(func.sum(Venda.total)).filter(
    Venda.empresa_documento == empresa_documento,
    Venda.status == "CONCLUIDA",
    func.date(Venda.data_venda) == ontem
).first()
vendas_ontem = float(vendas_ontem_q[0] or 0)

# ---- Vendas mes ----
vendas_mes_q = db.query(func.sum(Venda.total)).filter(
    Venda.empresa_documento == empresa_documento,
    Venda.status == "CONCLUIDA",
    func.date(Venda.data_venda) >= inicio_mes
).first()
vendas_mes = float(vendas_mes_q[0] or 0)

# ---- Estoque baixo ----
produtos_estoque_baixo = db.query(func.count(Produto.id)).filter(
    Produto.empresa_documento == empresa_documento,
    Produto.ativo == True,
    Produto.estoque_atual <= Produto.estoque_minimo
).scalar() or 0

# ---- Clientes ativos ----
clientes_ativos = db.query(func.count(Cliente.id)).filter(
    Cliente.empresa_documento == empresa_documento,
    Cliente.ativo == True
).scalar() or 0

# ---- Vendedores do dia (who made sales today) ----
vendedores_dia = db.query(
    Venda.vendedor_nome,
    func.count(Venda.id).label("qtd_vendas"),
    func.sum(Venda.total).label("total_vendido")
).filter(
    Venda.empresa_documento == empresa_documento,
    Venda.status == "CONCLUIDA",
    func.date(Venda.data_venda) == hoje
).group_by(Venda.vendedor_nome).order_by(func.sum(Venda.total).desc()).all()

# ---- Ultimas vendas ----
ultimas = db.query(Venda).filter(
    Venda.empresa_documento == empresa_documento,
    Venda.status == "CONCLUIDA"
).order_by(Venda.data_venda.desc()).limit(10).all()

return {
    "vendas_hoje": vendas_hoje,
    "vendas_ontem": vendas_ontem,
    "vendas_mes": vendas_mes,
    "quantidade_vendas_hoje": qtd_vendas_hoje,
    "ticket_medio_hoje": round(ticket_medio_hoje, 2),
    "produtos_estoque_baixo": produtos_estoque_baixo,
    "clientes_ativos": clientes_ativos,
    "vendedores_do_dia": [
        {
            "nome": v[0] or "Sem vendedor",
            "qtd_vendas": int(v[1]),
            "total_vendido": float(v[2])
        }
        for v in vendedores_dia
    ],
    "ultimas_vendas": [
        {
            "id": v.id,
            "total": float(v.total),
            "vendedor": v.vendedor_nome,
            "cliente_id": v.cliente_id,
            "data": str(v.data_venda)
        }
        for v in ultimas
    ]
}

```

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from database import get_db
from models_core import Cliente, Produto, Assinatura
from security import get_current_cliente

router = APIRouter(prefix="/painel", tags=["Painel do Cliente"])

@router.get("/produtos")
def listar_produtos(
    cliente: Cliente = Depends(get_current_cliente),
    db: Session = Depends(get_db)
):
    """
    Lista todos os produtos do catálogo com o status do cliente:
    - adquirido: produto que o cliente já tem ativo
    - bloqueado: produto que o cliente não adquiriu ou está inadimplente
    """
    produtos = db.query(Produto).filter(Produto.ativo == True).order_by(Produto.ordem).all()

    # Buscar assinaturas do cliente
    assinaturas = db.query(Assinatura).filter(
        Assinatura.cliente_id == cliente.id
    ).all()

    adquiridos = {a.produto_id: a for a in assinaturas if a.status == "ATIVA"}
    inadimplentes = {a.produto_id: a for a in assinaturas if a.status == "INADIMIENTE"}

    resultado = []
    for p in produtos:
        if p.id in adquiridos:
            status = "adquirido"
            assinatura = adquiridos[p.id]
        elif p.id in inadimplentes:
            status = "inadimplente"
            assinatura = inadimplentes[p.id]
        else:
            status = "bloqueado"
            assinatura = None

        resultado.append({
            "id": p.id,
            "nome": p.nome,
            "descricao": p.descricao,
            "preco_mensal": float(p.preco_mensal) if p.preco_mensal else 0,
            "preco_anual": float(p.preco_anual) if p.preco_anual else None,
            "icone": p.icone,
            "status": status,
            "data_aquisicao": str(assinatura.data_aquisicao) if assinatura else None,
            "data_expiracao": str(assinatura.data_expiracao) if assinatura and assinatura.data_expiracao else None,
        })

    return resultado

@router.get("/produto/{produto_id}")
def detalhe_produto(
    produto_id: int,
    cliente: Cliente = Depends(get_current_cliente),
    db: Session = Depends(get_db)
):
    """
    Detalhes de um produto específico.
    Se o cliente não adquiriu, mostra info + botão de compra.
    """
    produto = db.query(Produto).filter(Produto.id == produto_id, Produto.ativo == True).first()
    if not produto:
        raise HTTPException(status_code=404, detail="Produto não encontrado.")

    # Verificar se cliente tem assinatura
    assinatura = db.query(Assinatura).filter(
        Assinatura.cliente_id == cliente.id,
        Assinatura.produto_id == produto_id,
    ).first()

    if assinatura and assinatura.status == "ATIVA":
        status = "adquirido"
    elif assinatura and assinatura.status == "INADIMIENTE":
        status = "inadimplente"
    else:
        status = "bloqueado"

    return {
        "id": produto.id,
        "nome": produto.nome,
        "descricao": produto.descricao,
        "preco_mensal": float(produto.preco_mensal) if produto.preco_mensal else 0,
        "preco_anual": float(produto.preco_anual) if produto.preco_anual else None,
        "icone": produto.icone,
        "status": status,
        "data_aquisicao": str(assinatura.data_aquisicao) if assinatura else None,
        "data_expiracao": str(assinatura.data_expiracao) if assinatura and assinatura.data_expiracao else None,
    }

```

```

@router.get("/minhas-assinaturas")
def minhasassinaturas(
    cliente: Cliente = Depends(get_current_cliente),
    db: Session = Depends(get_db)
):
    """Lista todas as assinaturas do cliente."""
    assinaturas = db.query(Assinatura).filter(
        Assinatura.cliente_id == cliente.id
    ).all()

    resultado = []
    for a in assinaturas:
        produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
        resultado.append({
            "id": a.id,
            "produto_id": a.produto_id,
            "produto_nome": produto.nome if produto else "Desconhecido",
            "status": a.status,
            "valor_mensal": float(a.valor_mensal) if a.valor_mensal else 0,
            "data_aquisicao": str(a.data_aquisicao),
            "data_expiracao": str(a.data_expiracao) if a.data_expiracao else None,
        })

    return resultado

```

historico.py

backendPy/routers/historico.py • 199 linhas

```

"""
routers/historico.py - API de Histórico de Vendas

Endpoints para consultar o historico completo de vendas por empresa.
"""
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from sqlalchemy import func, and_, or_
from datetime import datetime, timedelta
from typing import Optional

from database import get_db_salto, get_db_core
from models import HistoricoVenda, Venda
from models_core import Usuario
from security import get_empresa_from_user, get_current_user, require_role

router = APIRouter(prefix="/historico", tags=["Historico de Vendas"])

@router.get("/vendas")
def listar_historico_vendas(
    empresa_documento: str = Depends(get_empresa_from_user),
    data_inicio: Optional[str] = None,
    data_fim: Optional[str] = None,
    usuario_id: Optional[int] = None,
    cliente_id: Optional[int] = None,
    status: Optional[str] = None,
    tipo_operacao: Optional[str] = None,
    busca: Optional[str] = None,
    limite: int = Query(50, ge=1, le=200),
    offset: int = Query(0, ge=0),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto),
):
    """Lista o historico de vendas da empresa com filtros."""
    query = db.query(HistoricoVenda).filter(
        HistoricoVenda.empresa_documento == empresa_documento
    )

    if data_inicio:
        try:
            di = datetime.strptime(data_inicio, "%Y-%m-%d")
            query = query.filter(HistoricoVenda.data_criacao >= di)
        except ValueError:
            pass

    if data_fim:
        try:
            df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
            query = query.filter(HistoricoVenda.data_criacao < df)
        except ValueError:
            pass

    if usuario_id:
        query = query.filter(HistoricoVenda.usuario_id == usuario_id)
    if cliente_id:
        query = query.filter(HistoricoVenda.cliente_id == cliente_id)
    if status:
        query = query.filter(HistoricoVenda.status == status)
    if tipo_operacao:
        query = query.filter(HistoricoVenda.tipo_operacao == tipo_operacao)
    if busca:
        query = query.filter(
            or_(

```

```

        HistoricoVenda.numero_pedido.ilike(f"%{busca}%"),
        HistoricoVenda.cliente_nome.ilike(f"%{busca}%"),
        HistoricoVenda.usuario_nome.ilike(f"%{busca}%"),
    )
)

total = query.count()
registros = query.order_by(HistoricoVenda.data_criacao.desc()).offset(offset).limit(limite).all()

return {
    "total": total,
    "offset": offset,
    "limite": limite,
    "registros": [
        {
            "id": r.id,
            "venda_id": r.venda_id,
            "numero_pedido": r.numero_pedido,
            "usuario_nome": r.usuario_nome,
            "usuario_papel": r.usuario_papel,
            "cliente_nome": r.cliente_nome,
            "valor_total": float(r.valor_total) if r.valor_total else 0,
            "valor_desconto": float(r.valor_desconto) if r.valor_desconto else 0,
            "forma_pagamento": r.forma_pagamento,
            "status": r.status,
            "tipo_operacao": r.tipo_operacao,
            "origem": r.origem,
            "data_criacao": str(r.data_criacao),
            "data_cancelamento": str(r.data_cancelamento) if r.data_cancelamento else None,
            "motivo_cancelamento": r.motivo_cancelamento,
            "observacao": r.observacao,
        }
        for r in registros
    ]
}

@router.get("/vendas/{historico_id}")
def detalhe_historico(
    historico_id: int,
    empresa_documento: str = Depends(get_empresa_from_user),
    current_user: Usuario = Depends(get_current_user),
    db: Session = Depends(get_db_salto),
):
    """Retorna os detalhes completos de um registro de historico."""
    registro = db.query(HistoricoVenda).filter(
        HistoricoVenda.id == historico_id,
        HistoricoVenda.empresa_documento == empresa_documento
    ).first()

    if not registro:
        raise HTTPException(status_code=404, detail="Registro de historico nao encontrado.")

    import json
    return {
        "id": registro.id,
        "venda_id": registro.venda_id,
        "numero_pedido": registro.numero_pedido,
        "empresa_documento": registro.empresa_documento,
        "usuario": {
            "id": registro.usuario_id,
            "nome": registro.usuario_nome,
            "papel": registro.usuario_papel,
        },
        "cliente": {
            "id": registro.cliente_id,
            "nome": registro.cliente_nome,
        },
        "financeiro": {
            "valor_total": float(registro.valor_total) if registro.valor_total else 0,
            "valor_desconto": float(registro.valor_desconto) if registro.valor_desconto else 0,
            "valor_recebido": float(registro.valor_recebido) if registro.valor_recebido else 0,
            "troco": float(registro.troco) if registro.troco else 0,
            "forma_pagamento": registro.forma_pagamento,
        },
        "itens": json.loads(registro.itens_json) if registro.itens_json else [],
        "status": registro.status,
        "tipo_operacao": registro.tipo_operacao,
        "origem": registro.origem,
        "observacao": registro.observacao,
        "data_criacao": str(registro.data_criacao),
        "data_alteracao": str(registro.data_alteracao) if registro.data_alteracao else None,
        "data_cancelamento": str(registro.data_cancelamento) if registro.data_cancelamento else None,
        "motivo_cancelamento": registro.motivo_cancelamento,
        "erro_detalhes": registro.erro_detalhes,
        "versao": registro.versao,
    }

@router.get("/estatisticas")
def estatisticas_historico(
    empresa_documento: str = Depends(get_empresa_from_user),
    data_inicio: Optional[str] = None,
    data_fim: Optional[str] = None,
    current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),

```

```

db: Session = Depends(get_db_salto),
):
    """Retorna estatísticas consolidadas do historico de vendas."""
    query = db.query(HistoricoVenda).filter(
        HistoricoVenda.empresa_documento == empresa_documento,
        HistoricoVenda.tipo_operacao == "VENDA"
    )

    if data_inicio:
        try:
            di = datetime.strptime(data_inicio, "%Y-%m-%d")
            query = query.filter(HistoricoVenda.data_criacao >= di)
        except ValueError:
            pass

    if data_fim:
        try:
            df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
            query = query.filter(HistoricoVenda.data_criacao < df)
        except ValueError:
            pass

    total_vendas = query.filter(HistoricoVenda.status == "CONCLUIDA").count()
    total_canceladas = query.filter(HistoricoVenda.status == "CANCELADA").count()

    faturamento = db.query(func.sum(HistoricoVenda.valor_total)).filter(
        HistoricoVenda.empresa_documento == empresa_documento,
        HistoricoVenda.status == "CONCLUIDA"
    ).scalar() or 0

    ticket_medio = faturamento / total_vendas if total_vendas > 0 else 0

    return {
        "total_vendas": total_vendas,
        "total_canceladas": total_canceladas,
        "faturamento_total": float(faturamento),
        "ticket_medio": round(float(ticket_medio), 2),
        "taxa_cancelamento": round(total_canceladas / (total_vendas + total_canceladas) * 100, 2) if (total_vendas + total_canceladas) > 0
    }
else 0,
}

```



router_admin_clientes.py

backendPy/routers/router_admin_clientes.py • 348 linhas

```

"""
router_admin_clientes.py - Cadastro e gestao de clientes (Corumba Sistemas Admin)
"""
from fastapi import APIRouter, Request
from fastapi.responses import HTMLResponse
from pydantic import BaseModel
from typing import Optional
from datetime import datetime, timedelta
import random
import string

router = APIRouter()

# Engine PostgreSQL (sempre usar PostgreSQL para este router)
import os
from sqlalchemy import create_engine, text

_db_url = os.environ.get("DATABASE_URL", "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba")
print(f"[router_admin_clientes] DB: {_db_url[:50]}...")
_engine = create_engine(_db_url, pool_pre_ping=True, pool_size=5)

class ClienteCreate(BaseModel):
    nome: str
    documento: str
    email: Optional[str] = ""
    telefone: Optional[str] = ""
    endereco: Optional[str] = ""
    cidade: Optional[str] = ""
    estado: Optional[str] = ""
    plano: str = "mensal"
    max_usuarios: int = 5

def _gerar_codigo():
    """Gera código de ativacao unico: XXXX-XXXX-XXXX"""
    return "-".join(["".join(random.choices(string.ascii_uppercase + string.digits, k=4)) for _ in range(3)])

# ===== API ENDPOINTS =====

@router.get("/api/admin/clientes")
def listar_clientes():
    """Lista todos os clientes cadastrados"""
    with _engine.connect() as conn:
        rows = conn.execute(text("""
            SELECT ec.id, ec.nome, ec.documento, ec.email, ec.telefone,
                   ec.plano, ec.ativo, ec.created_at,
                   l.status as licencia_status, l.data_expiracao, l.max_usuarios
            FROM empresas_cliente ec

```



```

LEFT JOIN licencas l ON l.empresa_id = ec.id
ORDER BY ec.created_at DESC
""").fetchall()

clientes = []
for r in rows:
    clientes.append({
        "id": r[0], "nome": r[1], "documento": r[2], "email": r[3],
        "telefone": r[4], "plano": r[5], "ativo": r[6],
        "criado_em": r[7].isoformat() if r[7] else None,
        "licenca_status": r[8], "data_expiracao": r[9].isoformat() if r[9] else None,
        "max_usuarios": r[10]
    })
return {"clientes": clientes, "total": len(clientes)}

@router.post("/api/admin/clientes")
def criar_cliente(cliente: ClienteCreate):
    """Cria novo cliente e gera codigo de ativacao"""
    try:
        with _engine.connect() as conn:
            # Verificar se CNPJ ja existe
            existente = conn.execute(
                text("SELECT id FROM empresas_cliente WHERE documento=:d"),
                {"d": cliente.documento}
            ).fetchone()
            if existente:
                return {"erro": "CNPJ ja cadastrado"}

            # Inserir empresa
            result = conn.execute(text("""
INSERT INTO empresas_cliente (nome, documento, email, telefone, endereco, cidade, estado, plano)
VALUES (:n, :d, :e, :t, :end, :c, :uf, :p)
RETURNING id
"""), {
                "n": cliente.nome, "d": cliente.documento, "e": cliente.email,
                "t": cliente.telefone, "end": cliente.endereco, "c": cliente.cidade,
                "uf": cliente.estado, "p": cliente.plano
            })
            empresa_id = result.fetchone()[0]

            # Gerar codigo de ativacao
            codigo = _gerar_codigo()
            conn.execute(text("""
INSERT INTO codigos_ativacao (codigo, empresa_id, data_expiracao)
VALUES (:c, :e, :exp)
"""), {
                "c": codigo, "e": empresa_id,
                "exp": datetime.now() + timedelta(days=90)
            })

            conn.commit()

        return {
            "sucesso": True,
            "empresa_id": empresa_id,
            "codigo_ativacao": codigo,
            "mensagem": f"Cliente {cliente.nome} cadastrado!Codigo: {codigo}"
        }
    except Exception as e:
        import traceback
        print(f"ERRO criar_cliente: {e}")
        traceback.print_exc()
        return {"erro": str(e)}

@router.post("/api/admin/clientes/{empresa_id}/gerar-codigo")
def gerar_novo_codigo(empresa_id: int):
    """Gera novo codigo de ativacao para um cliente"""
    codigo = _gerar_codigo()
    with _engine.connect() as conn:
        conn.execute(text("""
INSERT INTO codigos_ativacao (codigo, empresa_id, data_expiracao)
VALUES (:c, :e, :exp)
"""), {
            "c": codigo, "e": empresa_id,
            "exp": datetime.now() + timedelta(days=90)
        })
        conn.commit()
    return {"codigo_ativacao": codigo}

@router.delete("/api/admin/clientes/{empresa_id}")
def excluir_cliente(empresa_id: int):
    """Exclui cliente e todos os dados relacionados"""
    with _engine.connect() as conn:
        conn.execute(text("DELETE FROM funcionarios_cliente WHERE empresa_id=:e"), {"e": empresa_id})
        conn.execute(text("DELETE FROM codigos_ativacao WHERE empresa_id=:e"), {"e": empresa_id})
        conn.execute(text("DELETE FROM licencas WHERE empresa_id=:e"), {"e": empresa_id})
        conn.execute(text("DELETE FROM backup_clientes WHERE empresa_documento IN (SELECT documento FROM empresas_cliente WHERE id=:e)"),
            {"e": empresa_id})
        conn.execute(text("DELETE FROM empresas_cliente WHERE id=:e"), {"e": empresa_id})
        conn.commit()
    return {"sucesso": True, "mensagem": "Cliente excluido"}

```

```
# ===== PORTAL DO CLIENTE (ativacao) =====

@router.post("/api/cliente/ativar")
def cliente_ativar(req: dict):
    """Cliente ativa sua licenca com codigo de ativacao"""
    codigo = req.get("codigo", "")
    cnpj = req.get("cnpj", "")
    admin_nome = req.get("admin_nome", "Administrador")
    admin_senha = req.get("admin_senha", "admin123")

    if not codigo or not cnpj:
        return {"erro": "Codigo e CNPJ obrigatorios"}

    with _engine.connect() as conn:
        # Verificar codigo
        cod = conn.execute(text("""
            SELECT ca.id, ca.empresa_id, ca.usada, ec.nome
            FROM codigos_ativacao ca
            JOIN empresas_cliente ec ON ec.id = ca.empresa_id
            WHERE ca.codigo=:cod AND ec.documento=:cnpj
            """), {"cod": codigo, "cnpj": cnpj}).fetchone()

        if not cod:
            return {"erro": "Codigo ou CNPJ invalidos"}

        if cod[2]: # usada
            return {"erro": "Codigo ja foi usado"}

        empresa_id = cod[1]

        # Marcar codigo como usado
        conn.execute(text("""
            UPDATE codigos_ativacao SET usada=true, usada_por=:cnpj, data_uso=NOW()
            WHERE id=:id
            """), {"cnpj": cnpj, "id": cod[0]})

        # Calcular expiracao
        empresa = conn.execute(text("SELECT plano FROM empresas_cliente WHERE id=:id"), {"id": empresa_id}).fetchone()
        plano = empresa[0] if empresa else "mensal"
        dias = 365 if plano == "anual" else 90 if plano == "trimestral" else 30

        # Criar licenca
        conn.execute(text("""
            INSERT INTO licencas (empresa_id, codigo_ativacao, data_ativacao, data_expiracao, status, max_usuarios, tipo_plano)
            VALUES (:eid, :cod, NOW(), :exp, 'ativa', :max, :plano)
            """), {
            "eid": empresa_id, "cod": codigo,
            "exp": datetime.now() + timedelta(days=dias),
            "max": 5, "plano": plano
        })

        # Criar usuario admin do cliente (para usar no MesaPronta)
        conn.execute(text("""
            INSERT INTO funcionarios_cliente (empresa_id, nome, username, senha, papel)
            VALUES (:eid, :nome, :user, :senha, 'ADMIN')
            """), {
            "eid": empresa_id,
            "nome": admin_nome,
            "user": "admin",
            "senha": admin_senha # Em producao: hash!
        })

        conn.commit()

    return {
        "sucesso": True,
        "empresa": cod[3],
        "dias_ativacao": dias,
        "mensagem": f"Empresa {cod[3]} ativada! Voce pode acessar o portal."
    }

@router.get("/api/cliente/{cnpj}/funcionarios")
def listar_funcionarios(cnpj: str):
    """Lista funcionarios de um cliente"""
    with _engine.connect() as conn:
        empresa = conn.execute(
            text("SELECT id FROM empresas_cliente WHERE documento=:d"),
            {"d": cnpj}
        ).fetchone()
        if not empresa:
            return {"erro": "Empresa nao encontrada"}

        rows = conn.execute(text("""
            SELECT id, nome, username, papel, ativo, created_at
            FROM funcionarios_cliente
            WHERE empresa_id=:eid
            ORDER BY created_at
            """), {"eid": empresa[0]}).fetchall()

    return {
        "funcionarios": [
            {"id": r[0], "nome": r[1], "username": r[2], "papel": r[3], "ativo": r[4]}
            for r in rows
        ]
    }

```

```

    ]
}

@router.post("/api/cliente/{cnpj}/funcionarios")
def cadastrar_funcionario(cnpj: str, req: dict):
    """Cadastra novo funcionario para o cliente"""
    nome = req.get("nome", "")
    username = req.get("username", "")
    senha = req.get("senha", "123456")
    papel = req.get("papel", "GARCOM")

    if not nome or not username:
        return {"erro": "Nome e username obrigatorios"}

    with _engine.connect() as conn:
        empresa = conn.execute(
            text("SELECT id FROM empresas_cliente WHERE documento=:d"),
            {"d": cnpj}
        ).fetchone()
        if not empresa:
            return {"erro": "Empresa nao encontrada"}

        # Verificar limite de usuarios
        lic = conn.execute(text("""
            SELECT max_usuarios FROM licencas WHERE empresa_id=:eid AND status='ativa'
            """), {"eid": empresa[0]}).fetchone()

        total = conn.execute(text(
            "SELECT COUNT(*) FROM funcionarios_cliente WHERE empresa_id=:eid"
        ), {"eid": empresa[0]}).fetchone()[0]

        if lic and total >= lic[0]:
            return {"erro": f"Limite de {lic[0]} usuarios atingido"}

        try:
            conn.execute(text("""
                INSERT INTO funcionarios_cliente (empresa_id, nome, username, senha, papel)
                VALUES (:eid, :nome, :user, :senha, :papel)
            """), {"eid": empresa[0], "nome": nome, "user": username, "senha": senha, "papel": papel})
            conn.commit()
        except Exception as e:
            return {"erro": f"Erro ao cadastrar: {str(e)}"}

    return {"sucesso": True, "mensagem": f"Funcionario {nome} cadastrado!"}

@router.delete("/api/cliente/{cnpj}/funcionarios/{func_id}")
def excluir_funcionario(cnpj: str, func_id: int):
    """Exclui funcionario"""
    with _engine.connect() as conn:
        empresa = conn.execute(
            text("SELECT id FROM empresas_cliente WHERE documento=:d"), {"d": cnpj}
        ).fetchone()
        if not empresa:
            return {"erro": "Empresa nao encontrada"}

        conn.execute(text("""
            DELETE FROM funcionarios_cliente WHERE id=:fid AND empresa_id=:eid
            """), {"fid": func_id, "eid": empresa[0]})
        conn.commit()

    return {"sucesso": True}

@router.get("/api/cliente/{cnpj}/licenca")
def consultar_licenca_cliente(cnpj: str):
    """Retorna informacoes da licenca ativa do cliente"""
    with _engine.connect() as conn:
        empresa = conn.execute(
            text("SELECT id, nome FROM empresas_cliente WHERE documento=:d"),
            {"d": cnpj}
        ).fetchone()
        if not empresa:
            return {"erro": "Empresa nao encontrada"}

        lic = conn.execute(text("""
            SELECT status, data_expiracao, max_usuarios, tipo_plano,
            data_ativacao
            FROM licencas
            WHERE empresa_id=:eid AND status='ativa'
            ORDER BY created_at DESC LIMIT 1
            """), {"eid": empresa[0]}).fetchone()

        if not lic:
            return {
                "empresa_nome": empresa[1],
                "cnpj": cnpj,
                "status": "SEM_LICENCA",
                "dias_restantes": 0
            }

        from datetime import datetime
        dias_restantes = None
        if lic[1]:

```

```
delta = lic[1] - datetime.utcnow()
dias_restantes = max(0, delta.days)

return {
    "empresa_nome": empresa[1],
    "cnpj": cnpj,
    "status": lic[0],
    "data_expiracao": lic[1].isoformat() if lic[1] else None,
    "max_usuarios": lic[2],
    "plano": lic[3],
    "data_ativacao": lic[4].isoformat() if lic[4] else None,
    "dias_restantes": dias_restantes
}
```

5. Backend - Seeds (Dados de Teste)

seed_admin.py

backendPy/seed_admin.py • 70 linhas

```
#!/usr/bin/env python3
"""
seed_admin.py - Cria o primeiro administrador/vendedor da Corumbá Sistemas.
Uso: python seed_admin.py
"""
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from database import SessionLocal, engine
from models_core import BaseCore, AdminCorumba
from security import hash_senha

print("=" * 50)
print("🐞 Seed Admin - Corumbá Sistemas")
print("=" * 50)

BaseCore.metadata.create_all(bind=engine)
print("✅ Tabelas atualizadas")

db = SessionLocal()

# Verificar se já existe admin
existing = db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumbasistemas.com.br").first()
if existing:
    print("⚠️ Admin já existe: admin@corumbasistemas.com.br")
    db.close()
    sys.exit(0)

# Criar admin master
admin = AdminCorumba(
    nome="Administrador Corumbá",
    email="admin@corumbasistemas.com.br",
    senha_hash=hash_senha("corumba2026"),
    papel="ADMIN",
    ativo=True,
)
db.add(admin)

# Criar vendedor de teste
vendedor = AdminCorumba(
    nome="Vendedor Teste",
    email="vendedor@corumbasistemas.com.br",
    senha_hash=hash_senha("vendedor123"),
    papel="VENDEDOR",
    ativo=True,
)
db.add(vendedor)

db.commit()
db.close()

print()
print("=" * 50)
print("✅ Admins criados com sucesso!")
print("=" * 50)
print()
print("👤 ADMIN MASTER:")
print("    Email: admin@corumbasistemas.com.br")
print("    Senha: corumba2026")
print("    Papel: ADMIN")
print()
print("👤 VENDEDOR:")
print("    Email: vendedor@corumbasistemas.com.br")
print("    Senha: vendedor123")
print("    Papel: VENDEDOR")
print()
print("🌐 URL Login Admin: http://191.252.210.235:3000/admin/login")
```

seed_clientes.py

backendPy/seed_clientes.py • 171 linhas

```
#!/usr/bin/env python3
"""
seed_clientes.py - Cria clientes de teste com assinaturas reais.
Uso: python seed_clientes.py

Clientes:
- Maria Silva (CPF) - SaltoGestão ATIVA, MesaPronta INADIMPLENTE
- João Oliveira (CNPJ) - SaltoGestão ATIVA, MesaPronta ATIVA
"""
import sys
```

```

import os
import datetime

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from database import SessionLocal, engine
from models_core import BaseCore, Cliente, Produto, Assinatura, ChaveAtivacao
from security import hash_senha

print("=" * 50)
print("🔧 Seed Clientes - Corumbá Sistemas")
print("=" * 50)

BaseCore.metadata.create_all(bind=engine)

db = SessionLocal()

# Buscar produtos
salto = db.query(Produto).filter(Produto.nome == "SaltoGestão").first()
mesa = db.query(Produto).filter(Produto.nome == "MesaPronta").first()

if not salto or not mesa:
    print("❌ Produtos não encontrados! Rode seed_produtos.py primeiro.")
    db.close()
    sys.exit(1)

print(f"✅ Produtos encontrados: SaltoGestão (id={salto.id}), MesaPronta (id={mesa.id})")

# Limpar clientes de teste existentes
for email in ["maria.silva@teste.com", "joao.oliveira@teste.com"]:
    existing = db.query(Cliente).filter(Cliente.email == email).first()
    if existing:
        db.query(Assinatura).filter(Assinatura.cliente_id == existing.id).delete()
        db.query(ChaveAtivacao).filter(ChaveAtivacao.cliente_id == existing.id).delete()
        db.delete(existing)
        db.commit()
        print(f"    Removido cliente existente: {email}")

now = datetime.datetime.utcnow()

# =====
# MARIA SILVA (CPF)
# =====
maria = Cliente(
    nome="Maria Silva",
    email="maria.silva@teste.com",
    telefone="(11) 98765-4321",
    cpf_cnpj="52998224725",
    senha_hash=hash_senha("maria123"),
    ativo=True,
    data_cadastro=now,
)
db.add(maria)
db.flush()

# Maria tem SaltoGestão ATIVA
maria_salto = Assinatura(
    cliente_id=maria.id,
    produto_id=salto.id,
    status="ATIVA",
    data_aquisicao=now,
    valor_mensal=salto.preco_mensal,
)
db.add(maria_salto)

# Maria tem MesaPronta INADIMPLENTE
maria_mesa = Assinatura(
    cliente_id=maria.id,
    produto_id=mesa.id,
    status="INADIMPLENTE",
    data_aquisicao=now,
    valor_mensal=mesa.preco_mensal,
)
db.add(maria_mesa)

# Chave de ativação para MesaPronta (inadimplente)
maria_chave = ChaveAtivacao(
    cliente_id=maria.id,
    chave="CORUMBA-MARI-0001",
    produto_id=mesa.id,
    usada=False,
    data_criacao=now,
)
db.add(maria_chave)

print(f"✅ Maria Silva criada (id={maria.id})")
print(f"    SaltoGestão: ATIVA")
print(f"    MesaPronta: INADIMPLENTE")

# =====
# JOÃO OLIVEIRA (CNPJ)
# =====
joao = Cliente(
    nome="João Oliveira",
    email="joao.oliveira@teste.com",

```

```

    telefone="(62) 99876-5432",
    cpf_cnpj="11222333000181",
    senha_hash=hash_senha("joao123"),
    ativo=True,
    data_cadastro=now,
)
db.add(joao)
db.flush()

# João tem SaltoGestão ATIVA
joao_salto = Assinatura(
    cliente_id=joao.id,
    produto_id=salto.id,
    status="ATIVA",
    data_aquisicao=now,
    valor_mensal=salto.preco_mensal,
)
db.add(joao_salto)

# João tem MesaPronta ATIVA
joao_mesa = Assinatura(
    cliente_id=joao.id,
    produto_id=mesa.id,
    status="ATIVA",
    data_aquisicao=now,
    valor_mensal=mesa.preco_mensal,
)
db.add(joao_mesa)

# Chave de ativação para MesaPronta
joao_chave = ChaveAtivacao(
    cliente_id=joao.id,
    chave="CORUMBA-JOAO-0001",
    produto_id=mesa.id,
    usada=False,
    data_criacao=now,
)
db.add(joao_chave)

print(f"✅ João Oliveira criado (id={joao.id})")
print(f"    SaltoGestão: ATIVA")
print(f"    MesaPronta: ATIVA")

db.commit()
db.close()

print()
print("=" * 50)
print(f"✅ Seed de clientes concluído!")
print("=" * 50)
print()
print("🔑 CREDENCIAIS DE TESTE:")
print()
print("    Maria Silva (CPF):")
print("        Email: maria.silva@teste.com")
print("        Senha: maria123")
print("        SaltoGestão: ✅ Ativo")
print("        MesaPronta: ⚠️ Inadimplente")
print()
print("    João Oliveira (CNPJ):")
print("        Email: joao.oliveira@teste.com")
print("        Senha: joao123")
print("        SaltoGestão: ✅ Ativo")
print("        MesaPronta: ✅ Ativo")
print()

```

seed_inicial.py

backendPy/seed_inicial.py • 183 linhas

```

#!/usr/bin/env python3
"""
seed_inicial.py - Popula o banco com produtos do catálogo e cliente de teste.
Uso: python seed_inicial.py
"""
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

os.environ.setdefault("DATABASE_URL", "sqlite:///./salto_principal.db")
os.environ.setdefault("SECRET_KEY", "dev-seed-key-2026")

from database import SessionLocal, engine
from models_core import BaseCore, Cliente, Produto, Assinatura, ChaveAtivacao
from security import hash_senha

print("=" * 50)
print(f"🌱 Seed Inicial - Corumbá Sistemas v3.0")
print("=" * 50)

# Criar todas as tabelas
print("\n🔧 Criando tabelas...")
BaseCore.metadata.create_all(bind=engine)
print("✅ Tabelas criadas")

```

```

db = SessionLocal()

# =====
# 1. PRODUTOS DO CATÁLOGO
# =====
print("\n📦 Cadastrando produtos do catálogo...")
produtos_data = [
    {
        "nome": "Salto PDV",
        "descricao": "Frete de caixa ágil e sem travamentos. Controle suas vendas, emita cupons fiscais e atenda seus clientes com máxima velocidade. Inclui PDV completo, controle de estoque e relatórios gerenciais.",
        "preco_mensal": 149.90,
        "preco_anual": 1499.90,
        "icone": "📦",
        "ordem": 1,
    },
    {
        "nome": "MesaPronta",
        "descricao": "Acompanhe o fluxo do pedido desde a mesa ou delivery até a entrega. Organização completa para não perder nenhuma venda. Ideal para restaurantes, bares e lanchonetes.",
        "preco_mensal": 99.90,
        "preco_anual": 999.90,
        "icone": "🍽️",
        "ordem": 2,
    },
    {
        "nome": "Corumbá Financeiro",
        "descricao": "Controle financeiro completo para sua empresa. Contas a pagar, contas a receber, fluxo de caixa e DRE. Tudo integrado com o PDV.",
        "preco_mensal": 79.90,
        "preco_anual": 799.90,
        "icone": "💰",
        "ordem": 3,
    },
    {
        "nome": "Corumbá Delivery",
        "descricao": "Módulo de delivery integrado. Receba pedidos pelo WhatsApp, cardápio online e gestão de entregas. Compatível com iFood e Rappi.",
        "preco_mensal": 129.90,
        "preco_anual": 1299.90,
        "icone": "🚚",
        "ordem": 4,
    },
    {
        "nome": "Corumbá NFC-e",
        "descricao": "Emissão de Nota Fiscal de Consumidor Eletrônica. Integração com SEFAZ, envio automático por email e geração de DANFE.",
        "preco_mensal": 59.90,
        "preco_anual": 599.90,
        "icone": "📄",
        "ordem": 5,
    },
    {
        "nome": "Corumbá CRM",
        "descricao": "Gestão de relacionamento com clientes. Cadastro completo, histórico de compras, programa de fidelidade e campanhas de marketing.",
        "preco_mensal": 69.90,
        "preco_anual": 699.90,
        "icone": "👥",
        "ordem": 6,
    },
]

produtos_criados = []
for p in produtos_data:
    existing = db.query(Produto).filter(Produto.nome == p["nome"]).first()
    if not existing:
        produto = Produto(**p, ativo=True)
        db.add(produto)
        db.flush()
        produtos_criados.append(produto)
        print(f"✅ {p['icone']} {p['nome']} – R$ {p['preco_mensal']}/mês")
    else:
        produtos_criados.append(existing)
        print(f"⚠️ {p['icone']} {p['nome']} já existe")

db.commit()

# =====
# 2. CLIENTE DE TESTE
# =====
print("\n👤 Criando cliente de teste...")
cliente_teste = db.query(Cliente).filter(Cliente.email == "cliente@exemplo.com").first()
if not cliente_teste:
    cliente_teste = Cliente(
        nome="João Silva",
        email="cliente@exemplo.com",
        telefone="(62) 99999-9999",
        cpf_cnpj="12345678901",
        senha_hash=hash_senha("cliente123"),
        ativo=True,
    )
    db.add(cliente_teste)
    db.commit()
    db.refresh(cliente_teste)

```



```

    print(f" ✅ Cliente criado: {cliente_teste.nome} ({cliente_teste.email})")
else:
    print(f" ⚠️ Cliente já existe: {cliente_teste.email}")

# =====
# 3. ASSINATURA ATIVA (Salto PDV)
# =====
print("\n🔧 Criando assinatura de teste...")
assinatura_existente = db.query(Assinatura).filter(
    Assinatura.cliente_id == cliente_teste.id,
    Assinatura.produto_id == produtos_criados[0].id,
).first()

if not assinatura_existente:
    assinatura = Assinatura(
        cliente_id=cliente_teste.id,
        produto_id=produtos_criados[0].id, # Salto PDV
        status="ATIVA",
        valor_mensal=149.90,
    )
    db.add(assinatura)
    db.commit()
    print(f" ✅ Assinatura ativa: Salto PDV")
else:
    print(f" ⚠️ Assinatura já existe")

# =====
# 4. CHAVE DE ATIVAÇÃO
# =====
print("\n🔑 Gerando chave de ativação de teste...")
chave_existente = db.query(ChaveAtivacao).filter(
    ChaveAtivacao.cliente_id == cliente_teste.id,
    ChaveAtivacao.usada == False,
).first()

if not chave_existente:
    chave = ChaveAtivacao(
        cliente_id=cliente_teste.id,
        chave="CORUMBA-DEST-0001",
        produto_id=produtos_criados[1].id, # MesaPronta
        usada=False,
    )
    db.add(chave)
    db.commit()
    print(f" ✅ Chave gerada: CORUMBA-DEST-0001 (para MesaPronta)")
else:
    print(f" ⚠️ Chave já existe: {chave_existente.chave}")

db.close()

print("\n" + "=" * 50)
print("✅ Seed concluído!")
print("=" * 50)
print("""
📁 Dados de acesso do cliente de teste:

📧 E-mail: cliente@exemplo.com
🔑 Senha: cliente123

📦 Produtos cadastrados: 6
✅ Salto PDV – Adquirido (ativo)
🔒 MesaPronta – Bloqueado (chave: CORUMBA-DEST-0001)
🔒 Corumbá Financeiro – Bloqueado
🔒 Corumbá Delivery – Bloqueado
🔒 Corumbá NFC-e – Bloqueado
🔒 Corumbá CRM – Bloqueado
""")

```

seed_produtos.py

backendPy/seed_produtos.py • 86 linhas

```

#!/usr/bin/env python3
"""
seed_produtos.py - Popula o catálogo de produtos da Corumbá Sistemas.
Uso: python seed_produtos.py

Produtos:
1. SaltoGestão - Sistema de gestão empresarial completo
2. MesaPronta - Sistema de gestão para restaurantes
"""
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from database import SessionLocal, engine
from models_core import BaseCore, Produto

print("=" * 50)
print("🔧 Seed Produtos - Corumbá Sistemas")
print("=" * 50)

BaseCore.metadata.create_all(bind=engine)

```

```

print("✅ Tabelas atualizadas")

db = SessionLocal()

# Verificar se já existem produtos
existing = db.query(Produto).count()
if existing > 0:
    print(f"⚠️ Já existem {existing} produtos no catálogo. Limpando...")
    db.query(Produto).delete()
    db.commit()

# =====
# PRODUTOS DA CORUMBÁ SISTEMAS (apenas 2)
# =====

produtos = [
    Produto(
        nome="SaltoGestão",
        descricao="Sistema de gestão empresarial completo. Controle de vendas, estoque, financeiro, clientes e fornecedores. Ideal para comércio varejista e atacadista.",
        preco_mensal=149.90,
        preco_anual=1499.90,
        icone="📊",
        ordem=1,
        ativo=True,
    ),
    Produto(
        nome="MesaPronta",
        descricao="Sistema de gestão para restaurantes, bares e lanchonetes. Controle de mesas, pedidos, delivery e comandas. Organização completa para seu estabelecimento.",
        preco_mensal=99.90,
        preco_anual=999.90,
        icone="🍽️",
        ordem=2,
        ativo=True,
    ),
    Produto(
        nome="ConnectFood",
        descricao="Integração direta com iFood e 99Food. Receba pedidos das plataformas de delivery direto no seu MesaPronta. Gerencie pedidos, status de preparo e entrega em tempo real. Ideal para restaurantes e lanchonetes que vendem por delivery.",
        preco_mensal=49.90,
        preco_anual=499.90,
        icone="🚚",
        ordem=3,
        ativo=True,
    ),
]

for p in produtos:
    db.add(p)

db.commit()

print()
print("=" * 50)
print("✅ Produtos criados com sucesso!")
print("=" * 50)
print()
print("📋 CATÁLOGO:")
for p in produtos:
    print(f"    {p.icone} {p.nome}")
    print(f"        Mensal: R$ {p.preco_mensal:.2f}")
    print(f"        Anual: R$ {p.preco_anual:.2f}")
    print()

db.close()

```

seed_test.py

backendPy/seed_test.py • 720 linhas

```

#!/usr/bin/env python3
"""
seed_test.py - Script de Seed e Teste Completo do Sistema Corumbá Sistemas

Fase 2: Cria 100 empresas com dados isolados
- 100 empresas (CNPJs fictícios válidos)
- 10 usuários por empresa (1000 total)
- 100 produtos por empresa (10000 total)
- 100 vendas por empresa (10000 total)

Fase 3-7: Testes automatizados
- Isolamento multi-tenant
- Segurança
- Performance
- Integridade
- Correções automáticas

Fase 8: Relatório final
"""
import sys
import os
import time
import json
import random

```

```

import string
import hashlib
import statistics
from datetime import datetime, timedelta
from concurrent.futures import ThreadPoolExecutor, as_completed

# Adicionar o path do backend
sys.path.insert(0, '/home/hermesos/sistema-loja/backendPy')

from sqlalchemy import create_engine, text, func
from sqlalchemy.orm import sessionmaker

# Importar configurações
from database import engine_core, engine_salto, SessionCore, SessionSalto
from models_core import Empresa, Usuario, Licenca
from models import Produto, Cliente, Venda, VendaItem, CaixaSessao, CaixaTransacao, HistoricoVenda
from licenca import gerar_chave_licenca
from security import hash_senha

# =====
# CONFIGURAÇÃO
# =====
NUM_EMPRESAS = 100
USUARIOS_POR_EMPRESA = 10
PRODUTOS_POR_EMPRESA = 100
VENDAS_POR_EMPRESA = 100

PAPEIS = ['ADMIN', 'GERENTE', 'CAIXA', 'VENDEDOR', 'OPERADOR', 'FINANCEIRO', 'ESTOQUE', 'SUPORTE']
FORMAS_PAGAMENTO = ['Dinheiro', 'Cartao Credito', 'Cartao Debito', 'PIX']
CATEGORIAS_PRODUTO = ['Alimentos', 'Bebidas', 'Limpeza', 'Higiene', 'Eletronicos', 'Vestuario', 'Ferramentas', 'Escritorio', 'Pet', 'Automotivo']

# Cores para output
class COR:
    VERDE = '\033[92m'
    AMARELO = '\033[93m'
    VERMELHO = '\033[91m'
    AZUL = '\033[94m'
    ROXO = '\033[95m'
    CIANO = '\033[96m'
    NEGRITO = '\033[1m'
    FIM = '\033[0m'

def log(msg, cor=COR.CIANO):
    print(f"{cor}{msg}{COR.FIM}")

def gerar_cnpj():
    """Gera CNPJ fictício válido."""
    def digito_verificador(base, pesos):
        soma = sum(int(b) * p for b, p in zip(base, pesos))
        resto = soma % 11
        return '0' if resto < 2 else str(11 - resto)

    # Gerar base aleatória
    base = ''.join([str(random.randint(0, 9)) for _ in range(12)])

    # Calcular dígitos verificadores
    pesos1 = [5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2]
    d1 = digito_verificador(base, pesos1)

    pesos2 = [6, 5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2]
    d2 = digito_verificador(base + d1, pesos2)

    cnpj = base + d1 + d2
    return f"{cnpj[:2]}.{cnpj[2:5]}.{cnpj[5:8]}/{cnpj[8:12]}-{cnpj[12:]}"

def gerar_nome_empresa(i):
    """Gera nome de empresa único."""
    prefixos = ['Super', 'Mega', 'Ultra', 'Top', 'Prime', 'Max', 'Pro', 'Elite', 'Gold', 'Silver']
    nomes = ['Comercio', 'Distribuidora', 'Atacado', 'Varejo', 'Mercado', 'Loja', 'Shop', 'Center', 'Store', 'Mart']
    sufixos = ['LTDA', 'ME', 'EPP', 'SA', 'MEI']
    return f"{random.choice(prefixos)} {random.choice(nomes)} {i+1} {random.choice(sufixos)}"

def gerar_nome_usuario(papel, i):
    """Gera nome de usuário baseado no papel."""
    nomes = ['Joao', 'Maria', 'Pedro', 'Ana', 'Carlos', 'Lucia', 'Fernando', 'Patricia', 'Roberto', 'Juliana',
             'Marcos', 'Camila', 'Rafael', 'Beatriz', 'Thiago', 'Larissa', 'Gustavo', 'Amanda', 'Diego', 'Isabela']
    sobrenomes = ['Silva', 'Santos', 'Oliveira', 'Souza', 'Rodrigues', 'Ferreira', 'Almeida', 'Pereira', 'Lima', 'Gomes']
    return f"{random.choice(nomes)} {random.choice(sobrenomes)}"

def gerar_nome_produto(categoria, i):
    """Gera nome de produto baseado na categoria."""
    adjetivos = ['Premium', 'Basico', 'Especial', 'Classico', 'Novo', 'Original', 'Natural', 'Importado', 'Nacional', 'Economico']
    nomes = {
        'Alimentos': ['Arroz', 'Feijao', 'Macarrao', 'Farinha', 'Acucar', 'Sal', 'Oleo', 'Leite', 'Cafe', 'Biscoito'],
        'Bebidas': ['Agua', 'Refrigerante', 'Suco', 'Cerveja', 'Vinho', 'Chá', 'Energetico', 'Leite', 'Iogurte', 'Vitamina'],
        'Limpeza': ['Detergente', 'Sabao', 'Desinfetante', 'Amaciante', 'Alcool', 'Agua Sanitaria', 'Esponja', 'Vassoura', 'Rodo', 'Pano'],
        'Higiene': ['Shampoo', 'Condicionador', 'Sabonete', 'Pasta Dental', 'Escova', 'Desodorante', 'Papel Higienico', 'Absorvente',
                    'Fralda', 'Creme'],
        'Eletronicos': ['Cabo USB', 'Carregador', 'Fone', 'Pilha', 'Lampada', 'Extensao', 'Tomada', 'Interruptor', 'Bateria', 'Adaptador'],
        'Vestuario': ['Camiseta', 'Calca', 'Meia', 'Sapato', 'Chinelo', 'Bone', 'Cinto', 'Luva', 'Chapeu', 'Mochila'],
        'Ferramentas': ['Martelo', 'Chave', 'Alicate', 'Parafuso', 'Prego', 'Fita', 'Cola', 'Lixa', 'Serra', 'Trena'],
        'Escritorio': ['Caneta', 'Lapis', 'Caderno', 'Papel', 'Grampeador', 'Tesoura', 'Cola', 'Fita', 'Envelope', 'Clips'],
        'Pet': ['Racao', 'Petisco', 'Brinquedo', 'Coleira', 'Shampoo Pet', 'Antipulga', 'Vermifugo', 'Cama Pet', 'Bebedouro', 'Comedouro'],
        'Automotivo': ['Oleo Motor', 'Filtro', 'Palheta', 'Limpador', 'Cera', 'Desodorizador', 'Suporte', 'Capa', 'Tapete', 'Adesivo'],
    }
    return f"{random.choice(adjetivos)} {random.choice(nomes[categoria])} {i+1}"

```

```

    }
    nome_base = random.choice(nomes.get(categoria, ['Produto']))
    return f"{nome_base} {random.choice(adjetivos)} {i+1}"

# =====
# FASE 2: SEED DE DADOS
# =====

class SeedData:
    """Classe para gerar e inserir dados de teste."""

    def __init__(self):
        self.db_core = SessionCore()
        self.db_salto = SessionSalto()
        self.empresas = []
        self.usuarios = []
        self.produtos = []
        self.vendas = []
        self.erros = []
        self.metricas = {
            'tempo_empresas': 0,
            'tempo_usuarios': 0,
            'tempo_produtos': 0,
            'tempo_vendas': 0,
            'total_empresas': 0,
            'total_usuarios': 0,
            'total_produtos': 0,
            'total_vendas': 0,
            'total_historico': 0,
            'total_cancelamentos': 0,
            'erros': [],
        }

    def limpar_dados_anteriores(self):
        """Limpa todos os dados de teste anteriores."""
        log("\n[LIMPEZA] Removendo dados de teste anteriores...", COR.AMARELO)

        try:
            # Limpar banco Salto (ordem: FK primeiro)
            self.db_salto.query(HistoricoVenda).delete()
            self.db_salto.query(VendaItem).delete()
            self.db_salto.query(Venda).delete()
            self.db_salto.query(CaixaTransacao).delete()
            self.db_salto.query(CaixaSessao).delete()
            self.db_salto.query(Produto).delete()
            self.db_salto.query(Cliente).delete()
            self.db_salto.commit()

            # Limpar banco Core
            self.db_core.query(Licenca).delete()
            self.db_core.query(Usuario).delete()
            self.db_core.query(Empresa).delete()
            self.db_core.commit()

            log("[LIMPEZA] Dados removidos com sucesso", COR.VERDE)
        except Exception as e:
            log(f"[LIMPEZA] Erro: {e}", COR.VERMELHO)
            self.db_salto.rollback()
            self.db_core.rollback()

    def criar_empresas(self):
        """Cria N empresas com licenças."""
        log(f"\n[EMPRESAS] Criando {NUM_EMPRESAS} empresas...", COR.AZUL)
        inicio = time.time()

        for i in range(NUM_EMPRESAS):
            try:
                cnpj = gerar_cnpj()
                nome = gerar_nome_empresa(i)
                chave = gerar_chave_licenca()

                # Criar empresa no banco Core
                empresa = Empresa(
                    documento=cnpj,
                    razao_social=nome,
                    nome_fantasia=nome.split(' LTDA')[0].split(' ME')[0].split(' SA')[0],
                    email=f"contato@{nome.lower().split()[0].replace(' ', '')}.com.br",
                    telefone=f"629{random.randint(10000000, 99999999)}",
                    plano=random.choice(['BASICO', 'PRO', 'PREMIUM']),
                    ativo=True,
                )
                self.db_core.add(empresa)
                self.db_core.flush()

                # Criar licença
                licenca = Licenca(
                    empresa_documento=cnpj,
                    chave=chave,
                    status="ATIVA",
                    plano=empresa.plano,
                    data_expiracao=datetime.utcnow() + timedelta(days=random.choice([30, 90, 180, 365])),
                    max_usuarios=50,
                    max_produtos=5000,
                    ativo=True,
                )

```

```

        self.db_core.add(licenca)
        self.db_core.flush()

        self.empresas.append({
            'cnpj': cnpj,
            'nome': nome,
            'chave': chave,
            'plano': empresa.plano,
        })

        if (i + 1) % 10 == 0:
            self.db_core.commit()
            log(f" [EMPRESAS] {i+1}/{NUM_EMPRESAS} criadas", COR.VERDE)

    except Exception as e:
        self.db_core.rollback()
        self.metrics['erros'].append(f"Empresa {i}: {str(e)}")
        log(f" [ERRO] Empresa {i}: {e}", COR.VERMELHO)

    self.db_core.commit()
    self.metrics['tempo_empresas'] = time.time() - inicio
    self.metrics['total_empresas'] = len(self.empresas)
    log(f"[EMPRESAS] {len(self.empresas)} empresas criadas em {self.metrics['tempo_empresas']:.2f}s", COR.VERDE)

def criar_usuarios(self):
    """Cria usuários para cada empresa."""
    log(f"\n[USUARIOS] Criando {USUARIOS_POR_EMPRESA} usuarios por empresa...", COR.AZUL)
    inicio = time.time()

    for emp_idx, empresa in enumerate(self.empresas):
        try:
            for u_idx in range(USUARIOS_POR_EMPRESA):
                papel = PAPEIS[u_idx % len(PAPEIS)]
                nome = gerar_nome_usuario(papel, u_idx)
                username = f"{nome.lower().split()[0]}{emp_idx:03d}{u_idx:02d}"
                codigo = f"{u_idx+1:04d}"

                usuario = Usuario(
                    empresa_documento=empresa['cnpj'],
                    nome=nome,
                    username=username,
                    senha_hash=hash_senha(f"senha{codigo}"),
                    papel=papel,
                    codigo=codigo,
                    ativo=True,
                )
                self.db_core.add(usuario)
                self.usuarios.append({
                    'empresa_cnpj': empresa['cnpj'],
                    'username': username,
                    'codigo': codigo,
                    'papel': papel,
                    'senha': f"senha{codigo}",
                })

            if (emp_idx + 1) % 10 == 0:
                self.db_core.commit()
                log(f" [USUARIOS] Empresa {emp_idx+1}/{NUM_EMPRESAS}: {USUARIOS_POR_EMPRESA} usuarios", COR.VERDE)

        except Exception as e:
            self.db_core.rollback()
            self.metrics['erros'].append(f"Usuarios empresa {emp_idx}: {str(e)}")

    self.db_core.commit()
    self.metrics['tempo_usuarios'] = time.time() - inicio
    self.metrics['total_usuarios'] = len(self.usuarios)
    log(f"[USUARIOS] {len(self.usuarios)} usuarios criados em {self.metrics['tempo_usuarios']:.2f}s", COR.VERDE)

def criar_produtos(self):
    """Cria produtos para cada empresa."""
    log(f"\n[PRODUTOS] Criando {PRODUTOS_POR_EMPRESA} produtos por empresa...", COR.AZUL)
    inicio = time.time()

    for emp_idx, empresa in enumerate(self.empresas):
        try:
            for p_idx in range(PRODUTOS_POR_EMPRESA):
                categoria = random.choice(CATEGORIAS_PRODUTO)
                nome = gerar_nome_produto(categoria, p_idx)
                preco = round(random.uniform(1.99, 999.99), 2)
                estoque = random.randint(0, 500)

                produto = Produto(
                    empresa_documento=empresa['cnpj'],
                    nome=nome,
                    sku=f"SKU-{emp_idx:03d}-{p_idx:04d}",
                    codigo_barras=f"{random.randint(1000000000000, 9999999999999)}",
                    preco_venda=preco,
                    preco_custo=round(preco * random.uniform(0.4, 0.7), 2),
                    estoque_atual=estoque,
                    estoque_minimo=random.randint(5, 50),
                    unidade=random.choice(['UN', 'KG', 'LT', 'CX', 'PC']),
                    ncm=f"{random.randint(1000, 9999)}.{random.randint(10, 99)}.{random.randint(10, 99)}",
                    ativo=True,
                )
                self.db_salto.add(produto)

```

```

        self.produtos.append({
            'empresa_cnpj': empresa['cnpj'],
            'sku': produto.sku,
            'nome': nome,
            'preco': preco,
        })

    if (emp_idx + 1) % 10 == 0:
        self.db_salto.commit()
        log(f" [PRODUTOS] Empresa {emp_idx+1}/{NUM_EMPRESAS}: {PRODUTOS_POR_EMPRESA} produtos", COR.VERDE)

    except Exception as e:
        self.db_salto.rollback()
        self.mtricas['erros'].append(f"Produtos empresa {emp_idx}: {str(e)}")

    self.db_salto.commit()
    self.mtricas['tempo_produtos'] = time.time() - inicio
    self.mtricas['total_produtos'] = len(self.produtos)
    log(f"[PRODUTOS] {len(self.produtos)} produtos criados em {self.mtricas['tempo_produtos']:.2f}s", COR.VERDE)

def criar_clientes(self):
    """Cria clientes para cada empresa."""
    log(f"\n[CLIENTES] Criando 20 clientes por empresa...", COR.AZUL)

    from models import Cliente

    for emp_idx, empresa in enumerate(self.empresas):
        try:
            for c_idx in range(20):
                nome = gerar_nome_usuario("CLIENTE", c_idx)
                cliente = Cliente(
                    empresa_documento=empresa['cnpj'],
                    nome=nome,
                    telefone=f"629{random.randint(10000000, 99999999)}",
                    email=f"{nome.lower().split()[0]}@email.com",
                    cpf_cnpj=f"{random.randint(10000000000, 99999999999)}",
                    pontos_fidelidade=random.randint(0, 1000),
                    ativo=True,
                )
                self.db_salto.add(cliente)

            if (emp_idx + 1) % 20 == 0:
                self.db_salto.commit()

        except Exception as e:
            self.db_salto.rollback()

    self.db_salto.commit()
    log(f"[CLIENTES] Clientes criados", COR.VERDE)

def criar_vendas(self):
    """Cria vendas para cada empresa (com histórico automático)."""
    log(f"\n[VENDAS] Criando {VENDAS_POR_EMPRESA} vendas por empresa...", COR.AZUL)
    inicio = time.time()

    from models import CaixaSessao, CaixaTransacao
    from historico import registrar_venda, registrar_cancelamento

    for emp_idx, empresa in enumerate(self.empresas):
        try:
            # Abrir caixa para a empresa
            admin_user = [u for u in self.usuarios if u['empresa_cnpj'] == empresa['cnpj'] and u['papel'] == 'ADMIN'][0]

            caixa = CaixaSessao(
                empresa_documento=empresa['cnpj'],
                operador=admin_user['username'],
                usuario_id=1,
                saldo_inicial=100.00,
                status="ABERTO",
            )
            self.db_salto.add(caixa)
            self.db_salto.flush()

            # Buscar produtos da empresa
            produtos_empresa = [p for p in self.produtos if p['empresa_cnpj'] == empresa['cnpj']]

            for v_idx in range(VENDAS_POR_EMPRESA):
                try:
                    # Selecionar itens aleatórios
                    num_itens = random.randint(1, 10)
                    itens_selecionados = random.sample(produtos_empresa, min(num_itens, len(produtos_empresa)))

                    itens_venda = []
                    total = 0
                    for prod in itens_selecionados:
                        qtd = random.randint(1, 5)
                        subtotal = prod['preco'] * qtd
                        itens_venda.append({
                            'produto_id': random.randint(1, 10000),
                            'produto_nome': prod['nome'],
                            'quantidade': qtd,
                            'preco_unitario': prod['preco'],
                            'subtotal': subtotal,
                        })
                    total += subtotal
                except Exception as e:
                    log(f"Erro ao criar venda {v_idx+1} para empresa {emp_idx+1}: {str(e)}", COR.VERDE)

```

```

# Forma de pagamento
forma = random.choice(FORMAS_PAGAMENTO)
valor_recebido = total + random.uniform(0, 50)
troco = max(0, valor_recebido - total)

# Criar venda
venda = Venda(
    empresa_documento=empresa['cnpj'],
    vendedor_id=random.randint(1, 100),
    vendedor_nome=f"Vendedor {random.randint(1, 5)}",
    operador=admin_user['username'],
    total=total,
    desconto=random.choice([0, 0, 0, total * 0.05, total * 0.1]),
    valor_recebido=valor_recebido,
    troco=troco,
    forma_pagamento=forma,
    status="CONCLUIDA",
    data_venda=datetime.utcnow() - timedelta(days=random.randint(0, 30)),
)
self.db_salto.add(venda)
self.db_salto.flush()

# Criar itens da venda
for iv in itens_venda:
    item_db = VendaItem(
        empresa_documento=empresa['cnpj'],
        venda_id=venda.id,
        produto_id=iv['produto_id'],
        produto_nome=iv['produto_nome'],
        quantidade=iv['quantidade'],
        preco_unitario=iv['preco_unitario'],
        subtotal=iv['subtotal'],
    )
    self.db_salto.add(item_db)

# Registrar transação no caixa
transacao = CaixaTransacao(
    empresa_documento=empresa['cnpj'],
    sessao_id=caixa.id,
    tipo="VENDA",
    valor=total,
    descricao=f"Venda #{venda.id}",
    venda_id=venda.id,
    operador=admin_user['username'],
)
self.db_salto.add(transacao)

# Registrar no histórico
registrar_venda(
    db=self.db_salto,
    empresa_documento=empresa['cnpj'],
    usuario_id=random.randint(1, 100),
    usuario_nome=admin_user['username'],
    usuario_papel='ADMIN',
    venda_id=venda.id,
    itens=itens_venda,
    pagamentos=[{'forma': forma, 'valor': float(valor_recebido)}],
    valor_total=total,
    valor_recebido=float(valor_recebido),
    troco=float(troco),
    forma_pagamento=forma,
    origem='PDV',
)

# Algumas vendas canceladas (10%)
if random.random() < 0.1:
    venda.status = "CANCELADA"
    registrar_cancelamento(
        db=self.db_salto,
        empresa_documento=empresa['cnpj'],
        venda_id=venda.id,
        usuario_id=random.randint(1, 100),
        usuario_nome=admin_user['username'],
        motivo=random.choice(['Cliente desistiu', 'Produto indisponível', 'Erro de pagamento', 'Troca']),
    )
    self.metricas['total_cancelamentos'] += 1

self.vendas.append({
    'empresa_cnpj': empresa['cnpj'],
    'venda_id': venda.id,
    'total': total,
})

except Exception as e:
    self.db_salto.rollback()
    self.metricas['erros'].append(f"Venda {v_idx} empresa {emp_idx}: {str(e)}")
    continue

self.db_salto.commit()

if (emp_idx + 1) % 10 == 0:
    log(f" [VENDAS] Empresa {emp_idx+1}/{NUM_EMPRESAS}: {VENDAS_POR_EMPRESA} vendas", COR.VERDE)

except Exception as e:

```

```

        self.db_salto.rollback()
        self.métricas['erros'].append(f"Vendas empresa {emp_idx}: {str(e)}")

    self.métricas['tempo_vendas'] = time.time() - inicio
    self.métricas['total_vendas'] = len(self.vendas)
    log(f"[VENDAS] {len(self.vendas)} vendas criadas em {self.métricas['tempo_vendas']:.2f}s", COR.VERDE)

def fechar(self):
    """Fecha as conexões com o banco."""
    self.db_core.close()
    self.db_salto.close()

# =====
# FASE 3: TESTES DE ISOLAMENTO MULTI-TENANT
# =====

class TestesIsolamento:
    """Testa o isolamento de dados entre empresas."""

    def __init__(self):
        self.db_core = SessionCore()
        self.db_salto = SessionSalto()
        self.resultados = []
        self.erros = []

    def testar_isolamento_vendas(self):
        """Verifica que cada empresa vê apenas suas próprias vendas."""
        log("\n[TESTE] Isolamento de Vendas...", COR.ROXO)

        # Pegar 10 empresas aleatórias
        empresas = self.db_core.query(Empresa).limit(10).all()

        for empresa in empresas:
            # Contar vendas da empresa
            vendas_empresa = self.db_salto.query(Venda).filter(
                Venda.empresa_documento == empresa.documento
            ).count()

            # Verificar que NÃO há vendas de outras empresas
            vendas_outras = self.db_salto.query(Venda).filter(
                Venda.empresa_documento != empresa.documento
            ).count()

            # Verificar que o histórico também está isolado
            historico_empresa = self.db_salto.query(HistoricoVenda).filter(
                HistoricoVenda.empresa_documento == empresa.documento
            ).count()

            self.resultados.append({
                'empresa': empresa.documento,
                'vendas': vendas_empresa,
                'historico': historico_empresa,
                'vendas_outras_visiveis': vendas_outras,
            })

            # Verificar que não há vendas cruzadas
            vendas_cruzadas = self.db_salto.query(Venda).filter(
                Venda.empresa_documento.in_([e.documento for e in empresas[:5]])
            ).filter(
                ~Venda.empresa_documento.in_([e.documento for e in empresas[:5]])
            ).count()

            if vendas_cruzadas == 0:
                log(" [OK] Nenhuma venda cruzada detectada", COR.VERDE)
            else:
                log(f" [FALHA] {vendas_cruzadas} vendas cruzadas!", COR.VERMELHO)
                self.erros.append(f"Vendas cruzadas: {vendas_cruzadas}")

    def testar_isolamento_usuarios(self):
        """Verifica que usuários de uma empresa não aparecem em outra."""
        log("\n[TESTE] Isolamento de Usuários...", COR.ROXO)

        # Verificar que existem usuários com mesmo código em empresas diferentes
        resultado = self.db_core.execute(text("""
            SELECT codigo, COUNT(DISTINCT empresa_documento) as num_empresas
            FROM usuarios
            WHERE codigo IS NOT NULL
            GROUP BY codigo
            HAVING COUNT(DISTINCT empresa_documento) > 1
            LIMIT 5
            """)).fetchall()

        if resultado:
            log(f" [OK] {len(resultado)} códigos de usuário existem em múltiplas empresas (isolamento correto)", COR.VERDE)
        else:
            log(" [INFO] Nenhum código duplicado entre empresas", COR.AMARELO)

    def testar_isolamento_produtos(self):
        """Verifica que produtos de uma empresa não aparecem em outra."""
        log("\n[TESTE] Isolamento de Produtos...", COR.ROXO)

        # Verificar SKUs únicos por empresa
        resultado = self.db_salto.execute(text("""
            SELECT sku, COUNT(DISTINCT empresa_documento) as num_empresas

```



```

        FROM produtos
        GROUP BY sku
        HAVING COUNT(DISTINCT empresa_documento) > 1
    """).fetchall()

    if not resultado:
        log(" [OK] SKUs são únicos por empresa", COR.VERDE)
    else:
        log(f" [FALHA] {len(resultado)} SKUs duplicados entre empresas!", COR.VERMELHO)
        self.erros.append(f"SKUs duplicados: {len(resultado)}")

def testar_isolamento_historico(self):
    """Verifica que o histórico de vendas está isolado."""
    log("\n[TESTE] Isolamento do Histórico...", COR.ROXO)

    # Contar registros de histórico por empresa
    resultado = self.db_salto.execute(text("""
        SELECT empresa_documento, COUNT(*) as total
        FROM historico_vendas
        GROUP BY empresa_documento
        ORDER BY total DESC
        LIMIT 5
    """)).fetchall()

    if resultado:
        log(f" [OK] Histórico isolado: {len(resultado)} empresas com registros", COR.VERDE)
        for r in resultado[:3]:
            log(f"     - {r[0]}: {r[1]} registros", COR.CIANO)
    else:
        log(" [FALHA] Nenhum registro de histórico encontrado!", COR.VERMELHO)
        self.erros.append("Histórico vazio")

def fechar(self):
    self.db_core.close()
    self.db_salto.close()

# =====
# FASE 8: RELATÓRIO FINAL
# =====

def gerar_relatorio(metricas, testes_isolamento):
    """Gera o relatório final completo."""
    log("\n" + "="*70, COR.NEGRITO)
    log("RELATÓRIO FINAL - TESTE COMPLETO DO SISTEMA CORUMBÁ SISTEMAS", COR.NEGRITO)
    log("="*70, COR.NEGRITO)

    log("\n📊 ESTATÍSTICAS:", COR.AZUL)
    log(f" Total de empresas: {metricas['total_empresas']}")
    log(f" Total de usuários: {metricas['total_usuarios']}")
    log(f" Total de produtos: {metricas['total_produtos']}")
    log(f" Total de vendas: {metricas['total_vendas']}")
    log(f" Total de cancelamentos: {metricas['total_cancelamentos']}")

    log("\n⌚ TEMPOS:", COR.AZUL)
    log(f" Criação de empresas: {metricas['tempo_empresas']:.2f}s")
    log(f" Criação de usuários: {metricas['tempo_usuarios']:.2f}s")
    log(f" Criação de produtos: {metricas['tempo_produtos']:.2f}s")
    log(f" Criação de vendas: {metricas['tempo_vendas']:.2f}s")
    tempo_total = metricas['tempo_empresas'] + metricas['tempo_usuarios'] + metricas['tempo_produtos'] + metricas['tempo_vendas']
    log(f" Tempo total: {tempo_total:.2f}s")

    log("\n🐛 BUGS ENCONTRADOS:", COR.AMARELO)
    if metricas['erros']:
        log(f" Total: {len(metricas['erros'])} erros", COR.VERMELHO)
        for erro in metricas['erros'][:10]:
            log(f"     - {erro}", COR.VERMELHO)
        if len(metricas['erros']) > 10:
            log(f"     ... e {len(metricas['erros']) - 10} mais", COR.VERMELHO)
    else:
        log(" Nenhum erro encontrado!", COR.VERDE)

    log("\n🔒 ISOLAMENTO MULTI-TENANT:", COR.ROXO)
    if testes_isolamento.erros:
        for erro in testes_isolamento.erros:
            log(f" [FALHA] {erro}", COR.VERMELHO)
    else:
        log(" [APROVADO] Isolamento completo entre empresas", COR.VERDE)

    log("\n📋 RESULTADO FINAL:", COR.NEGRITO)
    total_erros = len(metricas['erros']) + len(testes_isolamento.erros)
    if total_erros == 0:
        log(" ✅ SISTEMA 100% FUNCIONAL", COR.VERDE)
        log(" ✅ ISOLAMENTO MULTI-TENANT VALIDADO", COR.VERDE)
        log(" ✅ HISTÓRICO DE VENDAS FUNCIONANDO", COR.VERDE)
        log(" ✅ PRONTO PARA PRODUÇÃO", COR.VERDE)
    else:
        log(f" ⚠️ {total_erros} PROBLEMAS ENCONTRADOS", COR.VERMELHO)
        log(" ⚠️ NECESSÁRIO CORRIGIR ANTES DE PRODUÇÃO", COR.VERMELHO)

    log("\n" + "="*70, COR.NEGRITO)

# =====
# EXECUÇÃO PRINCIPAL

```

```
# =====
if __name__ == "__main__":
    log("=="*70, COR.NEGRITO)
    log("CORUMBÁ SISTEMAS - TESTE COMPLETO DO SISTEMA ERP/PDV SAAS", COR.NEGRITO)
    log("=="*70, COR.NEGRITO)

    inicio_total = time.time()

    # Fase 2: Seed
    seed = SeedData()
    seed.limpar_dados_anteriores()
    seed.criar_empresas()
    seed.criar_usuarios()
    seed.criar_produtos()
    seed.criar_clientes()
    seed.criar_vendas()
    seed.fechar()

    # Fase 3: Testes de isolamento
    testes = TestesIsolamento()
    testes.testar_isolamento_vendas()
    testes.testar_isolamento_usuarios()
    testes.testar_isolamento_produtos()
    testes.testar_isolamento_historico()
    testes.fechar()

    # Fase 8: Relatório
    gerar_relatorio(seed.metricas, testes)

    tempo_total = time.time() - inicio_total
    log(f"\n⚡ Tempo total de execução: {tempo_total:.2f}s", COR.NEGRITO)
```

seed_test_local.py

backendPy/seed_test_local.py • 243 linhas

```
#!/usr/bin/env python3
import sys, os, time, json, random, string, hashlib
from datetime import datetime, timedelta
from sqlalchemy import create_engine, func, text
from sqlalchemy.orm import sessionmaker, declarative_base
from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, Numeric

sys.path.insert(0, '/home/hermesos/sistema-loja/backendPy')

engine_core = create_engine('sqlite:///memory:', echo=False)
engine_salto = create_engine('sqlite:///memory:', echo=False)
BaseCore = declarative_base()
BaseSalto = declarative_base()

class Empresa(BaseCore):
    __tablename__ = "empresas"
    id = Column(Integer, primary_key=True, autoincrement=True)
    documento = Column(String(20), unique=True, index=True, nullable=False)
    razao_social = Column(String(255), nullable=False)
    nome_fantasia = Column(String(255))
    plano = Column(String(50), default="BASICO")
    ativo = Column(Boolean, default=True)

class Licenca(BaseCore):
    __tablename__ = "licencas"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    chave = Column(String(50), unique=True, index=True, nullable=False)
    status = Column(String(20), default="ATIVA")
    plano = Column(String(50), default="BASICO")
    max_usuarios = Column(Integer, default=10)
    ativo = Column(Boolean, default=True)

class Usuario(BaseCore):
    __tablename__ = "usuarios"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    nome = Column(String(255), nullable=False)
    username = Column(String(100), index=True, nullable=False)
    senha_hash = Column(String(255), nullable=False)
    papel = Column(String(30), default="OPERADOR")
    codigo = Column(String(20), nullable=True)
    ativo = Column(Boolean, default=True)

class Produto(BaseSalto):
    __tablename__ = "produtos"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    nome = Column(String(255), nullable=False)
    preco_venda = Column(Numeric(10, 2), default=0)
    estoque_atual = Column(Numeric(10, 3), default=0)
    ativo = Column(Boolean, default=True)

class Venda(BaseSalto):
    __tablename__ = "vendas"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    total = Column(Numeric(10, 2), default=0)
```

```

    valor_recebido = Column(Numeric(10, 2), default=0)
    troco = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(String(50))
    status = Column(String(20), default="CONCLUIDA")
    data_venda = Column(DateTime, default=datetime.utcnow)

class VendaItem(BaseSalto):
    __tablename__ = "venda_itens"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    venda_id = Column(Integer, nullable=False)
    produto_id = Column(Integer, nullable=False)
    produto_nome = Column(String(255))
    quantidade = Column(Numeric(10, 3), nullable=False)
    preco_unitario = Column(Numeric(10, 2), nullable=False)
    subtotal = Column(Numeric(10, 2), nullable=False)

class HistoricoVenda(BaseSalto):
    __tablename__ = "historico_vendas"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    venda_id = Column(Integer, nullable=True)
    numero_pedido = Column(String(50), index=True)
    usuario_nome = Column(String(255), nullable=True)
    valor_total = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(String(50), nullable=True)
    itens_json = Column(Text, nullable=True)
    status = Column(String(20), default="CONCLUIDA")
    tipo_operacao = Column(String(30), default="VENDA")
    data_criacao = Column(DateTime, default=datetime.utcnow)
    motivo_cancelamento = Column(Text, nullable=True)

class CaixaSessao(BaseSalto):
    __tablename__ = "caixa_sesoes"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    operador = Column(String(255), nullable=False)
    status = Column(String(20), default="ABERTO")
    saldo_inicial = Column(Numeric(10, 2), default=0)

class CaixaTransacao(BaseSalto):
    __tablename__ = "caixa_transacoes"
    id = Column(Integer, primary_key=True, autoincrement=True)
    empresa_documento = Column(String(20), index=True, nullable=False)
    sessao_id = Column(Integer, nullable=False)
    tipo = Column(String(20), nullable=False)
    valor = Column(Numeric(10, 2), nullable=False)
    descricao = Column(String(255), nullable=True)
    operador = Column(String(255), nullable=True)

BaseCore.metadata.create_all(bind=engine_core)
BaseSalto.metadata.create_all(bind=engine_salto)
SessionCore = sessionmaker(bind=engine_core)
SessionSalto = sessionmaker(bind=engine_salto)

from historico import registrar_venda, registrar_cancelamento

class C:
    V='\033[92m';A='\033[93m';R='\033[91m';AZ='\033[94m'
    RO='\033[95m';CI='\033[96m';N='\033[1m';F='\033[0m'

def log(m,p=C.CI): print(f"{p}{m}{C.F}"); sys.stdout.flush()

def cnpj():
    b=''.join([str(random.randint(0,9)) for _ in range(12)])
    def d(b,p):
        s=sum(int(x)*y for x,y in zip(b,p));r=s%11;return '0' if r<2 else str(11-r)
    return f"{b[2:]}{b[2:5]}{b[5:8]}/{b[8:12]}-{d(b,[5,4,3,2,9,8,7,6,5,4,3,2])}{d(b+d(b,[5,4,3,2,9,8,7,6,5,4,3,2]),[6,5,4,3,2,9,8,7,6,5,4,3,2])}"

def hsh(s): return hashlib.sha256(s.encode()).hexdigest()

PAPEIS=['ADMIN','GERENTE','CAIXA','VENDEDOR','OPERADOR','FINANCEIRO','ESTOQUE','SUPORTE']
PAG=['Dinheiro','Cartao Credito','Cartao Debito','PIX']
R={'e':0,'u':0,'p':0,'v':0,'h':0,'ca':0,'er':[],'ta':0,'tf':0}

log(f"\n{' '*60}\nCORUMBA SISTEMAS - TESTE COMPLETO (SQLite)\n{' '*60}")
t0=time.time()
db_c=SessionCore(); db_s=SessionSalto()

log("\n[1/4] 100 empresas...",C.AZ)
t=time.time()
for i in range(100):
    try:
        cn=cnpj()
        db_c.add(Empresa(documento=cn,razao_social=f"Empresa {i+1} LTDA",nome_fantasia=f"Emp
{i+1}",plano=random.choice(['BASICO','PRO','PREMIUM'])))
        db_c.add(Licenca(empresa_documento=cn,chave=f"CORUMBA-{''.join(random.choices(string.ascii_uppercase+string.digits,k=4))}-
{''.join(random.choices(string.ascii_uppercase+string.digits,k=4))}-
{''.join(random.choices(string.ascii_uppercase+string.digits,k=4))}",status="ATIVA"))
        if (i+1)%25==0: db_c.commit()
    except Exception as e: db_c.rollback(); R['er'].append(f"E{i}:{e}")
db_c.commit(); R['e']=db_c.query(Empresa).count()
log(f" OK {R['e']} empresas em {time.time()-t:.1f}s",C.V)

```

```

log("\n[2/4] 1000 usuarios...",C.AZ)
t=time.time()
for em in db_c.query(Empresa).all():
    try:
        for u in range(10):
            db_c.add(Usuario(empresa_documento=em.documento,nome=f"User{u+1}",username=f"u{em.id:03d}{u:02d}",senha_hash=hsh(f"s{u+1:04d}"),papel=PAPEIS[u%8],codigo=f"u+1:04d"))
            db_c.commit()
        except Exception as e: db_c.rollback(); R['er'].append(f"U{e}:{e}")
R['u']=db_c.query(Usuario).count()
log(f" OK {R['u']} usuarios em {time.time()-t:.1f}s",C.V)

log("\n[3/4] 10000 produtos...",C.AZ)
t=time.time()
for em in db_c.query(Empresa).all():
    try:
        for p in range(100):
            db_s.add(Produto(empresa_documento=em.documento,nome=f"Prod {p+1}",preco_venda=round(random.uniform(1.99,999.99),2),estoque_atual=random.randint(0,500))
            db_s.commit()
        except Exception as e: db_s.rollback(); R['er'].append(f"P{e}:{e}")
R['p']=db_s.query(Produto).count()
log(f" OK {R['p']} produtos em {time.time()-t:.1f}s",C.V)

log("\n[4/4] 10000 vendas + historico...",C.AZ)
t=time.time()
for em in db_c.query(Empresa).all():
    try:
        cx=CaixaSessao(empresa_documento=em.documento,operador="admin",saldo_inicial=100,status="ABERTO")
        db_s.add(cx); db_s.flush()
        for v in range(100):
            ni=random.randint(1,5);itens=[];tot=0
            for _ in range(ni):
                pr=round(random.uniform(5,200),2);q=random.randint(1,3);st=pr*q;tot+=st
                itens.append({'produto_id':random.randint(1,10000),'produto_nome':f"P{random.randint(1,100)}",'quantidade':q,'preco_unitario':pr,'subtotal':st})
            vr=Venda(empresa_documento=em.documento,total=tot,valor_recebido=tot+random.uniform(0,20),forma_pagamento=random.choice(PAG),status="CONCLUIDA")
            db_s.add(vr); db_s.flush()
            for it in itens:
                db_s.add(VendaItem(empresa_documento=em.documento,venda_id=vr.id,produto_id=it['produto_id'],produto_nome=it['produto_nome'],quantidade=it['quantidade']))
            db_s.add(CaixaTransacao(empresa_documento=em.documento,sessao_id=cx.id,tipo="VENDA",valor=tot,descricao=f"V#{vr.id}",operador="admin"))

        registrar_venda(db=db_s,empresa_documento=em.documento,usuario_id=1,usuario_nome="Admin",usuario_papel="ADMIN",venda_id=vr.id,itens=itens,pagamento=vr.status)
        if random.random()<0.1:
            vr.status="CANCELADA"

        registrar_cancelamento(db=db_s,empresa_documento=em.documento,venda_id=vr.id,usuario_id=1,usuario_nome="Admin",motivo=random.choice(["Cliente desistiu","Produto indisponivel","Erro pagamento"]))
        R['ca']+=1
        db_s.commit()
        except Exception as e: db_s.rollback(); R['er'].append(f"V{e}:{e}");continue
R['v']=db_s.query(Venda).count();R['h']=db_s.query(HistoricoVenda).count()
log(f" OK {R['v']} vendas em {time.time()-t:.1f}s",C.V)
log(f" OK {R['h']} registros de historico",C.V)
log(f" OK {R['ca']} cancelamentos",C.V)

log(f"\n{' '*60}\nTESTES DE ISOLAMENTO MULTI-TENANT\n{' '*60}")
e1,e2=db_c.query(Empresa).limit(2).all()

v1=db_s.query(Venda).filter(Venda.empresa_documento==e1.documento).count()
v2=db_s.query(Venda).filter(Venda.empresa_documento==e2.documento).count()
if v1>0 and v2>0: log(f" OK Vendas isoladas: E1={v1}, E2={v2}",C.V);R['ta']+=1
else: log(f" FAIL Vendas: E1={v1}, E2={v2}",C.R);R['tf']+=1

h1=db_s.query(HistoricoVenda).filter(HistoricoVenda.empresa_documento==e1.documento).count()
h2=db_s.query(HistoricoVenda).filter(HistoricoVenda.empresa_documento==e2.documento).count()
if h1>0 and h2>0: log(f" OK Historico isolado: E1={h1}, E2={h2}",C.V);R['ta']+=1
else: log(f" FAIL Historico: E1={h1}, E2={h2}",C.R);R['tf']+=1

u1=db_c.query(Usuario).filter(Usuario.empresa_documento==e1.documento,Usuario.codigo=="0001").count()
u2=db_c.query(Usuario).filter(Usuario.empresa_documento==e2.documento,Usuario.codigo=="0001").count()
if u1>0 and u2>0: log(f" OK Codigo 0001 existe em ambas empresas",C.V);R['ta']+=1
else: log(f" INFO Codigo duplicado nao encontrado",C.A)

p1=db_s.query(Produto).filter(Produto.empresa_documento==e1.documento).count()
p2=db_s.query(Produto).filter(Produto.empresa_documento==e2.documento).count()
if p1==p2==100: log(f" OK Produtos isolados: E1={p1}, E2={p2}",C.V);R['ta']+=1
else: log(f" FAIL Produtos: E1={p1}, E2={p2}",C.R);R['tf']+=1

sd=db_s.execute(text("SELECT empresa_documento, COUNT(*) as c FROM historico_vendas GROUP BY empresa_documento ORDER BY c DESC LIMIT 5")).fetchall()
if sd: log(f" OK Historico por empresa: {len(sd)} empresas",C.V);R['ta']+=1
else: log(f" FAIL Sem registros de historico",C.R);R['tf']+=1

hc=db_s.query(HistoricoVenda).filter(HistoricoVenda.status=="CONCLUIDA").count()
hca=db_s.query(HistoricoVenda).filter(HistoricoVenda.status=="CANCELADA").count()
if hc>0 and hca>0: log(f" OK Consistencia: {hc} concluidos, {hca} cancelados",C.V);R['ta']+=1
else: log(f" FAIL Consistencia",C.R);R['tf']+=1

tt=time.time()-t0
log(f"\n{' '*60}\nRELATORIO FINAL\n{' '*60}")
log(f"Empresas: {R['e']} | Usuarios: {R['u']} | Produtos: {R['p']} | Vendas: {R['v']}")
log(f"Historico: {R['h']} | Cancelamentos: {R['ca']}")

```

```
log(f"Testes aprovados: {R['ta']} | Falhas: {R['tf']}")
log(f"Erros: {len(R['er'])} | Tempo: {tt:.1f}s")
tp=len(R['er'])+R['tf']
if tp==0: log(f"\nSISTEMA 100% FUNCIONAL - PRONTO PARA PRODUCAO",C.V)
else: log(f"\n{tp} PROBLEMAS ENCONTRADOS",C.R)
for e in R['er'][:5]: log(f" - {e}",C.R)
log(f"\n{' '*60}")
db_c.close(); db_s.close()
```

6. Backend - Templates e Static

 painel.html

backendPy/templates/painel.html • 100 linhas

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Painel Admin - Corumbá Sistemas</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
</head>
<body class="bg-slate-50 font-sans antialiased text-slate-800">

  <div class="flex h-screen overflow-hidden">

    <aside class="w-64 bg-slate-900 text-white flex flex-col shadow-xl">
      <div class="p-6 flex items-center justify-center border-b border-slate-700">
        <span class="text-2xl font-bold text-emerald-400"><i class="fa-solid fa-bolt mr-2"></i>Corumbá</span>
      </div>

      <nav class="flex-1 px-4 py-6 space-y-2">
        <a href="#" class="flex items-center px-4 py-3 bg-blue-600 text-white rounded-lg transition-colors">
          <i class="fa-solid fa-chart-line w-6"></i>
          <span class="font-medium">Dashboard</span>
        </a>
        <a href="#" class="flex items-center px-4 py-3 text-slate-300 hover:bg-slate-800 hover:text-white rounded-lg transition-colors">
          <i class="fa-solid fa-users-gear w-6"></i>
          <span class="font-medium">Usuários e Permissões</span>
        </a>
        <a href="#" class="flex items-center px-4 py-3 text-slate-300 hover:bg-slate-800 hover:text-white rounded-lg transition-colors">
          <i class="fa-solid fa-key w-6"></i>
          <span class="font-medium">Licenças (Máquinas)</span>
        </a>
        <a href="#" class="flex items-center px-4 py-3 text-slate-300 hover:bg-slate-800 hover:text-white rounded-lg transition-colors">
          <i class="fa-solid fa-store w-6"></i>
          <span class="font-medium">Lojas Clientes</span>
        </a>
      </nav>

      <div class="p-4 border-t border-slate-700">
        <div class="flex items-center gap-3">
          <div class="w-10 h-10 rounded-full bg-slate-700 flex items-center justify-center text-emerald-400 font-bold">
            AD
          </div>
          <div>
            <p class="text-sm font-medium">Administrador</p>
            <p class="text-xs text-slate-400">Master</p>
          </div>
        </div>
      </div>
    </aside>

    <main class="flex-1 relative overflow-y-auto focus:outline-none">
      <div class="p-8">
        <h1 class="text-3xl font-bold text-slate-800 mb-6">Visão Geral do Sistema</h1>

        <div class="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
          <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
            <div class="flex items-center justify-between mb-4">
              <h3 class="text-slate-500 font-medium">Lojas Ativas</h3>
              <span class="flex items-center justify-center w-8 h-8 rounded-full bg-blue-100 text-blue-600">
                <i class="fa-solid fa-building"></i>
              </span>
            </div>
            <p class="text-3xl font-bold text-slate-800">1</p>
          </div>

          <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
            <div class="flex items-center justify-between mb-4">
              <h3 class="text-slate-500 font-medium">Licenças Emitidas</h3>
              <span class="flex items-center justify-center w-8 h-8 rounded-full bg-emerald-100 text-emerald-600">
                <i class="fa-solid fa-key"></i>
              </span>
            </div>
            <p class="text-3xl font-bold text-slate-800">1</p>
          </div>

          <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
            <div class="flex items-center justify-between mb-4">
              <h3 class="text-slate-500 font-medium">Usuários Totais</h3>
              <span class="flex items-center justify-center w-8 h-8 rounded-full bg-purple-100 text-purple-600">
                <i class="fa-solid fa-users"></i>
              </span>
            </div>
            <p class="text-3xl font-bold text-slate-800">1</p>
          </div>
        </div>
      </div>
    </main>
  </div>
```

```

        <p class="text-3xl font-bold text-slate-800">1</p>
    </div>
</div>

<div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
    <h2 class="text-xl font-bold text-slate-800 mb-4">Gestão Rápida</h2>
    <p class="text-slate-600 mb-6">Selecione uma opção no menu lateral para gerenciar as permissões de acesso ao PDV Desktop,
ou criar novas chaves de licença para máquinas.</p>
    <button class="bg-slate-900 text-white px-6 py-2 rounded-lg font-medium hover:bg-slate-800 transition-colors">
        <i class="fa-solid fa-plus mr-2"></i> Nova Loja Cliente
    </button>
</div>

</div>
</main>
</div>

</body>
</html>

```

 main.py

backendPy/static/desktop/main.py • 675 linhas

```

"""
SALTO GESTAO Desktop v2.0 - ARQUIVO UNICO
Design Moderno & Clean - Paleta Corumba Sistemas
NAO precisa de tkinter! Usa Flet (Flutter-based).

Instalacao: pip install flet requests
Execucao: python main.py
"""

import flet as ft
import requests
import threading
from datetime import datetime

# =====
# PALETA DE CORES CORUMBA SISTEMAS
# =====
P      = "#0b1a30"
P80    = "#1a2d4a"
P50    = "#5a7ea8"
P20    = "#9bb5d0"
P10    = "#c5d5e8"
P5     = "#e2e8f0"
FUNDO  = "#f5f7fa"
BRANCO = "#ffffff"
TEXT0  = "#0b1a30"
TEXT080 = "#3c4a5e"
TEXT060 = "#6d7a8e"
TEXT040 = "#9eaabe"
OK     = "#22c55e"
ERRO   = "#ef4444"
ALERTA = "#f59e0b"
INFO   = "#3b82f6"
FUNDO_CARD = "#ffffff"
BORDA  = "#e2e8f0"

API_URL = "http://191.252.210.235:8080"

# =====
# CLIENTE API
# =====
class API:
    def __init__(self):
        self.base = API_URL
        self.token = None
        self.user = None
        self.h = {"Content-Type": "application/json"}

    def set_token(self, t):
        self.token = t
        self.h["Authorization"] = "Bearer " + t

    def login(self, u, p):
        try:
            r = requests.post(self.base + "/api/login", json={"username": u, "password": p}, timeout=10)
            if r.status_code == 200:
                d = r.json()
                self.set_token(d.get("access_token", ""))
                self.user = d.get("user", {})
                return {"ok": True}
            return {"ok": False, "err": r.json().get("detail", "Erro")}
        except requests.exceptions.ConnectionError:
            return {"ok": False, "err": "Sem conexao com o servidor"}
        except Exception as e:
            return {"ok": False, "err": str(e)}

    def get(self, ep):
        try:
            r = requests.get(self.base + ep, headers=self.h, timeout=15)
            return {"ok": True, "data": r.json()} if r.status_code == 200 else {"ok": False}

```

```

except:
    return {"ok": False}

def post(self, ep, data):
    try:
        r = requests.post(self.base + ep, json=data, headers=self.h, timeout=15)
        return {"ok": True} if r.status_code in (200, 201) else {"ok": False}
    except:
        return {"ok": False}

def put(self, ep, data):
    try:
        r = requests.put(self.base + ep, json=data, headers=self.h, timeout=15)
        return {"ok": True} if r.status_code == 200 else {"ok": False}
    except:
        return {"ok": False}

def delete(self, ep):
    try:
        r = requests.delete(self.base + ep, headers=self.h, timeout=15)
        return {"ok": True}
    except:
        return {"ok": False}

api = API()

# =====
# HELPER: Campo de entrada moderno
# =====
def campo(label, icon=None, pwd=False, hint=None, w=None, **kw):
    return ft.TextField(
        label=label, prefix_icon=icon, password=pwd,
        hint_text=hint, hint_style=ft.TextStyle(size=12, color=TEXT040),
        label_style=ft.TextStyle(size=11, color=TEXT060),
        text_style=ft.TextStyle(size=13, color=TEXT0),
        border_color=BORDA, focused_border_color=P, focused_border_width=2,
        border_radius=12, bgcolor=BRANCO,
        content_padding=ft.padding.only(left=16, right=16, top=12, bottom=12),
        width=w, **kw
    )

# =====
# HELPER: Botao primario
# =====
def btn(texto, icon=None, w=None, h=44, on_click=None, **kw):
    content = ft.Row([
        ft.Icon(icon, size=16, color=BRANCO) if icon else ft.Container(),
        ft.Text(texto, color=BRANCO, size=14, weight=ft.FontWeight.W_600),
    ], alignment=ft.MainAxisAlignment.CENTER, spacing=8) if icon else texto

    return ft.ElevatedButton(
        content=content, width=w, height=h, on_click=on_click,
        style=ft.ButtonStyle(
            bgcolor={ft.ControlState.DEFAULT: P, ft.ControlState.HOVERED: P80},
            color=BRANCO, shape=ft.RoundedRectangleBorder(radius=12),
            padding=ft.padding.symmetric(horizontal=24, vertical=12),
            elevation={ft.ControlState.DEFAULT: 0, ft.ControlState.HOVERED: 2},
            animation_duration=150,
        ), **kw
    )

# =====
# HELPER: Card
# =====
def card(content=None, padding=16, shadow=True, border=False, **kw):
    return ft.Container(
        content=content, bgcolor=BRANCO, border_radius=16, padding=padding,
        shadow=ft.BoxShadow(spread_radius=0, blur_radius=8,
            color=ft.Colors.with_opacity(0.06, TEXT0), offset=ft.Offset(0, 2)) if shadow else None,
        border=ft.border.all(1, BORDA) if border else None, **kw
    )

# =====
# COMPONENTE: Sidebar Item
# =====
def sidebar_item(label, icon, selected=False, on_click=None):
    return ft.Container(
        content=ft.Row([
            ft.Icon(icon, size=20, color=P if selected else TEXT060),
            ft.Container(width=4),
            ft.Text(label, size=13, weight=ft.FontWeight.W_600 if selected else ft.FontWeight.W_400,
                color=P if selected else TEXT080),
        ], spacing=12, alignment=ft.MainAxisAlignment.START),
        padding=ft.padding.symmetric(horizontal=16, vertical=10),
        bgcolor=ft.Colors.with_opacity(0.08, P) if selected else ft.Colors.TRANSPARENT,
        border_radius=12, on_click=on_click, animate=ft.Animation(150),
        margin=ft.padding.symmetric(horizontal=8, vertical=2),
    )

```



```

# =====
# COMPONENTE: Badge
# =====
def badge(texto, cor=P):
    return ft.Container(
        content=ft.Text(texto, size=10, weight=ft.FontWeight.W_600, color=cor),
        bgcolor=ft.Colors.with_opacity(0.1, cor), border_radius=9999,
        padding=ft.padding.symmetric(horizontal=8, vertical=3),
    )

# =====
# COMPONENTE: Stat Card
# =====
def stat_card(titulo, valor, icone, cor=P):
    return card(content=ft.Column([
        ft.Row([
            ft.Container(content=ft.Icon(icone, size=20, color=cor),
                bgcolor=ft.Colors.with_opacity(0.1, cor), border_radius=8, padding=8),
            ],
            ft.Container(height=4),
            ft.Text(titulo, size=11, color=TEXT060),
            ft.Text(str(valor), size=26, weight=ft.FontWeight.W_700, color=TEXT0),
        ], spacing=2), padding=16, width=200, height=90)

# =====
# TELA DE LOGIN
# =====
def tela_login(page, on_ok):
    user = campo("Usuario", icon=ft.Icons.PERSON_OUTLINED, hint="Digite seu usuario", w=360)
    pwd = campo("Senha", icon=ft.Icons.LOCK_OUTLINED, pwd=True, can_reveal_password=True, hint="Digite sua senha", w=360)
    msg = ft.Container(visible=False)

    def show(text, tipo="err"):
        cols = {"err": ERRO, "ok": OK}
        bgs = {"err": ft.Colors.with_opacity(0.08, ERRO), "ok": ft.Colors.with_opacity(0.08, OK)}
        icons = {"err": ft.Icons.ERROR_OUTLINED, "ok": ft.Icons.CHECK_CIRCLE_OUTLINED}
        msg.content = ft.Container(
            content=ft.Row([ft.Icon(icons.get(tipo), size=16, color=TEXT0), ft.Container(width=8,
                ft.Text(text, size=12, color=TEXT080)], spacing=8),
            bgcolor=bgs.get(tipo), border_radius=8, padding=ft.padding.symmetric(horizontal=12, vertical=8),
            border=ft.border.all(1, ft.Colors.with_opacity(0.15, cols.get(tipo))), w=360)
        msg.visible = True
        page.update()

    def do_login(e):
        if not user.value or not pwd.value:
            show("Preencha todos os campos"); return
        result = api.login(user.value, pwd.value)
        if result["ok"]:
            show("Login realizado!", "ok")
            page.update()
            import time; time.sleep(0.4)
            on_ok()
        else:
            show(result["err"])
            page.update()

    return ft.Container(
        alignment=ft.alignment.center, expand=True,
        gradient=ft.LinearGradient(
            begin=ft.alignment.top_center, end=ft.alignment.bottom_center,
            colors=[FUNDO, BRANCO]),
        content=card(content=ft.Column([
            # Logo
            ft.Container(
                content=ft.Column([
                    ft.Container(content=ft.Icon(ft.Icons.STORE_ROUNDED, size=40, color=P),
                        bgcolor=ft.Colors.with_opacity(0.1, P), border_radius=9999, padding=20,
                        width=80, height=80, alignment=ft.alignment.center),
                    ft.Container(height=8),
                    ft.Text("SaltoGestao", size=24, weight=ft.FontWeight.W_700, color=P),
                    ft.Text("Sistema de Gestao Empresarial", size=12, color=TEXT060),
                ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
                alignment=ft.alignment.center, padding=ft.padding.only(top=16)),
            ft.Container(height=32),
            user, ft.Container(height=8), pwd, ft.Container(height=8),
            msg, ft.Container(height=8),
            btn("Entrar", icon=ft.Icons.ARROW_FORWARD, w=360, h=48, on_click=do_login),
            ft.Container(height=16),
        ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=4),
        padding=40, w=420,
        shadow=ft.BoxShadow(spread_radius=0, blur_radius=24,
            color=ft.Colors.with_opacity(0.12, TEXT0), offset=ft.Offset(0, 8)))
    )

# =====
# TELA: DASHBOARD
# =====
def tela_dashboard(page):
    stats = ft.Row(spacing=12, wrap=True, scroll=ft.ScrollMode.AUTO)

    def load():

```

```

stats.controls.clear()
cr = api.get("/api/clientes/")
pr = api.get("/api/prodidos/")
vr = api.get("/api/vendas/")
nc = len(cr["data"]) if cr["ok"] else 0
np = len(pr["data"]) if pr["ok"] else 0
nv = len(vr["data"]) if vr["ok"] else 0
total = sum(v.get("total", 0) for v in vr["data"]) if vr["ok"] and vr["data"] else 0

for t, v, ic, c in [("Clientes", str(nc), ft.Icons.PEOPLE_OUTLINED, P),
                  ("Produtos", str(np), ft.Icons.INVENTORY_2_OUTLINED, INFO),
                  ("Vendas", str(nv), ft.Icons.POINT_OF_SALE, OK),
                  ("Faturamento", "R$ %.2f" % total, ft.Icons.ATTACH_MONEY, ALERTA)]:
    stats.controls.append(stat_card(t, v, ic, c))
page.update()

return ft.Container(
    content=ft.Column([
        ft.Container(
            content=ft.Row([
                ft.Column([
                    ft.Text("Dashboard", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                    ft.Text("Bem-vindo, " + str(api.user.get("nome", "Usuario")) + "!", size=13, color=TEXT060),
                ], spacing=4),
                ft.Row([ft.IconButton(icon=ft.Icons.REFRESH, icon_color=TEXT060,
                                     bgcolor=P5, on_click=lambda e: load(),
                                     style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8))),
                        expand=True, alignment=ft.MainAxisAlignment.END),
            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUNDO),
        stats,
        ft.Container(content=ft.Divider(height=1, color=BORDA), padding=ft.padding.symmetric(horizontal=24)),
        ft.Text("Resumo do Negocio", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
                padding=ft.padding.only(left=28, top=8)),
    ], spacing=16, expand=True),
    bgcolor=FUNDO, expand=True, on_mount=lambda e: load()
)

# =====
# TELA: CLIENTES
# =====
def tela_clientes(page):
    lista = ft.ListView(spacing=8, padding=0, expand=True)
    nome = campo("Nome completo", hint="Ex: Joao Silva")
    cpf = campo("CPF/CNPJ", hint="000.000.000-00")
    email = campo("E-mail", hint="email@exemplo.com")
    tel = campo("Telefone", hint="(62) 99999-9999")
    msg = ft.Container(visible=False)
    edit_id = {"v": None}

    def show(text, tipo="ok"):
        icons = {"ok": ft.Icons.CHECK_CIRCLE_OUTLINED, "err": ft.Icons.ERROR_OUTLINED,
                "info": ft.Icons.INFO_OUTLINED}
        bgs = {"ok": ft.Colors.with_opacity(0.08, OK), "err": ft.Colors.with_opacity(0.08, ERRO),
              "info": ft.Colors.with_opacity(0.08, INFO)}
        msg.content = ft.Container(
            content=ft.Row([ft.Text(text, size=12, color=TEXT080)], spacing=8),
            bgcolor=bgs.get(tipo), border_radius=8, padding=ft.padding.symmetric(horizontal=12, vertical=8),
            margin=ft.padding.only(bottom=8))
        msg.visible = True
        page.update()

    def load():
        lista.controls.clear()
        r = api.get("/api/clientes/")
        if r["ok"] and r["data"]:
            for c in r["data"]:
                lista.controls.append(card(
                    padding=0, shadow=False, border=True,
                    content=ft.Container(
                        content=ft.Row([
                            ft.CircleAvatar(content=ft.Text(c.get("nome", "?")[0].upper(), size=16,
                                                            weight=ft.FontWeight.W_600, color=BRANCO), bgcolor=P20, radius=20),
                            ft.Container(width=8),
                            ft.Column([
                                ft.Text(c.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=TEXT0),
                                ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=11, color=TEXT060),
                                ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=11, color=TEXT060),
                            ], spacing=2, expand=True),
                            ft.IconButton(icon=ft.Icons.EDIT_OUTLINED, icon_color=INFO,
                                         bgcolor=ft.Colors.with_opacity(0.08, INFO), tooltip="Editar",
                                         on_click=lambda e, cli=c: edit(cli),
                                         style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                            ft.IconButton(icon=ft.Icons.DELETE_OUTLINED, icon_color=ERRO,
                                         bgcolor=ft.Colors.with_opacity(0.08, ERRO), tooltip="Excluir",
                                         on_click=lambda e, cli=c: delete(cli),
                                         style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                        ], spacing=8, alignment=ft.MainAxisAlignment.START),
                        padding=ft.padding.symmetric(horizontal=16, vertical=10)))
            else:
                lista.controls.append(ft.Container(
                    content=ft.Column([
                        ft.Icon(ft.Icons.PEOPLE_OUTLINED, size=40, color=TEXT040),
                        ft.Text("Nenhum cliente cadastrado", size=14, color=TEXT060),
                    ]))

```

```

        ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
        alignment=ft.alignment.center, padding=40))
page.update()

def save(e):
    data = {"nome": nome.value or "", "cpf_cnpj": cpf.value or "", "email": email.value or "", "telefone": tel.value or ""}
    if not data["nome"]:
        show("Nome é obrigatorio", "err"); return
    if edit_id["v"]:
        r = api.put("/api/clientes/" + str(edit_id["v"]), data)
    else:
        r = api.post("/api/clientes/", data)
    if r["ok"]:
        show("Cliente salvo!", "ok")
        for f in [nome, cpf, email, tel]: f.value = ""
        edit_id["v"] = None
        load()
    else:
        show("Erro ao salvar", "err")

def edit(c):
    edit_id["v"] = c.get("id")
    nome.value = c.get("nome", "")
    cpf.value = c.get("cpf_cnpj", "")
    email.value = c.get("email", "")
    tel.value = c.get("telefone", "")
    show("Editando: " + str(c.get("nome", "")), "info")
    page.update()

def delete(c):
    def ok(e):
        if e.control.text == "Sim":
            r = api.delete("/api/clientes/" + str(c.get("id")))
            if r["ok"]: show("Excluido!", "ok"); load()
            else: show("Erro ao excluir", "err")
        dlg.open = False; page.update()
    dlg = ft.AlertDialog(
        title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
        content=ft.Text("Excluir '" + str(c.get("nome")) + "'?",
            actions=[ft.TextButton("Nao", on_click=ok),
                ft.TextButton("Sim", on_click=ok, style=ft.ButtonStyle(color=ERRO))])
    page.overlay.append(dlg); dlg.open = True; page.update()

def novo(e):
    edit_id["v"] = None
    for f in [nome, cpf, email, tel]: f.value = ""
    show("Novo cliente", "info"); page.update()

return ft.Container(
    content=ft.Column([
        ft.Container(
            content=ft.Row([
                ft.Column([ft.Text("Clientes", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                    ft.Text("Gerencie seus clientes", size=13, color=TEXT060)], spacing=4),
                ft.Row([btn("Novo", icon=ft.Icons.ADD, w=100, on_click=novo)], spacing=8,
                    expand=True, alignment=ft.MainAxisAlignment.END),
            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUNDO),
        # Form
        card(
            titulo="Formulario",
            content=ft.Column([
                ft.Row([nome, cpf], spacing=12),
                ft.Row([email, tel], spacing=12),
                ft.Row([btn("Salvar", icon=ft.Icons.SAVE_OUTLINED, w=140, on_click=save),
                    ft.OutlinedButton(text="Limpar", on_click=lambda e: novo(e),
                        style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=12),
                            side=ft.BorderSide(1.5, BORDA),
                            padding=ft.padding.symmetric(horizontal=20, vertical=12))]),
                spacing=8),
                msg,
            ], spacing=12), padding=16),
        ft.Text(" Clientes cadastrados", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
            padding=ft.padding.only(left=4, bottom=8)),
        ft.Container(content=lista, expand=True),
    ], spacing=16, expand=True),
    padding=ft.padding.all(24), bgcolor=FUNDO, expand=True, on_mount=lambda e: load()
)

# =====
# TELA: PRODUTOS
# =====
def tela_produtos(page):
    lista = ft.ListView(spacing=8, padding=0, expand=True)
    nome = campo("Nome do produto", hint="Ex: Arroz 5kg")
    preco = campo("Preco (R$)", hint="0.00")
    cat = campo("Categoria", hint="Alimentos")
    estoque = campo("Estoque", hint="0")
    msg = ft.Container(visible=False)
    edit_id = {"v": None}

    def show(text, tipo="ok"):
        icons = {"ok": ft.Icons.CHECK_CIRCLE_OUTLINED, "err": ft.Icons.ERROR_OUTLINED}
        bgs = {"ok": ft.Colors.with_opacity(0.08, OK), "err": ft.Colors.with_opacity(0.08, ERRO)}

```

```

msg.content = ft.Container(
    content=ft.Row([ft.Text(text, size=12, color=TEXT080)], spacing=8),
    bgcolor=bgs.get(tipo), border_radius=8,
    padding=ft.padding.symmetric(horizontal=12, vertical=8), margin=ft.padding.only(bottom=8))
msg.visible = True
page.update()

def load():
    lista.controls.clear()
    r = api.get("/api/produtos/")
    if r["ok"] and r["data"]:
        for p in r["data"]:
            lista.controls.append(card(
                padding=0, shadow=False, border=True,
                content=ft.Container(
                    content=ft.Row([
                        ft.Container(content=ft.Icon(ft.Icons.INVENTORY_2, size=16, color=INFO),
                                    bgcolor=ft.Colors.with_opacity(0.1, INFO), border_radius=8,
                                    padding=8, w=40, h=40, alignment=ft.alignment.center),
                        ft.Container(width=8),
                        ft.Column([
                            ft.Text(p.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=TEXT0),
                            ft.Text("R$ %.2f" % p.get("preco", 0), size=12, weight=ft.FontWeight.W_700, color=OK),
                            ft.Text(str(p.get("estoque", 0)) + " em estoque | " + str(p.get("categoria", "")), size=10, color=TEXT060),
                        ], spacing=2, expand=True),
                        ft.IconButton(icon=ft.Icons.EDIT_OUTLINED, icon_color=INFO,
                                    on_click=lambda e, pr=p: edit(pr),
                                    style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                        ft.IconButton(icon=ft.Icons.DELETE_OUTLINED, icon_color=ERRO,
                                    on_click=lambda e, pr=p: delete(pr),
                                    style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                        ], spacing=8, alignment=ft.MainAxisAlignment.START),
                padding=ft.padding.symmetric(horizontal=16, vertical=10)))
    else:
        lista.controls.append(ft.Container(
            content=ft.Column([ft.Icon(ft.Icons.INVENTORY_2_OUTLINED, size=40, color=TEXT040),
                                ft.Text("Nenhum produto", size=14, color=TEXT060)],
                                horizontal_alignment=ft.CrossAxisAlignment.CENTER),
            alignment=ft.alignment.center, padding=40))
    page.update()

def save(e):
    try:
        pr = float(preco.value or 0)
        es = int(estoque.value or 0)
    except ValueError:
        show("Preco e estoque devem ser numeros", "err"); return
    data = {"nome": nome.value or "", "preco": pr, "categoria": cat.value or "", "estoque": es}
    if not data["nome"]: show("Nome e obrigatorio", "err"); return
    if edit_id["v"]:
        r = api.put("/api/produtos/" + str(edit_id["v"]), data)
    else:
        r = api.post("/api/produtos/", data)
    if r["ok"]:
        show("Produto salvo!", "ok")
        for f in [nome, preco, cat, estoque]: f.value = ""
        edit_id["v"] = None; load()
    else:
        show("Erro ao salvar", "err")

def edit(p):
    edit_id["v"] = p.get("id"); nome.value = p.get("nome", "")
    preco.value = str(p.get("preco", "")); cat.value = p.get("categoria", "")
    estoque.value = str(p.get("estoque", "")); page.update()

def delete(p):
    def ok(e):
        if e.control.text == "Sim":
            r = api.delete("/api/produtos/" + str(p.get("id")))
            if r["ok"]: show("Excluido!", "ok"); load()
            dlg.open = False; page.update()
        dlg = ft.AlertDialog(title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
                             content=ft.Text("Excluir '" + str(p.get("nome")) + "'?",
                             actions=[ft.TextButton("Nao", on_click=ok), ft.TextButton("Sim", on_click=ok)])
        page.overlay.append(dlg); dlg.open = True; page.update()

return ft.Container(
    content=ft.Column([
        ft.Container(
            content=ft.Row([
                ft.Column([ft.Text("Produtos", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                            ft.Text("Gerencie seu estoque", size=13, color=TEXT060)], spacing=4),
                ft.Row([btn("Novo", icon=ft.Icons.ADD, w=100, on_click=lambda e: (edit_id.__setitem__("v", None), [f.__setattr__("value",
                spacing=8, expand=True, alignment=ft.MainAxisAlignment.END),
            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
            padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUND0),
            card(content=ft.Column([
                ft.Row([nome, preco], spacing=12), ft.Row([cat, estoque], spacing=12),
                ft.Row([btn("Salvar", icon=ft.Icons.SAVE_OUTLINED, w=140, on_click=save)], spacing=8),
                msg,
            ], spacing=12), padding=16),
            ft.Text("Produtos cadastrados", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
                    padding=ft.padding.only(left=4, bottom=8)),
            ft.Container(content=lista, expand=True),

```

```

], spacing=16, expand=True),
padding=ft.padding.all(24), bgcolor=FUNDO, expand=True, on_mount=lambda e: load()
)

# =====
# TELA: VENDAS
# =====
def tela_vendas(page):
    lista = ft.ListView(spacing=8, padding=0, expand=True)

    def load():
        lista.controls.clear()
        r = api.get("/api/vendas/")
        if r["ok"] and r["data"]:
            total = 0
            for v in r["data"]:
                t = v.get("total", 0); total += t
                lista.controls.append(card(
                    padding=0, shadow=False, border=True,
                    content=ft.Container(
                        content=ft.Row([
                            ft.Container(content=ft.Icon(ft.Icons.POINT_OF_SALE, size=16, color=OK),
                                bgcolor=ft.Colors.with_opacity(0.1, OK), border_radius=8,
                                padding=6, w=36, h=36, alignment=ft.alignment.center),
                            ft.Container(width=8),
                            ft.Column([
                                ft.Text("Venda #" + str(v.get("id",0)), size=13, weight=ft.FontWeight.W_600),
                                ft.Text(str(v.get("cliente_nome","N/A")), size=11, color=TEXT060),
                                ], spacing=2, expand=True),
                                ft.Text("R$ %.2f" % t, size=14, weight=ft.FontWeight.W_700, color=OK),
                                ], spacing=8, alignment=ft.MainAxisAlignment.START),
                                padding=ft.padding.symmetric(horizontal=16, vertical=10)))
                lista.controls.insert(0, card(
                    content=ft.Row([
                        ft.Text(str(len(r["data"])) + " vendas", size=12, color=TEXT060),
                        ft.Text("Total: R$ %.2f" % total, size=14, weight=ft.FontWeight.W_700, color=P),
                        ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
                        padding=16, shadow=False, bgcolor=ft.Colors.with_opacity(0.03, P),
                        border=ft.border.all(1, P20)))
            page.update()

    return ft.Container(
        content=ft.Column([
            ft.Container(
                content=ft.Column([ft.Text("Vendas", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                    ft.Text("Historico de vendas", size=13, color=TEXT060)], spacing=4),
                padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUNDO),
            ft.Container(content=lista, expand=True),
        ], spacing=16),
        padding=ft.padding.all(24), bgcolor=FUNDO, expand=True, on_mount=lambda e: load()
    )

# =====
# LAYOUT PRINCIPAL
# =====
def layout_principal():
    main = ft.Container(expand=True, bgcolor=FUNDO)
    sel = {"i": 0}

    def nav(i):
        sel["i"] = i
        telas = {0: tela_dashboard, 1: tela_clientes, 2: tela_produtos, 3: tela_vendas}
        if i in telas:
            main.content = telas[i](main.page)
            main.page.update()
        for j, item in enumerate(nav_items):
            item.bgcolor = ft.Colors.with_opacity(0.08, P) if j == i else ft.Colors.TRANSPARENT
        page.update()

    nav_items = [
        sidebar_item("Dashboard", ft.Icons.DASHBOARD, True, lambda e: nav(0)),
        sidebar_item("Clientes", ft.Icons.PEOPLE_OUTLINED, False, lambda e: nav(1)),
        sidebar_item("Produtos", ft.Icons.INVENTORY_2_OUTLINED, False, lambda e: nav(2)),
        sidebar_item("Vendas", ft.Icons.POINT_OF_SALE_OUTLINED, False, lambda e: nav(3)),
    ]

    sidebar = ft.Container(
        content=ft.Column([
            ft.Container(content=ft.Row([
                ft.Container(content=ft.Icon(ft.Icons.STORE, size=20, color=BRANCO),
                    bgcolor=P, border_radius=8, padding=8),
                ft.Container(width=6),
                ft.Column([ft.Text("SaltoGestao", size=13, weight=ft.FontWeight.W_700, color=BRANCO),
                    ft.Text("v2.0", size=8, color=P20)], spacing=0),
            ]), padding=ft.padding.symmetric(horizontal=12, vertical=16),
            margin=ft.padding.only(top=8, bottom=16)),
            ft.Column(nav_items, spacing=2),
            ft.Container(expand=True),
            ft.Container(content=ft.Text("Corumba Sistemas", size=8, color=P20),
                padding=ft.padding.all(12)),
        ], spacing=0),
        width=220, bgcolor=P, padding=ft.padding.symmetric(vertical=8))

```

```

return ft.Container(
    content=ft.Row([
        sidebar,
        ft.VerticalDivider(width=1, color=BORDA),
        main,
    ], expand=True, spacing=0),
    expand=True)

# =====
# APP PRINCIPAL
# =====
def main(page: ft.Page):
    page.title = "SaltoGestao v2.0"
    page.padding = 0
    page.window.width = 1280
    page.window.height = 800
    page.window.min_width = 1024
    page.window.min_height = 700
    page.bgcolor = FUNDO
    page.theme = ft.Theme(color_scheme_seed=P, font_family="Segoe UI", use_material3=True)

    mc = ft.Container(expand=True, bgcolor=FUNDO)

    def on_login():
        mc.content = layout_principal()
        page.update()

    mc.content = tela_login(page, on_login)
    page.add(mc)

if __name__ == "__main__":
    ft.app(target=main)

```

requirements.txt

backendPy/requirements.txt • 14 linhas

```

fastapi>=0.110.0
uvicorn[standard]>=0.29.0
sqlalchemy>=2.0.0
pydantic>=2.0.0
python-jose[cryptography]>=3.3.0
passlib>=1.7.4
python-dotenv>=1.0.0
alembic>=1.13.0
psycpg2-binary>=2.9.0
bcrypt>=4.0.0
python-multipart>=0.0.9
jinja2>=3.1.0
httpx>=0.27.0

```

7. Frontend Web - Next.js (Tiposcript/React)

layout.tsx

frontend-web/app/layout.tsx • 34 linhas

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "../globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "Create Next App",
  description: "Generated by create next app",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    

```

page.tsx

frontend-web/app/page.tsx • 9 linhas

```
export default function Home() {
  return (
    <div className="flex min-h-screen items-center justify-center bg-gray-100">
      <h1 className="text-4xl font-bold text-blue-600">
        Salto Gestão 2.0 - Front-end Iniciado
      </h1>
    </div>
  );
}
```

page.tsx

frontend-web/app/login/page.tsx • 110 linhas

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState('');

  const handleLogin = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    setIsLoading(true);
    setError('');

    try {
      const response = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
      });

      if (response.ok) {
        const data = await response.json();
        localStorage.setItem('corumba_token', data.access_token);
        localStorage.setItem('corumba_cliente', JSON.stringify(data.cliente));
        window.location.href = '/dashboard';
      } else {
        const errorData = await response.json();
        setError(errorData.error || 'Erro no login.');
```

```

    }
  } catch {
    setError('Erro ao conectar com o servidor.');
```

```

  } finally {
    setIsLoading(false);
  }
};
```

```

return (
```

```

  <div className="min-h-screen flex items-center justify-center bg-gray-50">
    <div className="max-w-md w-full p-10 bg-white rounded-xl shadow-lg border border-gray-100">
```

```

      {/* Logo */}
```

```

      <div className="flex flex-col items-center mb-10">
```

```

        
```

```

        <p className="text-sm text-[#0b1a30] tracking-widest uppercase font-medium">
```

```

          Portal do Cliente
```

```

        </p>
```

```

      </div>
```

```

      {error && (
```

```

        <div className="mb-4 p-3 bg-red-50 border border-red-200 text-red-700 text-sm rounded-md">
```

```

          {error}
```

```

        </div>
```

```

      )}
```

```

    <form onSubmit={handleLogin} className="space-y-6">
```

```

      <div className="space-y-5">
```

```

        <div>
```

```

          <label className="block text-sm font-semibold text-[#0b1a30] mb-2">
```

```

            E-mail
```

```

          </label>
```

```

          <input
```

```

            type="email"
```

```

            required
```

```

            className="w-full px-4 py-3 border border-gray-300 rounded-md focus:outline-none focus:ring-2 focus:ring-[#84cc16]
```

```

            focus:border-transparent transition-all text-gray-900"
```

```

            placeholder="seu@email.com"
```

```

            value={email}
```

```

            onChange={(e) => setEmail(e.target.value)}
```

```

          />
```

```

        </div>
```

```

        <div>
```

```

          <label className="block text-sm font-semibold text-[#0b1a30] mb-2">
```

```

            Senha
```

```

          </label>
```

```

          <input
```

```

            type="password"
```

```

            required
```

```

            className="w-full px-4 py-3 border border-gray-300 rounded-md focus:outline-none focus:ring-2 focus:ring-[#84cc16]
```

```

            focus:border-transparent transition-all text-gray-900"
```

```

            placeholder="....."
```

```

            value={password}
```

```

            onChange={(e) => setPassword(e.target.value)}
```

```

          />
```

```

        </div>
```

```

      </div>
```

```

      <button
```

```

        type="submit"
```

```

        disabled={isLoading}
```

```

        className="w-full flex justify-center py-3.5 px-4 rounded-md text-sm font-bold text-[#0b1a30] bg-[#84cc16] hover:bg-[#65a30d]
```

```

        hover:text-white transition-all disabled:opacity-70 mt-4"
```

```

      >
```

```

        {isLoading ? 'Autenticando...' : 'Entrar no Paine!}'
```

```

      </button>
```

```

    </form>
```

```

    <p className="text-xs text-center text-gray-400 mt-6">
```

```

      Não tem acesso? Contate a Corumbá Sistemas.
```

```

    </p>
```

```

  </div>
```

```

</div>
```

```

);
```

```

}
```

globals.css

frontend-web/app/globals.css • 39 linhas

```
@import "tailwindcss";
```

```
:root {
```

```
  --background: #ffffff;
```

```
  --foreground: #0b1a30;
```

```
}
```

```
@theme inline {
```

```
  --color-background: var(--background);
```

```
  --color-foreground: var(--foreground);
```



```

--font-sans: var(--font-geist-sans);
--font-mono: var(--font-geist-mono);
}

/* Forçar light mode - não usar dark mode */
@media (prefers-color-scheme: dark) {
  :root {
    --background: #ffffff;
    --foreground: #0b1a30;
  }
}

body {
  background: #ffffff;
  color: #0b1a30;
  font-family: Arial, Helvetica, sans-serif;
}

/* Garantir que inputs sempre tenham texto escuro */
input, select, textarea {
  color: #0b1a30 !important;
  background-color: #ffffff !important;
}

/* Garantir labels visíveis */
label {
  color: #0b1a30 !important;
}

```

page.tsx

frontend-web/app/dashboard/page.tsx • 304 linhas

```

'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';

interface Produto {
  id: number;
  nome: string;
  descricao: string;
  preco_mensal: number;
  preco_anual: number | null;
  icone: string;
  status: 'adquirido' | 'bloqueado' | 'inadimplente';
  data_aquisicao: string | null;
  data_expiracao: string | null;
}

export default function DashboardPage() {
  const router = useRouter();
  const [produtos, setProdutos] = useState<Produto[]>([]);
  const [cliente, setCliente] = useState<{ nome: string; email: string } | null>(null);
  const [carregando, setCarregando] = useState(true);
  const [produtoSelecionado, setProdutoSelecionado] = useState<Produto | null>(null);

  useEffect(() => {
    const token = localStorage.getItem('corumba_token');
    if (!token) {
      router.push('/login');
      return;
    }

    const clienteData = localStorage.getItem('corumba_cliente');
    if (clienteData) {
      setCliente(JSON.parse(clienteData));
    }

    // Buscar produtos do painel
    fetch('/api/painel/produtos', {
      headers: { Authorization: `Bearer ${token}` },
    })
      .then((res) => {
        if (res.ok) return res.json();
        throw new Error('Erro ao carregar produtos');
      })
      .then((data) => setProdutos(data))
      .catch((err) => console.error(err))
      .finally(() => setCarregando(false));
  }, [router]);

  const handleLogout = () => {
    localStorage.removeItem('corumba_token');
    localStorage.removeItem('corumba_cliente');
    router.push('/login');
  };

  const adquiridos = produtos.filter((p) => p.status === 'adquirido');
  const bloqueados = produtos.filter((p) => p.status === 'bloqueado');
  const inadimplentes = produtos.filter((p) => p.status === 'inadimplente');

  if (carregando) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-[#f8fafc]">

```

```

    <p className="text-gray-500">Carregando...</p>
  </div>
);
}

return (
  <div className="min-h-screen bg-[#f8fafc]">
    { /* Header */ }
    <header className="bg-white border-b border-gray-200 px-8 py-5">
      <div className="max-w-6xl mx-auto flex justify-between items-center">
        
        <div className="flex items-center gap-4">
          {cliente && (
            <div className="text-right">
              <p className="text-sm font-semibold text-[#0b1a30]">{cliente.nome}</p>
              <p className="text-xs text-gray-500">{cliente.email}</p>
            </div>
          )}
          <button
            onClick={handleLogout}
            className="text-sm text-gray-500 hover:text-red-600 transition-colors"
          >
            Sair
          </button>
        </div>
      </div>
    </header>

    { /* Conteúdo */ }
    <main className="max-w-6xl mx-auto px-8 py-10">
      { /* Título */ }
      <div className="mb-8">
        <h2 className="text-3xl font-bold text-[#0b1a30]">Nossas Soluções</h2>
        <p className="mt-2 text-gray-500">
          Gerencie seus produtos e adquira novos serviços
        </p>
      </div>

      { /* Produtos Adquiridos */ }
      {adquiridos.length > 0 && (
        <div className="mb-10">
          <h3 className="text-lg font-bold text-[#0b1a30] mb-4 flex items-center gap-2">
            <span className="w-3 h-3 rounded-full bg-green-500"></span>
            Seus Produtos
          </h3>
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            {adquiridos.map((p) => (
              <div
                key={p.id}
                className="bg-white p-6 rounded-xl shadow-sm border border-green-200 flex flex-col"
              >
                <div className="flex items-start justify-between mb-4">
                  <span className="text-4xl">{p.icone}</span>
                  <span className="bg-green-100 text-green-700 text-xs font-bold px-3 py-1 rounded-full uppercase">
                    Ativo
                  </span>
                </div>
                <h4 className="text-xl font-bold text-[#0b1a30] mb-2">{p.nome}</h4>
                <p className="text-gray-500 text-sm mb-4 flex-grow line-clamp-3">
                  {p.descricao}
                </p>
                <div className="flex items-center justify-between mt-auto pt-4 border-t border-gray-100">
                  <span className="text-lg font-bold text-[#0b1a30]">
                    R$ {p.preco_mensal.toFixed(2).replace('.', ',')}
                  <span className="text-xs text-gray-400 font-normal">/mês</span>
                </div>
                <button
                  onClick={() => {
                    if (p.nome === 'SaltoGestão') {
                      router.push('/dashboard/salto-gestao');
                    } else if (p.nome === 'MesaPronta') {
                      router.push('/dashboard/mesapronta');
                    } else if (p.nome === 'ConnectFood') {
                      alert('ConnectFood é um módulo integrado ao MesaPronta.\n\nAcesse pelo botão "🚚 Delivery" dentro do MesaPronta.\n\nReceba pedidos do iFood e 99Food direto no seu painel!');
                    }
                  }}
                  className="bg-[#84cc16] text-[#0b1a30] font-bold text-sm py-2 px-4 rounded-md hover:bg-[#65a30d] hover:text-white transition-colors"
                >
                  Acessar
                </button>
              </div>
            ))}
          </div>
        </div>
      )}

      { /* Produtos Bloqueados */ }
      {bloqueados.length > 0 && (

```

```

<div className="mb-10">
  <h3 className="text-lg font-bold text-[#0b1a30] mb-4 flex items-center gap-2">
    <span className="w-3 h-3 rounded-full bg-gray-400"></span>
    Disponíveis para Aquisição
  </h3>
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
    {bloqueados.map((p) => (
      <div
        key={p.id}
        className="bg-white p-6 rounded-xl shadow-sm border border-gray-200 flex flex-col opacity-80 hover:opacity-100 transition-
opacity"
      >
        <div className="flex items-start justify-between mb-4">
          <span className="text-4xl grayscale">{p.icone}</span>
          <span className="bg-gray-100 text-gray-500 text-xs font-bold px-3 py-1 rounded-full uppercase">
            Bloqueado
          </span>
        </div>
        <h4 className="text-xl font-bold text-[#0b1a30] mb-2">{p.nome}</h4>
        <p className="text-gray-400 text-sm mb-4 flex-grow line-clamp-3">
          {p.descricao}
        </p>
        <div className="flex items-center justify-between mt-auto pt-4 border-t border-gray-100">
          <span className="text-lg font-bold text-gray-400">
            R$ {p.preco_mensal.toFixed(2).replace('.', ',')}
          <span className="text-xs text-gray-400 font-normal">/mês</span>
        </div>
        <button
          onClick={() => setProdutoSelecioneado(p)}
          className="bg-[#0b1a30] text-white font-bold text-sm py-2 px-4 rounded-md hover:bg-slate-800 transition-colors"
        >
          Ver Detalhes
        </button>
      </div>
    ))}
  </div>
</div>
))}

```

```

/* Inadimplentes */
{inadimplentes.length > 0 && (
  <div className="mb-10">
    <h3 className="text-lg font-bold text-red-600 mb-4 flex items-center gap-2">
      <span className="w-3 h-3 rounded-full bg-red-500"></span>
      Pagamento Pendente
    </h3>
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
      {inadimplentes.map((p) => (
        <div
          key={p.id}
          className="bg-white p-6 rounded-xl shadow-sm border border-red-200 flex flex-col"
        >
          <div className="flex items-start justify-between mb-4">
            <span className="text-4xl">{p.icone}</span>
            <span className="bg-red-100 text-red-700 text-xs font-bold px-3 py-1 rounded-full uppercase">
              Inadimplente
            </span>
          </div>
          <h4 className="text-xl font-bold text-[#0b1a30] mb-2">{p.nome}</h4>
          <p className="text-gray-500 text-sm mb-4 flex-grow">
            Acesso bloqueado por inadimplência. Entre em contato com a Corumbá Sistemas para regularizar.
          </p>
          <button
            onClick={() => setProdutoSelecioneado(p)}
            className="w-full bg-red-600 text-white font-bold text-sm py-2 px-4 rounded-md hover:bg-red-700 transition-colors mt-4"
          >
            Regularizar Acesso
          </button>
        </div>
      ))}
    </div>
  </div>
))}

```

```

/* Modal de Detalhes do Produto */
{produtoSelecioneado && (
  <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
    <div className="bg-white rounded-2xl shadow-2xl max-w-lg w-full p-8">
      <div className="flex items-start justify-between mb-6">
        <div className="flex items-center gap-4">
          <span className="text-5xl">{produtoSelecioneado.icone}</span>
          <div>
            <h3 className="text-2xl font-bold text-[#0b1a30]">
              {produtoSelecioneado.nome}
            </h3>
            <p className="text-sm text-gray-500">Corumbá Sistemas</p>
          </div>
        </div>
        <button
          onClick={() => setProdutoSelecioneado(null)}
          className="text-gray-400 hover:text-gray-600 text-2xl"
        >
          x
        </button>
      </div>
    </div>
  </div>
)
}

```

```

        </button>
      </div>

      <p className="text-gray-600 mb-6 leading-relaxed">
        {produtoSelecionado.descricao}
      </p>

      <div className="bg-gray-50 rounded-lg p-4 mb-6">
        <div className="flex justify-between items-center mb-2">
          <span className="text-gray-500">Mensalidade</span>
          <span className="text-2xl font-bold text-[#0b1a30]">
            R$ {produtoSelecionado.preco_mensal.toFixed(2).replace('.', ',')}
          </span>
        </div>
        {produtoSelecionado.preco_anual && (
          <div className="flex justify-between items-center">
            <span className="text-gray-500">Plano Anual (com desconto)</span>
            <span className="text-lg font-bold text-green-600">
              R$ {produtoSelecionado.preco_anual.toFixed(2).replace('.', ',')}
              <span className="text-xs text-gray-400 font-normal">/ano</span>
            </span>
          </div>
        )}
      </div>

      <div className="flex gap-3">
        <button
          onClick={() => setProdutoSelecionado(null)}
          className="flex-1 py-3 border border-gray-300 text-gray-700 font-bold rounded-md hover:bg-gray-50 transition-colors"
        >
          Fechar
        </button>
        <button
          onClick={() => {
            alert('Para adquirir este produto, entre em contato com a Corumbá Sistemas:\n\ncontato@corumbasistemas.com.br\n(62) 99999-9999');
          }}
          className="flex-1 py-3 bg-[#84cc16] text-[#0b1a30] font-bold rounded-md hover:bg-[#65a30d] hover:text-white transition-colors"
        >
          Solicitar Contratação
        </button>
      </div>
    </div>
  </div>
)
}

/* Footer */
<footer className="bg-[#0b1a30] text-white py-8 text-center mt-auto">
  <h4 className="font-bold mb-2">Corumbá Sistemas</h4>
  <p className="text-sm text-gray-400">contato@corumbasistemas.com.br</p>
  <p className="text-sm text-gray-400">Goiânia, GO</p>
</footer>
</div>
);
}

```

```

"page.tsx"                                frontend-web/app/dashboard/mesapronta/page.tsx • 347 linhas

"use client";

import { useState, useEffect, useCallback } from "react";
import { useRouter } from "next/navigation";
import Image from "next/image";

const API_URL = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8080";

export default function MesaProntaDashboard() {
  const router = useRouter();
  const [activeTab, setActiveTab] = useState("visao-geral");
  const [documentoCliente, setDocumentoCliente] = useState("00.000.000/0001-00");
  const [nomeEmpresa, setNomeEmpresa] = useState("");
  const [funcionarios, setFuncionarios] = useState<any[]>([]);
  const [novoFuncionario, setNovoFuncionario] = useState({ nome: "", username: "", senha: "", papel: "GARCOM" });
  const [isSaving, setIsSaving] = useState(false);
  const [isLoadingUsers, setIsLoadingUsers] = useState(false);
  const [msgSucesso, setMsgSucesso] = useState("");
  const [licencaInfo, setLicencaInfo] = useState<any>(null);

  useEffect(() => {
    const token = localStorage.getItem("corumba_token");
    if (!token) { router.push("/login"); return; }
    const cnpjSalvo = localStorage.getItem("cnpj_cliente");
    if (cnpjSalvo) setDocumentoCliente(cnpjSalvo);
    const clienteData = localStorage.getItem("corumba_cliente");
    if (clienteData) {
      const c = JSON.parse(clienteData);
      if (c.nome) setNomeEmpresa(c.nome);
    }
  }, [router]);

  const cnpjLimpo = documentoCliente.replace(/[.\-\/]/g, "");

  const carregarFuncionarios = useCallback(async () => {

```

```

setIsLoadingUsers(true);
try {
  const res = await fetch(`${API_URL}/api/cliente/${cnpjLimpo}/funcionarios`);
  if (res.ok) {
    const data = await res.json();
    setFuncionarios(data.funcionarios || []);
  }
} catch (error) {
  console.error("Erro ao carregar funcionarios:", error);
} finally {
  setIsLoadingUsers(false);
}
}, [cnpjLimpo]);

const carregarLicenca = useCallback(async () => {
  try {
    const res = await fetch(`${API_URL}/api/licenca/validar`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ cnpj: documentoCliente }),
    });
    if (res.ok) setLicencaInfo(await res.json());
  } catch (e) { /* silencioso */ }
}, [documentoCliente]);

useEffect(() => {
  if (activeTab === "equipe") carregarFuncionarios();
  if (activeTab === "visao-geral") carregarLicenca();
}, [activeTab, carregarFuncionarios, carregarLicenca]);

const handleCriarFuncionario = async (e: React.FormEvent) => {
  e.preventDefault();
  setIsSaving(true);
  setMsgSucesso("");
  try {
    const res = await fetch(`${API_URL}/api/cliente/${cnpjLimpo}/funcionarios`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(novoFuncionario),
    });
    if (res.ok) {
      setMsgSucesso(`Funcionario "${novoFuncionario.nome}" cadastrado com sucesso!`);
      setNovoFuncionario({ nome: "", username: "", senha: "", papel: "GARCOM" });
      carregarFuncionarios();
      setTimeout(() => setMsgSucesso(""), 4000);
    } else {
      const err = await res.json();
      alert(err.erro || "Erro ao cadastrar funcionario.");
    }
  } catch (error: any) {
    alert("Erro de conexao: " + error.message);
  } finally {
    setIsSaving(false);
  }
};

const handleDeletarFuncionario = async (id: number, nome: string) => {
  if (!confirm(`Deseja remover o acesso de ${nome}?`)) return;
  try {
    const res = await fetch(`${API_URL}/api/cliente/${cnpjLimpo}/funcionarios/${id}`, { method: "DELETE" });
    if (res.ok) carregarFuncionarios();
    else alert("Nao foi possivel remover.");
  } catch (error) { alert("Erro de conexao."); }
};

return (
  <div className="flex h-screen bg-[#f8fafc] font-sans text-slate-800 overflow-hidden">
    <aside className="w-64 bg-[#0b1a30] text-white flex flex-col justify-between shadow-xl shrink-0">
      <div>
        <div className="p-6 border-b border-slate-800">
          <div className="relative w-full h-16">
            <Image src="/logo-corumba.png" alt="Corumba Sistemas" fill className="object-contain object-left filter brightness-0 invert"
priority />
          </div>
          <p className="text-[10px] text-orange-400 font-bold uppercase tracking-widest mt-2 px-1">MesaPronta – Gestao</p>
        </div>
        <nav className="px-4 py-6 space-y-1.5">
          {[ "visao-geral", "equipe", "delivery" ].map((tab) => (
            <button key={tab} onClick={() => setActiveTab(tab)}
              className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
                activeTab === tab ? "bg-orange-500/10 text-orange-400 border-l-4 border-orange-500" : "text-slate-300 hover:bg-slate-800/50
hover:text-white"
            }`}>
              <span className="text-lg">{tab === "visao-geral" ? "\u{1F4CA}" : tab === "equipe" ? "\u{1F465}" : "\u{1F6F5}"}</span>
              {tab === "visao-geral" ? "Visao Geral" : tab === "equipe" ? "Gestao de Equipe" : "ConnectFood"}
              {tab === "delivery" && <span className="text-[9px] bg-orange-500/20 text-orange-300 px-1.5 py-0.5 rounded ml-auto">Breve</
span>}
            </button>
          ))}
        </nav>
      </div>
      <div className="p-4 border-t border-slate-800">
        <button onClick={() => router.push("/dashboard")} className="w-full flex items-center gap-2 px-4 py-3 text-sm text-slate-400
hover:text-white font-medium transition-colors">
          <span>\u{1F6AA}</span> Voltar ao Hub

```

```

    </button>
  </div>
</aside>

<div className="flex-1 flex flex-col overflow-hidden">
  <header className="bg-white border-b border-gray-200 px-8 py-5 flex justify-between items-center shrink-0">
    <div>
      <h2 className="text-xl font-bold text-[#0b1a30]">
        {activeTab === "visao-geral" && "Visao Geral do MesaPronta"}
        {activeTab === "equipe" && "Gestao de Equipe"}
        {activeTab === "delivery" && "ConnectFood - Delivery"}
      </h2>
      {nomeEmpresa && <p className="text-xs text-gray-400 mt-0.5">{nomeEmpresa} - {documentoCliente}</p>}
    </div>
    <div className="flex items-center gap-2 text-xs font-bold text-green-700 bg-green-50 px-4 py-2 rounded-full">
      <span className="w-2 h-2 rounded-full bg-green-500 animate-pulse"></span>
      Licenca {licencaInfo?.status || "Ativa"}
    </div>
  </header>

  <main className="flex-1 overflow-y-auto p-8">
    {activeTab === "visao-geral" && (
      <div className="space-y-6 max-w-6xl mx-auto">
        <div className="bg-gradient-to-r from-[#0b1a30] to-[#1e3a5f] rounded-xl p-8 flex flex-col md:flex-row justify-between items-start md:items-center gap-6 shadow-lg">
          <div className="text-white max-w-2xl">
            <div className="flex items-center gap-3 mb-3">
              <span className="text-4xl">\u{1F37D}\uFE0F</span>
              <div>
                <h3 className="text-xl font-bold">MesaPronta - Corumba Food</h3>
                <p className="text-xs text-slate-400">Sistema de gestao para restaurantes, pizzarias e bares</p>
              </div>
            </div>
            <p className="text-slate-300 text-sm leading-relaxed">
              Controle completo: mesas, pedidos, cardapio, equipe e entregas. O aplicativo desktop e instalado no PC do restaurante e sincroniza com nossos servidores.
            </p>
          </div>
          <div className="flex flex-col gap-3 shrink-0">
            <button className="bg-[#84cc16] hover:bg-[#65a30d] text-[#0b1a30] font-bold py-3.5 px-6 rounded-md transition-all shadow-lg flex items-center gap-2">
              <span>\u{1F4E5}</span> Download para Windows
            </button>
            <button className="bg-orange-500/20 hover:bg-orange-500/30 text-orange-300 font-bold text-sm py-2 px-4 rounded-md transition-all flex items-center gap-2">
              <span>\u{1F4F1}</span> App Garcom (Em breve)
            </button>
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
            <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Equipe Ativa</p>
            <p className="text-3xl font-extrabold text-[#0b1a30]">{funcionarios.length}</p>
            <span className="text-xs text-green-600 font-semibold mt-1 inline-block">{funcionarios.filter(f => f.ativo).length} com acesso</span>
          </div>
          <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
            <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Sistema</p>
            <p className="text-lg font-extrabold text-[#0b1a30]">Desktop</p>
            <span className="text-xs text-blue-600 font-semibold mt-1 inline-block">Windows, JavaFX</span>
          </div>
          <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
            <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Banco de Dados</p>
            <p className="text-lg font-extrabold text-[#0b1a30]">Local + Nuvem</p>
            <span className="text-xs text-orange-600 font-semibold mt-1 inline-block">SQLite + PostgreSQL</span>
          </div>
        </div>

        <div className="bg-white rounded-xl border border-gray-200 p-8 shadow-sm">
          <h3 className="text-lg font-bold text-[#0b1a30] mb-4">\u{1F4CB} Sobre o MesaPronta</h3>
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            <div className="space-y-3">
              {[
                ["\u{1F37D}\uFE0F", "Gestao de Mesas", "Controle em tempo real: livre, ocupada, reservada."],
                ["\u{1F4DD}", "Pedidos Rapios", "Envie pedidos para cozinha/bar com um clique."],
                ["\u{1F4B0}", "Controle de Caixa", "Abertura, fechamento, sangria. Relatorios de vendas."],
              ].map((icon, title, desc), i) => (
                <div key={i} className="flex items-start gap-3">
                  <span className="text-xl">{icon}</span>
                  <div>
                    <p className="font-semibold text-[#0b1a30]">{title}</p>
                    <p className="text-xs text-gray-500">{desc}</p>
                  </div>
                </div>
              ))}
            </div>
            <div className="space-y-3">
              {[
                ["\u{1F373}", "Cardapio Digital", "Cadastro de produtos com foto, preco, categoria."],
                ["\u{1F6F5}", "Delivery Integrado", "Receba pedidos do iFood e 99Food (modulo ConnectFood)."],
                ["\u{2601}\uFE0F", "Backup na Nuvem", "Dados sincronizados. Recupere tudo se formatar o PC."],
              ].map((icon, title, desc), i) => (
                <div key={i} className="flex items-start gap-3">
                  <span className="text-xl">{icon}</span>
                  <div>
                    <p className="font-semibold text-[#0b1a30]">{title}</p>
                    <p className="text-xs text-gray-500">{desc}</p>
                  </div>
                </div>
              ))}
            </div>
          </div>
        </div>
      )
    )}
  </main>
</div>

```

```

    </div>
  </div>
</div>

<div className="bg-orange-50 border border-orange-200 rounded-xl p-6 shadow-sm">
  <h3 className="text-lg font-bold text-[#0b1a30] mb-4">\u{1F680} Como Comer</h3>
  <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
    {[
      ["1", "Cadastre a Equipe", "Va em Gestao de Equipe e cadastre garcons, cozinheiros e caixas"],
      ["2", "Baixe o Aplicativo", "Clique em Download para Windows e instale no PC"],
      ["3", "Ative a Licenca", "Na insira o codigo de ativacao recebido"],
      ["4", "Comece a Usar", "Sua equipe faz login com os dados cadastrados aqui"],
    ]
    .map(([, num, title, desc], i) => (
      <div key={i} className="text-center">
        <div className="w-10 h-10 bg-orange-500 text-white rounded-full flex items-center justify-center font-bold mx-auto
mb-2">{num}</div>
        <p className="text-sm font-semibold text-[#0b1a30]">{title}</p>
        <p className="text-xs text-gray-500">{desc}</p>
      </div>
    ))
  </div>
</div>
</div>
))

{activeTab === "equipe" && (
  <div className="max-w-6xl mx-auto space-y-8">
    {msgSucesso && (
      <div className="bg-green-50 border border-green-200 text-green-700 px-4 py-3 rounded-lg text-sm font-semibold">{msgSucesso}</
div>
    )}
    <div className="bg-white rounded-xl border border-gray-200 shadow-sm p-6">
      <h3 className="text-lg font-bold text-[#0b1a30]">Cadastrar Funcionario</h3>
      <p className="text-xs text-gray-400 mb-4">Vinculados ao CNPJ {documentoCliente}</p>
      <form onSubmit={handleCriarFuncionario} className="grid grid-cols-1 md:grid-cols-5 gap-4 items-end">
        <div>
          <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Nome</label>
          <input type="text" required value={novoFuncionario.nome} onChange={(e) => setNovoFuncionario({...novoFuncionario, nome:
e.target.value})} className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm outline-none focus:ring-2 focus:ring-orange-500 text-
gray-900" placeholder="Joao Silva" />
        </div>
        <div>
          <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Login</label>
          <input type="text" required value={novoFuncionario.username} onChange={(e) => setNovoFuncionario({...novoFuncionario,
username: e.target.value})} className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm outline-none focus:ring-2 focus:ring-
orange-500 text-gray-900" placeholder="joao" />
        </div>
        <div>
          <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Senha</label>
          <input type="password" required value={novoFuncionario.senha} onChange={(e) => setNovoFuncionario({...novoFuncionario,
senha: e.target.value})}
className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm outline-none focus:ring-2 focus:ring-orange-500 text-gray-900"
placeholder="*****" />
        </div>
        <div>
          <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Funcao</label>
          <select value={novoFuncionario.papel} onChange={(e) => setNovoFuncionario({...novoFuncionario, papel: e.target.value})}
className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm outline-none focus:ring-2 focus:ring-orange-500 bg-white text-gray-900
font-medium">
            <option value="ADMIN">Admin</option>
            <option value="GARCOM">Garcom</option>
            <option value="CAIXA">Caixa</option>
            <option value="COZINHA">Cozinha</option>
          </select>
        </div>
        <div>
          <button disabled={isSaving} type="submit" className="w-full bg-[#0b1a30] hover:bg-slate-800 text-white font-bold text-sm
py-2.5 px-4 rounded-md transition-colors disabled:opacity-50">
            {isSaving ? "Salvando..." : "Adicionar"}
          </button>
        </div>
      </form>
    </div>

    <div className="bg-white rounded-xl border border-gray-200 shadow-sm overflow-hidden">
      <div className="px-6 py-4 border-b border-gray-200 bg-gray-50/70">
        <h4 className="text-xs font-bold text-[#0b1a30] uppercase tracking-wider">{funcionarios.length} funcionario(s)
cadastrado(s)</h4>
      </div>
      <div className="overflow-x-auto">
        <table className="w-full text-left border-collapse">
          <thead><tr className="border-b border-gray-200 text-xs text-gray-400 uppercase bg-white">
            <th className="p-4 font-bold">Nome</th><th className="p-4 font-bold">Login</th><th className="p-4 font-bold">Funcao</
th><th className="p-4 font-bold">Status</th><th className="p-4 font-bold text-center">Acao</th>
          </tr></thead>
          <tbody className="divide-y divide-gray-100">
            {isLoadingUsers ? (
              <tr><td colspan={5} className="p-8 text-center text-sm text-gray-400">Carregando...</td></tr>
            ) : funcionarios.length > 0 ? (
              funcionarios.map((f: any) => (
                <tr key={f.id} className="hover:bg-gray-50/50">
                  <td className="p-4 text-sm font-bold text-[#0b1a30]">{f.nome}</td>
                  <td className="p-4 text-sm text-gray-600 font-mono">{f.username}</td>
                  <td className="p-4 text-sm"><span className={f.papel === "ADMIN" ? "bg-purple-50 text-purple-700 border border-purple-200" :

```

```

        f.papel === "CAIXA" ? "bg-blue-50 text-blue-700 border border-blue-200" :
        f.papel === "COZINHA" ? "bg-orange-50 text-orange-700 border border-orange-200" :
        "bg-green-50 text-green-700 border border-green-200"
      }`>{f.papel}</span></td>
      <td className="p-4 text-sm"><span className={`px-3 py-1 text-xs font-bold rounded-full ${f.ativo !== false ? "bg-
green-50 text-green-700 border border-green-200" : "bg-red-50 text-red-700 border border-red-200"}`}>{f.ativo !== false ? "Ativo" : "Inativo"}
</span></td>
      <td className="p-4 text-center"><button onClick={() => handleDeletarFuncionario(f.id, f.nome)} className="text-
red-500 hover:text-red-700 font-bold text-xs">Remover</button></td>
    </tr>
  )
) : (
  <tr><td colSpan={5} className="p-8 text-center"><p className="text-sm text-gray-500">Nenhum funcionario cadastrado.</
p><p className="text-xs text-gray-400 mt-1">Cadastre acima para liberar o acesso ao MesaPronta.</p></td></tr>
)
</tbody>
</table>
</div>
</div>

<div className="bg-blue-50 border border-blue-200 rounded-xl p-4 text-sm text-blue-700">
  <p className="font-semibold mb-1">\u{1F4A1} Como funciona?</p>
  <ul className="text-xs space-y-1 ml-4 list-disc">
    <li>Cadastre aqui os funcionarios que vao usar o MesaPronta</li>
    <li>Cada um recebe um login e senha proprios</li>
    <li>No app Desktop, fazem login com esses dados</li>
    <li>Admin tem acesso total; Garcom faz pedidos; Caixa fecha contas; Cozinha ve pedidos</li>
    <li>Se remover aqui, perde o acesso imediatamente</li>
  </ul>
</div>
</div>
)}

{activeTab === "delivery" && (
  <div className="max-w-6xl mx-auto space-y-6">
    <div className="bg-gradient-to-r from-orange-500 to-red-500 rounded-xl p-8 text-white shadow-lg">
      <div className="flex items-center gap-4 mb-4">
        <span className="text-5xl">\u{1F6F5}</span>
        <div><h3 className="text-2xl font-bold">ConnectFood</h3><p className="text-orange-100 text-sm">Integracao com iFood, 99Food
e outros</p></div>
      </div>
      <div className="bg-white rounded-xl border border-gray-200 p-8 text-center shadow-sm">
        <div className="text-6xl mb-4">\u{1F6A7}</div>
        <h4 className="text-xl font-bold text-[#0b1a30] mb-2">Modulo em Desenvolvimento</h4>
        <p className="text-gray-500 max-w-md mx-auto">Em breve voce podera conectar suas contas do iFood e 99Food diretamente pelo
painel.</p>
      </div>
    </div>
  )
}
</main>
</div>
</div>
);
}

```

page.tsx

frontend-web/app/dashboard/salto-gestao/page.tsx • 432 linhas

```

'use client';

import { useState, useEffect, useCallback } from 'react';
import { useRouter } from 'next/navigation';
import Image from 'next/image';

export default function SaltoGestaoDashboard() {
  const router = useRouter();
  const [activeTab, setActiveTab] = useState('visao-geral');

  // Identificação do Inquilino (Multi-tenant)
  const [documentoCliente, setDocumentoCliente] = useState('00.000.000/0001-00');

  // Estados de dados de Usuários (agora utilizando 'login' ao invés de 'email')
  const [usuarios, setUsuarios] = useState<any[]>([]);
  const [novoUsuario, setNovoUsuario] = useState({ nome: '', login: '', senha: '', papel: 'VENDEDOR' });
  const [isSaving, setIsSaving] = useState(false);
  const [isLoadingUsers, setIsLoadingUsers] = useState(false);

  // Estados para as Métricas da Visão Geral
  const [metricas, setMetricas] = useState({ faturamentoHoje: 0, pedidosHoje: 0, produtosCadastrados: 0 });
  const [carregandoMetricas, setCarregandoMetricas] = useState(false);

  // Verificação de autenticação ao montar a página
  useEffect(() => {
    const token = localStorage.getItem('corumba_token');
    if (!token) {
      router.push('/login');
      return;
    }
  });

  const cnpjSalvo = localStorage.getItem('cnpj_cliente');
  if (cnpjSalvo) {
    setDocumentoCliente(cnpjSalvo);
  }
}

```



```

}, [router]);

// Função para buscar métricas reais do banco SaltoGestao
const carregarMetricas = useCallback(async () => {
  setCarregandoMetricas(true);
  try {
    const res = await fetch(`/api/salto-gestao/metricas?documento=${documentoCliente}`);
    if (res.ok) {
      const data = await res.json();
      setMetricas(data);
    }
  } catch (error) {
    console.error("Erro ao carregar métricas da loja:", error);
  } finally {
    setCarregandoMetricas(false);
  }
}, [documentoCliente]);

// Carrega as métricas sempre que a aba Visão Geral for ativada
useEffect(() => {
  if (activeTab === 'visao-geral') {
    carregarMetricas();
  }
}, [activeTab, carregarMetricas]);

// Função para buscar usuários amarrados ao documento do cliente
const carregarUsuarios = useCallback(async () => {
  setIsLoadingUsers(true);
  try {
    const res = await fetch(`/api/usuarios?documento=${documentoCliente}`);
    if (res.ok) {
      const data = await res.json();
      setUsuarios(data);
    }
  } catch (error) {
    console.error("Erro ao carregar equipe:", error);
  } finally {
    setIsLoadingUsers(false);
  }
}, [documentoCliente]);

// Carrega a listagem sempre que a aba Configurações for ativada
useEffect(() => {
  if (activeTab === 'configuracoes') {
    carregarUsuarios();
  }
}, [activeTab, carregarUsuarios]);

// Handler para criar novo usuário/operador
const handleCriarUsuario = async (e: React.FormEvent) => {
  e.preventDefault();
  setIsSaving(true);
  try {
    const payload = {
      ...novoUsuario,
      empresa_documento: documentoCliente
    };

    const res = await fetch('/api/usuarios', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });

    if (res.ok) {
      alert("Operador cadastrado com sucesso!");
      // Limpa o formulário mantendo a estrutura de 'login'
      setNovoUsuario({ nome: '', login: '', senha: '', papel: 'VENDEDOR' });
      carregarUsuarios();
    } else {
      const err = await res.json();
      alert(err.error || "Erro ao salvar usuário.");
    }
  } catch (error) {
    alert("Falha na comunicação com o banco de dados.");
  } finally {
    setIsSaving(false);
  }
};

// Handler para remover usuário/operador
const handleDeletarUsuario = async (id: number, nome: string) => {
  if (!confirm("Deseja remover o acesso de ${nome}? Esta ação bloqueará o login dele imediatamente no PDV Desktop.")) {
    return;
  }

  try {
    const res = await fetch(`/api/usuarios?id=${id}`, { method: 'DELETE' });
    if (res.ok) {
      carregarUsuarios();
    } else {
      alert("Não foi possível remover o usuário.");
    }
  } catch (error) {
    alert("Erro ao conectar com o servidor.");
  }
};

```

```

}
};

return (
  <div className="flex h-screen bg-[#f8fafc] font-sans text-slate-800 overflow-hidden">

    {/* Sidebar Lateral Oficial */}
    <aside className="w-64 bg-[#0b1a30] text-white flex flex-col justify-between shadow-xl shrink-0">
      <div>
        {/* Logo Container */}
        <div className="p-6 border-b border-slate-800">
          <div className="relative w-full h-16">
            <Image
              src="/logo-corumba.png"
              alt="Salto Gestão"
              fill
              className="object-contain object-left filter brightness-0 invert"
              priority
            />
          </div>
          <p className="text-[10px] text-emerald-400 font-bold uppercase tracking-widest mt-2 px-1">
            Pánel de Gestão
          </p>
        </div>

        {/* Navegação por Abas */}
        <nav className="px-4 py-6 space-y-1.5">
          <button
            onClick={() => setActiveTab('visao-geral')}
            className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
              activeTab === 'visao-geral'
                ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
                : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
            }`>
            <span className="text-lg"><img alt="Visão Geral icon" data-bbox="321 384 331 394"/></span> Visão Geral
          </button>

          <button
            onClick={() => setActiveTab('vendas')}
            className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
              activeTab === 'vendas'
                ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
                : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
            }`>
            <span className="text-lg"><img alt="Vendas icon" data-bbox="321 484 331 494"/></span> Vendas
          </button>

          <button
            onClick={() => setActiveTab('produtos')}
            className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
              activeTab === 'produtos'
                ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
                : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
            }`>
            <span className="text-lg"><img alt="Produtos icon" data-bbox="321 584 331 594"/></span> Produtos
          </button>

          <button
            onClick={() => setActiveTab('configuracoes')}
            className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
              activeTab === 'configuracoes'
                ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
                : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
            }`>
            <span className="text-lg"><img alt="Configurações icon" data-bbox="321 684 331 694"/></span> Configurações
          </button>
        </nav>
      </div>

      {/* Rodapé da Sidebar */}
      <div className="p-4 border-t border-slate-800">
        <button
          onClick={() => router.push('/dashboard')}
          className="w-full flex items-center gap-2 px-4 py-3 text-sm text-slate-400 hover:text-white font-medium transition-colors"
        >
          <span><img alt="Voltar ao Hub icon" data-bbox="198 784 208 794"/></span> Voltar ao Hub
        </button>
      </div>
    </aside>

    {/* Área de Conteúdo Dinâmico */}
    <div className="flex-1 flex flex-col overflow-hidden">

      {/* Header Principal Superior */}
      <header className="bg-white border-b border-gray-200 px-8 py-5 flex justify-between items-center shrink-0">
        <h2 className="text-xl font-bold text-[#0b1a30] capitalize">
          {activeTab === 'visao-geral' ? 'Visão Geral da Loja' : activeTab}
        </h2>
        <div className="flex items-center gap-2 text-xs font-bold text-[#65a30d] bg-[#84cc16]/10 px-4 py-2 rounded-full">
          <span className="w-2 h-2 rounded-full bg-[#84cc16] animate-pulse"></span>
          Licença Ativa
        </div>
      </header>
    </div>
  </div>
)

```

```

    </div>
  </header>

  {/ * Visualização de Telas */}
  <main className="flex-1 overflow-y-auto p-8">

    {/ * TELA 1: VISÃO GERAL */}
    {activeTab === 'visao-geral' && (
      <div className="space-y-6 max-w-6xl mx-auto">
        {/ * Banner de Download */}
        <div className="bg-[#0b1a30] rounded-xl p-8 flex flex-col md:flex-row justify-between items-start md:items-center gap-6 shadow-md border border-slate-800">
          <div className="text-white max-w-2xl">
            <h3 className="text-xl font-bold mb-2">Instale o Salto PDV na sua máquina</h3>
            <p className="text-slate-300 text-sm leading-relaxed">
              Para operar o seu caixa com velocidade máxima, emitir cupons fiscais e usar atalhos de teclado rápidos, faça o download da versão Desktop estruturada para o seu negócio.
            </p>
          </div>
          <button className="bg-[#84cc16] hover:bg-[#65a30d] text-[#0b1a30] font-bold py-3.5 px-6 rounded-md transition-all shadow-lg shrink-0 flex items-center gap-2">
            <span><img alt="Download icon" data-bbox="235 258 248 268"/> Download para Windows</span>
          </button>
        </div>

        {/ * Cards de Indicadores */}
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
            <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Vendas Hoje</p>
            <p className="text-3xl font-extrabold text-[#0b1a30]">
              {carregandoMetricas ? '...' : new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' }).format(metricas.faturamentoHoje)}
            </p>
            <span className="text-xs text-emerald-600 font-semibold mt-1 inline-block">Métricas em tempo real</span>
          </div>

          <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
            <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Pedidos Fechados</p>
            <p className="text-3xl font-extrabold text-[#0b1a30]">
              {carregandoMetricas ? '...' : metricas.pedidosHoje}
            </p>
            <span className="text-xs text-gray-500 font-medium mt-1 inline-block">
              Ticket médio: {metricas.pedidosHoje > 0 ? new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' }).format(metricas.faturamentoHoje / metricas.pedidosHoje) : 'R$ 0,00'}
            </span>
          </div>

          <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
            <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Produtos Cadastrados</p>
            <p className="text-3xl font-extrabold text-[#0b1a30]">
              {carregandoMetricas ? '...' : metricas.produtosCadastrados}
            </p>
            <span className="text-xs text-blue-600 font-semibold mt-1 inline-block">Banco de Dados Sincronizado</span>
          </div>
        </div>

        {/ * Área do Gráfico */}
        <div className="bg-white rounded-xl border border-gray-200 p-12 text-center shadow-sm flex flex-col items-center justify-center min-h-[300px]">
          <div className="text-4xl mb-4 opacity-40"><img alt="Placeholder for chart" data-bbox="425 600 438 610"/></div>
          <h4 className="text-lg font-bold text-[#0b1a30] mb-1">Gráfico de Faturamento Semanal</h4>
          <p className="text-sm text-gray-400 max-w-sm">
            Os dados operacionais coletados no PDV aparecerão estruturados aqui assim que o caixa realizar a primeira sincronização de fechamento.
          </p>
        </div>
      </div>
    )}

    {/ * TELA 2: VENDAS (PLACEHOLDER) */}
    {activeTab === 'vendas' && (
      <div className="max-w-6xl mx-auto bg-white rounded-xl border border-gray-200 p-8 text-center shadow-sm">
        <p className="text-gray-500">Módulo de Histórico e Relatório Analítico de Vendas em desenvolvimento.</p>
      </div>
    )}

    {/ * TELA 3: PRODUTOS (PLACEHOLDER) */}
    {activeTab === 'produtos' && (
      <div className="max-w-6xl mx-auto bg-white rounded-xl border border-gray-200 p-8 text-center shadow-sm">
        <p className="text-gray-500">Módulo de Sincronização e ajuste de grade de Produtos em desenvolvimento.</p>
      </div>
    )}

    {/ * TELA 4: CONFIGURAÇÕES (GESTÃO REAL DE USUÁRIOS) */}
    {activeTab === 'configuracoes' && (
      <div className="max-w-6xl mx-auto space-y-8">

        {/ * Box de Criação */}
        <div className="bg-white rounded-xl border border-gray-200 shadow-sm p-6">
          <div className="mb-4">
            <h3 className="text-lg font-bold text-[#0b1a30]">Controle de Usuários da Loja</h3>
            <p className="text-xs text-gray-400">Cadastre os colaboradores para liberação de acessos individuais vinculados ao CNPJ {documentoCliente}</p>
          </div>
        </div>
      </div>
    )}
  </main>

```

```

<form onSubmit={handleCriarUsuario} className="grid grid-cols-1 md:grid-cols-4 gap-4 items-end">
  <div>
    <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Nome Completo</label>
    <input
      type="text"
      required
      value={novoUsuario.nome}
      onChange={(e) => setNovoUsuario({...novoUsuario, nome: e.target.value})}
      className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-[#84cc16] focus:border-transparent outline-none transition-all text-gray-900"
      placeholder="Ex: Carlos Alberto"
    />
  </div>
  <div>
    <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Login do PDV</label>
    <input
      type="text"
      required
      value={novoUsuario.login}
      onChange={(e) => setNovoUsuario({...novoUsuario, login: e.target.value})}
      className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-[#84cc16] focus:border-transparent outline-none transition-all text-gray-900"
      placeholder="Ex: carlos / 1001"
    />
  </div>
  <div>
    <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Senha de Acesso</label>
    <input
      type="password"
      required
      value={novoUsuario.senha}
      onChange={(e) => setNovoUsuario({...novoUsuario, senha: e.target.value})}
      className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-[#84cc16] focus:border-transparent outline-none transition-all text-gray-900"
      placeholder="*****"
    />
  </div>
  <div>
    <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Atribuição / Função</label>
    <select
      value={novoUsuario.papel}
      onChange={(e) => setNovoUsuario({...novoUsuario, papel: e.target.value})}
      className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-[#84cc16] focus:border-transparent outline-none bg-white transition-all text-gray-900 font-medium"
    >
      <option value="ADMIN">Admin</option>
      <option value="CAIXA">Caixa</option>
      <option value="VENDEDOR">Vendedor</option>
    </select>
  </div>
  <div className="md:col-span-4 flex justify-end mt-2">
    <button
      disabled={isSaving}
      type="submit"
      className="bg-[#0b1a30] hover:bg-slate-800 text-white font-bold text-sm py-2.5 px-6 rounded-md transition-colors shadow-sm disabled:opacity-50"
    >
      {isSaving ? 'Salvando no banco...' : 'Adicionar Usuário'}
    </button>
  </div>
</form>
</div>

/* Tabela Grid List */
<div className="bg-white rounded-xl border border-gray-200 shadow-sm overflow-hidden">
  <div className="px-6 py-4 border-b border-gray-200 bg-gray-50/70">
    <h4 className="text-xs font-bold text-[#0b1a30] uppercase tracking-wider">Usuários e Atribuições Ativas</h4>
  </div>

  <div className="overflow-x-auto">
    <table className="w-full text-left border-collapse">
      <thead>
        <tr className="border-b border-gray-200 text-xs text-gray-400 uppercase bg-white">
          <th className="p-4 font-bold">Colaborador</th>
          <th className="p-4 font-bold">Usuário de Login</th>
          <th className="p-4 font-bold">Nível de Acesso</th>
          <th className="p-4 font-bold text-center">Ação</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-gray-100">
        {isLoadingUsers ? (
          <tr>
            <td colspan={4} className="p-8 text-center text-sm text-gray-400">Consultando tabelas Neon...</td>
          </tr>
        ) : usuarios.map((user) => (
          <tr key={user.id} className="hover:bg-gray-50/50 transition-colors">
            <td className="p-4 text-sm font-bold text-[#0b1a30]">{user.nome}</td>
            <td className="p-4 text-sm text-gray-600">{user.login}</td>
            <td className="p-4 text-sm">
              <span className={`px-3 py-1 text-xs font-extrabold rounded-full ${
                user.papel === 'ADMIN' ? 'bg-purple-50 text-purple-700 border border-purple-200' :
                user.papel === 'CAIXA' ? 'bg-blue-50 text-blue-700 border border-blue-200' :
                'bg-amber-50 text-amber-700 border border-amber-200'
              }`}>
                {user.papel}
              </span>
            </td>
            <td>

```

```

        </span>
      </td>
      <td className="p-4 text-center">
        <button
          onClick={() => handleDeletarUsuario(user.id, user.nome)}
          className="text-red-500 hover:text-red-700 font-bold text-xs transition-colors"
        >
          Remover Acesso
        </button>
      </td>
    </tr>
  )}
  {!isLoadingUsers && usuarios.length === 0 && (
    <tr>
      <td colSpan={4} className="p-8 text-center text-sm text-gray-400">Nenhum funcionário cadastrado sob este CNPJ.</td>
    </tr>
  )}
</tbody>
</table>
</div>
</div>
</div>
  </main>
</div>
</div>
);
}

```

route.ts frontend-web/app/dashboard/salto-gestao/metricas/route.ts • 49 linhas

```
import { NextResponse } from 'next/server';
import { Pool } from 'pg';

// ⚠️ ATENÇÃO: Coloque aqui a URL de conexão do banco de dados SALTO GESTAO do Neon
// (Diferente da URL do CorumbaCore que usamos nos usuários)
const pool = new Pool({
  connectionString: "postgresql://neondb_owner:neondb_owner@ep-weathered-thunder-ap2gc8n9-pooler.c-7.us-east-1.aws.neon.tech/salto_gestao?sslmode=require"
});

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const documento = searchParams.get('documento');

    if (!documento) {
      return NextResponse.json({ error: 'CNPJ não informado.' }, { status: 400 });
    }

    // 1. Busca as Vendas de HOJE desta empresa
    // Ajuste o nome da coluna de data (ex: data_venda, criado_em) de acordo com o seu Python
    const vendasResult = await pool.query(`
      SELECT
        COALESCE(SUM(total), 0) as faturamento_hoje,
        COUNT(id) as pedidos_hoje
      FROM vendas
      WHERE empresa_documento = $1
      AND DATE(data_venda) = CURRENT_DATE
    `, [documento]);

    // 2. Busca o total de Produtos cadastrados por esta empresa
    const produtosResult = await pool.query(`
      SELECT COUNT(id) as total_produtos
      FROM produtos
      WHERE empresa_documento = $1
    `, [documento]);

    const metricas = {
      faturamentoHoje: Number(vendasResult.rows[0].faturamento_hoje),
      pedidosHoje: Number(vendasResult.rows[0].pedidos_hoje),
      produtosCadastrados: Number(produtosResult.rows[0].total_produtos)
    };

    return NextResponse.json(metricas);

  } catch (error) {
    console.error("Erro ao buscar métricas:", error);
    return NextResponse.json({ error: 'Erro ao buscar métricas do Salto Gestão' }, { status: 500 });
  }
}
```

page.tsx frontend-web/app/admin/login/page.tsx • 114 linhas

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
```

```

export default function AdminLoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState('');

  const handleLogin = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    setIsLoading(true);
    setError('');

    try {
      const response = await fetch('/api/admin/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, senha }),
      });

      if (response.ok) {
        const data = await response.json();
        localStorage.setItem('admin_token', data.access_token);
        localStorage.setItem('admin_data', JSON.stringify(data.admin));
        window.location.href = '/admin/dashboard';
      } else {
        const errorData = await response.json();
        setError(errorData.detail || 'Erro no login.');
```

```

        disabled={isLoading}
        className="w-full flex justify-center py-3.5 px-4 rounded-md text-sm font-bold text-[#0b1a30] bg-[#84cc16] hover:bg-[#65a30d]
        hover:text-white transition-all disabled:opacity-70"
      >
        {isLoading ? 'Entrando...' : 'Entrar'}
      </button>
    </form>

    <div className="mt-6 p-3 bg-gray-50 rounded-lg">
      <p className="text-xs text-gray-500 text-center">
        <strong>Demo:</strong> admin@corumbasistemas.com.br / corumba2026
      </p>
    </div>
  </div>
</div>
);
}

```

page.tsx

frontend-web/app/admin/dashboard/page.tsx • 152 linhas

```

'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

interface DashboardData {
  clientes: { total: number; ativos: number };
  assinaturas: { total: number; ativas: number; inadimplentes: number; canceladas: number };
  chaves_ativacao: { total: number; usadas: number; pendentes: number };
  receita_mensal_estimada: number;
}

export default function AdminDashboardPage() {
  const router = useRouter();
  const [dash, setDash] = useState<DashboardData | null>(null);
  const [admin, setAdmin] = useState<{ nome: string; papel: string } | null>(null);
  const [carregando, setCarregando] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('admin_token');
    if (!token) {
      router.push('/admin/login');
      return;
    }

    const adminData = localStorage.getItem('admin_data');
    if (adminData) {
      setAdmin(JSON.parse(adminData));
    }

    fetch('/api/admin/dashboard', {
      headers: { Authorization: `Bearer ${token}` },
    })
      .then((res) => (res.ok ? res.json() : null))
      .then((data) => setDash(data))
      .catch(() => {})
      .finally(() => setCarregando(false));
  }, [router]);

  const handleLogout = () => {
    localStorage.removeItem('admin_token');
    localStorage.removeItem('admin_data');
    router.push('/admin/login');
  };

  if (carregando) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-[#f8fafc]">
        <p className="text-gray-500">Carregando...</p>
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-[#f8fafc]">
      </* Header */>
      <header className="bg-[#0b1a30] text-white px-8 py-5">
        <div className="max-w-6xl mx-auto flex justify-between items-center">
          <div>
            <h1 className="text-2xl font-bold">Corumbá Sistemas</h1>
            <p className="text-sm text-slate-400">Painel Administrativo — {admin?.nome}</p>
          </div>
          <div className="flex items-center gap-4">
            <span className="bg-[#84cc16] text-[#0b1a30] text-xs font-bold px-3 py-1 rounded-full uppercase">
              {admin?.papel}
            </span>
            <button
              onClick={handleLogout}
              className="text-sm text-slate-400 hover:text-white transition-colors"
            >
              Sair
            </button>
          </div>
        </div>
      </header>
    </div>
  );
}

```

```

    </div>
  </div>
</header>

<main className="max-w-6xl mx-auto px-8 py-8">
  { /* Stats */ }
  <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-8">
    <div className="bg-white rounded-xl border border-gray-200 p-6">
      <p className="text-xs font-bold text-gray-400 uppercase">Clientes</p>
      <p className="text-3xl font-extrabold text-[#0b1a30]">{dash?.clientes.total || 0}</p>
      <p className="text-xs text-green-600 font-semibold mt-1">{dash?.clientes.ativos || 0} ativos</p>
    </div>
    <div className="bg-white rounded-xl border border-gray-200 p-6">
      <p className="text-xs font-bold text-gray-400 uppercase">Assinaturas Ativas</p>
      <p className="text-3xl font-extrabold text-green-600">{dash?.assinaturas.ativas || 0}</p>
      <p className="text-xs text-gray-500 font-medium mt-1">de {dash?.assinaturas.total || 0} total</p>
    </div>
    <div className="bg-white rounded-xl border border-gray-200 p-6">
      <p className="text-xs font-bold text-gray-400 uppercase">Inadimplentes</p>
      <p className="text-3xl font-extrabold text-red-600">{dash?.assinaturas.inadimplentes || 0}</p>
      <p className="text-xs text-red-500 font-medium mt-1">requerem atenção</p>
    </div>
    <div className="bg-white rounded-xl border border-gray-200 p-6">
      <p className="text-xs font-bold text-gray-400 uppercase">Receita Mensal</p>
      <p className="text-3xl font-extrabold text-[#0b1a30]">
        R$ {(dash?.receita_mensal_estimada || 0).toFixed(2).replace('.', ',')}
      </p>
      <p className="text-xs text-blue-600 font-semibold mt-1">estimada</p>
    </div>
  </div>

  { /* Ações Rápidas */ }
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mb-8">
    <Link
      href="/admin/clientes/listar"
      className="bg-white p-6 rounded-xl border border-gray-200 hover:border-[#84cc16] hover:shadow-md transition-all group"
    >
      <div className="text-3xl mb-3">👤</div>
      <h3 className="text-lg font-bold text-[#0b1a30] group-hover:text-[#65a30d]">Clientes</h3>
      <p className="text-sm text-gray-500 mt-1">Gerenciar clientes cadastrados</p>
    </Link>

    <Link
      href="/admin/clientes/criar"
      className="bg-white p-6 rounded-xl border border-gray-200 hover:border-[#84cc16] hover:shadow-md transition-all group"
    >
      <div className="text-3xl mb-3">➕</div>
      <h3 className="text-lg font-bold text-[#0b1a30] group-hover:text-[#65a30d]">Novo Cliente</h3>
      <p className="text-sm text-gray-500 mt-1">Cadastrar novo cliente e vincular produtos</p>
    </Link>

    <div className="bg-white p-6 rounded-xl border border-gray-200">
      <div className="text-3xl mb-3">🔑</div>
      <h3 className="text-lg font-bold text-[#0b1a30]">Chaves de Ativação</h3>
      <p className="text-sm text-gray-500 mt-1">
        {dash?.chaves_ativacao.pendentes || 0} chaves pendentes
      </p>
    </div>
  </div>

  { /* Últimos clientes */ }
  <div className="bg-white rounded-xl border border-gray-200 overflow-hidden">
    <div className="px-6 py-4 border-b border-gray-200 flex justify-between items-center">
      <h3 className="text-sm font-bold text-[#0b1a30] uppercase tracking-wider">Últimos Clientes</h3>
      <Link href="/admin/clientes/listar" className="text-sm text-[#65a30d] font-bold hover:underline">
        Ver todos →
      </Link>
    </div>
    <div className="p-6 text-center text-gray-400 text-sm">
      <Link href="/admin/clientes/criar" className="text-[#84cc16] font-bold hover:underline">
        Cadastre o primeiro cliente →
      </Link>
    </div>
  </div>
</main>
</div>
);
}

```

page.tsx

frontend-web/app/admin/clientes/criar/page.tsx • 256 linhas

```

'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

interface Produto {
  id: number;
  nome: string;
  descricao: string;
  preco_mensal: number;
  icone: string;
}

```



```

}

export default function AdminCriarClientePage() {
  const router = useRouter();
  const [produtos, setProdutos] = useState<Produto[]>([]);
  const [produtosSelecionados, setProdutosSelecionados] = useState<number[]>([]);
  const [carregandoProdutos, setCarregandoProdutos] = useState(true);

  const [form, setForm] = useState({
    nome: '',
    email: '',
    telefone: '',
    cpf_cnpj: '',
    senha: '',
  });

  const [isSaving, setIsSaving] = useState(false);
  const [erro, setErro] = useState('');
  const [sucesso, setSucesso] = useState('');

  // Carregar produtos do catálogo
  useState(() => {
    fetch('/api/admin/produtos')
      .then((res) => res.json())
      .then((data) => setProdutos(data))
      .catch(() => {})
      .finally(() => setCarregandoProdutos(false));
  });

  const toggleProduto = (id: number) => {
    setProdutosSelecionados((prev) =>
      prev.includes(id) ? prev.filter((p) => p !== id) : [...prev, id]
    );
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsSaving(true);
    setErro('');
    setSucesso('');

    try {
      const res = await fetch('/api/admin/clientes/criar', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          nome: form.nome,
          email: form.email,
          telefone: form.telefone,
          cpf_cnpj: form.cpf_cnpj,
          senha: form.senha,
          produto_ids: produtosSelecionados,
        }),
      });

      if (res.ok) {
        const data = await res.json();
        setSucesso(`Cliente ${data.nome} cadastrado com sucesso! ID: ${data.cliente_id}`);
        setForm({ nome: '', email: '', telefone: '', cpf_cnpj: '', senha: '' });
        setProdutosSelecionados([]);
      } else {
        const err = await res.json();
        setErro(err.detail || 'Erro ao cadastrar cliente.');
      }
    } catch {
      setErro('Erro de conexão com o servidor.');
```

```

{erro && (
  <div className="mb-6 p-4 bg-red-50 border border-red-200 text-red-700 rounded-lg">
    X {erro}
  </div>
)}

<form onSubmit={handleSubmit} className="space-y-6">
  /* Dados Pessoais */
  <div className="bg-white rounded-xl border border-gray-200 p-6">
    <h3 className="text-lg font-bold text-[#0b1a30] mb-4">🏠 Dados do Cliente</h3>
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div>
        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Nome Completo</label>
        <input
          type="text"
          required
          value={form.nome}
          onChange={(e) => setForm({ ...form, nome: e.target.value })}
          className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
          placeholder="Ex: Maria Silva"
        />
      </div>
      <div>
        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">E-mail</label>
        <input
          type="email"
          required
          value={form.email}
          onChange={(e) => setForm({ ...form, email: e.target.value })}
          className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
          placeholder="cliente@email.com"
        />
      </div>
      <div>
        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Telefone</label>
        <input
          type="text"
          value={form.telefone}
          onChange={(e) => setForm({ ...form, telefone: e.target.value })}
          className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
          placeholder="(62) 99999-9999"
        />
      </div>
      <div>
        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">CPF ou CNPJ</label>
        <input
          type="text"
          required
          value={form.cpf_cnpj}
          onChange={(e) => setForm({ ...form, cpf_cnpj: e.target.value })}
          className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
          placeholder="000.000.000-00 ou 00.000.000/0000-00"
        />
      </div>
      <div className="md:col-span-2">
        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Senha de Acesso</label>
        <input
          type="password"
          required
          value={form.senha}
          onChange={(e) => setForm({ ...form, senha: e.target.value })}
          className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
          placeholder="Senha para acesso ao painel"
        />
      </div>
    </div>
  </div>

  /* Produtos */
  <div className="bg-white rounded-xl border border-gray-200 p-6">
    <h3 className="text-lg font-bold text-[#0b1a30] mb-4">🛒 Produtos / Serviços</h3>
    <p className="text-sm text-gray-500 mb-4">Selecione os produtos que o cliente está adquirindo:</p>

    {carregandoProdutos ? (
      <p className="text-gray-400">Carregando produtos...</p>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
        {produtos.map((p) => (
          <div
            key={p.id}
            onClick={() => toggleProduto(p.id)}
            className={`p-4 rounded-lg border-2 cursor-pointer transition-all ${
              produtosSelecionados.includes(p.id)
                ? 'border-[#84cc16] bg-green-50'
                : 'border-gray-200 hover:border-gray-300'
            }`}
          >
            <div className="flex items-center gap-3">
              <span className="text-2xl">{p.icone}</span>
              <div className="flex-1">
                <p className="font-bold text-[#0b1a30]">{p.nome}</p>
                <p className="text-xs text-gray-500">
                  R$ {p.preco_mensal.toFixed(2).replace('.', ',')} / mês
                </p>
              </div>
            </div>
          </div>
        ))}
      </div>
    )}
  </div>

```

```

<div className={`w-6 h-6 rounded-full border-2 flex items-center justify-center ${
  produtosSelecionados.includes(p.id)
    ? 'border-[#84cc16] bg-[#84cc16]'
    : 'border-gray-300'
}`}>
  {produtosSelecionados.includes(p.id) && (
    <span className="text-white text-xs">{p.preco_mensal}</span>
  )}
</div>
</div>
</div>
</div>
))}
</div>
)}

{produtosSelecionados.length > 0 && (
  <div className="mt-4 p-3 bg-green-50 rounded-lg">
    <p className="text-sm text-green-700 font-medium">
      {produtosSelecionados.length} produto(s) selecionado(s)
    </p>
    {produtosSelecionados.length > 1 ? 'Total: R$' : 'Total: R$'}
    {produtosSelecionados
      .filter((p) => produtosSelecionados.includes(p.id))
      .reduce((sum, p) => sum + p.preco_mensal, 0)
      .toFixed(2)
      .replace('.', ',')}
    /mês
  </p>
</div>
)}
</div>

{ /* Botões */ }
<div className="flex gap-4">
  <Link
    href="/admin/clientes/listar"
    className="flex-1 py-3 border border-gray-300 text-gray-700 font-bold rounded-md text-center hover:bg-gray-50 transition-colors"
  >
    Cancelar
  </Link>
  <button
    type="submit"
    disabled={!isSaving}
    className="flex-1 py-3 bg-[#84cc16] text-[#0b1a30] font-bold rounded-md hover:bg-[#65a30d] hover:text-white transition-colors disabled:opacity-50"
  >
    {isSaving ? 'Cadastrando...' : 'Cadastrar Cliente'}
  </button>
</div>
</form>
</main>
</div>
);
}

```

page.tsx

frontend-web/app/admin/clientes/listar/page.tsx • 193 linhas

```

'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

interface Cliente {
  id: number;
  nome: string;
  email: string;
  telefone: string;
  cpf_cnpj: string;
  ativo: boolean;
  tem_inadimplencia: boolean;
  total_produtos: number;
}

export default function AdminClientesPage() {
  const router = useRouter();
  const [clientes, setClientes] = useState<Cliente[]>([]);
  const [carregando, setCarregando] = useState(true);
  const [busca, setBusca] = useState('');
  const [filtro, setFiltro] = useState<string>('TODOS');
  const [erro, setErro] = useState('');

  const carregarClientes = async () => {
    try {
      let url = '/api/admin/clientes';
      if (filtro === 'INADIMIENTE') url += '?status=INADIMIENTE';
      if (busca) url += (url.includes('?') ? '&' : '?') + `busca=${encodeURIComponent(busca)}`;

      const res = await fetch(url);
      if (res.ok) {
        const data = await res.json();
        setClientes(data);
      } else {

```

```

        setErro('Erro ao carregar clientes');
    }
} catch {
    setErro('Erro de conexão com o servidor');
} finally {
    setCarregando(false);
}
};

useEffect(() => {
    carregarClientes();
}, [filtro, busca]);

return (
    <div className="min-h-screen bg-[#f8fafc]">
        {/* Header */}
        <header className="bg-[#0b1a30] text-white px-8 py-5">
            <div className="max-w-6xl mx-auto flex justify-between items-center">
                <div>
                    <h1 className="text-2xl font-bold">Corumbá Sistemas</h1>
                    <p className="text-sm text-slate-400">Área do Vendedor – Gestão de Clientes</p>
                </div>
                <Link
                    href="/admin/clientes/criar"
                    className="bg-[#84cc16] text-[#0b1a30] font-bold py-2 px-6 rounded-md hover:bg-[#65a30d] hover:text-white transition-colors"
                >
                    + Novo Cliente
                </Link>
            </div>
        </header>

        <main className="max-w-6xl mx-auto px-8 py-8">
            {/* Filtros */}
            <div className="bg-white rounded-xl border border-gray-200 p-6 mb-6">
                <div className="flex flex-col md:flex-row gap-4">
                    <div className="flex-1">
                        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Buscar Cliente</label>
                        <input
                            type="text"
                            placeholder="Nome, email ou CPF/CNPJ..."
                            value={busca}
                            onChange={(e) => setBusca(e.target.value)}
                            className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] focus:border-transparent outline-none"
                        />
                    </div>
                    <div className="md:w-48">
                        <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Status</label>
                        <select
                            value={filtro}
                            onChange={(e) => setFiltro(e.target.value)}
                            className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none bg-white"
                        >
                            <option value="TODOS">Todos</option>
                            <option value="ATIVO">Ativos</option>
                            <option value="INADIMIENTE">Inadimplentes</option>
                        </select>
                    </div>
                </div>
            </div>

            {/* Stats rápidos */}
            <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
                <div className="bg-white rounded-xl border border-gray-200 p-5">
                    <p className="text-xs font-bold text-gray-400 uppercase">Total Clientes</p>
                    <p className="text-3xl font-extrabold text-[#0b1a30]">{clientes.length}</p>
                </div>
                <div className="bg-white rounded-xl border border-gray-200 p-5">
                    <p className="text-xs font-bold text-gray-400 uppercase">Ativos</p>
                    <p className="text-3xl font-extrabold text-green-600">{clientes.filter(c => !c.tem_inadimplencia).length}</p>
                </div>
                <div className="bg-white rounded-xl border border-gray-200 p-5">
                    <p className="text-xs font-bold text-gray-400 uppercase">Inadimplentes</p>
                    <p className="text-3xl font-extrabold text-red-600">{clientes.filter(c => c.tem_inadimplencia).length}</p>
                </div>
            </div>

            {/* Tabela */}
            <div className="bg-white rounded-xl border border-gray-200 overflow-hidden">
                <div className="px-6 py-4 border-b border-gray-200 bg-gray-50">
                    <h3 className="text-sm font-bold text-[#0b1a30] uppercase tracking-wider">Clientes Cadastrados</h3>
                </div>

                {erro && (
                    <div className="p-4 bg-red-50 border-b border-red-200 text-red-700 text-sm">{erro}</div>
                )}

                {carregando ? (
                    <div className="p-12 text-center text-gray-400">Carregando clientes...</div>
                ) : clientes.length === 0 ? (

```

```

<div className="p-12 text-center text-gray-400">
  Nenhum cliente encontrado.
  <Link href="/admin/clientes/criar" className="text-[#84cc16] font-bold hover:underline ml-1">
    Cadastre o primeiro cliente.
  </Link>
</div>
) : (
  <div className="overflow-x-auto">
    <table className="w-full text-left">
      <thead>
        <tr className="border-b border-gray-200 text-xs text-gray-500 uppercase bg-gray-50">
          <th className="p-4 font-bold">Cliente</th>
          <th className="p-4 font-bold">Contato</th>
          <th className="p-4 font-bold">CPF/CNPJ</th>
          <th className="p-4 font-bold">Produtos</th>
          <th className="p-4 font-bold">Status</th>
          <th className="p-4 font-bold text-center">Ações</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>
            <div className="divide-y divide-gray-100">
              {clientes.map((c) => (
                <tr key={c.id} className="hover:bg-gray-50 transition-colors">
                  <td className="p-4">
                    <p className="text-sm font-bold text-[#0b1a30]">{c.nome}</p>
                    <p className="text-xs text-gray-500">{c.email}</p>
                  </td>
                  <td className="p-4 text-sm text-gray-600">{c.telefone}</td>
                  <td className="p-4 text-sm text-gray-600 font-mono">{c.cpf_cnpj}</td>
                  <td className="p-4 text-sm text-center">
                    <span className="bg-blue-50 text-blue-700 font-bold px-2 py-1 rounded">
                      {c.total_produtos}
                    </span>
                  </td>
                  <td className="p-4">
                    {c.tem_inadimplencia ? (
                      <span className="bg-red-50 text-red-700 text-xs font-bold px-3 py-1 rounded-full border border-red-200">
                        ⚠ Inadimplente
                      </span>
                    ) : (
                      <span className="bg-green-50 text-green-700 text-xs font-bold px-3 py-1 rounded-full border border-green-200">
                        ✅ Ativo
                      </span>
                    )}
                  </td>
                  <td className="p-4 text-center">
                    <button
                      onClick={() => router.push(`/admin/clientes/detalhe?id=${c.id}`)}
                      className="text-[#0b1a30] hover:text-[#84cc16] font-bold text-sm transition-colors"
                    >
                      Ver Detalhes
                    </button>
                  </td>
                </tr>
              )})}
            </div>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>
</main>
</div>
);
}

```

8. Frontend Web - API Routes (Proxy)

route.ts

frontend-web/app/api/login/route.ts • 33 linhas

```
import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function POST(request: Request) {
  try {
    const { email, password } = await request.json();

    const res = await fetch(`${API_URL}/api/auth/cliente/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, senha: password }),
    });

    if (res.ok) {
      const data = await res.json();
      return NextResponse.json(data);
    }

    const err = await res.json().catch(() => ({ detail: 'Erro no login' }));
    return NextResponse.json(
      { error: err.detail || 'Credenciais inválidas' },
      { status: res.status }
    );
  } catch (error) {
    console.error('Erro no Login:', error);
    return NextResponse.json(
      { error: 'Erro ao conectar com o servidor. Verifique se o backend está rodando.' },
      { status: 500 }
    );
  }
}
```

route.ts

frontend-web/app/api/admin/auth/login/route.ts • 26 linhas

```
import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function POST(request: Request) {
  try {
    const { email, senha } = await request.json();

    const res = await fetch(`${API_URL}/api/admin/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, senha }),
    });

    const data = await res.json();

    if (res.ok) {
      return NextResponse.json(data);
    }
    return NextResponse.json(data, { status: res.status });
  } catch (error) {
    console.error('Erro no login admin:', error);
    return NextResponse.json({ error: 'Erro de conexão' }, { status: 500 });
  }
}
```

route.ts

frontend-web/app/api/admin/clientes/route.ts • 50 linhas

```
import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const status = searchParams.get('status');
    const busca = searchParams.get('busca');

    let url = `${API_URL}/api/admin/clientes`;
    const params = new URLSearchParams();
    if (status) params.set('status', status);
    if (busca) params.set('busca', busca);
    if (params.toString()) url += `?${params.toString()}`;

    const res = await fetch(url);
```

```

    if (res.ok) {
      const data = await res.json();
      return NextResponse.json(data);
    }
    return NextResponse.json([], { status: res.status });
  } catch (error) {
    console.error('Erro ao buscar clientes:', error);
    return NextResponse.json([], { status: 500 });
  }
}

export async function POST(request: Request) {
  try {
    const body = await request.json();

    const res = await fetch(`${API_URL}/api/admin/clientes/criar`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body),
    });

    const data = await res.json();

    if (res.ok) {
      return NextResponse.json(data);
    }
    return NextResponse.json(data, { status: res.status });
  } catch (error) {
    console.error('Erro ao criar cliente:', error);
    return NextResponse.json({ error: 'Erro de conexão' }, { status: 500 });
  }
}

```

route.ts

frontend-web/app/api/admin/clientes/criar/route.ts • 26 linhas

```

import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function POST(request: Request) {
  try {
    const body = await request.json();

    const res = await fetch(`${API_URL}/api/admin/clientes/criar`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body),
    });

    const data = await res.json();

    if (res.ok) {
      return NextResponse.json(data);
    }
    return NextResponse.json(data, { status: res.status });
  } catch (error) {
    console.error('Erro ao criar cliente:', error);
    return NextResponse.json({ error: 'Erro de conexão com o servidor.' }, { status: 500 });
  }
}

```

route.ts

frontend-web/app/api/admin/dashboard/route.ts • 28 linhas

```

import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function GET(request: Request) {
  try {
    const authHeader = request.headers.get('authorization') || '';
    const token = authHeader.replace('Bearer ', '');

    if (!token) {
      return NextResponse.json({ error: 'Não autorizado' }, { status: 401 });
    }

    const res = await fetch(`${API_URL}/api/admin/dashboard`, {
      headers: { Authorization: `Bearer ${token}` },
    });

    if (res.ok) {
      const data = await res.json();
      return NextResponse.json(data);
    }
    return NextResponse.json({ error: 'Erro ao buscar dashboard' }, { status: res.status });
  } catch (error) {
    console.error('Erro na API dashboard admin:', error);
    return NextResponse.json({ error: 'Erro interno' }, { status: 500 });
  }
}

```

```

}
}

```

route.ts

frontend-web/app/api/admin/produtos/route.ts • 18 linhas

```

import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function GET() {
  try {
    const res = await fetch(`${API_URL}/api/admin/produtos`);
    if (res.ok) {
      const data = await res.json();
      return NextResponse.json(data);
    }
    return NextResponse.json([], { status: res.status });
  } catch (error) {
    console.error('Erro ao buscar produtos:', error);
    return NextResponse.json([], { status: 500 });
  }
}

```

route.ts

frontend-web/app/api/cliente/funcionarios/route.ts • 73 linhas

```

/**
 * API Route: Gerenciar funcionários de um cliente (Portal do Cliente)
 * GET /api/cliente/funcionarios?cnpj=XXXXXXXXXXXX
 * POST /api/cliente/funcionarios?cnpj=XXXXXXXXXXXX
 */
import { NextRequest, NextResponse } from 'next/server';

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const cnpj = searchParams.get('cnpj');

    if (!cnpj) {
      return NextResponse.json({ erro: 'CNPJ obrigatório' }, { status: 400 });
    }

    const apiUrl = process.env.NEXT_PUBLIC_API_URL || process.env.API_URL || 'http://localhost:8080';
    const res = await fetch(`${apiUrl}/api/cliente/${cnpj}/funcionarios`);

    if (!res.ok) {
      if (res.status === 500 || res.status === 404) {
        return NextResponse.json({ funcionarios: [] });
      }
      const err = await res.json().catch(() => ({}));
      return NextResponse.json({ erro: err.erro || 'Erro ao buscar' }, { status: res.status });
    }

    const data = await res.json();
    return NextResponse.json(data);
  } catch (error: any) {
    console.error('Erro ao buscar funcionários:', error.message);
    return NextResponse.json({
      erro: 'Erro de conexão com o servidor',
      detail: error.message,
      funcionarios: [],
    }, { status: 500 });
  }
}

export async function POST(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const cnpj = searchParams.get('cnpj');

    if (!cnpj) {
      return NextResponse.json({ erro: 'CNPJ obrigatório' }, { status: 400 });
    }

    const body = await request.json();
    const apiUrl = process.env.NEXT_PUBLIC_API_URL || process.env.API_URL || 'http://localhost:8080';

    const res = await fetch(`${apiUrl}/api/cliente/${cnpj}/funcionarios`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body),
    });

    if (!res.ok) {
      const err = await res.json().catch(() => ({}));
      return NextResponse.json({ erro: err.erro || 'Erro ao cadastrar' }, { status: res.status });
    }

    const data = await res.json();
    return NextResponse.json(data, { status: 201 });
  } catch (error: any) {

```



```

    console.error('Erro ao cadastrar funcionário:', error.message);
    return NextResponse.json({
      erro: 'Erro de conexão com o servidor',
      detail: error.message
    }, { status: 500 });
  }
}

```

route.ts

frontend-web/app/api/cliente/funcionarios/cadastrar/route.ts • 53 linhas

```

/**
 * API Route: Cadastrar funcionário (Portal do Cliente)
 * POST /api/cliente/funcionarios?cnpj=XXXXXXXXXXXXX
 */
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const cnpj = searchParams.get('cnpj');

    if (!cnpj) {
      return NextResponse.json({ erro: 'CNPJ obrigatório' }, { status: 400 });
    }

    const body = await request.json();

    if (!body.nome || !body.username || !body.senha) {
      return NextResponse.json({ erro: 'Nome, username e senha são obrigatórios' }, { status: 400 });
    }

    const apiUrl = process.env.NEXT_PUBLIC_API_URL || process.env.API_URL || 'http://localhost:8080';

    console.log(`[cadastrar-funcionario] POST ${apiUrl}/api/cliente/${cnpj}/funcionarios`);

    const res = await fetch(`${apiUrl}/api/cliente/${cnpj}/funcionarios`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        nome: body.nome,
        username: body.username,
        senha: body.senha,
        papel: body.papel || 'GARCOM',
      }),
    });

    if (!res.ok) {
      const err = await res.json().catch(() => ({}));
      console.error('[cadastrar-funcionario] Erro:', res.status, err);
      return NextResponse.json({ erro: err.erro || 'Erro ao cadastrar' }, { status: res.status });
    }

    const data = await res.json();
    return NextResponse.json(data);
  } catch (error: any) {
    console.error('[cadastrar-funcionario] Erro:', error.message);
    return NextResponse.json({
      erro: 'Erro de conexão com o servidor',
      detail: error.message
    }, { status: 500 });
  }
}

```

route.ts

frontend-web/app/api/cliente/funcionarios/[id]/route.ts • 36 linhas

```

/**
 * API Route: Remover funcionário (Portal do Cliente)
 * DELETE /api/cliente/funcionarios/[id]?cnpj=XXXXXXXXXXXXX
 */
import { NextRequest, NextResponse } from 'next/server';

export async function DELETE(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { searchParams } = new URL(request.url);
    const cnpj = searchParams.get('cnpj');
    const { id } = await params;

    if (!cnpj) {
      return NextResponse.json({ erro: 'CNPJ obrigatório' }, { status: 400 });
    }

    const apiUrl = process.env.NEXT_PUBLIC_API_URL || process.env.API_URL || 'http://localhost:8080';

    const res = await fetch(`${apiUrl}/api/cliente/${cnpj}/funcionarios/${id}`, {
      method: 'DELETE',
    });
  }
}

```

```

    if (!res.ok) {
      return NextResponse.json({ erro: 'Erro ao remover funcionário' }, { status: res.status });
    }

    return NextResponse.json({ sucesso: true });
  } catch (error: any) {
    console.error('[remover-funcionario] Erro:', error.message);
    return NextResponse.json({ erro: 'Erro de conexão' }, { status: 500 });
  }
}

```

route.ts

frontend-web/app/api/cliente/licenca/route.ts • 50 linhas

```

/**
 * API Route: Consultar licença do cliente
 * GET /api/cliente/licenca?cnpj=XXXXXXXXXXXXX
 *
 * Proxy para o backend FastAPI: /api/cliente/{cnpj}/licenca
 */
import { NextRequest, NextResponse } from 'next/server';

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const cnpj = searchParams.get('cnpj');

    if (!cnpj) {
      return NextResponse.json({ erro: 'CNPJ obrigatório' }, { status: 400 });
    }

    const apiUrl = process.env.NEXT_PUBLIC_API_URL || process.env.API_URL || 'http://localhost:8080';
    const res = await fetch(`${apiUrl}/api/cliente/${cnpj}/licenca`);

    if (!res.ok) {
      if (res.status === 404 || res.status === 500) {
        // Fallback: retorna licença ativa genérica
        return NextResponse.json({
          cnpj,
          status: 'ATIVA',
          dias_restantes: 30,
          max_usuarios: 5,
          plano: 'mensal',
        });
      }
      const err = await res.json().catch(() => ({}));
      return NextResponse.json({ erro: err.erro || 'Erro ao consultar' }, { status: res.status });
    }

    const data = await res.json();
    return NextResponse.json(data);
  } catch (error: any) {
    console.error('Erro ao consultar licença:', error.message);
    // Fallback silencioso
    return NextResponse.json({
      status: 'ATIVA',
      dias_restantes: 30,
      max_usuarios: 5,
      plano: 'mensal',
      offline: true,
    });
  }
}

```

route.ts

frontend-web/app/api/painel/produtos/route.ts • 35 linhas

```

import { NextResponse } from 'next/server';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export async function GET(request: Request) {
  try {
    const authHeader = request.headers.get('authorization') || '';
    const token = authHeader.replace('Bearer ', '');

    if (!token) {
      return NextResponse.json({ error: 'Não autorizado' }, { status: 401 });
    }

    const res = await fetch(`${API_URL}/api/painel/produtos`, {
      headers: { Authorization: `Bearer ${token}` },
    });

    if (res.ok) {
      const data = await res.json();
      return NextResponse.json(data);
    }

    return NextResponse.json(
      { error: 'Erro ao buscar produtos' },
      { status: res.status }
    );
  }
}

```

```

    });
  } catch (error) {
    console.error('Erro na API do painel:', error);
    return NextResponse.json(
      { error: 'Erro interno' },
      { status: 500 }
    );
  }
}

```

route.ts

frontend-web/app/api/proxy/mesapronta/[...slug]/route.ts • 46 linhas

```

/**
 * API Proxy: Repassa requisições para o backend FastAPI na VPS
 * GET /api/proxy/mesapronta/funcionarios/:cnpj
 * POST /api/proxy/mesapronta/funcionarios/:cnpj
 * DELETE /api/proxy/mesapronta/funcionarios/:cnpj/:id
 */
import { NextRequest, NextResponse } from 'next/server';

const BACKEND_URL = process.env.NEXT_PUBLIC_API_URL || process.env.API_URL || 'http://localhost:8080';

async function handler(req: NextRequest, { params }: { params: Promise<any> }) {
  const resolved = await params;
  const slug = resolved.slug as string[];
  const path = slug.join('/');

  const url = `${BACKEND_URL}/api/mesapronta/${path}`;

  console.log(`[proxy-mesapronta] ${req.method} ${url}`);

  try {
    const init: RequestInit = {
      method: req.method,
      headers: { 'Content-Type': 'application/json' },
    };

    if (req.method === 'POST') {
      const body = await req.json();
      init.body = JSON.stringify(body);
    }

    const res = await fetch(url, init);
    const data = await res.json().catch(() => ({}));

    return NextResponse.json(data, { status: res.status });
  } catch (error: any) {
    console.error(`[proxy-mesapronta] Erro:`, error.message);
    return NextResponse.json({
      erro: 'Erro de conexão com o servidor',
      detail: error.message,
      funcionarios: []
    }, { status: 502 });
  }
}

export { handler as GET, handler as POST, handler as DELETE };

```

route.ts

frontend-web/app/api/seed/route.ts • 47 linhas

```

import { NextResponse } from 'next/server';
import { Pool } from 'pg';
import bcrypt from 'bcryptjs';

const pool = new Pool({
  connectionString: "postgresql://neondb_owner:npg_uvBP7hjZdC9M@ep-weathered-thunder-ap2gc8n9-pooler.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require"
});

export async function GET() {
  try {
    const email = 'admin@corumbasistemas.com.br';
    const senhaPura = 'admin123';

    // 1. Gera o hash seguro da senha usando o bcryptjs
    const salt = await bcrypt.genSalt(10);
    const senhaHash = await bcrypt.hash(senhaPura, salt);

    // 2. Verifica se o admin já existe para não duplicar
    const checkUser = await pool.query('SELECT id FROM usuarios WHERE username = $1', [email]);

    if (checkUser.rows.length > 0) {
      return NextResponse.json({ message: 'O usuário administrador já existe no banco de dados.' });
    }

    // 3. Insere o primeiro administrador master do sistema
    await pool.query(
      'INSERT INTO usuarios (nome, username, senha_hash, papel, ativo) VALUES ($1, $2, $3, $4, $5)',
      ['Admin Corumbá', email, senhaHash, 'ADMIN', true]
    );
  }
}

```

```

    });

    return NextResponse.json({
      success: true,
      message: 'Primeiro acesso criado com sucesso!',
      usuario: email,
      senha: textToStars(senhaPura)
    });

  } catch (error) {
    console.error(error);
    return NextResponse.json({ error: 'Erro ao criar conta master: ' + error }, { status: 500 });
  }
}

function textToStars(text: string) {
  return '•'.repeat(text.length);
}

```

route.ts

frontend-web/app/api/usuarios/route.ts • 75 linhas

```

import { NextResponse } from 'next/server';
import { Pool } from 'pg';
import bcrypt from 'bcryptjs';

const pool = new Pool({
  connectionString: "postgresql://neondb_owner:npg_n0pgcEzU2JM7@ep-cold-field-aqxfpb3m-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require"
});

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const documento = searchParams.get('documento');

    if (!documento) {
      return NextResponse.json({ error: 'O CNPJ ou CPF da empresa é obrigatório.' }, { status: 400 });
    }

    const { rows } = await pool.query(
      'SELECT id, nome, username as login, papel, ativo FROM usuarios WHERE empresa_documento = $1 ORDER BY id ASC',
      [documento]
    );

    return NextResponse.json(rows);
  } catch (error) {
    return NextResponse.json({ error: 'Erro ao buscar usuários' }, { status: 500 });
  }
}

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { nome, login, senha, papel, empresa_documento } = body;

    // 1. TRAVA DE SEGURANÇA: Verifica se o frontend enviou tudo
    if (!nome || !login || !senha || !empresa_documento) {
      return NextResponse.json({
        error: 'Dados incompletos! Recebido -> Nome: ${nome}, Login: ${login}, Doc: ${empresa_documento}'
      }, { status: 400 });
    }

    // 2. Verifica se o login já existe na loja
    const check = await pool.query('SELECT id FROM usuarios WHERE username = $1 AND empresa_documento = $2', [login, empresa_documento]);
    if (check.rows.length > 0) {
      return NextResponse.json({ error: 'Esse login já está em uso na sua loja.' }, { status: 400 });
    }

    // 3. Criptografa a senha para o PDV
    const salt = await bcrypt.genSalt(10);
    const senhaHash = await bcrypt.hash(senha, salt);

    // 4. Insere no banco
    await pool.query(
      'INSERT INTO usuarios (nome, username, senha_hash, papel, ativo, empresa_documento) VALUES ($1, $2, $3, $4, $5, $6)',
      [nome, login, senhaHash, papel, true, empresa_documento]
    );

    return NextResponse.json({ success: true });
  } catch (error: any) {
    // 🚨 AGORA ELE MOSTRA O ERRO EXATO DO BANCO DE DADOS
    console.error("ERRO DETALHADO NO BANCO:", error);
    return NextResponse.json({ error: `Erro no Postgres: ${error.message}` }, { status: 500 });
  }
}

export async function DELETE(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const id = searchParams.get('id');

    await pool.query('DELETE FROM usuarios WHERE id = $1', [id]);
    return NextResponse.json({ success: true });
  }
}

```

```
    } catch (error) {  
      return NextResponse.json({ error: 'Erro ao deletar usuário' }, { status: 500 });  
    }  
  }  
}
```

9. Frontend Web - Configuracao

next.config.ts

frontend-web/next.config.ts • 10 linhas

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  allowedDevOrigins: ['191.252.210.235', 'localhost'],
  images: {
    unoptimized: true,
  }
};

export default nextConfig;
```

package.json

frontend-web/package.json • 33 linhas

```
{
  "name": "frontend-web",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "jsonwebtoken": "^9.0.3",
    "next": "16.2.6",
    "pg": "^8.21.0",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/bcryptjs": "^2.4.6",
    "@types/jsonwebtoken": "^9.0.10",
    "@types/node": "^20",
    "@types/pg": "^8.20.0",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.2.6",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

tsconfig.json

frontend-web/tsconfig.json • 38 linhas

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": false,
    "noEmit": true,
    "incremental": true,
    "module": "esnext",
    "esModuleInterop": true,
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "plugins": [
      {
        "name": "next"
      }
    ]
  },
  "include": [
    "next-env.d.ts",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts",
    "**/*.ts",
    "**/*.tsx"
  ]
}
```

```
  ],  
  "exclude": [  
    "node_modules"  
  ]  
}
```

AGENTS.md

frontend-web/AGENTS.md • 6 linhas

```
<!-- BEGIN:nextjs-agent-rules -->  
# This is NOT the Next.js you know  
  
This version has breaking changes – APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in  
'node_modules/next/dist/docs/' before writing any code. Heed deprecation notices.  
<!-- END:nextjs-agent-rules -->
```

next-env.d.ts

frontend-web/next-env.d.ts • 7 linhas

```
/// <reference types="next" />  
/// <reference types="next/image-types/global" />  
import "next/dev/types/routes.d.ts";  
  
// NOTE: This file should not be edited  
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

10. Desktop - Flet (Python)

 main.py

desktop/main.py • 675 linhas

```
"""
SALTO GESTAO Desktop v2.0 - ARQUIVO UNICO
Design Moderno & Clean - Paleta Corumba Sistemas
NAO precisa de tkinter! Usa Flet (Flutter-based).

Instalacao: pip install flet requests
Execucao: python main.py
"""

import flet as ft
import requests
import threading
from datetime import datetime

# =====
# PALETA DE CORES CORUMBA SISTEMAS
# =====
P      = "#0b1a30"
P80    = "#1a2d4a"
P50    = "#5a7ea8"
P20    = "#9bb5d0"
P10    = "#c5d5e8"
P5     = "#e2e8f0"
FUNDO  = "#f5f7fa"
BRANCO = "#ffffff"
TEXTO  = "#0b1a30"
TEXT080 = "#3c4a5e"
TEXT060 = "#6d7a8e"
TEXT040 = "#9eaabe"
OK      = "#22c55e"
ERRO    = "#ef4444"
ALERTA  = "#f59e0b"
INFO    = "#3b82f6"
FUNDO_CARD = "#ffffff"
BORDA   = "#e2e8f0"

API_URL = "http://191.252.210.235:8080"

# =====
# CLIENTE API
# =====
class API:
    def __init__(self):
        self.base = API_URL
        self.token = None
        self.user = None
        self.h = {"Content-Type": "application/json"}

    def set_token(self, t):
        self.token = t
        self.h["Authorization"] = "Bearer " + t

    def login(self, u, p):
        try:
            r = requests.post(self.base + "/api/login", json={"username": u, "password": p}, timeout=10)
            if r.status_code == 200:
                d = r.json()
                self.set_token(d.get("access_token", ""))
                self.user = d.get("user", {})
                return {"ok": True}
            return {"ok": False, "err": r.json().get("detail", "Erro")}
        except requests.exceptions.ConnectionError:
            return {"ok": False, "err": "Sem conexao com o servidor"}
        except Exception as e:
            return {"ok": False, "err": str(e)}

    def get(self, ep):
        try:
            r = requests.get(self.base + ep, headers=self.h, timeout=15)
            return {"ok": True, "data": r.json()} if r.status_code == 200 else {"ok": False}
        except:
            return {"ok": False}

    def post(self, ep, data):
        try:
            r = requests.post(self.base + ep, json=data, headers=self.h, timeout=15)
            return {"ok": True} if r.status_code in (200, 201) else {"ok": False}
        except:
            return {"ok": False}

    def put(self, ep, data):
        try:
            r = requests.put(self.base + ep, json=data, headers=self.h, timeout=15)
            return {"ok": True} if r.status_code == 200 else {"ok": False}
```



```

except:
    return {"ok": False}

def delete(self, ep):
    try:
        r = requests.delete(self.base + ep, headers=self.h, timeout=15)
        return {"ok": True}
    except:
        return {"ok": False}

api = API()

# =====
# HELPER: Campo de entrada moderno
# =====
def campo(label, icon=None, pwd=False, hint=None, w=None, **kw):
    return ft.TextField(
        label=label, prefix_icon=icon, password=pwd,
        hint_text=hint, hint_style=ft.TextStyle(size=12, color=TEXT040),
        label_style=ft.TextStyle(size=11, color=TEXT060),
        text_style=ft.TextStyle(size=13, color=TEXT0),
        border_color=BORDA, focused_border_color=P, focused_border_width=2,
        border_radius=12, bgcolor=BRANCO,
        content_padding=ft.padding.only(left=16, right=16, top=12, bottom=12),
        width=w, **kw
    )

# =====
# HELPER: Botao primario
# =====
def btn(texto, icon=None, w=None, h=44, on_click=None, **kw):
    content = ft.Row([
        ft.Icon(icon, size=16, color=BRANCO) if icon else ft.Container(),
        ft.Text(texto, color=BRANCO, size=14, weight=ft.FontWeight.W_600),
    ], alignment=ft.MainAxisAlignment.CENTER, spacing=8) if icon else texto

    return ft.ElevatedButton(
        content=content, width=w, height=h, on_click=on_click,
        style=ft.ButtonStyle(
            bgcolor={ft.ControlState.DEFAULT: P, ft.ControlState.HOVERED: P80},
            color=BRANCO, shape=ft.RoundedRectangleBorder(radius=12),
            padding=ft.padding.symmetric(horizontal=24, vertical=12),
            elevation={ft.ControlState.DEFAULT: 0, ft.ControlState.HOVERED: 2},
            animation_duration=150,
        ), **kw
    )

# =====
# HELPER: Card
# =====
def card(content=None, padding=16, shadow=True, border=False, **kw):
    return ft.Container(
        content=content, bgcolor=BRANCO, border_radius=16, padding=padding,
        shadow=ft.BoxShadow(spread_radius=0, blur_radius=8,
            color=ft.Colors.with_opacity(0.06, TEXT0), offset=ft.Offset(0, 2)) if shadow else None,
        border=ft.border.all(1, BORDA) if border else None, **kw
    )

# =====
# COMPONENTE: Sidebar Item
# =====
def sidebar_item(label, icon, selected=False, on_click=None):
    return ft.Container(
        content=ft.Row([
            ft.Icon(icon, size=20, color=P if selected else TEXT060),
            ft.Container(width=4),
            ft.Text(label, size=13, weight=ft.FontWeight.W_600 if selected else ft.FontWeight.W_400,
                color=P if selected else TEXT080),
        ], spacing=12, alignment=ft.MainAxisAlignment.START),
        padding=ft.padding.symmetric(horizontal=16, vertical=10),
        bgcolor=ft.Colors.with_opacity(0.08, P) if selected else ft.Colors.TRANSPARENT,
        border_radius=12, on_click=on_click, animate=ft.Animation(150),
        margin=ft.padding.symmetric(horizontal=8, vertical=2),
    )

# =====
# COMPONENTE: Badge
# =====
def badge(texto, cor=P):
    return ft.Container(
        content=ft.Text(texto, size=10, weight=ft.FontWeight.W_600, color=cor),
        bgcolor=ft.Colors.with_opacity(0.1, cor), border_radius=9999,
        padding=ft.padding.symmetric(horizontal=8, vertical=3),
    )

# =====
# COMPONENTE: Stat Card
# =====

```

```

def stat_card(titulo, valor, icone, cor=P):
    return card(content=ft.Column([
        ft.Row([
            ft.Container(content=ft.Icon(icone, size=20, color=cor),
                bgcolor=ft.Colors.with_opacity(0.1, cor), border_radius=8, padding=8),
            ],
            ft.Container(height=4),
            ft.Text(titulo, size=11, color=TEXT060),
            ft.Text(str(valor), size=26, weight=ft.FontWeight.W_700, color=TEXT0),
        ], spacing=2), padding=16, width=200, height=90)

# =====
# TELA DE LOGIN
# =====
def tela_login(page, on_ok):
    user = campo("Usuario", icon=ft.Icons.PERSON_OUTLINED, hint="Digite seu usuario", w=360)
    pwd = campo("Senha", icon=ft.Icons.LOCK_OUTLINED, pwd=True, can_reveal_password=True, hint="Digite sua senha", w=360)
    msg = ft.Container(visible=False)

    def show(text, tipo="err"):
        cols = {"err": ERRO, "ok": OK}
        bgs = {"err": ft.Colors.with_opacity(0.08, ERRO), "ok": ft.Colors.with_opacity(0.08, OK)}
        icons = {"err": ft.Icons.ERROR_OUTLINED, "ok": ft.Icons.CHECK_CIRCLE_OUTLINED}
        msg.content = ft.Container(
            content=ft.Row([ft.Icon(icons.get(tipo), size=16, color=TEXT0), ft.Container(width=8),
                ft.Text(text, size=12, color=TEXT080)], spacing=8),
            bgcolor=bgs.get(tipo), border_radius=8, padding=ft.padding.symmetric(horizontal=12, vertical=8),
            border=ft.Border.all(1, ft.Colors.with_opacity(0.15, cols.get(tipo))), w=360)
        msg.visible = True
        page.update()

    def do_login(e):
        if not user.value or not pwd.value:
            show("Preencha todos os campos"); return
        result = api.login(user.value, pwd.value)
        if result["ok"]:
            show("Login realizado!", "ok")
            page.update()
            import time; time.sleep(0.4)
            on_ok()
        else:
            show(result["err"])
            page.update()

    return ft.Container(
        alignment=ft.alignment.center, expand=True,
        gradient=ft.LinearGradient(
            begin=ft.alignment.top_center, end=ft.alignment.bottom_center,
            colors=[FUNDO, BRANCO]),
        content=card(content=ft.Column([
            # Logo
            ft.Container(
                content=ft.Column([
                    ft.Container(content=ft.Icon(ft.Icons.STORE_ROUNDED, size=40, color=P),
                        bgcolor=ft.Colors.with_opacity(0.1, P), border_radius=9999, padding=20,
                        width=80, height=80, alignment=ft.alignment.center),
                    ft.Container(height=8),
                    ft.Text("Salto Gestao", size=24, weight=ft.FontWeight.W_700, color=P),
                    ft.Text("Sistema de Gestao Empresarial", size=12, color=TEXT060),
                ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
                alignment=ft.alignment.center, padding=ft.padding.only(top=16)),
            ft.Container(height=32),
            user, ft.Container(height=8), pwd, ft.Container(height=8),
            msg, ft.Container(height=8),
            btn("Entrar", icon=ft.Icons.ARROW_FORWARD, w=360, h=48, on_click=do_login),
            ft.Container(height=16),
        ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=4),
        padding=40, w=420,
        shadow=ft.BoxShadow(spread_radius=0, blur_radius=24,
            color=ft.Colors.with_opacity(0.12, TEXT0), offset=ft.Offset(0, 8)))
    )

# =====
# TELA: DASHBOARD
# =====
def tela_dashboard(page):
    stats = ft.Row(spacing=12, wrap=True, scroll=ft.ScrollMode.AUTO)

    def load():
        stats.controls.clear()
        cr = api.get("/api/clientes/")
        pr = api.get("/api/prodidos/")
        vr = api.get("/api/vendas/")
        nc = len(cr["data"]) if cr["ok"] else 0
        np = len(pr["data"]) if pr["ok"] else 0
        nv = len(vr["data"]) if vr["ok"] else 0
        total = sum(v.get("total", 0) for v in vr["data"] if vr["ok"] and vr["data"] else 0)

        for t, v, ic, c in [("Clientes", str(nc), ft.Icons.PEOPLE_OUTLINED, P),
            ("Produtos", str(np), ft.Icons.INVENTORY_2_OUTLINED, INFO),
            ("Vendas", str(nv), ft.Icons.POINT_OF_SALE, OK),
            ("Faturamento", "R$ %.2f" % total, ft.Icons.ATTACH_MONEY, ALERTA)]:
            stats.controls.append(stat_card(t, v, ic, c))

```

```

page.update()

return ft.Container(
    content=ft.Column([
        ft.Container(
            content=ft.Row([
                ft.Column([
                    ft.Text("Dashboard", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                    ft.Text("Bem-vindo, " + str(api.user.get("nome", "Usuario")) + "!", size=13, color=TEXT060),
                ], spacing=4),
                ft.Row([ft.IconButton(icon=ft.Icons.REFRESH, icon_color=TEXT060,
                    bgcolor=P5, on_click=lambda e: load(),
                    style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8))),
                    expand=True, alignment=ft.MainAxisAlignment.END),
            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUND0),
            stats,
            ft.Container(content=ft.Divider(height=1, color=BORDA), padding=ft.padding.symmetric(horizontal=24)),
            ft.Text("Resumo do Negocio", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
                padding=ft.padding.only(left=28, top=8)),
        ], spacing=16, expand=True),
        bgcolor=FUND0, expand=True, on_mount=lambda e: load()
    )

# =====
# TELA: CLIENTES
# =====
def tela_clientes(page):
    lista = ft.ListView(spacing=8, padding=0, expand=True)
    nome = campo("Nome completo", hint="Ex: Joao Silva")
    cpf = campo("CPF/CNPJ", hint="000.000.000-00")
    email = campo("E-mail", hint="email@exemplo.com")
    tel = campo("Telefone", hint="(62) 99999-9999")
    msg = ft.Container(visible=False)
    edit_id = {"v": None}

    def show(text, tipo="ok"):
        icons = {"ok": ft.Icons.CHECK_CIRCLE_OUTLINED, "err": ft.Icons.ERROR_OUTLINED,
            "info": ft.Icons.INFO_OUTLINED}
        bgs = {"ok": ft.Colors.with_opacity(0.08, OK), "err": ft.Colors.with_opacity(0.08, ERRO),
            "info": ft.Colors.with_opacity(0.08, INFO)}
        msg.content = ft.Container(
            content=ft.Row([ft.Text(text, size=12, color=TEXT080)], spacing=8),
            bgcolor=bgs.get(tipo), border_radius=8, padding=ft.padding.symmetric(horizontal=12, vertical=8),
            margin=ft.padding.only(bottom=8))
        msg.visible = True
        page.update()

    def load():
        lista.controls.clear()
        r = api.get("/api/clientes/")
        if r["ok"] and r["data"]:
            for c in r["data"]:
                lista.controls.append(card(
                    padding=0, shadow=False, border=True,
                    content=ft.Container(
                        content=ft.Row([
                            ft.CircleAvatar(content=ft.Text(c.get("nome", "?")[0].upper(), size=16,
                                weight=ft.FontWeight.W_600, color=BRANCO), bgcolor=P20, radius=20),
                            ft.Container(width=8),
                            ft.Column([
                                ft.Text(c.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=TEXT0),
                                ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=11, color=TEXT060),
                                ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=11, color=TEXT060),
                            ], spacing=2, expand=True),
                            ft.IconButton(icon=ft.Icons.EDIT_OUTLINED, icon_color=INFO,
                                bgcolor=ft.Colors.with_opacity(0.08, INFO), tooltip="Editar",
                                on_click=lambda e, cli=c: edit(cli),
                                style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                            ft.IconButton(icon=ft.Icons.DELETE_OUTLINED, icon_color=ERRO,
                                bgcolor=ft.Colors.with_opacity(0.08, ERRO), tooltip="Excluir",
                                on_click=lambda e, cli=c: delete(cli),
                                style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                        ], spacing=8, alignment=ft.MainAxisAlignment.START,
                        padding=ft.padding.symmetric(horizontal=16, vertical=10)))
                else:
                    lista.controls.append(ft.Container(
                        content=ft.Column([
                            ft.Icon(ft.Icons.PEOPLE_OUTLINED, size=40, color=TEXT040),
                            ft.Text("Nenhum cliente cadastrado", size=14, color=TEXT060),
                        ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
                        alignment=ft.alignment.center, padding=40))
        page.update()

    def save(e):
        data = {"nome": nome.value or "", "cpf_cnpj": cpf.value or "", "email": email.value or "", "telefone": tel.value or ""}
        if not data["nome"]:
            show("Nome e obrigatorio", "err"); return
        if edit_id["v"]:
            r = api.put("/api/clientes/" + str(edit_id["v"]), data)
        else:
            r = api.post("/api/clientes/", data)
        if r["ok"]:
            show("Cliente salvo!", "ok")

```

```

        for f in [nome, cpf, email, tel]: f.value = ""
        edit_id["v"] = None
        load()
    else:
        show("Erro ao salvar", "err")

def edit(c):
    edit_id["v"] = c.get("id")
    nome.value = c.get("nome", "")
    cpf.value = c.get("cpf_cnpj", "")
    email.value = c.get("email", "")
    tel.value = c.get("telefone", "")
    show("Editando: " + str(c.get("nome", "")), "info")
    page.update()

def delete(c):
def ok(e):
    if e.control.text == "Sim":
        r = api.delete("/api/clientes/" + str(c.get("id")))
        if r["ok"]: show("Excluido!", "ok"); load()
        else: show("Erro ao excluir", "err")
        dlg.open = False; page.update()
    dlg = ft.AlertDialog(
        title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
        content=ft.Text("Excluir '" + str(c.get("nome")) + "'?"),
        actions=[ft.TextButton("Nao", on_click=ok),
                 ft.TextButton("Sim", on_click=ok, style=ft.ButtonStyle(color=ERRO))]
    )
    page.overlay.append(dlg); dlg.open = True; page.update()

def novo(e):
    edit_id["v"] = None
    for f in [nome, cpf, email, tel]: f.value = ""
    show("Novo cliente", "info"); page.update()

return ft.Container(
    content=ft.Column([
        ft.Container(
            content=ft.Row([
                ft.Column([ft.Text("Clientes", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                             ft.Text("Gerencie seus clientes", size=13, color=TEXT060)], spacing=4),
                ft.Row([btn("Novo", icon=ft.Icons.ADD, w=100, on_click=novo)], spacing=8,
                        expand=True, alignment=ft.MainAxisAlignment.END),
            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
              padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUNDO),
        # Form
        card(
            titulo="Formulario",
            content=ft.Column([
                ft.Row([nome, cpf], spacing=12),
                ft.Row([email, tel], spacing=12),
                ft.Row([btn("Salvar", icon=ft.Icons.SAVE_OUTLINED, w=140, on_click=save),
                        ft.OutlinedButton(text="Limpar", on_click=lambda e: novo(e),
                                          style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=12),
                                                            side=ft.BorderSide(1.5, BORDA),
                                                            padding=ft.padding.symmetric(horizontal=20, vertical=12)))]),
                spacing=8),
            msg,
            ], spacing=12, padding=16),
        ft.Text(" Clientes cadastrados", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
                padding=ft.padding.only(left=4, bottom=8)),
        ft.Container(content=lista, expand=True),
    ], spacing=16, expand=True),
    padding=ft.padding.all(24), bgcolor=FUNDO, expand=True, on_mount=lambda e: load()
)

# =====
# TELA: PRODUTOS
# =====
def tela_produtos(page):
    lista = ft.ListView(spacing=8, padding=0, expand=True)
    nome = campo("Nome do produto", hint="Ex: Arroz 5kg")
    preco = campo("Preco (R$)", hint="0.00")
    cat = campo("Categoria", hint="Alimentos")
    estoque = campo("Estoque", hint="0")
    msg = ft.Container(visible=False)
    edit_id = {"v": None}

    def show(text, tipo="ok"):
        icons = {"ok": ft.Icons.CHECK_CIRCLE_OUTLINED, "err": ft.Icons.ERROR_OUTLINED}
        bgs = {"ok": ft.Colors.with_opacity(0.08, OK), "err": ft.Colors.with_opacity(0.08, ERRO)}
        msg.content = ft.Container(
            content=ft.Row([ft.Text(text, size=12, color=TEXT080)], spacing=8),
            bgcolor=bgs.get(tipo), border_radius=8,
            padding=ft.padding.symmetric(horizontal=12, vertical=8), margin=ft.padding.only(bottom=8))
        msg.visible = True
        page.update()

    def load():
        lista.controls.clear()
        r = api.get("/api/produtos/")
        if r["ok"] and r["data"]:
            for p in r["data"]:
                lista.controls.append(card(
                    padding=0, shadow=False, border=True,

```

```

        content=ft.Container(
            content=ft.Row([
                ft.Container(content=ft.Icon(ft.Icons.INVENTORY_2, size=16, color=INFO),
                    bgcolor=ft.Colors.with_opacity(0.1, INFO), border_radius=8,
                    padding=8, w=40, h=40, alignment=ft.alignment.center),
                ft.Container(width=8),
                ft.Column([
                    ft.Text(p.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=TEXT0),
                    ft.Text("R$ %.2f" % p.get("preco", 0), size=12, weight=ft.FontWeight.W_700, color=OK),
                    ft.Text(str(p.get("estoque", 0)) + " em estoque | " + str(p.get("categoria", "")), size=10, color=TEXT060),
                ], spacing=2, expand=True),
                ft.IconButton(icon=ft.Icons.EDIT_OUTLINED, icon_color=INFO,
                    on_click=lambda e, pr=p: edit(pr),
                    style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
                ft.IconButton(icon=ft.Icons.DELETE_OUTLINED, icon_color=ERROR,
                    on_click=lambda e, pr=p: delete(pr),
                    style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
            ], spacing=8, alignment=ft.MainAxisAlignment.START,
            padding=ft.padding.symmetric(horizontal=16, vertical=10)))
    else:
        lista.controls.append(ft.Container(
            content=ft.Column([ft.Icon(ft.Icons.INVENTORY_2_OUTLINED, size=40, color=TEXT040),
                ft.Text("Nenhum produto", size=14, color=TEXT060)],
            horizontal_alignment=ft.CrossAxisAlignment.CENTER),
            alignment=ft.alignment.center, padding=40))
    page.update()

def save(e):
    try:
        pr = float(preco.value or 0)
        es = int(estoque.value or 0)
    except ValueError:
        show("Preco e estoque devem ser numeros", "err"); return
    data = {"nome": nome.value or "", "preco": pr, "categoria": cat.value or "", "estoque": es}
    if not data["nome"]: show("Nome e obrigatorio", "err"); return
    if edit_id["v"]:
        r = api.put("/api/produtos/" + str(edit_id["v"]), data)
    else:
        r = api.post("/api/produtos/", data)
    if r["ok"]:
        show("Produto salvo!", "ok")
        for f in [nome, preco, cat, estoque]: f.value = ""
        edit_id["v"] = None; load()
    else:
        show("Erro ao salvar", "err")

def edit(p):
    edit_id["v"] = p.get("id"); nome.value = p.get("nome", "")
    preco.value = str(p.get("preco", "")); cat.value = p.get("categoria", "")
    estoque.value = str(p.get("estoque", "")); page.update()

def delete(p):
    def ok(e):
        if e.control.text == "Sim":
            r = api.delete("/api/produtos/" + str(p.get("id")))
            if r["ok"]: show("Excluido!", "ok"); load()
            dlg.open = False; page.update()
        dlg = ft.AlertDialog(title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
            content=ft.Text("Excluir '" + str(p.get("nome")) + "'?"),
            actions=[ft.TextButton("Nao", on_click=ok), ft.TextButton("Sim", on_click=ok)])
        page.overlay.append(dlg); dlg.open = True; page.update()

    return ft.Container(
        content=ft.Column([
            ft.Container(
                content=ft.Row([
                    ft.Column([ft.Text("Produtos", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                        ft.Text("Gerencie seu estoque", size=13, color=TEXT060)], spacing=4),
                    ft.Row([btn("Novo", icon=ft.Icons.ADD, w=100, on_click=lambda e: (edit_id.__setitem__("v", None), [f.__setattr__("value",
                        "")) for f in [nome, preco, cat, estoque]), page.update())], spacing=8, expand=True, alignment=ft.MainAxisAlignment.END),
                ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
                padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUNDO),
            card(content=ft.Column([
                ft.Row([nome, preco], spacing=12), ft.Row([cat, estoque], spacing=12),
                ft.Row([btn("Salvar", icon=ft.Icons.SAVE_OUTLINED, w=140, on_click=save)], spacing=8),
            ], spacing=12), padding=16),
            ft.Text("Produtos cadastrados", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
                padding=ft.padding.only(left=4, bottom=8)),
            ft.Container(content=lista, expand=True),
        ], spacing=16, expand=True),
        padding=ft.padding.all(24), bgcolor=FUNDO, expand=True, on_mount=lambda e: load())

# =====
# TELA: VENDAS
# =====
def tela_vendas(page):
    lista = ft.ListView(spacing=8, padding=0, expand=True)

    def load():
        lista.controls.clear()
        r = api.get("/api/vendas/")

```

```

if r["ok"] and r["data"]:
    total = 0
    for v in r["data"]:
        t = v.get("total", 0); total += t
        lista.controls.append(card(
            padding=0, shadow=False, border=True,
            content=ft.Container(
                content=ft.Row([
                    ft.Container(content=ft.Icon(ft.Icons.POINT_OF_SALE, size=16, color=OK),
                                bgcolor=ft.Colors.with_opacity(0.1, OK), border_radius=8,
                                padding=6, w=36, h=36, alignment=ft.alignment.center),
                    ft.Container(width=8),
                    ft.Column([
                        ft.Text("Venda #" + str(v.get("id",0)), size=13, weight=ft.FontWeight.W_600),
                        ft.Text(str(v.get("cliente_nome","N/A")), size=11, color=TEXT060),
                    ], spacing=2, expand=True),
                    ft.Text("R$ %.2f" % t, size=14, weight=ft.FontWeight.W_700, color=OK),
                ], spacing=8, alignment=ft.MainAxisAlignment.START,
                padding=ft.padding.symmetric(horizontal=16, vertical=10)))
        lista.controls.insert(0, card(
            content=ft.Row([
                ft.Text(str(len(r["data"])) + " vendas", size=12, color=TEXT060),
                ft.Text("Total: R$ %.2f" % total, size=14, weight=ft.FontWeight.W_700, color=P),
            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            padding=16, shadow=False, bgcolor=ft.Colors.with_opacity(0.03, P),
            border=ft.border.all(1, P20)))
    page.update()

return ft.Container(
    content=ft.Column([
        ft.Container(
            content=ft.Column([ft.Text("Vendas", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
                                ft.Text("Historico de vendas", size=13, color=TEXT060)], spacing=4),
            padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUNDO),
        ft.Container(content=lista, expand=True),
    ], spacing=16),
    padding=ft.padding.all(24), bgcolor=FUNDO, expand=True, on_mount=lambda e: load())
)

# =====
# LAYOUT PRINCIPAL
# =====
def layout_principal():
    main = ft.Container(expand=True, bgcolor=FUNDO)
    sel = {"i": 0}

    def nav(i):
        sel["i"] = i
        telas = {0: tela_dashboard, 1: tela_clientes, 2: tela_produtos, 3: tela_vendas}
        if i in telas:
            main.content = telas[i](main.page)
            main.page.update()
        for j, item in enumerate(nav_items):
            item.bgcolor = ft.Colors.with_opacity(0.08, P) if j == i else ft.Colors.TRANSPARENT
        page.update()

    nav_items = [
        sidebar_item("Dashboard", ft.Icons.DASHBOARD, True, lambda e: nav(0)),
        sidebar_item("Clientes", ft.Icons.PEOPLE_OUTLINED, False, lambda e: nav(1)),
        sidebar_item("Produtos", ft.Icons.INVENTORY_2_OUTLINED, False, lambda e: nav(2)),
        sidebar_item("Vendas", ft.Icons.POINT_OF_SALE_OUTLINED, False, lambda e: nav(3)),
    ]

    sidebar = ft.Container(
        content=ft.Column([
            ft.Container(content=ft.Row([
                ft.Container(content=ft.Icon(ft.Icons.STORE, size=20, color=BRANCO),
                            bgcolor=P, border_radius=8, padding=8),
                ft.Container(width=6),
                ft.Column([ft.Text("SaltoGestao", size=13, weight=ft.FontWeight.W_700, color=BRANCO),
                            ft.Text("v2.0", size=8, color=P20)], spacing=0),
            ]), padding=ft.padding.symmetric(horizontal=12, vertical=16),
            margin=ft.padding.only(top=8, bottom=16)),
            ft.Column(nav_items, spacing=2),
            ft.Container(expand=True),
            ft.Container(content=ft.Text("Corumba Sistemas", size=8, color=P20),
                        padding=ft.padding.all(12)),
        ], spacing=0),
        width=220, bgcolor=P, padding=ft.padding.symmetric(vertical=8))

    return ft.Container(
        content=ft.Row([
            sidebar,
            ft.VerticalDivider(width=1, color=BORDA),
            main,
        ], expand=True, spacing=0),
        expand=True)

# =====
# APP PRINCIPAL
# =====
def main(page: ft.Page):
    page.title = "SaltoGestao v2.0"

```

```

page.padding = 0
page.window.width = 1280
page.window.height = 800
page.window.min_width = 1024
page.window.min_height = 700
page.bgcolor = FUNDO
page.theme = ft.Theme(color_scheme_seed=P, font_family="Segoe UI", use_material3=True)

mc = ft.Container(expand=True, bgcolor=FUNDO)

def on_login():
    mc.content = layout_principal()
    page.update()

mc.content = tela_login(page, on_login)
page.add(mc)

if __name__ == "__main__":
    ft.run(main, view=ft.AppView.FLET_APP)

```

main_flet.py

desktop/main_flet.py • 910 linhas

```

"""
SaltoGestao - Sistema Desktop com Flet
Desenvolvido por Corumba Sistemas
Interface moderna para gestao de empresas

Instalacao: pip install flet requests
Execucao: flet run main_flet.py
          ou: python main_flet.py
"""

import flet as ft
import requests
import json
import threading
from datetime import datetime

# =====
# CONFIGURACAO
# =====
API_URL = "http://191.252.210.235:8080"

# =====
# CLIENTE DA API
# =====
class APIClient:
    """Cliente para comunicacao com a API FastAPI do SaltoGestao"""

    def __init__(self):
        self.base_url = API_URL
        self.token = None
        self.user = None
        self.headers = {"Content-Type": "application/json"}

    def set_token(self, token):
        self.token = token
        self.headers["Authorization"] = "Bearer " + token

    def login(self, username, password):
        try:
            resp = requests.post(
                self.base_url + "/api/login",
                json={"username": username, "password": password},
                timeout=10,
            )
            if resp.status_code == 200:
                data = resp.json()
                self.set_token(data.get("access_token", ""))
                self.user = data.get("user", {})
                return {"success": True, "data": data}
            return {"success": False, "error": resp.json().get("detail", "Erro no login")}
        except requests.exceptions.ConnectionError:
            return {"success": False, "error": "Nao foi possivel conectar ao servidor"}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def get(self, endpoint):
        try:
            resp = requests.get(self.base_url + endpoint, headers=self.headers, timeout=15)
            if resp.status_code == 200:
                return {"success": True, "data": resp.json()}
            return {"success": False, "error": resp.text}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def post(self, endpoint, data):
        try:
            resp = requests.post(
                self.base_url + endpoint, json=data, headers=self.headers, timeout=15
            )

```

```

        if resp.status_code in (200, 201):
            return {"success": True, "data": resp.json()}
        return {"success": False, "error": resp.text}
    except Exception as e:
        return {"success": False, "error": str(e)}

def put(self, endpoint, data):
    try:
        resp = requests.put(
            self.base_url + endpoint, json=data, headers=self.headers, timeout=15
        )
        if resp.status_code == 200:
            return {"success": True, "data": resp.json()}
        return {"success": False, "error": resp.text}
    except Exception as e:
        return {"success": False, "error": str(e)}

def delete(self, endpoint):
    try:
        resp = requests.delete(self.base_url + endpoint, headers=self.headers, timeout=15)
        if resp.status_code in (200, 204):
            return {"success": True}
        return {"success": False, "error": resp.text}
    except Exception as e:
        return {"success": False, "error": str(e)}

api = APIClient()

# =====
# TELA DE LOGIN
# =====
def criar_tela_login(page, on_login_success):
    username_field = ft.TextField(
        label="Usuário",
        prefix_icon=ft.Icons.PERSON,
        border_radius=10,
        width=350,
        text_size=15,
    )
    password_field = ft.TextField(
        label="Senha",
        prefix_icon=ft.Icons.LOCK,
        password=True,
        can_reveal_password=True,
        border_radius=10,
        width=350,
        text_size=15,
    )
    msg_text = ft.Text("", size=14, weight=ft.FontWeight.BOLD)
    loading = ft.ProgressRing(width=20, height=20, visible=False)

    def do_login(e):
        if not username_field.value or not password_field.value:
            msg_text.value = "Preencha todos os campos!"
            msg_text.color = ft.Colors.ORANGE
            page.update()
            return
        loading.visible = True
        msg_text.value = "Conectando..."
        msg_text.color = ft.Colors.BLUE
        page.update()
        result = api.login(username_field.value, password_field.value)
        loading.visible = False
        if result["success"]:
            msg_text.value = "Login realizado com sucesso!"
            msg_text.color = ft.Colors.GREEN
            page.update()
            on_login_success()
        else:
            msg_text.value = result["error"]
            msg_text.color = ft.Colors.RED
            page.update()

    return ft.Container(
        content=ft.Column(
            [
                ft.Icon(ft.Icons.STORE, size=80, color=ft.Colors.BLUE_700),
                ft.Text("SaltoGestao", size=36, weight=ft.FontWeight.BOLD, color=ft.Colors.BLUE_900),
                ft.Text("Sistema de Gestao Empresarial", size=14, color=ft.Colors.GREY_600),
                ft.Divider(height=20, color=ft.Colors.TRANSPARENT),
                username_field,
                ft.Container(height=5),
                password_field,
                ft.Container(height=10),
                ft.ElevatedButton(
                    content=ft.Row(
                        [ft.Icon(ft.Icons.LOGIN), ft.Text("Entrar", size=16, weight=ft.FontWeight.BOLD)],
                        alignment=ft.MainAxisAlignment.CENTER,
                    ),
                    width=350,
                    height=50,
                    bgcolor=ft.Colors.BLUE_700,
                    color=ft.Colors.WHITE,
                ),
            ],
        ),
    )

```



```

        style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=10)),
        on_click=do_login,
    ),
    ft.Container(height=10),
    ft.Row([loading, msg_text], alignment=ft.MainAxisAlignment.CENTER),
    ft.Divider(height=20),
    ft.Text("Corumba Sistemas (c) 2026", size=11, color=ft.Colors.GREY_500),
],
horizontal_alignment=ft.CrossAxisAlignment.CENTER,
alignment=ft.MainAxisAlignment.CENTER,
),
alignment=ft.alignment.center,
expand=True,
)

# =====
# TELA DE CLIENTES
# =====
def criar_tela_clientes(page):
    clientes_list = ft.ListView(spacing=5, padding=10, expand=True)
    search_field = ft.TextField(
        label="Buscar cliente...",
        prefix_icon=ft.Icons.SEARCH,
        border_radius=10,
        expand=True,
    )
    nome_field = ft.TextField(label="Nome completo", border_radius=8, expand=True)
    cpf_field = ft.TextField(label="CPF/CNPJ", border_radius=8, expand=True)
    email_field = ft.TextField(label="E-mail", border_radius=8, expand=True)
    telefone_field = ft.TextField(label="Telefone", border_radius=8, expand=True)
    endereco_field = ft.TextField(label="Endereco", border_radius=8, expand=True)
    msg_text = ft.Text("", size=13, weight=ft.FontWeight.BOLD)
    edit_id = {"value": None}

    def carregar_clientes():
        clientes_list.controls.clear()
        result = api.get("/api/clientes/")
        if result["success"]:
            for c in result["data"]:
                clientes_list.controls.append(
                    ft.Card(
                        content=ft.Container(
                            content=ft.Row(
                                [
                                    ft.Icon(ft.Icons.PERSON, color=ft.Colors.BLUE_600),
                                    ft.Column(
                                        [
                                            ft.Text(c.get("nome", ""), weight=ft.FontWeight.BOLD, size=14),
                                            ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=12, color=ft.Colors.GREY_600),
                                            ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=12, color=ft.Colors.GREY_600),
                                        ],
                                        expand=True,
                                        spacing=2,
                                    ),
                                    ft.IconButton(
                                        icon=ft.Icons.EDIT,
                                        icon_color=ft.Colors.BLUE_600,
                                        tooltip="Editar",
                                        on_click=lambda e, cli=c: editar_cliente(cli),
                                    ),
                                    ft.IconButton(
                                        icon=ft.Icons.DELETE,
                                        icon_color=ft.Colors.RED_600,
                                        tooltip="Excluir",
                                        on_click=lambda e, cli=c: excluir_cliente(cli),
                                    ),
                                ],
                                alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
                            ),
                            padding=12,
                        ),
                        elevation=2,
                    )
                )
            else:
                clientes_list.controls.append(
                    ft.Text("Erro ao carregar: " + str(result["error"]), color=ft.Colors.RED)
                )
        page.update()

    def salvar_cliente(e):
        data = {
            "nome": nome_field.value or "",
            "cpf_cnpj": cpf_field.value or "",
            "email": email_field.value or "",
            "telefone": telefone_field.value or "",
            "endereco": endereco_field.value or "",
        }
        if not data["nome"]:
            msg_text.value = "Nome é obrigatório!"
            msg_text.color = ft.Colors.ORANGE
            page.update()
            return
        if edit_id["value"]:

```

```

        result = api.put("/api/clientes/" + str(edit_id["value"]), data)
    else:
        result = api.post("/api/clientes/", data)
    if result["success"]:
        msg_text.value = "Cliente salvo com sucesso!"
        msg_text.color = ft.Colors.GREEN
        limpar_form()
        carregar_clientes()
    else:
        msg_text.value = "Erro: " + str(result["error"])[0:80]
        msg_text.color = ft.Colors.RED
        page.update()

def editar_cliente(cliente):
    edit_id["value"] = cliente.get("id")
    nome_field.value = cliente.get("nome", "")
    cpf_field.value = cliente.get("cpf_cnpj", "")
    email_field.value = cliente.get("email", "")
    telefone_field.value = cliente.get("telefone", "")
    endereco_field.value = cliente.get("endereco", "")
    msg_text.value = "Editando cliente..."
    msg_text.color = ft.Colors.BLUE
    page.update()

def excluir_cliente(cliente):
    def confirmar(e):
        if e.control.text == "Sim":
            result = api.delete("/api/clientes/" + str(cliente.get("id")))
            if result["success"]:
                msg_text.value = "Cliente excluido!"
                msg_text.color = ft.Colors.GREEN
                carregar_clientes()
            else:
                msg_text.value = "Erro: " + str(result["error"])[0:80]
                msg_text.color = ft.Colors.RED
        dlg.open = False
        page.update()
    dlg = ft.AlertDialog(
        title=ft.Text("Confirmar exclusao"),
        content=ft.Text("Excluir cliente " + str(cliente.get("nome")) + "?" ),
        actions=[
            ft.TextButton("Nao", on_click=confirmar),
            ft.TextButton("Sim", on_click=confirmar),
        ],
    )
    page.overlay.append(dlg)
    dlg.open = True
    page.update()

def limpar_form():
    edit_id["value"] = None
    nome_field.value = ""
    cpf_field.value = ""
    email_field.value = ""
    telefone_field.value = ""
    endereco_field.value = ""

def novo_cliente(e):
    limpar_form()
    msg_text.value = "Novo cliente"
    msg_text.color = ft.Colors.BLUE
    page.update()

return ft.Container(
    content=ft.Column(
        [
            ft.Row(
                [
                    ft.Icon(ft.Icons.PEOPLE, size=32, color=ft.Colors.BLUE_700),
                    ft.Text("Gestao de Clientes", size=24, weight=ft.FontWeight.BOLD),
                    ft.ElevatedButton(
                        "Novo",
                        icon=ft.Icons.ADD,
                        on_click=novo_cliente,
                        bgcolor=ft.Colors.GREEN_600,
                        color=ft.Colors.WHITE,
                    ),
                ],
                alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            ),
            ft.Divider(),
            ft.Row([search_field]),
            ft.Card(
                content=ft.Container(
                    content=ft.Column(
                        [
                            ft.Text("Formulario", weight=ft.FontWeight.BOLD, size=16),
                            ft.Row([nome_field, cpf_field]),
                            ft.Row([email_field, telefone_field]),
                            endereco_field,
                            ft.Row(
                                [
                                    ft.ElevatedButton(
                                        "Salvar",
                                        icon=ft.Icons.SAVE,

```

```

        on_click=salvar_cliente,
        bgcolor=ft.Colors.BLUE_700,
        color=ft.Colors.WHITE,
    ),
    ft.OutlinedButton(
        "Limpar",
        on_click=lambda e: (limpar_form(), page.update()),
    ),
    msg_text,
],
alignment=ft.MainAxisAlignment.START,
spacing=10,
),
],
spacing=8,
),
padding=15,
),
elevation=3,
),
ft.Text("Clientes cadastrados:", weight=ft.FontWeight.BOLD, size=14),
clientes_list,
],
spacing=10,
expand=True,
),
padding=15,
expand=True,
on_mount=lambda e: carregar_clientes(),
)

# =====
# TELA DE PRODUTOS
# =====
def criar_tela_produtos(page):
    produtos_list = ft.ListView(spacing=5, padding=10, expand=True)
    nome_field = ft.TextField(label="Nome do produto", border_radius=8, expand=True)
    preco_field = ft.TextField(label="Preço (R$)", border_radius=8, expand=True, keyboard_type=ft.KeyboardType.NUMBER)
    categoria_field = ft.TextField(label="Categoria", border_radius=8, expand=True)
    estoque_field = ft.TextField(label="Estoque", border_radius=8, expand=True, keyboard_type=ft.KeyboardType.NUMBER)
    descricao_field = ft.TextField(label="Descrição", border_radius=8, multiline=True, min_lines=2)
    msg_text = ft.Text("", size=13, weight=ft.FontWeight.BOLD)
    edit_id = {"value": None}

    def carregar_produtos():
        produtos_list.controls.clear()
        result = api.get("/api/produtos/")
        if result["success"]:
            for p in result["data"]:
                produtos_list.controls.append(
                    ft.Card(
                        content=ft.Container(
                            content=ft.Row(
                                [
                                    ft.Icon(ft.Icons.INVENTORY_2, color=ft.Colors.PURPLE_600),
                                    ft.Column(
                                        [
                                            ft.Text(p.get("nome", ""), weight=ft.FontWeight.BOLD, size=14),
                                            ft.Text("R$ %.2f" % p.get("preco", 0), size=13, color=ft.Colors.GREEN_700),
                                            ft.Text("Estoque: %d | %s" % (p.get("estoque", 0), p.get("categoria", "")), size=11,
weight=ft.FontWeight.BOLD),
color=ft.Colors.GREY_600),
                                        ],
                                    expand=True,
                                    spacing=2,
                                ),
                            ),
                        ft.IconButton(
                            icon=ft.Icons.EDIT,
                            icon_color=ft.Colors.BLUE_600,
                            on_click=lambda e, prod=p: editar_produto(prod),
                        ),
                        ft.IconButton(
                            icon=ft.Icons.DELETE,
                            icon_color=ft.Colors.RED_600,
                            on_click=lambda e, prod=p: excluir_produto(prod),
                        ),
                    ],
                ),
                padding=12,
            ),
            elevation=2,
        )
    else:
        produtos_list.controls.append(ft.Text("Erro: " + str(result["error"]), color=ft.Colors.RED))
    page.update()

def salvar_produto(e):
    try:
        preco = float(preco_field.value or 0)
        estoque = int(estoque_field.value or 0)
    except ValueError:
        msg_text.value = "Preço e estoque devem ser numeros!"
        msg_text.color = ft.Colors.ORANGE

```

```

        page.update()
        return
    data = {
        "nome": nome_field.value or "",
        "preco": preco,
        "categoria": categoria_field.value or "",
        "estoque": estoque,
        "descricao": descricao_field.value or "",
    }
    if not data["nome"]:
        msg_text.value = "Nome e obrigatorio!"
        msg_text.color = ft.Colors.ORANGE
        page.update()
        return
    if edit_id["value"]:
        result = api.put("/api/produtos/" + str(edit_id["value"]), data)
    else:
        result = api.post("/api/produtos/", data)
    if result["success"]:
        msg_text.value = "Produto salvo!"
        msg_text.color = ft.Colors.GREEN
        limpar_form()
        carregar_produtos()
    else:
        msg_text.value = str(result["error"])[1:80]
        msg_text.color = ft.Colors.RED
    page.update()

def editar_produto(produto):
    edit_id["value"] = produto.get("id")
    nome_field.value = produto.get("nome", "")
    preco_field.value = str(produto.get("preco", ""))
    categoria_field.value = produto.get("categoria", "")
    estoque_field.value = str(produto.get("estoque", ""))
    descricao_field.value = produto.get("descricao", "")
    msg_text.value = "Editando produto..."
    msg_text.color = ft.Colors.BLUE
    page.update()

def excluir_produto(produto):
    def confirmar(e):
        if e.control.text == "Sim":
            result = api.delete("/api/produtos/" + str(produto.get("id")))
            if result["success"]:
                msg_text.value = "Produto excluido!"
                msg_text.color = ft.Colors.GREEN
                carregar_produtos()
            dlg.open = False
            page.update()
    dlg = ft.AlertDialog(
        title=ft.Text("Confirmar"),
        content=ft.Text("Excluir '" + str(produto.get("nome")) + "'?"),
        actions=[ft.TextButton("Nao", on_click=confirmar), ft.TextButton("Sim", on_click=confirmar)],
    )
    page.overlay.append(dlg)
    dlg.open = True
    page.update()

def limpar_form():
    edit_id["value"] = None
    nome_field.value = ""
    preco_field.value = ""
    categoria_field.value = ""
    estoque_field.value = ""
    descricao_field.value = ""

return ft.Container(
    content=ft.Column(
        [
            ft.Row(
                [
                    ft.Icon(ft.Icons.INVENTORY, size=32, color=ft.Colors.PURPLE_700),
                    ft.Text("Gestao de Produtos", size=24, weight=ft.FontWeight.BOLD),
                    ft.ElevatedButton(
                        "Novo",
                        icon=ft.Icons.ADD,
                        on_click=lambda e: (limpar_form(), page.update()),
                        bgcolor=ft.Colors.GREEN_600,
                        color=ft.Colors.WHITE,
                    ),
                ],
                alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
            ),
            ft.Divider(),
            ft.Card(
                content=ft.Container(
                    content=ft.Column(
                        [
                            ft.Text("Formulario", weight=ft.FontWeight.BOLD, size=16),
                            ft.Row([nome_field, preco_field]),
                            ft.Row([categoria_field, estoque_field]),
                            descricao_field,
                            ft.Row(
                                [
                                    ft.ElevatedButton("Salvar", icon=ft.Icons.SAVE, on_click=salvar_produto,

```

```

bgcolor=ft.Colors.PURPLE_700, color=ft.Colors.WHITE),
        ft.OutlinedButton("Limpar", on_click=lambda e: (limpar_form(), page.update())),
        msg_text,
    ],
    spacing=10,
),
],
spacing=8,
),
padding=15,
),
elevation=3,
),
ft.Text("Produtos cadastrados:", weight=ft.FontWeight.BOLD, size=14),
produtos_list,
],
spacing=10,
expand=True,
),
padding=15,
expand=True,
on_mount=lambda e: carregar_produtos(),
)

# =====
# TELA DE VENDAS
# =====
def criar_tela_vendas(page):
    vendas_list = ft.ListView(spacing=5, padding=10, expand=True)

    def carregar_vendas():
        vendas_list.controls.clear()
        result = api.get("/api/vendas/")
        if result["success"]:
            total_geral = 0
            for v in result["data"]:
                total = v.get("total", 0)
                total_geral += total
                vendas_list.controls.append(
                    ft.Card(
                        content=ft.Container(
                            content=ft.Row(
                                [
                                    ft.Icon(ft.Icons.SHOPPING_CART, color=ft.Colors.GREEN_600),
                                    ft.Column(
                                        [
                                            ft.Text("Venda #" + str(v.get("id", 0)), weight=ft.FontWeight.BOLD, size=14),
                                            ft.Text("Cliente: " + str(v.get("cliente_nome", "N/A")), size=12),
                                            ft.Text("Data: " + str(v.get("data", "N/A")), size=11, color=ft.Colors.GREY_600),
                                        ],
                                        expand=True,
                                        spacing=2,
                                    ),
                                ],
                                ft.Text("R$ %.2f" % total, size=16, weight=ft.FontWeight.BOLD, color=ft.Colors.GREEN_700),
                            ),
                            padding=12,
                        ),
                        elevation=2,
                    )
                )
            vendas_list.controls.insert(
                0,
                ft.Container(
                    content=ft.Text(
                        "Total: R$ %.2f | %d vendas" % (total_geral, len(result["data"])),
                        weight=ft.FontWeight.BOLD, size=16, color=ft.Colors.BLUE_900,
                    ),
                    padding=10,
                    bgcolor=ft.Colors.BLUE_50,
                    border_radius=8,
                ),
            )
        else:
            vendas_list.controls.append(ft.Text("Erro: " + str(result["error"]), color=ft.Colors.RED))
            page.update()

    return ft.Container(
        content=ft.Column(
            [
                ft.Row(
                    [
                        ft.Icon(ft.Icons.POINT_OF_SALE, size=32, color=ft.Colors.GREEN_700),
                        ft.Text("Vendas Realizadas", size=24, weight=ft.FontWeight.BOLD),
                    ],
                    alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
                ),
                ft.Divider(),
                vendas_list,
            ],
            spacing=10,
            expand=True,
        ),
        padding=15,
    )

```

```

        expand=True,
        on_mount=lambda e: carregar_vendas(),
    )

# =====
# TELA DE DASHBOARD
# =====
def criar_tela_dashboard(page):
    stats_column = ft.Column(spacing=10)

    def carregar_stats():
        stats_column.controls.clear()
        clientes_r = api.get("/api/clientes/")
        produtos_r = api.get("/api/produtos/")
        vendas_r = api.get("/api/vendas/")

        n_clientes = len(clientes_r["data"]) if clientes_r["success"] else 0
        n_produtos = len(produtos_r["data"]) if produtos_r["success"] else 0
        n_vendas = len(vendas_r["data"]) if vendas_r["success"] else 0
        total_vendas = 0
        if vendas_r["success"]:
            for v in vendas_r["data"]:
                total_vendas += v.get("total", 0)

        cards = [
            ("Clientes", str(n_clientes), ft.Colors.BLUE_700, ft.Colors.BLUE_50),
            ("Produtos", str(n_produtos), ft.Colors.PURPLE_700, ft.Colors.PURPLE_50),
            ("Vendas", str(n_vendas), ft.Colors.GREEN_700, ft.Colors.GREEN_50),
            ("Faturamento", "R$ %.2f" % total_vendas, ft.Colors.ORANGE_700, ft.Colors.ORANGE_50),
        ]

        for titulo, valor, cor, bg in cards:
            stats_column.controls.append(
                ft.Card(
                    content=ft.Container(
                        content=ft.Column(
                            [
                                ft.Text(titulo, size=14, color=ft.Colors.GREY_700),
                                ft.Text(valor, size=32, weight=ft.FontWeight.BOLD, color=cor),
                            ],
                            horizontal_alignment=ft.CrossAxisAlignment.CENTER,
                            spacing=5,
                        ),
                        padding=20,
                        bgcolor=bg,
                        border_radius=12,
                        width=200,
                    ),
                    elevation=4,
                )
            )
        page.update()

    return ft.Container(
        content=ft.Column(
            [
                ft.Row(
                    [
                        ft.Icon(ft.Icons.DASHBOARD, size=32, color=ft.Colors.BLUE_700),
                        ft.Text("Dashboard", size=24, weight=ft.FontWeight.BOLD),
                        ft.IconButton(
                            icon=ft.Icons.REFRESH,
                            icon_color=ft.Colors.BLUE_700,
                            tooltip="Atualizar",
                            on_click=lambda e: carregar_stats(),
                        ),
                    ],
                    alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
                ),
                ft.Divider(),
                ft.Text("Bem-vindo, " + str(api.user.get("nome", "Usuario")) + "!", size=16, color=ft.Colors.GREY_700),
                ft.Text("Empresa: " + str(api.user.get("empresa_nome", "N/A")), size=13, color=ft.Colors.GREY_600),
                ft.Divider(height=10),
                ft.Text("Resumo do Negocio", size=18, weight=ft.FontWeight.BOLD),
                ft.Row([stats_column], wrap=True, spacing=15, run_spacing=15),
            ],
            spacing=10,
            expand=True,
        ),
        padding=15,
        expand=True,
        on_mount=lambda e: carregar_stats(),
    )

# =====
# TELA DE TESTE 100 CLIENTES
# =====
def criar_tela_teste(page):
    result_text = ft.Text("", size=13, selectable=True)
    progress_bar = ft.ProgressBar(width=400, visible=False)
    progress_text = ft.Text("", size=12)

    def executar_teste(e):

```

```

def run_test():
    progress_bar.visible = True
    result_text.value = "Iniciando teste de 100 clientes..."
    page.update()
    sucessos = 0
    falhas = 0
    for i in range(1, 101):
        data = {
            "nome": "Cliente Teste %03d" % i,
            "cpf_cnpj": "%011d" % i,
            "email": "cliente%03d@teste.com" % i,
            "telefone": "(62) 9%08d" % i,
            "endereco": "Rua Teste, %d - Centro" % i,
        }
        result = api.post("/api/clientes/", data)
        if result["success"]:
            sucessos += 1
        else:
            falhas += 1
        if i % 10 == 0:
            progress_bar.value = i / 100
            progress_text.value = "Progresso: %d/100" % i
            page.update()
        result_text.value = (
            "Teste concluido!\n\n"
            "Resultados:\n"
            " Sucessos: %d/100\n"
            " Falhas: %d/100\n"
            " Taxa de sucesso: %d%%" % (sucessos, falhas, sucessos)
        )
    progress_bar.visible = False
    page.update()
    threading.Thread(target=run_test, daemon=True).start()

return ft.Container(
    content=ft.Column(
        [
            ft.Row(
                [
                    ft.Icon(ft.Icons.SCIENCE, size=32, color=ft.Colors.AMBER_700),
                    ft.Text("Teste de Carga - 100 Clientes", size=24, weight=ft.FontWeight.BOLD),
                ],
            ),
            ft.Divider(),
            ft.Text(
                "Este teste insere 100 clientes de exemplo no sistema para validar a performance da API.",
                size=14, color=ft.Colors.GREY_700,
            ),
            ft.Container(height=10),
            ft.ElevatedButton(
                "Executar Teste de 100 Clientes",
                icon=ft.Icons.PLAY_ARROW,
                on_click=executar_teste,
                bgcolor=ft.Colors.AMBER_700,
                color=ft.Colors.WHITE,
                style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=10)),
                height=50,
            ),
            ft.Container(height=15),
            progress_bar,
            progress_text,
            ft.Divider(),
            ft.Container(
                content=ft.Column([result_text]),
                padding=15,
                bgcolor=ft.Colors.GREY_100,
                border_radius=10,
                width=500,
            ),
        ],
        spacing=10,
        expand=True,
    ),
    padding=15,
    expand=True,
)

# =====
# APLICACAO PRINCIPAL
# =====
def main(page: ft.Page):
    page.title = "SaltoGestao - Sistema de Gestao Empresarial"
    page.theme_mode = ft.ThemeMode.LIGHT
    page.padding = 0
    page.window.width = 1200
    page.window.height = 750
    page.window.min_width = 900
    page.window.min_height = 600
    page.theme = ft.Theme(color_scheme_seed=ft.Colors.BLUE, font_family="Segoe UI")

    main_content = ft.Container(expand=True)

    def mostrar_tela(tela_nome):
        telas = {

```

```

        "dashboard": criar_tela_dashboard,
        "clientes": criar_tela_clientes,
        "produtos": criar_tela_produtos,
        "vendas": criar_tela_vendas,
        "teste": criar_tela_teste,
    }
    if tela_nome in telas:
        main_content.content = telas[tela_nome](page)
        page.update()

def on_login_success():
    main_content.content = criar_layout_principal()
    page.update()

def criar_layout_principal():
    def navegar(e):
        idx = e.control.selected_index
        telas_map = {0: "dashboard", 1: "clientes", 2: "produtos", 3: "vendas", 4: "teste"}
        if idx in telas_map:
            mostrar_tela(telas_map[idx])

    sidebar = ft.NavigationRail(
        selected_index=0,
        label_type=ft.NavigationRailLabelType.ALL,
        min_width=100,
        min_extended_width=200,
        group_alignment=-0.9,
        destinations=[
            ft.NavigationRailDestination(
                icon=ft.Icons.DASHBOARD_OUTLINED,
                selected_icon=ft.Icons.DASHBOARD,
                label="Dashboard",
            ),
            ft.NavigationRailDestination(
                icon=ft.Icons.PEOPLE_OUTLINED,
                selected_icon=ft.Icons.PEOPLE,
                label="Clientes",
            ),
            ft.NavigationRailDestination(
                icon=ft.Icons.INVENTORY_2_OUTLINED,
                selected_icon=ft.Icons.INVENTORY_2,
                label="Produtos",
            ),
            ft.NavigationRailDestination(
                icon=ft.Icons.POINT_OF_SALE_OUTLINED,
                selected_icon=ft.Icons.POINT_OF_SALE,
                label="Vendas",
            ),
            ft.NavigationRailDestination(
                icon=ft.Icons.SCIENCE_OUTLINED,
                selected_icon=ft.Icons.SCIENCE,
                label="Teste 100",
            ),
        ],
        on_change=navegar,
        bgcolor=ft.Colors.BLUE_50,
    )

    content_area = ft.Container(
        content=criar_tela_dashboard(page),
        expand=True,
        padding=0,
    )

    nonlocal main_content
    main_content = content_area

    return ft.Container(
        content=ft.Row(
            [sidebar, ft.VerticalDivider(width=1), content_area],
            expand=True,
            spacing=0,
        ),
        expand=True,
    )

    main_content.content = criar_tela_login(page, on_login_success)
    page.add(main_content)

if __name__ == "__main__":
    ft.app(target=main)

```

 **main_v2.py**

desktop/main_v2.py • 902 linhas

```

"""
SALTO GESTAO Desktop v2.0 - Design Moderno & Clean
Paleta: Corumbá Sistemas (#0b1a30)
"""

import flet as ft
import requests
import json

```



```

import threading
from datetime import datetime

# Importar design system
from design_system import Cores, Espacamento, Bordas, DS

# Configuracao
API_URL = "http://191.252.210.235:8080"

# =====
# CLIENTE API
# =====

class APIClient:
    def __init__(self):
        self.base_url = API_URL
        self.token = None
        self.user = None
        self.headers = {"Content-Type": "application/json"}

    def set_token(self, token):
        self.token = token
        self.headers["Authorization"] = "Bearer " + token

    def login(self, username, password):
        try:
            resp = requests.post(
                self.base_url + "/api/login",
                json={"username": username, "password": password},
                timeout=10,
            )
            if resp.status_code == 200:
                data = resp.json()
                self.set_token(data.get("access_token", ""))
                self.user = data.get("user", {})
                return {"success": True, "data": data}
            return {"success": False, "error": resp.json().get("detail", "Erro no login")}
        except requests.exceptions.ConnectionError:
            return {"success": False, "error": "Nao foi possivel conectar ao servidor"}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def get(self, endpoint):
        try:
            resp = requests.get(self.base_url + endpoint, headers=self.headers, timeout=15)
            if resp.status_code == 200:
                return {"success": True, "data": resp.json()}
            return {"success": False, "error": resp.text}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def post(self, endpoint, data):
        try:
            resp = requests.post(self.base_url + endpoint, json=data, headers=self.headers, timeout=15)
            if resp.status_code in (200, 201):
                return {"success": True, "data": resp.json()}
            return {"success": False, "error": resp.text}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def put(self, endpoint, data):
        try:
            resp = requests.put(self.base_url + endpoint, json=data, headers=self.headers, timeout=15)
            if resp.status_code == 200:
                return {"success": True, "data": resp.json()}
            return {"success": False, "error": resp.text}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def delete(self, endpoint):
        try:
            resp = requests.delete(self.base_url + endpoint, headers=self.headers, timeout=15)
            if resp.status_code in (200, 204):
                return {"success": True}
            return {"success": False, "error": resp.text}
        except Exception as e:
            return {"success": False, "error": str(e)}

api = APIClient()

# =====
# TELA DE LOGIN v2.0
# =====

def criar_tela_login(page, on_login_success):
    """Tela de login com design moderno e clean"""

    username_field = DS.campo(
        label="Usuario",
        prefix_icon=ft.Icons.PERSON_OUTLINED,
        hint_text="Digite seu usuario",
        width=360,
    )

```

```

)
password_field = DS.campo(
    label="Senha",
    prefix_icon=ft.Icons.LOCK_OUTLINED,
    password=True,
    can_reveal_password=True,
    hint_text="Digite sua senha",
    width=360,
)

msg_banner = ft.Container(visible=False, animate=ft.Animation(200, ft.AnimationCurve.EASE_OUT))
loading = DS.loading_spinner(size=20)
loading.visible = False

def show_message(text, msg_type="error"):
    icons = {"error": ft.Icons.ERROR_OUTLINED, "success": ft.Icons.CHECK_CIRCLE_OUTLINED}
    colors = {"error": Cores.ERRO, "success": Cores.SUCESSO}
    bg_colors = {"error": ft.Colors.with_opacity(0.08, Cores.ERRO), "success": ft.Colors.with_opacity(0.08, Cores.SUCESSO)}

    msg_banner.content = ft.Container(
        content=ft.Row([
            ft.Icon(icons.get(msg_type), size=16, color=colors.get(msg_type)),
            ft.Container(width=8,
                ft.Text(text, size=12, color=Cores.TEXT0_80),
            ], spacing=8),
        bgcolor=bg_colors.get(msg_type),
        border_radius=Bordas.SM,
        padding=ft.padding.symmetric(horizontal=12, vertical=8),
        border=ft.border.all(1, ft.Colors.with_opacity(0.15, colors.get(msg_type))),
        width=360,
    )
    msg_banner.visible = True
    page.update()

def do_login(e):
    if not username_field.value or not password_field.value:
        show_message("Preencha todos os campos", "error")
        return

    loading.visible = True
    msg_banner.visible = False
    page.update()

    result = api.login(username_field.value, password_field.value)
    loading.visible = False

    if result["success"]:
        show_message("Login realizado com sucesso!", "success")
        page.update()
        # Pequena pausa para mostrar mensagem de sucesso
        import time
        time.sleep(0.5)
        on_login_success()
    else:
        show_message(result["error"], "error")
        page.update()

# Layout do login
login_card = DS.card(
    content=ft.Column([
        # Logo area
        ft.Container(
            content=ft.Column([
                # Icon com fundo circular
                ft.Container(
                    content=ft.Icon(ft.Icons.STORE_ROUNDED, size=40, color=Cores.P),
                    bgcolor=ft.Colors.with_opacity(0.1, Cores.P),
                    border_radius=Bordas.FULL,
                    padding=20,
                    width=80,
                    height=80,
                    alignment=ft.alignment.center,
                ),
                ft.Container(height=Espacamento.SM),
                DS.titulo("SaltoGestao", size=24, weight=ft.FontWeight.W_700, color=Cores.P),
                ft.Container(height=2),
                DS.subtitulo("Sistema de Gestao Empresarial", size=12),
            ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=2),
            alignment=ft.alignment.center,
            padding=ft.padding.only(top=Espacamento.MD),
        ),
        ft.Container(height=Espacamento.LG),

        # Campos
        username_field,
        ft.Container(height=Espacamento.SM),
        password_field,
        ft.Container(height=Espacamento.SM),

        # Mensagem
        msg_banner,
        ft.Container(height=Espacamento.SM),
    ]),

```

```

        # Botao
        DS.botoao(
            texto="Entrar",
            icon=ft.Icons.ARROW_FORWARD,
            width=360,
            height=48,
            on_click=do_login,
        ),

        ft.Container(height=Espacamento.SM),

        # Loading row
        ft.Row([loading], alignment=ft.MainAxisAlignment.CENTER),

        ft.Container(height=Espacamento.MD),
    ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=4),
    padding=Espacamento.XL,
    width=420,
    shadow=ft.BoxShadow(
        spread_radius=0,
        blur_radius=24,
        color=ft.Colors.with_opacity(0.12, Cores.ESCUR0),
        offset=ft.Offset(0, 8),
    ),
)

return ft.Container(
    content=login_card,
    alignment=ft.alignment.center,
    expand=True,
    gradient=ft.LinearGradient(
        begin=ft.alignment.top_center,
        end=ft.alignment.bottom_center,
        colors=[Cores.FUNDO, Cores.FUNDO_CARD],
    ),
)

# =====
# TELA DE CLIENTES v2.0
# =====

def criar_tela_clientes(page):
    """Tela de clientes com design moderno"""

    clientes_list = ft.ListView(spacing=Espacamento.SM, padding=0, expand=True)
    search_field = DS.campo(
        label="Buscar...",
        prefix_icon=ft.Icons.SEARCH,
        hint_text="Nome, CPF, telefone...",
    )

    # Formulário
    nome_field = DS.campo(label="Nome completo", hint_text="Ex: Joao Silva")
    cpf_field = DS.campo(label="CPF/CNPJ", hint_text="000.000.000-00")
    email_field = DS.campo(label="E-mail", hint_text="email@exemplo.com")
    telefone_field = DS.campo(label="Telefone", hint_text="(62) 99999-9999")

    msg_banner = ft.Container(visible=False, animate=ft.Animation(200, ft.AnimationCurve.EASE_OUT))
    edit_id = {"value": None}

    def show_msg(text, msg_type="success"):
        icons = {"success": ft.Icons.CHECK_CIRCLE_OUTLINED, "error": ft.Icons.ERROR_OUTLINED, "info": ft.Icons.INFO_OUTLINED}
        colors = {"success": Cores.SUCESS0, "error": Cores.ERRO, "info": Cores.INFO}
        bg = {"success": ft.Colors.with_opacity(0.08, Cores.SUCESS0), "error": ft.Colors.with_opacity(0.08, Cores.ERRO), "info":
ft.Colors.with_opacity(0.08, Cores.INFO)}
        msg_banner.content = ft.Container(
            content=ft.Row(
                [
                    ft.Icon(icons.get(msg_type), size=16, color=Cores.TEXT0),
                    ft.Container(width=8),
                    ft.Text(text, size=12, color=Cores.TEXT0_B0),
                ], spacing=8),
            bgcolor=bg.get(msg_type),
            border_radius=Bordas.SM,
            padding=ft.padding.symmetric(horizontal=12, vertical=8),
            margin=ft.padding.only(bottom=Espacamento.SM),
        )
        msg_banner.visible = True
        page.update()

    def carregar_clientes():
        clientes_list.controls.clear()
        result = api.get("/api/clientes/")

        if result["success"]:
            clientes = result["data"]
            if not clientes:
                clientes_list.controls.append(DS.empty_state(
                    icon=ft.Icons.PEOPLE_OUTLINED,
                    title="Nenhum cliente cadastrado",
                    subtitle="Clique em 'Novo' para adicionar",
                ))
            for c in clientes:
                clientes_list.controls.append(DS.card(

```

```

padding=0,
shadow=False,
border=ft.border.all(1, Cores.BORDA),
content=ft.Container(
    content=ft.Row([
        ft.CircleAvatar(
            content=ft.Text(
                c.get("nome", "?").split()[0][0].upper() if c.get("nome") else "?",
                size=16, weight=ft.FontWeight.W_600, color=Cores.FUNDO_CARD,
            ),
            bgcolor=Cores.P20,
            radius=20,
        ),
        ft.Container(width=Espacamento.SM),
        ft.Column([
            ft.Text(c.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0),
            ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=11, color=Cores.TEXT0_60),
            ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=11, color=Cores.TEXT0_60),
        ], spacing=2, expand=True),
        DS.icone_botao(ft.Icons.EDIT_OUTLINED, "Editar", on_click=lambda e, cli=c: editar_cliente(cli)),
        DS.icone_botao(ft.Icons.DELETE_OUTLINED, "Excluir", icon_color=Cores.ERRO, on_click=lambda e, cli=c:
excluir_cliente(cli)),
    ], spacing=Espacamento.SM, alignment=ft.MainAxisAlignment.START),
padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
    ),
))
else:
    clientes_list.controls.append(
        ft.Text("Erro: " + str(result["error"])[80], color=Cores.ERRO, size=12)
    )
page.update()

def salvar_cliente(e):
    data = {
        "nome": nome_field.value or "",
        "cpf_cnpj": cpf_field.value or "",
        "email": email_field.value or "",
        "telefone": telefone_field.value or "",
    }
    if not data["nome"]:
        show_msg("Nome e obrigatorio", "error")
        return

    if edit_id["value"]:
        result = api.put("/api/clientes/" + str(edit_id["value"]), data)
    else:
        result = api.post("/api/clientes/", data)

    if result["success"]:
        show_msg("Cliente salvo com sucesso!", "success")
        limpar_form()
        carregar_clientes()
    else:
        show_msg("Erro: " + str(result["error"])[80], "error")

def editar_cliente(cliente):
    edit_id["value"] = cliente.get("id")
    nome_field.value = cliente.get("nome", "")
    cpf_field.value = cliente.get("cpf_cnpj", "")
    email_field.value = cliente.get("email", "")
    telefone_field.value = cliente.get("telefone", "")
    show_msg("Editando: " + str(cliente.get("nome", "")), "info")
    page.update()

def excluir_cliente(cliente):
    def confirmar(e):
        if e.control.text == "Sim":
            result = api.delete("/api/clientes/" + str(cliente.get("id")))
            if result["success"]:
                show_msg("Cliente excluido!", "success")
                carregar_clientes()
            else:
                show_msg("Erro ao excluir", "error")
        dlg.open = False
        page.update()

    dlg = ft.AlertDialog(
        title=ft.Text("Confirmar exclusao", weight=ft.FontWeight.W_600),
        content=ft.Text("Excluir '" + str(cliente.get("nome")) + "'?"),
        actions=[
            ft.TextButton("Nao", on_click=confirmar),
            ft.TextButton("Sim", on_click=confirmar, style=ft.ButtonStyle(color=Cores.ERRO)),
        ],
    )
    page.overlay.append(dlg)
    dlg.open = True
    page.update()

def limpar_form():
    edit_id["value"] = None
    nome_field.value = cpf_field.value = email_field.value = telefone_field.value = ""

def Novo_cliente(e):
    limpar_form()
    show_msg("Novo cliente - preencha os campos", "info")

```

```

page.update()

# Layout
return ft.Container(
    content=ft.Column([
        DS.page_header("Clientes", subtitle="Gerencie seus clientes", actions=[
            DS.botoao_outline("Limpar", icon=ft.Icons.REFRESH, on_click=lambda e: (limpar_form(), page.update())),
            DS.botoao("Novo Cliente", icon=ft.Icons.ADD, on_click=Novo_cliente),
        ]),

        # Busca
        DS.card(content=search_field, padding=Espacamento.SM, shadow=False),

        # Formulário
        DS.card(
            titulo="Formulário",
            content=ft.Column([
                ft.Row([nome_field, cpf_field], spacing=Espacamento.SM),
                ft.Row([email_field, telefone_field], spacing=Espacamento.SM),
                ft.Row([
                    DS.botoao("Salvar", icon=ft.Icons.SAVE_OUTLINED, on_click=salvar_cliente),
                    DS.botoao_outline("Limpar", on_click=lambda e: (limpar_form(), page.update())),
                ], spacing=Espacamento.SM),
                msg_banner,
            ], spacing=Espacamento.SM),
            padding=Espacamento.MD,
        ),

        # Lista
        ft.Container(
            content=ft.Text(str("Cadastrados"), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60),
            padding=ft.padding.only(left=4, bottom=Espacamento.SM),
        ),
        ft.Container(
            content=clientes_list,
            expand=True,
        ),
    ], spacing=Espacamento.MD, expand=True),
    padding=Espacamento.LG,
    bgcolor=Cores.FUNDO,
    expand=True,
    on_mount=lambda e: carregar_clientes(),
)

# =====
# TELA DE PRODUTOS v2.0
# =====

def criar_tela_produtos(page):
    produtos_list = ft.ListView(spacing=Espacamento.SM, padding=0, expand=True)

    nome_field = DS.campo(label="Nome do produto", hint_text="Ex: Arroz 5kg")
    preco_field = DS.campo(label="Preço (R$)", hint_text="0.00")
    categoria_field = DS.campo(label="Categoria", hint_text="Alimentos")
    estoque_field = DS.campo(label="Estoque", hint_text="0")
    descricao_field = DS.campo(label="Descricao", multiline=True, height=72)

    msg_banner = ft.Container(visible=False)
    edit_id = {"value": None}

    def show_msg(text, msg_type="success"):
        colors = {"success": Cores.SUCESSO, "error": Cores.ERRO, "info": Cores.INFO}
        bg = {"success": ft.Colors.with_opacity(0.08, Cores.SUCESSO), "error": ft.Colors.with_opacity(0.08, Cores.ERRO), "info":
ft.Colors.with_opacity(0.08, Cores.INFO)}
        msg_banner.content = ft.Container(
            content=ft.Row([ft.Text(text, size=12, color=Cores.TEXT0_80)], spacing=8),
            bgcolor=bg.get(msg_type),
            border_radius=Bordas.SM,
            padding=ft.padding.symmetric(horizontal=12, vertical=8),
            margin=ft.padding.only(bottom=Espacamento.SM),
        )
        msg_banner.visible = True
        page.update()

    def carregar_produtos():
        produtos_list.controls.clear()
        result = api.get("/api/produtos/")
        if result["success"]:
            produtos = result["data"]
            if not produtos:
                produtos_list.controls.append(DS.empty_state(
                    icon=ft.Icons.INVENTORY_2_OUTLINED,
                    title="Nenhum produto cadastrado",
                ))
            for p in produtos:
                preco = p.get("preco", 0)
                estoque = p.get("estoque", 0)
                produtos_list.controls.append(DS.card(
                    padding=0,
                    shadow=False,
                    border=ft.border.all(1, Cores.BORDA),
                    content=ft.Container(
                        content=ft.Row([
                            ft.Container(

```

```

        content=ft.Icon(ft.Icons.INVENTORY_2_OUTLINED, size=18, color=Cores.INFO),
        bgcolor=ft.Colors.with_opacity(0.1, Cores.INFO),
        border_radius=Bordas.SM,
        padding=8,
        width=40, height=40,
        alignment=ft.alignment.center,
    ),
    ft.Container(width=Espacamento.SM,
    ft.Column([
        ft.Text(p.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0),
        ft.Text("R$ %.2f" % preco, size=12, weight=ft.FontWeight.W_700, color=Cores.SUCESSO, font_family="Courier
New")),
        ft.Text(str(estoque) + " em estoque | " + str(p.get("categoria", "")), size=10, color=Cores.TEXT0_60),
    ], spacing=2, expand=True),
    DS.icone_botao(ft.Icons.EDIT_OUTLINED, "Editar", on_click=lambda e, prod=p: editar_produto(prod)),
    DS.icone_botao(ft.Icons.DELETE_OUTLINED, "Excluir", icon_color=Cores.ERRO, on_click=lambda e, prod=p:
excluir_produto(prod)),
    ], spacing=Espacamento.SM, alignment=ft.MainAxisAlignment.START),
    padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
),
))
else:
    produtos_list.controls.append(ft.Text("Erro: " + str(result["error"])[0:80], color=Cores.ERRO, size=12))
page.update()

def salvar_produto(e):
    try:
        preco = float(preco_field.value or 0)
        estoque = int(estoque_field.value or 0)
    except ValueError:
        show_msg("Preço e estoque devem ser numeros", "error")
        return
    data = {"nome": nome_field.value or "", "preco": preco, "categoria": categoria_field.value or "", "estoque": estoque, "descricao":
descricao_field.value or ""}
    if not data["nome"]:
        show_msg("Nome e obrigatorio", "error")
        return
    if edit_id["value"]:
        result = api.put("/api/produtos/" + str(edit_id["value"]), data)
    else:
        result = api.post("/api/produtos/", data)
    if result["success"]:
        show_msg("Produto salvo!", "success")
        for f in [nome_field, preco_field, categoria_field, estoque_field, descricao_field]:
            f.value = ""
        edit_id["value"] = None
        carregar_produtos()
    else:
        show_msg("Erro: " + str(result["error"])[0:80], "error")

def editar_produto(produto):
    edit_id["value"] = produto.get("id")
    nome_field.value = produto.get("nome", "")
    preco_field.value = str(produto.get("preco", ""))
    categoria_field.value = produto.get("categoria", "")
    estoque_field.value = str(produto.get("estoque", ""))
    descricao_field.value = produto.get("descricao", "")
    show_msg("Editando: " + str(produto.get("nome", "")), "info")
    page.update()

def excluir_produto(produto):
    def confirmar(e):
        if e.control.text == "Sim":
            result = api.delete("/api/produtos/" + str(produto.get("id")))
            if result["success"]:
                show_msg("Produto excluido!", "success")
                carregar_produtos()
            dlg.open = False
            page.update()
        dlg = ft.AlertDialog(
            title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
            content=ft.Text("Excluir '" + str(produto.get("nome")) + "'?",
            actions=[ft.TextButton("Nao", on_click=confirmar), ft.TextButton("Sim", on_click=confirmar)],
        )
        page.overlay.append(dlg)
        dlg.open = True
        page.update()

    return ft.Container(
        content=ft.Column([
            DS.page_header("Produtos", subtitle="Gerencie seu estoque", actions=[
                DS.botao_outline("Limpar", icon=ft.Icons.REFRESH, on_click=lambda e: page.update()),
                DS.botao("Novo Produto", icon=ft.Icons.ADD, on_click=lambda e: None),
            ]),
            DS.card(
                titulo="Formulario",
                content=ft.Column([
                    ft.Row([nome_field, preco_field], spacing=Espacamento.SM),
                    ft.Row([categoria_field, estoque_field], spacing=Espacamento.SM),
                    descricao_field,
                    ft.Row([
                        DS.botao("Salvar", icon=ft.Icons.SAVE_OUTLINED, on_click=salvar_produto),
                        DS.botao_outline("Limpar", on_click=lambda e: page.update()),
                    ], spacing=Espacamento.SM),
                    msg_banner,

```

```

        ], spacing=Espacamento.SM),
        padding=Espacamento.MD,
    ),
    ft.Container(
        content=ft.Text("Produtos cadastrados", size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60),
        padding=ft.padding.only(left=4, bottom=Espacamento.SM),
    ),
    ft.Container(content=produtos_list, expand=True),
], spacing=Espacamento.MD, expand=True),
padding=Espacamento.LG,
bgcolor=Cores.FUNDO,
expand=True,
on_mount=lambda e: carregar_produtos(),
)

# =====
# TELA DE VENDAS v2.0
# =====

def criar_tela_vendas(page):
    vendas_list = ft.ListView(spacing=Espacamento.SM, padding=0, expand=True)

    def carregar_vendas():
        vendas_list.controls.clear()
        result = api.get("/api/vendas/")
        if result["success"]:
            vendas = result["data"]
            total_geral = 0
            for v in vendas:
                total = v.get("total", 0)
                total_geral += total
                vendas_list.controls.append(DS.card(
                    padding=0,
                    shadow=False,
                    border=ft.border.all(1, Cores.BORDA),
                    content=ft.Container(
                        content=ft.Row([
                            ft.Container(
                                content=ft.Icon(ft.Icons.POINT_OF_SALE, size=16, color=Cores.SUCESSO),
                                bgcolor=ft.Colors.with_opacity(0.1, Cores.SUCESSO),
                                border_radius=Bordas.SM,
                                padding=6, width=36, height=36,
                                alignment=ft.alignment.center,
                            ),
                            ft.Container(width=Espacamento.SM),
                            ft.Column([
                                ft.Text("Venda #" + str(v.get("id", 0)), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0),
                                ft.Text(str(v.get("cliente_nome", "N/A")), size=11, color=Cores.TEXT0_60),
                                ft.Text(str(v.get("data", "N/A")), size=10, color=Cores.TEXT0_40),
                            ], spacing=2, expand=True),
                                ft.Text("R$ %.2f" % total, size=14, weight=ft.FontWeight.W_700, color=Cores.SUCESSO, font_family="Courier New"),
                            ], spacing=Espacamento.SM, alignment=ft.MainAxisAlignment.START),
                        padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
                    ),
                ))
            vendas_list.controls.insert(0, DS.card(
                content=ft.Row([
                    ft.Text(str(len(vendas)) + " vendas", size=12, color=Cores.TEXT0_60),
                    ft.Text("Total: R$ %.2f" % total_geral, size=14, weight=ft.FontWeight.W_700, color=Cores.P, font_family="Courier New"),
                ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
                padding=Espacamento.MD,
                shadow=False,
                border=ft.border.all(1, Cores.P20),
                bgcolor=ft.Colors.with_opacity(0.03, Cores.P),
            ))
        page.update()

    return ft.Container(
        content=ft.Column([
            DS.page_header("Vendas", subtitle="Historico de vendas"),
            ft.Container(content=vendas_list, expand=True),
        ], spacing=Espacamento.MD),
        padding=Espacamento.LG,
        bgcolor=Cores.FUNDO,
        expand=True,
        on_mount=lambda e: carregar_vendas(),
    )

# =====
# TELA DE DASHBOARD v2.0
# =====

def criar_tela_dashboard(page):
    stats_column = ft.Row(spacing=Espacamento.SM, wrap=True, scroll=ft.ScrollMode.AUTO)

    def carregar_stats():
        stats_column.controls.clear()

        clientes_r = api.get("/api/clientes/")
        produtos_r = api.get("/api/produtos/")
        vendas_r = api.get("/api/vendas/")

        n_clientes = len(clientes_r["data"]) if clientes_r["success"] else 0

```

```

n_produtos = len(produtos_r["data"]) if produtos_r["success"] else 0
n_vendas = len(vendas_r["data"]) if vendas_r["success"] else 0

total_vendas = 0
if vendas_r["success"]:
    for v in vendas_r["data"]:
        total_vendas += v.get("total", 0)

stats = [
    ("Clientes", str(n_clientes), ft.Icons.PEOPLE_OUTLINED, Cores.P),
    ("Produtos", str(n_produtos), ft.Icons.INVENTORY_2_OUTLINED, Cores.INFO),
    ("Vendas", str(n_vendas), ft.Icons.POINT_OF_SALE, Cores.SUCESSO),
    ("Faturamento", "R$ %.2f" % total_vendas, ft.Icons.ATTACH_MONEY, Cores.ALERTA),
]

for titulo, valor, icone, cor in stats:
    stats_column.controls.append(DS.stat_card(titulo, valor, icone, cor))

page.update()

return ft.Container(
    content=ft.Column([
        DS.page_header("Dashboard",
            subtitle="Bem-vindo, " + str(api.user.get("nome", "Usuario")) + "!",
            actions=[DS.icone_botao(ft.Icons.REFRESH, "Atualizar", on_click=lambda e: carregar_stats())]),
        stats_column,
        DS.divider(),
        DS.subtitulo("Resumo do Negocio", size=13),
    ], spacing=Espacamento.MD,
    padding=Espacamento.LG,
    bgcolor=Cores.FUNDO,
    expand=True,
    on_mount=lambda e: carregar_stats(),
)

# =====
# TELA DE TESTE v2.0
# =====

def criar_tela_teste(page):
    result_text = ft.Text("", size=12, selectable=True, color=Cores.TEXT0_80)
    progress_bar = ft.ProgressBar(width=300, visible=False, color=Cores.P, bgcolor=Cores.P10)
    progress_text = ft.Text("", size=11, color=Cores.TEXT0_60)

    def executar_teste(e):
        def run_test():
            progress_bar.visible = True
            result_text.value = "Iniciando teste de 100 clientes..."
            page.update()
            sucessos = 0
            falhas = 0
            for i in range(1, 101):
                data = {
                    "nome": "Cliente Teste %03d" % i,
                    "cpf_cnpj": "%011d" % i,
                    "email": "cliente%03d@teste.com" % i,
                    "telefone": "(62) 9%08d" % i,
                    "endereco": "Rua Teste, %d" % i,
                }
                result = api.post("/api/clientes/", data)
                if result["success"]:
                    sucessos += 1
                else:
                    falhas += 1
            if i % 10 == 0:
                progress_bar.value = i / 100
                progress_text.value = "Progresso: %d/100" % i
                page.update()
            result_text.value = "Teste concluido!\n Sucessos: %d/100\n Falhas: %d" % (sucessos, falhas)
            progress_bar.visible = False
            page.update()
            threading.Thread(target=run_test, daemon=True).start()

        return ft.Container(
            content=ft.Column([
                DS.page_header("Teste de Carga", subtitle="Insira 100 clientes de teste"),
                DS.card(
                    content=ft.Column([
                        DS.paragrafo("Este teste insere 100 clientes de exemplo para validar a performance da API."),
                        ft.Container(height=Espacamento.SM),
                        DS.botao("Executar Teste - 100 Clientes", icon=ft.Icons.PLAY_ARROW_ROUNDED, on_click=executar_teste),
                        ft.Container(height=Espacamento.SM),
                        progress_bar,
                        progress_text,
                    ], spacing=Espacamento.SM,
                    padding=Espacamento.MD,
                ),
                ft.Container(height=Espacamento.SM),
                DS.titulo("Resultado", size=14),
                DS.card(
                    content=result_text,
                    padding=Espacamento.MD,
                    shadow=False,

```



```

        border=ft.border.all(1, Cores.BORDA),
    ),
    ], spacing=Espacamento.MD),
    padding=Espacamento.LG,
    bgcolor=Cores.FUNDO,
    expand=True,
)

# =====
# LAYOUT PRINCIPAL v2.0
# =====

def criar_layout_principal():
    """Layout principal com sidebar moderna"""

    main_content = ft.Container(
        content=criar_tela_dashboard(None),
        expand=True,
        bgcolor=Cores.FUNDO,
    )

    # Indice selecionado (usando closure)
    state = {"selected": 0}

    def navegar(index):
        state["selected"] = index
        telas = {
            0: criar_tela_dashboard,
            1: criar_tela_clientes,
            2: criar_tela_produtos,
            3: criar_tela_vendas,
            4: criar_tela_teste,
        }
        if index in telas:
            main_content.content = telas[index](main_content.page)
            main_content.page.update()

    # Sidebar items
    sidebar_items = [
        ("Dashboard", ft.Icons.DASHBOARD_OUTLINED, ft.Icons.DASHBOARD),
        ("Clientes", ft.Icons.PEOPLE_OUTLINED, ft.Icons.PEOPLE),
        ("Produtos", ft.Icons.INVENTORY_2_OUTLINED, ft.Icons.INVENTORY_2),
        ("Vendas", ft.Icons.POINT_OF_SALE_OUTLINED, ft.Icons.POINT_OF_SALE),
        ("Teste 100", ft.Icons.SCIENCE_OUTLINED, ft.Icons.SCIENCE),
    ]

    sidebar_controls = []
    for i, (label, icon_out, icon_fill) in enumerate(sidebar_items):
        idx = i
        is_selected = i == 0

        sidebar_controls.append(DS.sidebar_item(
            label=label,
            icon=icon_fill if is_selected else icon_out,
            selected=is_selected,
            on_click=lambda e, i=idx: navegar(i),
        ))

    # Sidebar
    sidebar = ft.Container(
        content=ft.Column([
            # Logo
            ft.Container(
                content=ft.Row([
                    ft.Container(
                        content=ft.Icon(ft.Icons.STORE, size=22, color=Cores.FUNDO_CARD),
                        bgcolor=Cores.P,
                        border_radius=Bordas.SM,
                        padding=8,
                    ),
                    ft.Container(width=8),
                ], spacing=0),
                ft.Column([
                    ft.Text("SaltoGestao", size=14, weight=ft.FontWeight.W_700, color=Cores.FUNDO_CARD, font_family="Segoe UI"),
                    ft.Text("v2.0", size=9, color=Cores.P20, font_family="Segoe UI"),
                ], spacing=0),
            ], spacing=4, alignment=ft.MainAxisAlignment.START),
            padding=ft.padding.symmetric(horizontal=Espacamento.SM, vertical=Espacamento.MD),
            margin=ft.padding.only(top=Espacamento.SM, bottom=Espacamento.MD),
        ),
        ft.Container(height=Espacamento.SM),

        # Nav items
        ft.Column(sidebar_controls, spacing=2),

        ft.Container(expand=True),

        # Footer sidebar
        ft.Container(
            content=DS.subtitulo("Corumba Sistemas (c) 2026", size=9),
            padding=ft.padding.all(Espacamento.SM),
        ),
    ], spacing=0),
    width=220,

```

```

        bgcolor=Cores.P,
        padding=ft.padding.symmetric(vertical=Espacamento.SM),
    )

    return ft.Container(
        content=ft.Row([
            sidebar,
            ft.VerticalDivider(width=1, color=Cores.BORDA),
            main_content,
        ], expand=True, spacing=0),
        expand=True,
    )

# =====
# APP PRINCIPAL
# =====

def main(page: ft.Page):
    page.title = "SaltoGestao v2.0"
    page.padding = 0
    page.window.width = 1280
    page.window.height = 800
    page.window.min_width = 1024
    page.window.min_height = 700
    page.bgcolor = Cores.FUNDO
    page.theme = ft.Theme(
        color_scheme_seed=Cores.P,
        font_family="Segoe UI",
        use_material3=True,
    )

    main_content = ft.Container(expand=True, bgcolor=Cores.FUNDO)

    def on_login_success():
        main_content.content = criar_layout_principal()
        page.update()

    main_content.content = criar_tela_login(page, on_login_success)
    page.add(main_content)

if __name__ == "__main__":
    ft.app(target=main)

```

design_system.py

desktop/design_system.py • 716 linhas

```

"""
=====
SALTO GESTAO - SISTEMA DESKTOP v2.0
Design Moderno e Clean - Paleta Corumba Sistemas
=====

Paleta de cores:
--cor-primaria:      #0b1a30 (azul escuro - marca Corumba)
--cor-primaria-80:   #1a2d4a (variante 80%)
--cor-primaria-60:   #2d4a6e (variante 60%)
--cor-primaria-40:   #5a7ea8 (variante 40%)
--cor-primaria-20:   #9bb5d0 (variante 20%)
--cor-primaria-10:   #c5d5e8 (variante 10%)

--cor-fundo:         #f5f7fa (cinza muito claro - fundo app)
--cor-fundo-card:    #ffffff (branco puro - cards)
--cor-fundo-hover:   #ebeeef (hover sutil)

--cor-texto:         #0b1a30 (texto principal)
--cor-texto-80:      #3c4a5e (texto secundario)
--cor-texto-60:      #6d7a8e (texto terciario)
--cor-texto-40:      #9eaabe (texto desabilitado)

--cor-sucesso:       #22c55e (verde)
--cor-alerta:        #f59e0b (amarelo/laranja)
--cor-erro:          #ef4444 (vermelho)
--cor-info:          #3b82f6 (azul claro)

--cor-borda:         #e2e8f0 (bordas sutis)
--cor-borda-focus:   #0b1a30 (borda em foco)

Tipografia:
--font-family: 'Segoe UI', 'Inter', system-ui, sans-serif
--font-mono: 'Cascadia Code', 'Consolas', monospace

Espacamento:
--space-xs: 4px
--space-sm: 8px
--space-md: 16px
--space-lg: 24px
--space-xl: 32px
--space-2xl: 48px

Borda:
--radius-sm: 8px
--radius-md: 12px

```

```

--radius-lg: 16px
--radius-xl: 20px
--radius-full: 9999px

Sombras:
--shadow-sm: 0 1px 3px rgba(11,26,48,0.08)
--shadow-md: 0 4px 12px rgba(11,26,48,0.10)
--shadow-lg: 0 8px 24px rgba(11,26,48,0.12)
--shadow-xl: 0 16px 48px rgba(11,26,48,0.16)

Transicoes:
--transition-fast: 150ms ease
--transition-normal: 250ms ease
--transition-slow: 350ms ease
"""

import flet as ft
import requests
import json
import threading
from datetime import datetime

# =====
# DESIGN SYSTEM - CONSTANTES
# =====

# Paleta Corumba Sistemas
class Cores:
    """Paleta de cores Corumba Sistemas"""
    # Primaria
    P = "#0b1a30" # Primaria (marca)
    P80 = "#1a2d4a"
    P60 = "#2d4a6e"
    P40 = "#5a7ea8"
    P20 = "#9bb5d0"
    P10 = "#c5d5e8"
    P5 = "#e2e8f0"

    # Fundos
    FUNDO = "#f5f7fa"
    FUNDO_CARD = "ffffff"
    FUNDO_HOVER = "#ebeeef"

    # Texto
    TEXT0 = "#0b1a30"
    TEXT0_80 = "#3c4a5e"
    TEXT0_60 = "#6d7a8e"
    TEXT0_40 = "#9eaabe"

    # Status
    SUCESSO = "#22c55e"
    ALERTA = "#f59e0b"
    ERRO = "#ef4444"
    INFO = "#3b82f6"

    # Bordas
    BORDA = "#e2e8f0"
    BORDA_FOCUS = "#0b1a30"
    ESCURO = "#0b1a30"

    # Transparentes
    OVERLAY = "rgba(11,26,48,0.5)"

class Espacamento:
    """Escala de espacamento"""
    XS = 4
    SM = 8
    MD = 16
    LG = 24
    XL = 32
    XXL = 48

class Bordas:
    """Raios de borda"""
    SM = 8
    MD = 12
    LG = 16
    XL = 20
    FULL = 9999

# =====
# COMPONENTES BASE
# =====

class DS:
    """Design System - Componentes reutilizaveis"""

    @staticmethod
    def container(
        content=None,
        bgcolor=Cores.FUNDO_CARD,
        border_radius=Bordas.MD,

```

```

padding=Espacamento.MD,
margin=None,
width=None,
height=None,
border=None,
shadow=True,
expand=False,
col=None,
):
    """Card/container base com estilo padrao"""
    return ft.Container(
        content=content,
        bgcolor=bgcolor,
        border_radius=border_radius,
        padding=padding,
        margin=margin,
        width=width,
        height=height,
        border=border,
        expand=expand,
        col=col,
        shadow=ft.BoxShadow(
            spread_radius=0,
            blur_radius=8,
            color=ft.Colors.with_opacity(0.06, Cores.ESCURO),
            offset=ft.Offset(0, 2),
        ) if shadow else None,
    )

@staticmethod
def titulo(texto, size=16, weight=ft.FontWeight.W_600, color=Cores.TEXT0):
    """Titulo padronizado"""
    return ft.Text(
        texto,
        size=size,
        weight=weight,
        color=color,
        font_family="Segoe UI",
        selectable=True,
    )

@staticmethod
def subtitulo(texto, size=12, color=Cores.TEXT0_60):
    """Subtitulo padronizado"""
    return ft.Text(
        texto,
        size=size,
        color=color,
        font_family="Segoe UI",
        selectable=True,
    )

@staticmethod
def paragrafo(texto, size=13, color=Cores.TEXT0_80, selectable_text=True):
    """Paragrafo padronizado"""
    return ft.Text(
        texto,
        size=size,
        color=color,
        font_family="Segoe UI",
        selectable=selectable_text,
        weight=ft.FontWeight.W_400,
    )

@staticmethod
def campo(
    label,
    prefix_icon=None,
    password=False,
    can_reveal_password=False,
    width=None,
    height=48,
    text_size=14,
    hint_text=None,
    on_change=None,
    on_focus=None,
    suffix=None,
):
    """Campo de entrada padronizado estilo moderno"""
    return ft.TextField(
        label=label,
        prefix_icon=prefix_icon,
        password=password,
        can_reveal_password=can_reveal_password,
        width=width,
        height=height,
        text_size=text_size,
        hint_text=hint_text,
        hint_style=ft.TextStyle(
            size=13,
            color=Cores.TEXT0_40,
            font_family="Segoe UI",
        ),
        label_style=ft.TextStyle(
            size=12,

```

```

        color=Cores.TEXT0_60,
        font_family="Segoe UI",
    ),
    text_style=ft.TextStyle(
        size=text_size,
        color=Cores.TEXT0,
        font_family="Segoe UI",
    ),
    border_color=Cores.BORDA,
    focused_border_color=Cores.P,
    focused_border_width=2,
    border_radius=Bordas.MD,
    bgcolor=Cores.FUNDO_CARD,
    content_padding=ft.padding.only(left=16, right=16, top=12, bottom=12),
    on_change=on_change,
    on_focus=on_focus,
    suffix=suffix,
)

@staticmethod
def botao(
    texto,
    on_click=None,
    bgcolor=Cores.P,
    color=Cores.FUNDO_CARD,
    width=None,
    height=44,
    icon=None,
    disabled=False,
    style=None,
    expand=False,
):
    """Botao padrao com estilo moderno"""
    btn_style = ft.ButtonStyle(
        bgcolor={
            ft.ControlState.DEFAULT: bgcolor,
            ft.ControlState.HOVERED: Cores.P80,
            ft.ControlState.DISABLED: Cores.TEXT0_40,
        },
        color=color,
        shape=ft.RoundedRectangleBorder(radius=Bordas.MD),
        padding=ft.padding.symmetric(horizontal=24, vertical=12),
        elevation={
            ft.ControlState.DEFAULT: 0,
            ft.ControlState.HOVERED: 2,
        },
        animation_duration=150,
        text_style=ft.TextStyle(
            size=14,
            weight=ft.FontWeight.W_600,
            font_family="Segoe UI",
        ),
    ),
)

content = texto
if icon:
    content = ft.Row(
        [
            ft.Icon(icon, size=18, color=color),
            ft.Text(texto, color=color, size=14, weight=ft.FontWeight.W_600),
        ],
        alignment=ft.MainAxisAlignment.CENTER,
        spacing=8,
    )

return ft.ElevatedButton(
    content=content,
    on_click=on_click,
    width=width,
    height=height,
    disabled=disabled,
    style=btn_style if style is None else style,
    expand=expand,
)

@staticmethod
def botao_outline(
    texto,
    on_click=None,
    color=Cores.P,
    width=None,
    height=44,
    icon=None,
    expand=False,
):
    """Botao secundario outline"""
    style = ft.ButtonStyle(
        bgcolor={
            ft.ControlState.DEFAULT: Cores.FUNDO_CARD,
            ft.ControlState.HOVERED: Cores.P5,
        },
        color={
            ft.ControlState.DEFAULT: color,
            ft.ControlState.HOVERED: Cores.P,
        },
    ),

```

```

        shape=ft.RoundedRectangleBorder(radius=Bordas.MD),
        side=ft.BorderSide(width=1.5, color=Cores.BORDA),
        padding=ft.padding.symmetric(horizontal=24, vertical=12),
        animation_duration=150,
        text_style=ft.TextStyle(
            size=14,
            weight=ft.FontWeight.W_500,
            font_family="Segoe UI",
        ),
    )

    content = texto
    if icon:
        content = ft.Row(
            [
                ft.Icon(icon, size=18, color=color),
                ft.Text(texto, color=color, size=14, weight=ft.FontWeight.W_500),
            ],
            alignment=ft.MainAxisAlignment.CENTER,
            spacing=8,
        )

    return ft.OutlinedButton(
        content=content,
        on_click=on_click,
        width=width,
        height=height,
        style=style,
        expand=expand,
    )

@staticmethod
def badge(texto, color=Cores.P20, text_color=Cores.P, border_radius=Bordas.FULL):
    """Badge/tag padrao"""
    return ft.Container(
        content=ft.Text(
            texto,
            size=11,
            weight=ft.FontWeight.W_500,
            color=text_color,
            font_family="Segoe UI",
        ),
        bgcolor=color,
        border_radius=border_radius,
        padding=ft.padding.symmetric(horizontal=10, vertical=4),
    )

@staticmethod
def icone_botao(icon, tooltip=None, on_click=None, icon_color=Cores.TEXT0_60, size=20):
    """Botao de icone circular"""
    return ft.IconButton(
        icon=icon,
        tooltip=tooltip,
        icon_color=icon_color,
        icon_size=size,
        on_click=on_click,
        style=ft.ButtonStyle(
            shape=ft.CircleBorder(),
            padding=8,
            bgcolor={
                ft.ControlState.DEFAULT: Cores.P5,
                ft.ControlState.HOVERED: Cores.P10,
            },
        ),
    )

@staticmethod
def card(
    titulo=None,
    subtitulo=None,
    content=None,
    padding=Espacamento.MD,
    margin=None,
    width=None,
    height=None,
    on_click=None,
    header_icon=None,
    header_color=Cores.P,
    header_actions=None,
    expand=False,
    col=None,
):
    """Card com estilo moderno"""
    # Header do card
    header = None
    if titulo:
        header_content = []
        if header_icon:
            header_content.append(ft.Icon(header_icon, size=18, color=header_color))
        header_content.append(DS.titulo(titulo, size=14, color=Cores.TEXT0))
        if subtitulo:
            header_content.append(ft.Container(width=8))
            header_content.append(DS.subtitulo(subtitulo))
        if header_actions:
            header_content.append(ft.Row(expand=True, controls=[], alignment=ft.MainAxisAlignment.END))

```

```

        for action in header_actions:
            header_content.append(action)

        header = ft.Container(
            content=ft.Row(header_content, spacing=8, alignment=ft.MainAxisAlignment.START),
            padding=ft.padding.only(left=padding, right=padding, top=padding, bottom=Espacamento.SM) if content else
ft.padding.all(padding),
            border=ft.border.only(bottom=ft.BorderSide(1, Cores.BORDA)) if content else None,
        )

        # Footer com conteudo
        body = None
        if content:
            body = ft.Container(
                content=content,
                padding=ft.padding.all(padding),
                expand=True,
            )

        children = [c for c in [header, body] if c]

        card = ft.Container(
            content=ft.Column(children, spacing=0, expand=True) if len(children) > 1 else children[0],
            bgcolor=Cores.FUNDO_CARD,
            border_radius=Bordas.LG,
            padding=0,
            margin=margin,
            width=width,
            height=height,
            expand=expand,
            col=col,
            shadow=ft.BoxShadow(
                spread_radius=0,
                blur_radius=8,
                color=ft.Colors.with_opacity(0.06, Cores.ESCUR0),
                offset=ft.Offset(0, 2),
            ),
            animate=ft.Animation(150, ft.AnimationCurve.EASE_OUT),
            on_hover=lambda e: None, # Placeholder para hover animation
            on_click=on_click,
        )

        return card

    @staticmethod
    def divider():
        """Divisor padrao"""
        return ft.Container(
            height=1,
            bgcolor=Cores.BORDA,
            margin=ft.padding.symmetric(vertical=Espacamento.SM),
        )

    @staticmethod
    def loading_spinner(size=24, color=Cores.P):
        """Spinner de carregamento"""
        return ft.ProgressRing(
            width=size,
            height=size,
            stroke_width=3,
            color=color,
            bgcolor=ft.Colors.with_opacity(0.1, color),
        )

    @staticmethod
    def stat_card(titulo, valor, icone=None, cor=Cores.P, subtítulo=None, tendencia=None):
        """Card de estatística/dashboard"""
        children = []

        # Icon + Titulo row
        titulo_row = []
        if icone:
            titulo_row.append(ft.Container(
                content=ft.Icon(icone, size=20, color=cor),
                bgcolor=ft.Colors.with_opacity(0.1, cor),
                border_radius=Bordas.SM,
                padding=8,
            ))
            titulo_row.append(ft.Container(width=4))
        titulo_row.append(DS.subtitulo(titulo, size=11))
        children.append(ft.Row(titulo_row, spacing=8))

        children.append(ft.Container(height=4))

        # Valor grande
        children.append(ft.Text(
            str(valor),
            size=26,
            weight=ft.FontWeight.W_700,
            color=Cores.TEXT0,
            font_family="Segoe UI",
        ))

        # Subtitulo/Tendencia
        if subtítulo or tendencia:

```

```

        extras = []
        if subtítulo:
            extras.append(DS.subtítulo(subtítulo, size=10))
        if tendencia:
            cor_trend = Cores.SUCESSO if tendencia > 0 else Cores.ERRO
            extras.append(ft.Text(
                ("+" if tendencia > 0 else "") + str(tendencia) + "%",
                size=10,
                weight=ft.FontWeight.W_600,
                color=cor_trend,
            ))
        children.append(ft.Row(extras, spacing=4))

    return DS.card(
        content=ft.Column(children, spacing=2, alignment=ft.MainAxisAlignment.START),
        padding=Espacamento.MD,
        width=200,
        height=90,
    )

    @staticmethod
    def tabela_header(*headers):
        """Header de tabela"""
        cells = [ft.Text(h, size=11, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60, font_family="Segoe UI") for h in headers]
        return ft.Container(
            content=ft.Row(cells, spacing=Espacamento.MD, alignment=ft.MainAxisAlignment.START),
            padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=Espacamento.SM),
            bgcolor=Cores.FUNDO,
            border_radius=ft.border_radius.only(top_left=Bordas.MD, top_right=Bordas.MD),
        )

    @staticmethod
    def tabela_row(*cells, selected=False, hover=True):
        """Linha de tabela"""
        controls = []
        for cell in cells:
            if isinstance(cell, str):
                controls.append(ft.Text(cell, size=12, color=Cores.TEXT0_80, font_family="Segoe UI"))
            else:
                controls.append(cell)

        return ft.Container(
            content=ft.Row(controls, spacing=Espacamento.MD, alignment=ft.MainAxisAlignment.START),
            padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
            bgcolor=Cores.P5 if selected else Cores.FUNDO_CARD,
            border=ft.border.only(bottom=ft.BorderSide(0.5, Cores.BORDA)),
            animate=ft.Animation(100, ft.AnimationCurve.EASE_OUT),
        )

    @staticmethod
    def sidebar_item(label, icon, selected=False, on_click=None, badge_count=None):
        """Item da sidebar de navegacao"""
        bg = ft.Colors.with_opacity(0.08, Cores.P) if selected else ft.Colors.TRANSPARENT
        text_weight = ft.FontWeight.W_600 if selected else ft.FontWeight.W_400
        icon_color = Cores.P if selected else Cores.TEXT0_60
        text_color = Cores.P if selected else Cores.TEXT0_80

        row_controls = [
            ft.Icon(icon, size=20, color=icon_color),
            ft.Container(width=4),
            ft.Text(label, size=13, weight=text_weight, color=text_color, font_family="Segoe UI"),
        ]

        if badge_count:
            row_controls.append(
                ft.Container(expand=True, content=None, alignment=ft.alignment.center_right)
            )
            row_controls.append(DS.badge(str(badge_count), color=Cores.ERRO, text_color=Cores.FUNDO_CARD))

        return ft.Container(
            content=ft.Row(row_controls, spacing=12, alignment=ft.MainAxisAlignment.START),
            padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
            bgcolor=bg,
            border_radius=Bordas.MD,
            on_click=on_click,
            animate=ft.Animation(150, ft.AnimationCurve.EASE_OUT),
            on_hover=lambda e: (
                e.control.__setattr__('bgcolor', ft.Colors.with_opacity(0.08, Cores.P) if e.data == "true" and not selected else None),
                e.control.update(),
            ) if True else None,
            margin=ft.padding.symmetric(horizontal=Espacamento.SM, vertical=2),
        )

    @staticmethod
    def nav_bar(title, page, user_name=None, avatar_url=None, actions=None):
        """Barra de navegacao superior"""
        left = [
            DS.titulo(title, size=18, color=Cores.P),
        ]

        right = []
        if actions:
            right.extend(actions)

        if user_name:

```



```

        if avatar_url is None:
            right.append(
                ft.Row([
                    ft.CircleAvatar(
                        content=ft.Text(user_name[0].upper(), size=14, weight=ft.FontWeight.W_600, color=Cores.FUNDO_CARD),
                        bgcolor=Cores.P,
                        radius=16,
                    ),
                    ft.Container(width=4,
                        ft.Text(user_name.split()[0], size=12, weight=ft.FontWeight.W_500, color=Cores.TEXT0_80, font_family="Segoe UI"),
                    ], spacing=8, alignment=ft.MainAxisAlignment.CENTER)
                )

    return ft.Container(
        content=ft.Row(
            [
                ft.Row(left, spacing=Espacamento.MD),
                ft.Row(right, spacing=Espacamento.SM, expand=True, alignment=ft.MainAxisAlignment.END),
            ],
            alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
        ),
        padding=ft.padding.symmetric(horizontal=Espacamento.LG, vertical=Espacamento.SM),
        bgcolor=Cores.FUNDO_CARD,
        border=ft.border.only(bottom=ft.BorderSide(1, Cores.BORDA)),
        shadow=ft.BoxShadow(
            spread_radius=0,
            blur_radius=4,
            color=ft.Colors.with_opacity(0.04, Cores.ESCURO),
            offset=ft.Offset(0, 1),
        ),
    ),
)

@staticmethod
def page_header(title, subtitle=None, actions=None):
    """Header de pagina"""
    left = ft.Column(
        [ft.Text(title, size=22, weight=ft.FontWeight.W_700, color=Cores.TEXT0, font_family="Segoe UI")]
        + ([ft.Container(height=2, ft.Text(subtitle, size=13, color=Cores.TEXT0_60, font_family="Segoe UI")] if subtitle else []),
        spacing=4,
    )

    return ft.Container(
        content=ft.Row(
            [
                left,
                ft.Row(actions or [], spacing=Espacamento.SM, expand=True, alignment=ft.MainAxisAlignment.END),
            ],
            alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
        ),
        padding=ft.padding.only(left=Espacamento.LG, right=Espacamento.LG, top=Espacamento.LG, bottom=Espacamento.SM),
        bgcolor=Cores.FUNDO,
    )

@staticmethod
def empty_state(icon=ft.Icons.INBOX_OUTLINED, title="Nenhum dado encontrado", subtitle=None, action=None):
    """Estado vazio elegante"""
    return ft.Container(
        content=ft.Column(
            [ft.Icon(icon, size=48, color=Cores.TEXT0_40)]
            + [ft.Container(height=Espacamento.MD)]
            + [ft.Text(title, size=16, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60, text_align=ft.TextAlign.CENTER)]
            + [ft.Container(height=Espacamento.XS)]
            + [ft.Text(subtitle or "", size=12, color=Cores.TEXT0_40, text_align=ft.TextAlign.CENTER)]
            + ([ft.Container(height=Espacamento.MD), action] if action else []),
            alignment=ft.MainAxisAlignment.CENTER,
            horizontal_alignment=ft.CrossAxisAlignment.CENTER,
        ),
        alignment=ft.alignment.center,
        padding=Espacamento.XL,
    )

@staticmethod
def snackbar(message, type="info", duration=3000):
    """Notificacao toast"""
    type_config = {
        "success": {"icon": ft.Icons.CHECK_CIRCLE, "color": Cores.SUCESSO, "bg": ft.Colors.with_opacity(0.1, Cores.SUCESSO)},
        "error": {"icon": ft.Icons.ERROR_OUTLINED, "color": Cores.ERRO, "bg": ft.Colors.with_opacity(0.1, Cores.ERRO)},
        "warning": {"icon": ft.Icons.WARNING_OUTLINED, "color": Cores.ALERTA, "bg": ft.Colors.with_opacity(0.1, Cores.ALERTA)},
        "info": {"icon": ft.Icons.INFO_OUTLINED, "color": Cores.INFO, "bg": ft.Colors.with_opacity(0.1, Cores.INFO)},
    }
    cfg = type_config.get(type, type_config["info"])

    return ft.Container(
        content=ft.Row([
            ft.Icon(cfg["icon"], size=18, color=cfg["color"]),
            ft.Container(width=8),
            ft.Text(message, size=13, color=Cores.TEXT0_80, font_family="Segoe UI", expand=True),
        ], spacing=8),
        bgcolor=cfg["bg"],
        border_radius=Bordas.MD,
        padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=Espacamento.SM),
        border=ft.border.all(1, ft.Colors.with_opacity(0.2, cfg["color"])),
        shadow=ft.BoxShadow(
            spread_radius=0,
            blur_radius=12,

```

```

        color=ft.Colors.with_opacity(0.1, Cores.ESCUR0),
        offset=ft.Offset(0, 4),
    ),
    animate=ft.Animation(200, ft.AnimationCurve.EASE_OUT),
)

```

testar_100_clientes.py

desktop/testar_100_clientes.py • 357 linhas

```

"""
=====
Teste de 100 Clientes - SaltoGestão Desktop
=====
Insere 100 clientes via API e valida cada operação.
Uso: python3 testar_100_clientes.py
"""

import requests
import json
import time
import sys

# =====
# CONFIGURAÇÃO
# =====
# Tenta local primeiro, depois VPS
URLS = [
    "http://localhost:8080",
    "http://191.252.210.235:8080",
    "http://192.168.100.68:8080",
]

API_BASE = None
for url in URLS:
    try:
        r = requests.get(f"{url}/health", timeout=3)
        if r.status_code == 200:
            API_BASE = url
            print(f"✅ API conectada em: {url}")
            break
    except Exception:
        continue

if not API_BASE:
    print("❌ Nenhuma API encontrada. Inicie o backend primeiro.")
    sys.exit(1)

# =====
# LOGIN PARA OBTER TOKEN
# =====
def fazer_login():
    """Faz login e retorna o token JWT."""
    credenciais = [
        {"email": "admin@corumba.com", "senha": "admin123"},
        {"email": "admin@salgestao.com", "senha": "admin123"},
        {"email": "admin@saltogestao.com", "senha": "admin123"},
    ]
    for cred in credenciais:
        try:
            r = requests.post(f"{API_BASE}/api/auth/login", json=cred, timeout=5)
            if r.status_code == 200:
                data = r.json()
                token = data.get("access_token") or data.get("token")
                if token:
                    print(f"✅ Login OK com: {cred['email']}")
                    return token
        except Exception:
            continue
    # Tenta registrar novo admin
    try:
        dados = {
            "nome": "Admin Teste",
            "email": "admin@teste.com",
            "senha": "teste123",
            "tipo": "admin",
            "cnpj_empresa": "00.000.000/0001-00"
        }
        r = requests.post(f"{API_BASE}/api/auth/register", json=dados, timeout=5)
        if r.status_code in [200, 201]:
            token = r.json().get("access_token") or r.json().get("token")
            if token:
                print("✅ Admin registrado e logado")
                return token
    except Exception:
        pass
    print("❌ Falha no login. Verifique as credenciais.")
    return None

# =====
# GERA DADOS DE 100 CLIENTES
# =====
def gerar_clientes_100():
    """Gera 100 clientes com dados variados e realistas."""
    nomes_base = [

```

```

"João", "Maria", "José", "Ana", "Pedro", "Carla", "Lucas", "Fernanda",
"Marcos", "Juliana", "Rafael", "Patricia", "Bruno", "Camila", "Diego",
"Amanda", "Gustavo", "Larissa", "Thiago", "Beatriz", "Felipe", "Mariana",
"Rodrigo", "Letícia", "André", "Vanessa", "Marcelo", "Tatiana", "Leonardo",
"Priscila", "Eduardo", "Sabrina", "Ricardo", "Isabela", "Fábio", "Natália",
"Alexandre", "Daniela", "Guilherme", "Bianca", "Vinicius", "Lorena",
"Matheus", "Gabriela", "Samuel", "Helena", "Isaac", "Valentina", "Daniel",
"Laura", "Henrique", "Manuela", "Caio", "Sophia", "Enzo", "Heloísa",
"Miguel", "Alice", "Arthur", "Júlia", "Bernardo", "Luísa", "Heitor",
"Isabella", "Davi", "Lara", "Lorenzo", "Cecília", "Theo", "Eloá",
"Gabriel", "Yasmin", "Noah", "Rafaela", "Benjamin", "Giovanna", "Samuel",
"Clara", "Anthony", "Livia", "Joaquim", "Melissa", "Nicolas", "Rebeca",
"Lucca", "Agatha", "Ryan", "Esther", "Ian", "Nicole", "David", "Sarah",
"Pedro Henrique", "Vitória", "Bruno Henrique", "Luna", "Cauã", "Marina",
"Kaique", "Valentim", "Yuri", "Antonella"
]

sobrenomes = [
"Silva", "Santos", "Oliveira", "Souza", "Rodrigues", "Ferreira", "Alves",
"Pereira", "Lima", "Gomes", "Costa", "Ribeiro", "Martins", "Carvalho",
"Araújo", "Melo", "Barbosa", "Rocha", "Dias", "Nascimento", "Andrade",
"Moreira", "Nunes", "Marques", "Machado", "Mendes", "Freitas", "Cardoso",
" Ramos", "Vieira", "Monteiro", "Cunha", "Guimarães", "Figueiredo",
"Pinto", "Correia", "Teixeira", "Caldeira", "Borges", "Miranda"
]

cidades = [
("Aparecida de Goiânia", "GO"), ("Goiânia", "GO"), ("Anápolis", "GO"),
("Rio Verde", "GO"), ("Luziânia", "GO"), ("Águas Lindas", "GO"),
("Valparaíso", "GO"), ("Trindade", "GO"), ("Novo Gama", "GO"),
("Senador Canedo", "GO"), ("Itumbiara", "GO"), ("Jataí", "GO"),
("Formosa", "GO"), ("Catalão", "GO"), ("Caldas Novas", "GO")
]

bairros = [
"Centro", "Jardim América", "Setor Oeste", "Setor Bueno",
"Jardim Goiás", "Vila Brasília", "Setor Central", "Campinas",
"Vila Aurora", "Jardim Europa", "Setor Marista", "PQ Oeste",
"Porto Grande", "Jardim Novo Mundo", "Vila Aurora"
]

ruas = [
"Rua das Flores", "Av. Brasil", "Rua São João", "Av. Goiás",
"Rua 15 de Novembro", "Rua da Independência", "Av. Anhanguera",
"Rua São Paulo", "Rua Minas Gerais", "Av. Tóquio",
"Rua Vitória", "Av. Independência", "Rua Boa Vista",
"Av. Presidente Vargas", "Rua das Palmeiras"
]

clientes = []
usados = set()

for i in range(100):
    import random
    random.seed(i + 42) # Reprodutível

    nome = nomes_base[i % len(nomes_base)]
    sobrenome1 = sobrenomes[i % len(sobrenomes)]
    sobrenome2 = sobrenomes[(i * 3 + 7) % len(sobrenomes)]
    nome_completo = f"{nome} {sobrenome1} {sobrenome2}"

    cidade, estado = cidades[i % len(cidades)]
    bairro = bairros[i % len(bairros)]
    rua = ruas[i % len(ruas)]
    numero = random.randint(1, 999)
    cep = f"{random.randint(10000, 99999)}-{random.randint(100, 999)}"

    # Gera documento único
    doc_num = f"{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(0, 99):02d}"
    doc_key = doc_num[:9]
    if doc_key in usados:
        doc_num = f"{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(0, 99):02d}"
        usados.add(doc_num[:9])

    email = f"{nome.lower()}_{i+1}@email.com"

    # Telefone variado
    ddd = random.choice(["62", "11", "21", "31", "41", "51", "61", "71", "81", "91"])
    telefone = f"{ddd}9{random.randint(8000, 9999)}-{random.randint(1000, 9999)}"

    cliente = {
        "nome": nome_completo.strip(),
        "documento": doc_num,
        "email": email,
        "telefone": telefone,
        "endereço": {
            "rua": f"{rua}, {numero}",
            "bairro": bairro,
            "cidade": cidade,
            "estado": estado,
            "cep": cep,
            "complemento": f"Apto {random.randint(1, 20)}" if random.random() > 0.5 else ""
        },
        "observacoes": f"Cliente teste #{i+1} - gerado automaticamente"
    }

```

```

        clientes.append(cliente)

    return clientes

# =====
# TESTE PRINCIPAL
# =====
def main():
    print("=" * 50)
    print("🚀 TESTE: 100 CLIENTES - SALTOGESTÃO DESKTOP")
    print("=" * 50)

    # 1. Login
    print("\n🔥 ETAPA 1: Login")
    token = fazer_login()
    if not token:
        return

    headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}

    # 2. Limpa clientes de teste anteriores
    print("\n🔥 ETAPA 2: Limpando clientes de teste anteriores")
    try:
        r = requests.get(f"{API_BASE}/api/admin/clientes", headers=headers, timeout=10)
        if r.status_code == 200:
            existentes = r.json()
            for c in existentes:
                if "teste" in c.get("email", "") or "Cliente teste" in str(c.get("observacoes", "")):
                    cid = c.get("id")
                    requests.delete(f"{API_BASE}/api/admin/clientes/{cid}", headers=headers, timeout=5)
            print(f"🔥 Clientes existentes verificados: {len(existentes)}")
    except Exception as e:
        print(f"🔥 Aviso: {e}")

    # 3. Gera 100 clientes
    print("\n🔥 ETAPA 3: Gerando 100 clientes")
    clientes = gerar_clientes_100()
    print(f"✅ {len(clientes)} clientes gerados")

    # 4. Insere todos
    print("\n🔥 ETAPA 4: Inserindo clientes via API")
    inseridos = 0
    falhas = 0
    erros_detalhados = []
    ids_inseridos = []

    inicio = time.time()
    for i, cliente in enumerate(clientes):
        try:
            r = requests.post(
                f"{API_BASE}/api/admin/clientes",
                json=cliente,
                headers=headers,
                timeout=10
            )
            if r.status_code in [200, 201]:
                inseridos += 1
                resp_data = r.json()
                cid = resp_data.get("id") or resp_data.get("cliente", {}).get("id")
                if cid:
                    ids_inseridos.append(cid)
            else:
                falhas += 1
                erros_detalhados.append(f"#{i+1} {cliente['nome']}: HTTP {r.status_code} - {r.text[:100]}")
        except Exception as e:
            falhas += 1
            erros_detalhados.append(f"#{i+1} {cliente['nome']}: {str(e)[:100]}")

    # Progress bar
    if (i + 1) % 10 == 0:
        pct = (i + 1) % 10
        print(f"📊 {i+1}/100 inseridos... ({inseridos} OK, {falhas} falhas)")

    tempo = time.time() - inicio
    print(f"\n✅ Inseridos: {inseridos}/100")
    if falhas > 0:
        print(f"❌ Falhas: {falhas}")
        for err in erros_detalhados[:10]:
            print(f"🔥 {err}")
    print(f"🕒 Tempo: {tempo:.1f}s ({tempo/max(inseridos,1)*1000:.0f}ms/cliente)")

    # 5. Validação: Busca todos
    print("\n🔥 ETAPA 5: Validação - Buscando todos os clientes")
    try:
        r = requests.get(f"{API_BASE}/api/admin/clientes", headers=headers, timeout=10)
        if r.status_code == 200:
            todos = r.json()
            total = len(todos) if isinstance(todos, list) else todos.get("total", 0)
            print(f"✅ Total na API: {total} clientes")

            # Verifica dados de 5 clientes aleatórios
            if isinstance(todos, list) and len(todos) > 0:
                print("\n🔍 Verificação de 5 clientes aleatórios:")
                import random

```

```

        amostra = random.sample(todos, min(5, len(todos)))
        for c in amostra:
            nome = c.get("nome", "?")[:30]
            email = c.get("email", "?")[:25]
            doc = c.get("documento", "?")[:14]
            cli_id = c.get("id", "?")
            print(f"      ✅ ID:{cli_id} | {nome} | {email} | {doc}")
        else:
            print(f"      ❌ Erro na busca: HTTP {r.status_code}")
    except Exception as e:
        print(f"      ❌ Erro: {e}")

# 6. Teste de busca individual
if ids_inseridos:
    print("\n🔴 ETAPA 6: Teste de busca por ID")
    test_id = ids_inseridos[0]
    try:
        r = requests.get(f"{API_BASE}/api/admin/clientes/{test_id}", headers=headers, timeout=5)
        if r.status_code == 200:
            c = r.json()
            print(f"      ✅ Busca por ID {test_id}: {c.get('nome', '?')[:40]}")
        else:
            print(f"      ❌ Falha: HTTP {r.status_code}")
    except Exception as e:
        print(f"      ❌ Erro: {e}")

# 7. Teste de atualização
if ids_inseridos:
    print("\n🔴 ETAPA 7: Teste de atualização")
    test_id = ids_inseridos[1] if len(ids_inseridos) > 1 else ids_inseridos[0]
    try:
        r = requests.put(
            f"{API_BASE}/api/admin/clientes/{test_id}",
            json={"telefone": "6299999-8888", "observacoes": "Atualizado via teste"},
            headers=headers,
            timeout=5
        )
        if r.status_code in [200, 201]:
            print(f"      ✅ Atualização ID {test_id}: OK")
        else:
            print(f"      ❌ Falha: HTTP {r.status_code}")
    except Exception as e:
        print(f"      ❌ Erro: {e}")

# 8. RESULTADO FINAL
print("\n" + "=" * 50)
print("📊 RESULTADO FINAL")
print("=" * 50)
print(f"API:           {API_BASE}")
print(f"Inseridos:      {inseridos}/100")
print(f>Falhas:         {falhas}")
print(f>Tempo total:    {tempo:.1f}s")
print(f>Performance:   {tempo/max(inseridos,1)*1000:.0f}ms/cliente")

if inseridos == 100:
    print("\n🎉 PASSOU! Todos os 100 clientes inseridos com sucesso!")
else:
    print(f"\n⚠️ ATENÇÃO: {falhas} clientes falharam")

# Teste de paginação (se suportado)
print("\n🔴 ETAPA 8: Teste de paginação")
for page in [1, 2, 5]:
    try:
        r = requests.get(
            f"{API_BASE}/api/admin/clientes?page={page}&limit=20",
            headers=headers, timeout=5
        )
        if r.status_code == 200:
            dados = r.json()
            if isinstance(dados, list):
                print(f"      ✅ Página {page}: {len(dados)} registros")
            else:
                print(f"      ✅ Página {page}: {dados}")
    except Exception as e:
        print(f"      ⚠️ Página {page}: {e}")

print("\n✅ Teste concluído!")

if __name__ == "__main__":
    main()

```

 **__init__.py**

desktop/utils/__init__.py • 2 linhas

SaltoGestão Desktop - Utilitários

 **api_client.py**

desktop/utils/api_client.py • 250 linhas

```

"""
=====

```

```

SaltoGestão - Sistema Desktop
Módulo de conexão com API (FastAPI Backend)
=====
"""
import requests
import json
from datetime import datetime
from typing import Optional

class APIClient:
    """Cliente para comunicação com o backend FastAPI."""

    def __init__(self, base_url: str = "http://localhost:8080"):
        self.base_url = base_url.rstrip("/")
        self.token: Optional[str] = None
        self.usuario = None
        self.session = requests.Session()
        self.timeout = 10

    # =====
    # AUTENTICAÇÃO
    # =====

    def login_admin(self, email: str, senha: str) -> dict:
        """Login como administrador (Corumbá Sistemas)."""
        resp = self._post("/api/admin/auth/login", {
            "email": email,
            "senha": senha
        })
        if resp.get("access_token"):
            self.token = resp["access_token"]
            self.usuario = resp.get("usuario", {"tipo": "admin"})
            self.session.headers.update({"Authorization": f"Bearer {self.token}"})
        return resp

    def login_cliente(self, email: str, senha: str) -> dict:
        """Login como cliente (empresa)."""
        resp = self._post("/api/auth/cliente/login", {
            "email": email,
            "senha": senha
        })
        if resp.get("access_token"):
            self.token = resp["access_token"]
            self.usuario = resp.get("usuario", {"tipo": "cliente"})
            self.session.headers.update({"Authorization": f"Bearer {self.token}"})
        return resp

    def login_vendedor(self, email: str, senha: str) -> dict:
        """Login como vendedor."""
        resp = self._post("/api/vendedor/auth/login", {
            "email": email,
            "senha": senha
        })
        if resp.get("access_token"):
            self.token = resp["access_token"]
            self.usuario = resp.get("usuario", {"tipo": "vendedor"})
            self.session.headers.update({"Authorization": f"Bearer {self.token}"})
        return resp

    def logout(self):
        """Limpa sessão."""
        self.token = None
        self.usuario = None
        self.session.headers.pop("Authorization", None)

    @property
    def autenticado(self) -> bool:
        return self.token is not None

    @property
    def tipo_usuario(self) -> str:
        if self.usuario:
            return self.usuario.get("tipo", "desconhecido")
        return "desconhecido"

    # =====
    # ADMIN - DASHBOARD
    # =====

    def admin_dashboard(self) -> dict:
        """Estatísticas do painel admin."""
        return self._get("/api/admin/dashboard")

    # =====
    # ADMIN - PRODUTOS (Catálogo)
    # =====

    def admin_listar_produtos(self) -> list:
        """Lista todos os produtos do catálogo."""
        return self._get("/api/admin/produtos") or []

    # =====
    # ADMIN - CLIENTES
    # =====

```

```

def admin_listar_clientes(self, pagina: int = 1, por_pagina: int = 50) -> dict:
    """Lista clientes com paginação."""
    return self._get(f"/api/admin/clientes?pagina={pagina}&por_pagina={por_pagina}")

def admin_criar_cliente(self, dados: dict) -> dict:
    """Cria um novo cliente (empresa)."""
    return self._post("/api/admin/clientes/criar", dados)

def admin_buscar_cliente(self, cliente_id: int) -> dict:
    """Busca dados de um cliente específico."""
    return self._get(f"/api/admin/clientes/{cliente_id}")

def admin_editar_cliente(self, cliente_id: int, dados: dict) -> dict:
    """Edita dados de um cliente."""
    return self._put(f"/api/admin/clientes/{cliente_id}", dados)

def admin_excluir_cliente(self, cliente_id: int) -> bool:
    """Exclui um cliente."""
    resp = self._delete(f"/api/admin/clientes/{cliente_id}")
    return resp is not None

# =====
# ADMIN - ASSINATURAS
# =====

def admin_listar_assinaturas(self) -> list:
    """Lista todas as assinaturas."""
    return self._get("/api/admin/assinaturas") or []

# =====
# ADMIN - VENDAS
# =====

def admin_listar_vendas(self, pagina: int = 1) -> dict:
    """Lista vendas com paginação."""
    return self._get(f"/api/admin/vendas?pagina={pagina}")

# =====
# VENDEDOR
# =====

def vendedor_listar_clientes(self) -> list:
    """Lista clientes do vendedor."""
    return self._get("/api/vendedor/clientes") or []

def vendedor_cadastrar_cliente(self, dados: dict) -> dict:
    """Vendedor cadastra novo cliente."""
    return self._post("/api/vendedor/clientes/criar", dados)

def vendedor gerar_chave(self, cliente_id: int, produto_id: int) -> dict:
    """Gera chave de ativação para cliente."""
    return self._post("/api/vendedor/gerar-chave", {
        "cliente_id": cliente_id,
        "produto_id": produto_id
    })

# =====
# CLIENTE
# =====

def cliente_dashboard(self) -> dict:
    """Dashboard do cliente."""
    return self._get("/api/cliente/dashboard")

def cliente_meus_dados(self) -> dict:
    """Dados da empresa cliente."""
    return self._get("/api/cliente/meus-dados")

def cliente_produtos_ativos(self) -> list:
    """Produtos ativos do cliente."""
    return self._get("/api/cliente/produtos-ativos") or []

def cliente_gerar_chave(self, produto_id: int) -> dict:
    """Cliente gera chave de ativação."""
    return self._post("/api/cliente/gerar-chave", {"produto_id": produto_id})

# =====
# GERAIS
# =====

def health_check(self) -> bool:
    """Verifica se a API está online."""
    try:
        resp = self._get("/health")
        return resp is not None
    except Exception:
        return False

# =====
# MÉTODOS HTTP INTERNOS
# =====

def _get(self, endpoint: str) -> any:
    try:

```

```

        resp = self.session.get(f"{self.base_url}{endpoint}", timeout=self.timeout)
        if resp.status_code == 401:
            self.logout()
            raise Exception("Sessão expirada. Faça login novamente.")
        if resp.status_code >= 400:
            return {"erro": resp.text, "status": resp.status_code}
        return resp.json() if resp.text else None
    except requests.ConnectionError:
        raise Exception(f"Não foi possível conectar ao servidor ({self.base_url})")
    except requests.Timeout:
        raise Exception("Tempo de conexão esgotado. Verifique sua internet.")
    except Exception as e:
        raise Exception(f"Erro na requisição: {str(e)}")

def _post(self, endpoint: str, dados: dict) -> dict:
    try:
        resp = self.session.post(f"{self.base_url}{endpoint}", json=dados, timeout=self.timeout)
        if resp.status_code == 401:
            self.logout()
            raise Exception("Sessão expirada. Faça login novamente.")
        if resp.status_code >= 400:
            return {"erro": resp.text, "status": resp.status_code}
        return resp.json() if resp.text else {}
    except requests.ConnectionError:
        raise Exception(f"Não foi possível conectar ao servidor ({self.base_url})")
    except requests.Timeout:
        raise Exception("Tempo de conexão esgotado.")
    except Exception as e:
        raise Exception(f"Erro na requisição: {str(e)}")

def _put(self, endpoint: str, dados: dict) -> dict:
    try:
        resp = self.session.put(f"{self.base_url}{endpoint}", json=dados, timeout=self.timeout)
        if resp.status_code == 401:
            self.logout()
            raise Exception("Sessão expirada.")
        if resp.status_code >= 400:
            return {"erro": resp.text, "status": resp.status_code}
        return resp.json() if resp.text else {}
    except requests.ConnectionError:
        raise Exception(f"Não foi possível conectar ao servidor.")
    except Exception as e:
        raise Exception(f"Erro na requisição: {str(e)}")

def _delete(self, endpoint: str) -> any:
    try:
        resp = self.session.delete(f"{self.base_url}{endpoint}", timeout=self.timeout)
        if resp.status_code == 401:
            self.logout()
            raise Exception("Sessão expirada.")
        if resp.status_code >= 400:
            return None
        return resp.json() if resp.text else {"sucesso": True}
    except requests.ConnectionError:
        raise Exception(f"Não foi possível conectar ao servidor.")
    except Exception as e:
        raise Exception(f"Erro na requisição: {str(e)}")

```

config.py

desktop/utils/config.py • 120 linhas

```

"""
=====
SaltoGestão - Configurações e Tema
=====
"""
import json
import os

# Caminho do arquivo de configuração
CONFIG_DIR = os.path.join(os.path.expanduser("~"), ".saltogestao")
CONFIG_FILE = os.path.join(CONFIG_DIR, "config.json")

# =====
# CORES DO TEMA (Dark Mode Profissional)
# =====
CORES = {
    # Primárias
    "primaria": "#1E88E5",          # Azul principal
    "primaria_hover": "#1565C0",    # Azul hover
    "primaria_clara": "#42A5F5",     # Azul claro

    # Backgrounds
    "bg_principal": "#0B1A30",      # Azul escuro (fundo principal)
    "bg_card": "#0F2138",           # Azul mais claro (cards)
    "bg_input": "#162D4A",          # Fundo dos inputs
    "bg_header": "#0A1628",         # Fundo do header

    # Textos
    "texto_primario": "FFFFFF",      # Texto principal (branco)
    "texto_secundario": "#90A4AE",   # Texto secundário (cinza azulado)
    "texto_terciario": "#546E7A",    # Texto terciário (cinza escuro)

    # Status

```



```

"sucesso": "#43A047",      # Verde sucesso
"sucesso_bg": "#1B5E20",   # Fundo verde escuro
"erro": "#E53935",         # Vermelho erro
"erro_bg": "#B71C1C",      # Fundo vermelho escuro
"aviso": "#FB8C00",        # Laranja aviso
"aviso_bg": "#E65100",     # Fundo laranja escuro
"info": "#039BE5",         # Azul info

# Bordas
"borda": "#1E3A5F",        # Borda padrão
"borda_foco": "#1E88E5",   # Borda com foco

# Botões
"btn_primario": "#1E88E5",
"btn_primario_hover": "#1565C0",
"btn_sucesso": "#43A047",
"btn_sucesso_hover": "#2E7D32",
"btn_erro": "#E53935",
"btn_erro_hover": "#C62828",
"btn_secundario": "#37474F",
"btn_secundario_hover": "#455A64",

# Tabela
"tabela_header": "#0F2138",
"tabela_linha_par": "#0D1E36",
"tabela_linha_impar": "#112540",
"tabela_linha_hover": "#162D4A",
"tabela_selecionada": "#1A3A5C",
}

# =====
# FONTE
# =====
FONTE = "Segoe UI"
FONTE_MONO = "Consolas"

# =====
# DIMENSÕES
# =====
JANELA_LARGURA = 1280
JANELA_ALTURA = 780
SIDEBAR_LARGURA = 240
HEADER_ALTURA = 60

# =====
# CONFIGURAÇÃO DO SERVIDOR
# =====
DEFAULT_API_URL = "http://localhost:8080"

def carregar_config() -> dict:
    """Carrega configurações salvas."""
    if os.path.exists(CONFIG_FILE):
        try:
            with open(CONFIG_FILE, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            pass
    return {"api_url": DEFAULT_API_URL, "ultimo_email": ""}

def salvar_config(config: dict):
    """Salva configurações."""
    os.makedirs(CONFIG_DIR, exist_ok=True)
    with open(CONFIG_FILE, "w", encoding="utf-8") as f:
        json.dump(config, f, ensure_ascii=False, indent=2)

def obter_api_url() -> str:
    """Retorna a URL da API configurada."""
    config = carregar_config()
    return config.get("api_url", DEFAULT_API_URL)

def salvar_api_url(url: str):
    """Salva a URL da API."""
    config = carregar_config()
    config["api_url"] = url
    salvar_config(config)

def salvar_ultimo_email(email: str):
    """Salva o último email usado no login."""
    config = carregar_config()
    config["ultimo_email"] = email
    salvar_config(config)

```

 **README.md**

desktop/README.md • 44 linhas

SaltoGestão - Sistema Desktop

Sistema de gestão comercial desktop desenvolvido em Python com CustomTkinter.

```
## Como Executar

```bash
cd SaltoGestao/desktop
chmod +x run.sh
./run.sh
```

Ou manualmente:

```bash
cd SaltoGestao/desktop
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 main.py
```

## Requisitos

- Python 3.10+
- tkinter (geralmente já vem com o Python)
- customtkinter (instalado automaticamente pelo run.sh)

## Telas

- 🚪 **Login** - Admin, Vendedor ou Cliente
- 📊 **Dashboard** - Estatísticas e atividades recentes
- 👤 **Clientes** - CRUD completo com busca
- 📦 **Produtos** - Catálogo de produtos (grid visual)
- 💰 **Vendas** - Histórico e nova venda
- 📝 **Assinaturas** - Gestão de assinaturas
- 📋 **Relatórios** - Exportação de relatórios

## Configuração

O sistema conecta ao backend FastAPI. A URL do servidor pode ser configurada na tela de login.

URL padrão: `http://localhost:8080`
```

requirements.txt

desktop/requirements.txt • 3 linhas

```
flet==0.24.1
requests>=2.31.0
```

11. Testes e Documentação

 **testar_100_clientes_api.py**

testar_100_clientes_api.py • 370 linhas

```
"""
=====
Teste de 100 Clientes - SaltoGestão Backend
=====
Insere 100 clientes via API operacional (empresa CNPJ: 12.345.678/0001-90)
Pré-requisitos:
    1. Backend rodando em localhost:8080
    2. Usuário colaborador criado na empresa
Uso: python3 testar_100_clientes.py
"""

import requests
import json
import time
import sys

# =====
# CONFIGURAÇÃO - API do SaltoGestão
# =====
API_BASE = "http://localhost:8080"
CNPJ_EMPRESA = "12.345.678/0001-90" # CNPJ da empresa de teste

# =====
# ETAPA 1: CRIAR EMPRESA E USUÁRIO
# =====
def preparar_ambiente():
    """Cria empresa e usuário colaborador se não existirem."""
    # Cria um admin primeiro para gerar token de cadastro
    # Usa o endpoint de login operacional
    try:
        # Tenta criar um usuário colaborador via seed direto no banco
        import os
        os.chdir("/opt/projects/SaltoGestao/backendPy")
        sys.path.insert(0, "/opt/projects/SaltoGestao/backendPy")

        from database import SessionLocal, engine
        from models_core import BaseCore, Usuario, AdminCorumba, Empresa
        from security import hash_senha

        BaseCore.metadata.create_all(bind=engine)
        db = SessionLocal()

        # Cria empresa se não existe
        empresa = db.query(Empresa).filter(Empresa.documento == CNPJ_EMPRESA).first()
        if not empresa:
            empresa = Empresa(
                razao_social="Empresa Teste LTDA",
                nome_fantasia="Empresa Teste",
                documento=CNPJ_EMPRESA,
                email="contato@empresateste.com",
                telefone="62999990000",
                ativo=True
            )
            db.add(empresa)
            db.commit()
            print(f"✅ Empresa criada: {empresa.nome_fantasia}")

        # Cria usuário admin da empresa se não existe
        user = db.query(Usuario).filter(
            Usuario.username == "admin_empresa",
            Usuario.empresa_documento == CNPJ_EMPRESA
        ).first()
        if not user:
            user = Usuario(
                nome="Admin Empresa",
                username="admin_empresa",
                email="admin@empresateste.com",
                senha_hash=hash_senha("empresa123"),
                papel="ADMIN",
                empresa_documento=CNPJ_EMPRESA,
                ativo=True
            )
            db.add(user)
            db.commit()
            print(f"✅ Usuário criado: admin_empresa / empresa123 (ADMIN)")

        # Se não existe admin Corumbá, cria um
        admin = db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumba.com").first()
        if not admin:
            admin = AdminCorumba(
                nome="Admin Corumbá",
                email="admin@corumba.com",
                senha_hash=hash_senha("corumba123"),
                papel="ADMIN",
                ativo=True
            )
            db.add(admin)
            db.commit()
            print(f"✅ Admin Corumbá criado: admin@corumba.com (ADMIN)")
```

```

        db.add(admin)
        db.commit()
        print(f"✅ Admin Corumbá criado: admin@corumba.com / corumba123")

    db.close()
    return True
except Exception as e:
    print(f"❌ Erro ao preparar ambiente: {e}")
    return False

# =====
# ETAPA 2: LOGIN
# =====
def fazer_login():
    """Faz login como colaborador da empresa."""
    credenciais = [
        {"username": "admin_empresa", "senha": "empresa123"},
        {"email": "admin@corumbasistemas.com.br", "senha": "corumba2026"},
        {"email": "vendedor@corumbasistemas.com.br", "senha": "vendedor123"},
    ]
    for cred in credenciais:
        try:
            r = requests.post(f"{API_BASE}/api/login", json=cred, timeout=5)
            if r.status_code == 200:
                data = r.json()
                token = data.get("access_token") or data.get("token")
                if token:
                    print(f"✅ Login OK: {cred}")
                    return token, data.get("usuario", data.get("admin", {}))
        except Exception:
            continue
    print(f"❌ Falha no login. Verifique se o backend está rodando.")
    return None, None

# =====
# GERA DADOS DE 100 CLIENTES
# =====
def gerar_clientes_100():
    """Gera 100 clientes com dados variados."""
    import random
    random.seed(42)

    nomes_base = [
        "João", "Maria", "José", "Ana", "Pedro", "Carla", "Lucas", "Fernanda",
        "Marcos", "Juliana", "Rafael", "Patricia", "Bruno", "Camila", "Diego",
        "Amanda", "Gustavo", "Larissa", "Thiago", "Beatriz", "Felipe", "Mariana",
        "Rodrigo", "Leticia", "André", "Vanessa", "Marcelo", "Tatiana", "Leonardo",
        "Priscila", "Eduardo", "Sabrina", "Ricardo", "Isabela", "Fábio", "Natália",
        "Alexandre", "Daniela", "Guilherme", "Bianca", "Vinicius", "Lorena",
        "Matheus", "Gabriela", "Samuel", "Helena", "Isaac", "Valentina", "Daniel",
        "Laura", "Henrique", "Manuela", "Caio", "Sophia", "Enzo", "Heloísa",
        "Miguel", "Alice", "Arthur", "Júlia", "Bernardo", "Luísa", "Heitor",
        "Isabella", "Davi", "Lara", "Lorenzo", "Cecilia", "Theo", "Eloá",
        "Gabriel", "Yasmin", "Noah", "Rafaela", "Benjamin", "Giovanna", "Anthony",
        "Clara", "Joaquim", "Melissa", "Nicolas", "Rebeca", "Lucca", "Agatha",
        "Ryan", "Esther", "Ian", "Nicole", "David", "Sarah", "Pedro", "Vitória"
    ]

    sobrenomes = [
        "Silva", "Santos", "Oliveira", "Souza", "Rodrigues", "Ferreira",
        "Alves", "Pereira", "Lima", "Gomes", "Costa", "Ribeiro", "Martins",
        "Carvalho", "Araújo", "Melo", "Barbosa", "Rocha", "Dias", "Nascimento"
    ]

    clientes = []
    usados_docs = set()

    for i in range(100):
        nome = nomes_base[i % len(nomes_base)]
        sobrenome = sobrenomes[i % len(sobrenomes)]
        nome_completo = f"{nome} {sobrenome} {sobrenomes[(i*3+7) % len(sobrenomes)]}"

        # Gera CPF único
        while True:
            cpf = f"{random.randint(100,999)}{random.randint(100,999)}{random.randint(100,999)}{random.randint(0,99):02d}"
            if cpf not in usados_docs:
                usados_docs.add(cpf)
                break

        bairro = random.choice([
            "Centro", "Jardim América", "Setor Oeste", "Setor Bueno",
            "Jardim Goiás", "Campinas", "Vila Aurora", "Jardim Europa"
        ])

        cidade = random.choice(["Aparecida de Goiânia", "Goiânia", "Anápolis"])
        estado = "GO"
        cep = f"{random.randint(10000,99999)}-{random.randint(100,999)}"

        ddd = random.choice(["62", "11", "21", "31", "41", "51"])
        telefone = f"{ddd}9{random.randint(8000,9999)}-{random.randint(1000,9999)}"

        email = f"{nome.lower()}@i+1@email.com"

```

```

    cliente = {
        "nome": nome_completo.strip(),
        "telefone": telefone,
        "email": email,
        "cpf_cnpj": cpf,
        "endereco": f"Rua {random.choice(['Flores', 'Brasil', 'São João', 'Goiás', 'Palmeiras'])} {random.randint(1,999)}",
        "cidade": cidade,
        "estado": estado,
        "cep": cep,
        "data_nascimento": f"{random.randint(1960,2005)}-{random.randint(1,12):02d}-{random.randint(1,28):02d}"
    }
    clientes.append(cliente)

return clientes

# =====
# TESTE PRINCIPAL
# =====
def main():
    print("=" * 60)
    print("🚀 TESTE: 100 CLIENTES - SALTOGESTÃO (API Operacional)")
    print("=" * 60)

    # 1. Prepara ambiente
    print("\n🔥 ETAPA 1: Preparando ambiente...")
    preparar_ambiente()

    # 2. Login
    print("\n🔥 ETAPA 2: Login")
    token, usuario = fazer_login()
    if not token:
        print("❌ Não foi possível fazer login. Abortando.")
        return

    headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
    print(f"  Usuário: {usuario.get('nome', '?')} | Papel: {usuario.get('papel', '?')}")
    empresa_doc = usuario.get("empresa_documento", CNPJ_EMPRESA)
    print(f"  Empresa: {empresa_doc}")

    # 3. Gera 100 clientes
    print("\n🔥 ETAPA 3: Gerando 100 clientes")
    clientes = gerar_clientes_100()
    print(f"  ✅ {len(clientes)} clientes gerados")

    # 4. Insere todos via POST /api/clientes/
    print("\n🔥 ETAPA 4: Inserindo clientes via POST /api/clientes/")
    inseridos = 0
    falhas = 0
    erros_detalhados = []
    ids_inseridos = []

    inicio = time.time()
    for i, cliente in enumerate(clientes):
        try:
            r = requests.post(
                f"{API_BASE}/api/clientes/",
                json=cliente,
                headers=headers,
                timeout=10
            )
            if r.status_code in [200, 201]:
                inseridos += 1
                resp_data = r.json()
                cid = resp_data.get("id")
                if cid:
                    ids_inseridos.append(cid)
            else:
                falhas += 1
                if falhas <= 5:
                    erros_detalhados.append(f"#{i+1} {cliente['nome'][:20]}: HTTP {r.status_code} - {r.text[:80]}")
        except Exception as e:
            falhas += 1
            if falhas <= 5:
                erros_detalhados.append(f"#{i+1} {cliente['nome'][:20]}: {str(e)[:80]}")

    if (i + 1) % 20 == 0:
        print(f"  📊 {i+1}/100 inseridos... ({inseridos} OK, {falhas} falhas)")

    tempo = time.time() - inicio
    print(f"\n  ✅ Inseridos: {inseridos}/100")
    if falhas > 0:
        print(f"  ❌ Falhas: {falhas}")
        for err in erros_detalhados:
            print(f"    {err}")
    print(f"  ⌚ Tempo: {tempo:.1f}s ({tempo/max(inseridos,1)*1000:.0f}ms/cliente)")

    # 5. Validação: Busca todos
    print("\n🔥 ETAPA 5: Validação - GET /api/clientes/")
    try:
        r = requests.get(
            f"{API_BASE}/api/clientes/?ativo=true&ordem=nome",
            headers=headers, timeout=10
        )
        if r.status_code == 200:

```

```

    todos = r.json()
    print(f"✅ Total na API: {len(todos)} clientes")

    # Mostra 5 primeiros
    print("\n🔍 Primeiros 5 clientes:")
    for c in todos[:5]:
        print(f"✅ ID:{c.get('id')} | {c.get('nome','?')[:30]} | {c.get('cpf_cnpj','?')}")

    print(f"\n🔍 Últimos 5 clientes:")
    for c in todos[-5:]:
        print(f"✅ ID:{c.get('id')} | {c.get('nome','?')[:30]} | {c.get('cpf_cnpj','?')}")
    else:
        print(f"❌ Erro: HTTP {r.status_code} - {r.text[:100]}")
except Exception as e:
    print(f"❌ Erro: {e}")

# 6. Teste de busca por ID
if ids_inseridos:
    print("\n🔍 ETAPA 6: Teste de busca por ID")
    test_id = ids_inseridos[0]
    try:
        r = requests.get(f"{API_BASE}/api/clientes/{test_id}", headers=headers, timeout=5)
        if r.status_code == 200:
            c = r.json()
            print(f"✅ GET /api/clientes/{test_id}: {c.get('nome','?')[:40]}")
        else:
            print(f"❌ Falha: HTTP {r.status_code}")
    except Exception as e:
        print(f"❌ Erro: {e}")

# 7. Teste de atualização
if len(ids_inseridos) > 1:
    print("\n🔍 ETAPA 7: Teste de atualização (PUT)")
    test_id = ids_inseridos[1]
    try:
        r = requests.put(
            f"{API_BASE}/api/clientes/{test_id}",
            json={"telefone": "6299999-8888"},
            headers=headers,
            timeout=5
        )
        if r.status_code in [200, 201]:
            c = r.json()
            print(f"✅ PUT /api/clientes/{test_id}: tel={c.get('telefone','?')}")
        else:
            print(f"❌ Falha: HTTP {r.status_code}")
    except Exception as e:
        print(f"❌ Erro: {e}")

# 8. Teste de desativação
if len(ids_inseridos) > 2:
    print("\n🔍 ETAPA 8: Teste de desativação (DELETE)")
    test_id = ids_inseridos[2]
    try:
        r = requests.delete(f"{API_BASE}/api/clientes/{test_id}", headers=headers, timeout=5)
        if r.status_code in [200, 204]:
            print(f"✅ DELETE /api/clientes/{test_id}: desativado")
            # Verifica se está inativo
            r2 = requests.get(
                f"{API_BASE}/api/clientes/?ativo=false",
                headers=headers, timeout=5
            )
            if r2.status_code == 200:
                inativos = r2.json()
                print(f"✅ Clientes inativos: {len(inativos)}")
            else:
                print(f"❌ Falha: HTTP {r.status_code}")
        except Exception as e:
            print(f"❌ Erro: {e}")

# 9. RESULTADO FINAL
print("\n" + "=" * 60)
print("📊 RESULTADO FINAL")
print("=" * 60)
print(f"API: {API_BASE}")
print(f"Empresa: {empresa_doc}")
print(f>Inseridos: {inseridos}/100")
print(f>Falhas: {falhas}")
print(f>Tempo total: {tempo:.1f}s")
print(f>Performance: {tempo/max(inseridos,1)*1000:.0f}ms/cliente")

if inseridos == 100:
    print("\n🎉 PASSOU! Todos os 100 clientes inseridos com sucesso!")
    print("✅ API operacional SaltoGestão funcionando corretamente!")
elif inseridos > 90:
    print(f"\n✅ QUASE LÁ! {inseridos}/100 clientes inseridos.")
else:
    print(f"\n⚠️ PROBLEMA: {falhas} clientes falharam")

print("\n✅ Teste concluído!")

if __name__ == "__main__":
    main()

```

README.md

documentos/README.md • 26 linhas

```
# 📁 Documentos

Esta pasta armazena documentos que não são código-fonte do projeto.

## O que vai aqui

- PDFs – manuais, guias, propostas comerciais, contratos
- Planilhas – Excel (.xlsx), CSV, Google Sheets exportadas
- Apresentações – PowerPoint (.pptx), PDFs de slides
- Imagens de referência – wireframes, mockups, prints de tela
- Outros documentos – documentos fiscais, certificados, etc.

## Organização

Crie subpastas por tipo ou projeto se necessário:

...
documentos/
├─ guias/
├─ propostas/
├─ planilhas/
└─ ...
...

> ⚠️ Nota: Não coloque código-fonte aqui. Use as pastas `backendPy/`, `frontend-web/`, `desktop-app/` para isso.
```

README.md

sites/README.md • 26 linhas

```
# 🌐 Sites

Esta pasta armazena sites completos criados para o Corumbá Sistemas / SaltoGestão.

## O que vai aqui

- Landing pages – páginas de vendas, captação de leads
- Sites institucionais – páginas para barbearias, clientes, etc.
- Páginas de demonstração – demos, portfolios, Catálogos online

## Estrutura por site

Cada site fica em sua própria subpasta:

...
sites/
├─ barbearia-template-1/
├─ barbearia-template-2/
├─ landing-saltdgestao/
└─ ...
...

Cada subpasta é um site independente com seus próprios arquivos HTML, CSS, JS, imagens, etc.

> ⚠️ Nota: O frontend principal do SaltoGestão fica em `frontend-web/`. Esta pasta é para sites separados criados sob demanda.
```